Secoinsa S.O. Multi-usuario 6.2

Guía práctica del sistema operativo del ordenador Serie 20 de Secoinsa

Uso del sistema • Administración • Programación • Referencia del sistema

Guía y documentación independientes - 27 de septiembre de 2026

Eduardo Cruz - https://github.com/ArcadeHacker/

Contenido

- Secoinsa S.O. Multi-usuario 6.2
- Acerca de este documento
- Prefacio
- Cómo usar este documento

Parte I — El sistema

- Capítulo 1 — Conoce el sistema operativo
- Capítulo 2 — Visión general de la arquitectura
- Capítulo 3 — Arranque, conexión y salida

Parte II — Uso del sistema

- Capítulo 4 — Uso del terminal
- Capítulo 5 — Comandos, nombres de archivo y discos
- Capítulo 6 — Tu primer archivo
- Capítulo 7 — Edición de archivos y ejecución de programas
- Capítulo 8 — Automatización del trabajo con EXEC
- Capítulo 9 — Cuentas, mensajes y hábitos cotidianos

Parte III — Administración del sistema

- Capítulo 10 — Administración de cuentas
- Capítulo 11 — Gestión del almacenamiento
- Capítulo 12 — Dispositivos, impresión y particiones
- Capítulo 13 — Copias de seguridad y recuperación
- Capítulo 14 — Mantenimiento del arranque y fin del trabajo

Parte IV — Programación

- Capítulo 15 — Tu primer programa en BASIC
- Capítulo 16 — Valores, decisiones y bucles en BASIC
- Capítulo 17 — Programación con archivos
- Capítulo 18 — Escritura de programas en código máquina Z80
- Capítulo 19 — Uso de los servicios del sistema

Parte V — Fundamentos del sistema

- Capítulo 20 — Organización y memoria
- Capítulo 21 — Arquitectura de arranque
- Capítulo 22 — Particiones, identidad y protección
- Capítulo 23 — Conceptos de almacenamiento y dispositivos

Parte VI — Funcionamiento interno del sistema operativo

- Capítulo 24 — Entrada al núcleo y estructuras compartidas
- Capítulo 25 — Planificación, devoluciones de llamada y sincronización
- Capítulo 26 — Implementación del sistema de archivos
- Capítulo 27 — E/S de disco y traducción física
- Capítulo 28 — Controladores de terminal, serie, impresora y cinta
- Capítulo 29 — Impresión diferida y mensajes
- Capítulo 30 — Procesamiento de comandos, carga y terminación

Parte VII — Referencia

- Capítulo 31 — Referencia de comandos
- Capítulo 32 — Referencia de servicios del sistema
- Capítulo 33 — Diagnósticos y mensajes
- Capítulo 34 — Dispositivos y límites del sistema
- Capítulo 35 — Referencia de grafía de comandos y de lenguajes

- Apéndice A. Formatos de programa, de lenguaje y de archivado
- Apéndice B. Mapas de memoria, E/S, interrupciones y rutinas
- Apéndice C. Archivos del sistema operativo y componentes de los lenguajes
- Apéndice D. Comandos, instrucciones y funciones de BASIC
- Apéndice E. Atlas de estructuras binarias
- Glosario
- Índice alfabético

- Emulación en MAME

Acerca de este documento

Esta guía independiente describe el sistema operativo S.O. Multi-usuario, versión 6.2, del ordenador Secoinsa Serie 20 de fabricación española. Este sistema operativo podría ser la copia superviviente más antigua que se conoce de un sistema operativo en español: sus nombres de comandos, mensajes y textos de ayuda datan de 1981 y tienen importancia histórica.

El documento existe porque, en el momento de su publicación, no se tiene constancia de que se conserve ningún juego de manuales técnicos del sistema operativo de esta máquina. La guía es fruto de la ingeniería inversa de un ordenador Secoinsa Serie 20 superviviente y del examen del comportamiento y la implementación de su sistema operativo. Se trata de una reconstrucción independiente, no de una reimpresión de un manual del fabricante.

Los nombres originales de productos, comandos y mensajes conservan su grafía histórica. Su inclusión no implica que el fabricante sea autor de esta edición ni que la respalde. Este documento no concede ningún derecho sobre el software del sistema operativo.

Prefacio

Secoinsa (Sociedad Española de Comunicaciones e Informática, S.A.) se fundó en 1975 como parte del esfuerzo de España por crear una industria informática nacional. Sus socios fundadores fueron el Instituto Nacional de Industria (INI), el grupo industrial del Estado, junto con Telefónica y Fujitsu. La idea era combinar el diseño y la fabricación españoles con tecnología extranjera. Fujitsu aportó a la empresa vínculos industriales internacionales e invirtió en su fábrica de Málaga.

Gran parte de la base técnica de Secoinsa procedía de Telesincro, una empresa electrónica española fundada en 1963 que ya fabricaba ordenadores de gestión mucho antes de que existiera Secoinsa. A su máquina de facturación Winner/Contafac le siguieron la Factor Q y la Factor R y, a principios de los años setenta, la Factor S programable. Todas ellas se construyeron para la contabilidad, la facturación y el trabajo de oficina cotidiano. Secoinsa negoció el control mayoritario de Telesincro en 1975 y completó la compra en 1976. Telesincro conservó su propio nombre y sus productos dentro del grupo, y aportó una gama de productos consolidada y una larga experiencia en el suministro de equipos de gestión.

La gama propia de Secoinsa se basó en diseños tanto de Telesincro como de Fujitsu. Una de sus máquinas fue el ordenador Secoinsa Serie 20, un sistema de gestión compacto basado en terminales que se presentó a principios de los años ochenta. Se suministraba con terminales Secoinsa IS 9502, cada uno con teclado y una pantalla monocroma verde de 24 líneas de 80 caracteres. Para quienes lo usaban, el terminal era en la práctica el ordenador: los comandos, los programas y el contenido de los archivos aparecían allí en forma de texto.

Tras una serie de dificultades comerciales, entre ellas un intento fallido de entrar en el mercado de los ordenadores personales y educativos en 1984-1985 con la versión del FM-7 de Fujitsu comercializada con la marca Secoinsa, en 1986 la empresa aceptó fusionarse con Fujitsu España, que la absorbió junto con todos sus activos y obligaciones.

Parece que se ha conservado muy poco del trabajo informático de Secoinsa. Sus máquinas se construyeron para oficinas, bancos y organismos públicos, no para hogares, y cuando quedaban obsoletas normalmente se desguazaban en lugar de conservarse. Se conocen pocos ejemplares supervivientes de la mayoría de los modelos, y el software y la documentación están en una situación aún peor. Solo se conoce la existencia de un puñado de folletos, descripciones técnicas y registros oficiales.

Para una empresa que estuvo en el centro del intento de España de crear su propia industria informática, es un legado muy escaso, y merece la pena documentar cada máquina, manual o listado recuperado antes de que también desaparezca.

El valor histórico de la conservación de este ordenador Secoinsa Serie 20 reside en el entorno de trabajo completo, que incluye su raro sistema operativo. El sistema basado en cpu Z80 que se describe aquí combina varios usuarios en terminales, memoria por bancos, archivos de gestión estructurados y

un entorno BASIC integrado. Explicar cómo funcionan juntas estas funciones preserva una parte de la historia de la informática española que una cronología de la empresa o una especificación del hardware no pueden transmitir por sí solas.

Este documento enseña y describe el sistema operativo «S.O. Multi-usuario» del ordenador Secoinsa Serie 20, sus nombres de comandos en español y su notación de archivos, así como secuencias cotidianas como conectarse al sistema, acceder a archivos, realizar tareas básicas y desconectar la sesión. Los capítulos posteriores convierten esas mismas operaciones en archivos de comandos e introducen la programación en BASIC. Los capítulos técnicos explican cómo llegan esas operaciones visibles al ejecutivo y a los dispositivos.

Cómo usar este documento

Si es tu primera sesión, empieza por los capítulos 1 a 5. El capítulo 6 presenta las operaciones con archivos. El capítulo 7 trata la edición de texto y la ejecución de BASIC compilado; el capítulo 8 presenta los archivos de comandos de EXEC, desde un primer saludo hasta las variables y los bucles. El capítulo 9 trata las cuentas y los mensajes. Sigue los ejemplos en orden; los ejemplos posteriores usan archivos creados anteriormente. Antes de practicar esos ejemplos, pide una cuenta de prácticas normal y privilegios suficientes para usar EDITAR y BASIC.

Los administradores deberían continuar con los capítulos 10 a 14. Los programadores pueden pasar de los ejemplos de operaciones con archivos a los capítulos 15 a 19: un primer programa en BASIC, valores y control de flujo, E/S de archivos, código máquina Z80 y, por último, servicios del sistema y extensiones de BASIC. Los capítulos 20 a 30 explican el sistema con una profundidad creciente. Los capítulos 31 a 35 ofrecen referencias de comandos, servicios, errores, dispositivos y límites, seguidas de la grafía de los comandos y de los vocabularios independientes del editor, de EXEC y del lenguaje de consulta. Emulación en MAME, después del índice alfabético, ofrece instrucciones para emular y arrancar el sistema en MAME. Los apéndices recogen los formatos de programa, los mapas de hardware, las explicaciones de los archivos del sistema, el vocabulario de BASIC y las estructuras binarias.

Introducción de los ejemplos

Un bloque de código precedido de una instrucción para *introducir* texto contiene exactamente el texto que debes escribir. Pulsa Intro después de cada comando. No escribas el indicador salvo que las instrucciones digan expresamente que forma parte de la entrada. El procesador de comandos muestra `>`, EDITAR muestra `*` y BASIC muestra `-`. Comprueba el indicador antes de introducir un comando: el `RUN` de BASIC y el `RUN` del procesador de comandos son operaciones distintas.

Cada comando introductorio se presenta por separado. Lee el párrafo que lo sigue antes de continuar. Los listados de programa más largos son ejemplos completos que se introducen línea numerada a línea numerada; la explicación que los acompaña da cuenta de cada instrucción.

Los nombres como `NOTA.TEXTO:S`, `COPIA.TEXTO:S` y `HOLA.BASICOBJ:S` son nombres de práctica. Antes de crear un nombre, comprueba que no esté en uso. `S` es una asociación lógica de disco, no un directorio. Los ejemplos necesitan una asociación S accesible y con permiso de escritura; si la tuya es distinta, pregunta al administrador qué asociación debes usar.

Notación sintáctica

Notación	Significado
`LISDIR`	Nombre literal del comando
`nombre.tipo:unidad`	Sustituye estos campos por tu especificación de archivo
`[elemento]`	Elemento opcional; no escribas los corchetes
`elemento...`	Repetición admitida por ese comando
`(VERIFI)`	Paréntesis literales y palabra clave de opción; escríbelos
`A` o `B` en el texto	Elige la alternativa indicada; no escribas la palabra «o»
`0x1234`	Número hexadecimal en la explicación
`08h`	Constante hexadecimal en notación de ensamblador
`Ctrl + X`	Mantén pulsada Ctrl y pulsa X
`Esc, Q`	Pulsa Esc, suéltala y pulsa Q

Las comillas son literales en BASIC. No se presupone que CSI (el procesador de comandos, también llamado indicador del sistema) admita de forma general el entrecomillado ni la continuación de comandos. Mientras aprendes, usa los nombres completos de los comandos. El capítulo 5 explica los nombres de comandos y las limitaciones de las abreviaturas.

Una advertencia antes de un ejemplo de mantenimiento señala un cambio real o una operación destructiva. Lee sus requisitos previos antes de escribir. Los ejemplos de referencia que afectan al dispositivo A requieren que un administrador establezca qué designa A. No son instrucciones para formatear un disco cualquiera que esté conectado.

Niveles de detalle

Los procedimientos de usuario describen lo que aparece en el terminal. Las secciones de programación describen la interfaz que utiliza un programa. Las secciones de funcionamiento interno describen la implementación. Las notas de las tablas identifican interpretaciones probables, campos desconocidos y los límites de una interfaz. Estas distinciones evitan que un contrato incompleto se confunda con una regla general.

Parte I — El sistema

Capítulo 1 — Conoce el sistema operativo

S.O. Multi-usuario versión 6.2 es un sistema operativo multiusuario para el ordenador Secoinsa Serie 20, basado en el Z80. Trabajas escribiendo comandos en un terminal. El sistema responde con texto, realiza la operación solicitada y normalmente vuelve a mostrar un indicador para el siguiente comando.

El procesador de comandos se llama CSI (procesador de comandos, también conocido como indicador del sistema). Entiende nombres de comandos como `AYUDA`, `LISDIR` y `COPIFI`. La mayoría de los comandos son programas que se cargan cuando se solicitan. Aprender estos nombres es más útil que buscar equivalencias con los comandos de otro sistema operativo: la puntuación, las opciones y las reglas de archivos son propias de este sistema.

Tu terminal, tu cuenta y tu partición

Tu cuenta identifica tus archivos y tus privilegios. Tu partición contiene un programa en ejecución, su memoria, sus asignaciones de dispositivos y su estado de conexión. Pueden funcionar varias particiones a la vez. El ejecutivo alterna entre ellas para que un usuario pueda trabajar mientras se ejecuta el programa de otro.

Estos términos describen cosas distintas. Cambiar de cuenta cambia la identidad; detener una partición detiene un contexto de ejecución. Cerrar la sesión pone fin a tu conexión. Ninguna de esas acciones significa que se haya apagado toda la máquina.

Qué puedes hacer

El sistema permite listar archivos, editar texto, copiar, imprimir, mantener cuentas y administrar el almacenamiento. Los archivos de comandos de EXEC automatizan secuencias de comandos. BASIC incluye un editor, un intérprete, un compilador y un entorno de ejecución. Los programas en código máquina pueden llamar a servicios del ejecutivo para la entrada de consola, los archivos, los dispositivos y la planificación.

Los archivos usan un nombre de dos partes, como `NOTA.TEXTO`, seguido cuando es necesario de una asociación de disco: `NOTA.TEXTO:S`. No hay rutas de directorios jerárquicas que aprender. La propiedad y las asociaciones de disco determinan qué archivos puede encontrar un comando.

Qué significa la versión

El sistema operativo se identifica como versión 6.2. CSI (el procesador de comandos, también llamado indicador del sistema) y EXEC tienen su propio número de versión, 5.5, y BASIC S20 tiene el número de versión 1.0. El número de un componente no tiene por qué coincidir con el del sistema operativo. Este documento conserva tal cual los nombres y mensajes en español que verás en el terminal y explica su funcionamiento.

Empieza por el arranque, en el capítulo 3. No necesitas entender la conmutación de bancos de memoria ni los formatos ejecutables para listar un archivo. El capítulo siguiente ofrece un mapa del sistema para que esos detalles encajen cuando llegues a ellos.

Capítulo 2 — Visión general de la arquitectura

El terminal, el entorno de comandos y el almacenamiento colaboran a través del sistema operativo. Un comando como LISDIR empieza siendo texto escrito en un terminal. CSI selecciona el programa del comando, ese programa pide al sistema operativo información de archivos y el resultado aparece en el terminal. Este capítulo identifica los componentes de ese recorrido; sus direcciones de memoria y sus interfaces internas se tratan más adelante.

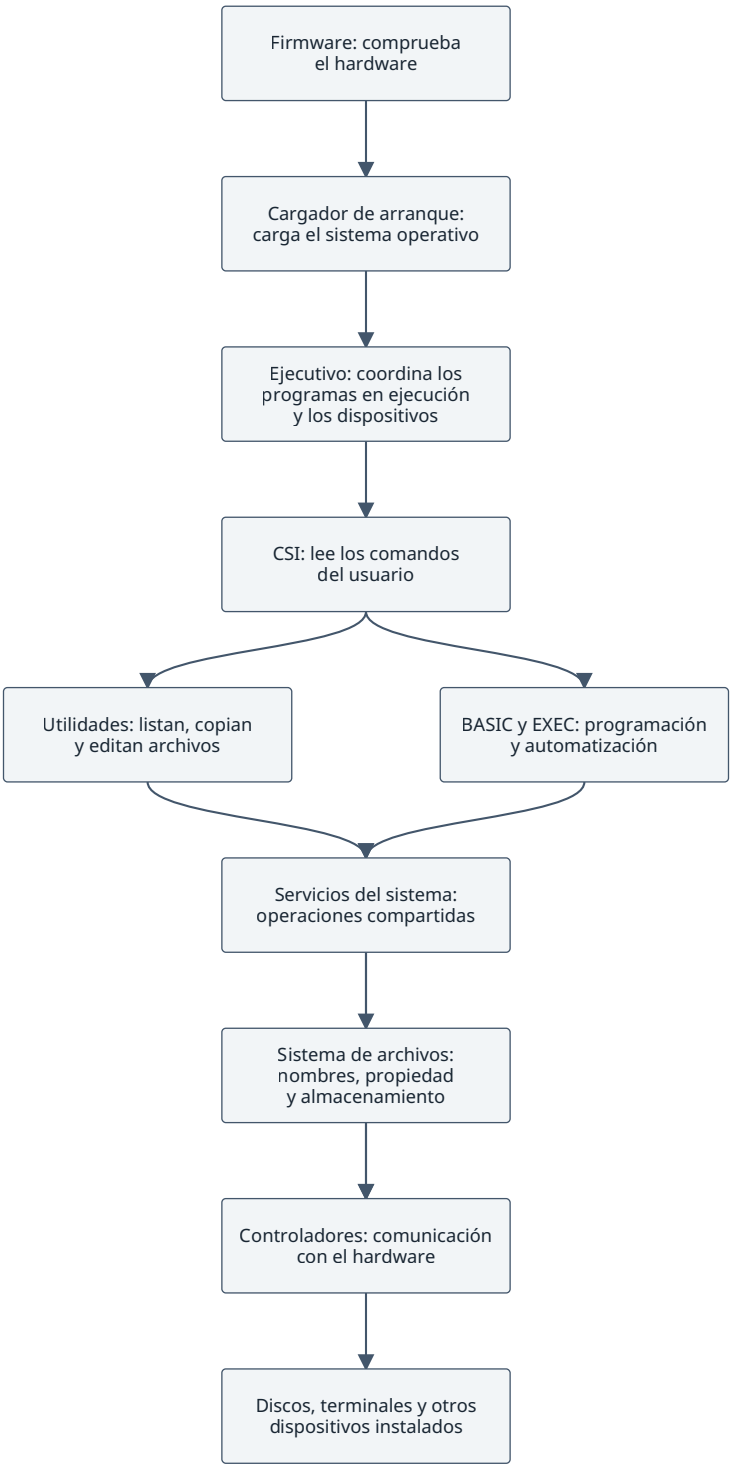


Figura 2.1. Del arranque a la ejecución de comandos. Una definición de dispositivo no garantiza por sí sola que el hardware correspondiente esté instalado.

Arranque del sistema operativo

El firmware es el programa inicial que se guarda en la memoria de solo lectura del ordenador. Comprueba el hardware y lee de disco un pequeño programa llamado cargador de arranque (bootstrap). El cargador de arranque carga SISTEMA.NUCLEO, el ejecutivo del sistema operativo. SISTEMA.STARTUP aporta código de inicialización adicional. Es un componente ejecutable, no un archivo de texto con comandos que el operador pueda editar.

El ejecutivo sigue encargándose de coordinar la ejecución y el acceso a los dispositivos. Una vez completadas la inicialización y la conexión de la cuenta, el usuario llega normalmente al indicador de comandos de CSI. El capítulo 3 explica esa secuencia visible para el usuario; el capítulo 21 describe en detalle el cargador de arranque y la inicialización.

Cuentas, particiones y trabajo compartido

Una cuenta identifica al usuario y los archivos a los que puede acceder. Una partición proporciona un entorno de ejecución, que incluye memoria y asignaciones de terminal. El ejecutivo alterna entre particiones para que varios usuarios puedan avanzar en el mismo ordenador.

Una parte de la memoria contiene rutinas compartidas por las particiones. Otras partes se seleccionan a medida que se ejecutan distintos programas. Esta organización de la memoria se denomina conmutación de bancos. Permite al sistema usar más memoria de la que el procesador puede direccionar en un instante dado. No necesitas elegir bancos para listar o editar un archivo; la organización de la memoria se explica en el capítulo 20.

Comandos, lenguajes y servicios del sistema

CSI lee la línea de comandos y selecciona el programa solicitado. La mayoría de las utilidades son archivos independientes de tipo COMANDO. Así, EDITAR y COPIFI son programas que CSI pone en marcha, no operaciones de edición y copia integradas en el propio indicador.

BASIC tiene su propio editor, intérprete, compilador y entorno de ejecución para programas en BASIC. EXEC lee archivos de comandos que automatizan el trabajo en el nivel de comandos. Cada uno tiene sus propias instrucciones y su propio contexto de entrada. El indicador te dice qué componente está aceptando entrada en ese momento; el capítulo 4 enseña esa distinción antes de que empieces a introducir programas.

Los programas solicitan operaciones comunes mediante servicios del sistema: mostrar caracteres, abrir archivos u obtener información sobre la cuenta actual. El ejecutivo lleva a cabo esas peticiones y devuelve el resultado correspondiente. El capítulo 19 presenta la interfaz de llamada para programadores; el capítulo 24 explica su mecanismo interno de despacho.

Archivos y dispositivos

El sistema de archivos relaciona el nombre de un archivo con su propiedad, su organización y su contenido almacenado. Las utilidades lo usan para encontrar los datos que necesitan. Un nombre no determina por sí solo cómo debe interpretarse ese contenido: el texto normal, el código fuente BASIC y los programas compilados requieren operaciones distintas.

Un nombre lógico de dispositivo permite a un programa seleccionar un dispositivo sin nombrar sus conexiones de hardware. Por ejemplo, S selecciona la asociación del disco del sistema e IMPR1 selecciona una asociación de impresora. El administrador relaciona los nombres lógicos con definiciones físicas de dispositivos. Después, un controlador aporta las operaciones específicas del hardware. Esta separación permite a un programa usar una impresora asignada sin conocer los puertos de su controladora.

El capítulo 5 presenta la notación de archivos que ve el usuario. El capítulo 11 y el capítulo 12 explican la administración del almacenamiento y de los dispositivos; el capítulo 23 desarrolla el modelo subyacente de almacenamiento y dispositivos. Los capítulos posteriores distinguen entre una operación aceptada por el software y los datos realmente escritos o impresos, distinción importante al diagnosticar salidas incompletas.

Capítulo 3 — Arranque, conexión y salida

Empieza con el ordenador Secoinsa Serie 20 encendido y configurado por su administrador. Durante el arranque, los diagnósticos de la ROM preceden al mensaje `Carga del S.O.` y al mensaje de presentación del sistema. Espera a que el sistema pida la cuenta; no escribas comandos mientras la carga esté en curso.

Conéctate a tu cuenta

El sistema pregunta:

```
Nombre de cuenta a conectarse:
```

La cuenta de administración que se usa en esta guía es SISTEMA. Escribe su nombre y pulsa Intro:

```
SISTEMA
```

SISTEMA es la cuenta de administración real, no una cuenta de prácticas que haya que crear. Para el trabajo habitual, procura usar una cuenta con menos privilegios. Al iniciar sesión, la contraseña no se muestra mientras la escribes. Una conexión correcta lleva normalmente al indicador de CSI, >. Una cuenta también puede iniciar automáticamente un archivo de comandos, por lo que puede ser normal ver un menú tras iniciar sesión.

Haz tu primera consulta

En >, introduce:

```
AYUDA
```

Como su nombre indica, AYUDA muestra ayuda. Sin tema, muestra el resumen de comandos. Si la presentación se detiene, pulsa Intro para continuar. Has consultado al sistema sin modificar ningún archivo.

Ahora consulta un comando concreto:

```
AYUDA LISDIR
```

Se muestra la ayuda de LISDIR, el comando que lista el directorio. Usarás ese comando en los capítulos siguientes. La distinción es importante: `AYUDA LISDIR` describe LISDIR; `LISDIR` realiza un listado.

Normalmente, AYUDA muestra el tema correspondiente. Si la búsqueda falla, puede invocar la utilidad indicada con AYUD como argumento. AYUDA BORRAR sigue esta vía alternativa y muestra `Falta tipo del fichero` en lugar de la ayuda. Consulta la referencia de BORRAR para obtener instrucciones de borrado; no des por hecho que toda petición de un tema sea una simple consulta pasiva.

Por último, comprueba tu identidad:

```
ESTADO QUIEN
```

ESTADO informa sobre una parte concreta del estado del sistema; QUIEN selecciona la identidad actual. A diferencia de AYUDA, ESTADO requiere indicar qué se consulta. Si escribes ESTADO solo, se produce un error de sintaxis.

Finaliza una sesión

Guarda tu trabajo y sal de la aplicación con su propio comando de finalización. En el indicador de CSI, introduce:

```
FIN
```

FIN desconecta la cuenta y vuelve al proceso de inicio de sesión. Es un comando para cerrar la sesión. No garantiza que se cierre la salida pendiente de otro programa ni que se apague toda la máquina.

Reinicio e interrupción

Esc seguida de Q solicita una interrupción. El manejador actual de la aplicación determina cuándo y cómo responde; la vuelta a CSI no tiene por qué ser inmediata. Es preferible usar el comando de salida de la propia aplicación y comprobar qué indicador vuelve a aparecer. Una interrupción forzada puede dejar salidas incompletas. El capítulo 7 explica el comportamiento de la interrupción pendiente en EDITAR.

Esc seguida de I pregunta `Desea IPL (S/N) -`. S confirma el reinicio; N rechaza el reinicio, pero finaliza el comando interrumpido en lugar de reanudarlo. Ninguna de las dos respuestas guarda nada. El reinicio afecta también a las demás particiones. El capítulo 14 trata el mantenimiento y los límites de la interfaz de apagado.

Parte II — Uso del sistema

Capítulo 4 — Uso del terminal

Un terminal envía caracteres al ordenador y muestra los caracteres que este le devuelve. El indicador actual te dice qué programa está leyendo esos caracteres. Antes de escribir, comprueba si estás en CSI (>), en EDITAR (*) o en BASIC (-).

El ordenador Secoinsa Serie 20 se acompañaba del terminal informático Secoinsa IS 9502, con teclado independiente y una pantalla monocroma verde de 24 líneas de 80 caracteres. El terminal envía las pulsaciones de teclas al ordenador; CSI, el editor o el programa en ejecución deciden qué significan esas pulsaciones. El teclado y la pantalla no forman un ordenador independiente que ejecute este sistema operativo. El prefacio ofrece el contexto histórico del hardware.

Introduce y corrige una línea

En CSI, escribe lo siguiente y pulsa Intro:

```
AYUDA
```

Intro envía la línea. Para ello, la conexión del terminal usa el retorno de carro, el carácter 0x0D. Si te equivocas al escribir un carácter antes de enviar la línea, Retroceso o Supr borran el carácter anterior. Ctrl + X borra toda la línea de entrada para que puedas empezar de nuevo. Esc seguida de una c minúscula también cancela la línea de entrada actual de CSI.

Normalmente, CSI convierte las minúsculas en mayúsculas. Usa letras mayúsculas y dígitos en los nombres de archivo de práctica. El guion bajo no es adecuado aquí: la ruta de entrada de CSI puede tratarlo como Supr. Las cadenas entre comillas de BASIC tienen sus propias reglas, así que no deduzcas su comportamiento a partir de la edición de entrada de CSI.

Tecla	Operación en CSI
Intro	Envía la línea actual; continúa un listado paginado
Retroceso o Ctrl + H	Borra el carácter anterior
Supr	Borra el carácter anterior
Ctrl + X	Descarta la línea de entrada actual
Esc, c minúscula	Cancela la línea de entrada actual

Ctrl + Z no es una tecla de CSI para recuperar el comando anterior. La tecla física BREAK no tiene, en esta guía, ninguna operación general suficientemente especificada. El comando BREAK de BASIC es una función de depuración independiente.

Pausa, reanudación y restauración de la salida

Esc seguida de Q solicita una interrupción a través del manejador actual de la aplicación. No es una salida inmediata universal. EDITAR puede conservar la petición hasta una operación posterior; usa su comando de finalización y comprueba que aparece el indicador de CSI. Véase el capítulo 7.

Esc seguida de S pone en pausa la partición que lo solicita. Pulsa de nuevo el mismo par de teclas para reanudarla. No escribas el resto de un comando mientras esté en pausa: la entrada normal en cola puede descartarse al reanudar. Las demás particiones siguen ejecutándose.

Esc seguida de O activa o desactiva la salida normal y el eco del teclado. El programa puede seguir ejecutándose mientras su presentación está suprimida. Si el terminal se queda en silencio tras ese control, repítelo para restaurar la salida. Esto es distinto tanto de una pausa como de una interrupción.

Reconoce un problema de configuración del terminal

Si los comandos se aceptan, pero el movimiento del cursor o la disposición de la pantalla son incorrectos, puede que el sistema esté usando una clase de terminal equivocada (el componente que traduce los controles de pantalla para ese terminal). Si los caracteres aparecen alterados, puede que los parámetros de comunicación no coincidan. Comunica el síntoma al administrador en lugar de cambiar la configuración del terminal mientras aprendes el lenguaje de comandos.

Si al pegar texto rápidamente se pierden caracteres, deja de enviar, borra el comando incompleto y vuelve a introducirlo más despacio. El administrador debería comprobar el control de flujo y los parámetros de comunicación del terminal. Que la pantalla se vea correctamente no garantiza por sí solo una entrada fiable y sostenida.

Las dimensiones normales de la consola son L79/P23. No las cambies: reducir L puede acortar la cantidad de entrada que se acepta en una línea de comandos, además de afectar a la presentación. Si la salida se ha suprimido, restáurala con Esc seguida de O, y distingue esa situación de un fallo de comunicación.

El capítulo 12 explica los parámetros de comunicación, las clases de terminal y su configuración. El capítulo 28 explica cómo llegan los controles del terminal al dispositivo.

Capítulo 5 — Comandos, nombres de archivo y discos

La línea de comandos le dice a CSI qué hacer y qué objeto usar. Empieza leyendo un ejemplo concreto de izquierda a derecha:

```
LISDIR NOTA.TEXTO:S
```

LISDIR es la operación. NOTA es el primer campo del nombre de archivo. TEXTO es su campo de tipo. El punto separa esos campos. El sufijo `:S` selecciona el volumen del sistema mediante su asociación lógica de disco, S. En estos ejemplos, léelo como «en el disco del sistema»: NOTA.TEXTO:S significa el archivo NOTA.TEXTO de ese volumen. Este comando pide la entrada de catálogo de un archivo; no muestra el contenido del archivo.

S es un nombre lógico de disco, no la etiqueta literal de volumen SISTEMA. SISTEMA es también el nombre de la cuenta de administración y el primer campo de muchos nombres de archivo del sistema. Un volumen de almacenamiento contiene archivos; en este sistema operativo, una partición es un entorno de memoria y de ejecución, como se explica en el capítulo 9. El sufijo `:S` selecciona almacenamiento, no una partición de ejecución.

Cada campo del nombre de archivo tiene como máximo ocho caracteres. Mientras aprendes, usa nombres sencillos en mayúsculas, como NOTA.TEXTO y HOLA.BASICOBJ. Un tipo es un nombre, no una garantía de contenido: llamar BASICOBJ a un archivo no convierte un texto cualquiera en un programa BASIC.

Lista varios archivos

Introduce:

```
LISDIR *.*:S
```

El primer asterisco coincide con los nombres; el segundo, con los tipos. Así se solicitan los archivos de S visibles para tu cuenta. La propiedad puede ocultar los archivos privados de otra cuenta. Por tanto, un resultado vacío no significa que todo el volumen esté vacío.

Para acotar la selección, introduce:

```
LISDIR SISTEMA.*:S
```

Así se selecciona el nombre SISTEMA con cualquier tipo. El sistema usa esa familia para componentes como SISTEMA.CSI y SISTEMA.AYUDA. El apéndice C explica esos componentes uno por uno.

El comodín de signo de interrogación puede coincidir con una posición en un patrón de listado, pero la compatibilidad con comodines depende de cada comando. No uses un patrón de listado en un comando de copia o de mantenimiento salvo que ese comando lo admita.

Añade una opción

Introduce:

```
LISDIR *.*:S (SORT)
```

Los paréntesis encierran las opciones. SORT solicita un listado ordenado. Estos paréntesis son caracteres literales, a diferencia de los corchetes que marcan el material opcional en las descripciones de sintaxis.

La unidad S es una asociación que mantiene el sistema. Las letras A a G pueden designar otros discos. No son subdirectorios. Si omites la unidad, puede seleccionarse una unidad de trabajo que no se ha montado; en estos ejemplos, escribir `:S` deja explícito el destino.

Interpreta los mensajes de finalización

Un comando puede terminar con `CR = n`, una hora y `TE = ... minutos.`. CR es su resultado de finalización; TE es el tiempo transcurrido. CR 0 indica que el comando se ha ejecutado correctamente. Lee también los mensajes anteriores: un verificador puede terminar correctamente aunque informe de problemas en el sistema de archivos.

Ante un error de sintaxis, lo habitual es comprobar el nombre del comando, los espacios, los dos puntos y los paréntesis antes de volver a intentarlo. Ante un mensaje de archivo no encontrado, comprueba el nombre, la asociación de disco y la cuenta actual. El capítulo 33 distingue los resultados de los comandos de los errores de BASIC y de los números de mensaje del sistema.

Organizaciones de archivo

Organización	Para qué sirve
S	Texto secuencial o registros serializados
SR	Almacenamiento secuencial con protección de lectura
D	Registros de longitud fija seleccionados por número
I	Registros con clave y un índice ordenado
K	Registros con clave sin el índice ordenado del formato I
R	Programas en código máquina reubicables
A	Programas situados en direcciones absolutas

Normalmente elegirás un programa adecuado al archivo, en lugar de manipular directamente estas letras. LISTAR muestra texto secuencial; BASIC carga su propio código fuente guardado; RUN ejecuta BASIC compilado. Sus formatos internos se explican después de los capítulos de programación.

Nombres completos y abreviaturas

Este documento usa los nombres completos de los comandos. Con la abreviatura estándar activada, la mayoría de los comandos de la lista también pueden acortarse hasta su prefijo mínimo especificado; se aceptan los prefijos intermedios. Por ejemplo, ASOCIAR puede introducirse como ASO, ASOC, ASOCI, ASOCIA o ASOCIAR. Todas estas formas solicitan la misma operación de asociación. ASIGNAR es un comando distinto: entre sus formas cortas están A y AS, que no seleccionan ASOCIAR.

El mínimo se define por separado para cada comando. No siempre son tres letras, y algunos nombres de comando deben escribirse completos. Hay dos excepciones importantes a la regla de los prefijos intermedios: B selecciona BASIC, pero BA selecciona BACKUP, mientras que BAS y BASI seleccionan BASIC; EXEC debe escribirse siempre completo, porque EX y EXE pueden seleccionar un archivo de comandos equivocado. Usa los nombres completos hasta que estas diferencias te resulten familiares.

Las opciones tienen sus propias reglas de escritura. En ESTADO MEMORIA, ESTADO es el comando y MEMORIA selecciona la información que se muestra. MEM es una abreviatura admitida de esa opción, incluso cuando la abreviatura de nombres de comando está desactivada. Una palabra clave completa es la grafía que acepta el sistema, que puede ser a su vez una palabra española abreviada; no la alargues hasta la palabra que supongas completa.

El capítulo 35 enumera los nombres completos, las formas cortas admitidas y los ajustes. También explica las excepciones causadas por sinónimos, nombres de archivo y coincidencias entre nombres de opciones.

Capítulo 6 — Tu primer archivo

En este capítulo se crea un pequeño archivo de texto y se le aplican las operaciones habituales con archivos. Usa una cuenta de prácticas que pueda ejecutar EDITAR y escribir archivos en S. Antes de nada, asegúrate de que NOTA.TEXTO, COPIA.TEXTO y OTRO.TEXTO no son nombres que ya estés usando.

1. Comprueba los nombres de práctica

Introduce:

```
LISDIR NOTA.TEXTO:S
```

Así se pregunta si NOTA.TEXTO es visible en S. Si ya existe, elige otro nombre que no esté en uso y sustitúyelo de forma coherente en los pasos siguientes. Un archivo que reconoces como tuyo no es prescindible solo porque su nombre coincida con el de un ejemplo.

2. Inicia el editor

Introduce:

```
EDITAR NOTA.TEXTO:S
```

EDITAR abre una sesión de edición de texto para ese nombre. Su indicador es `*`. Ahora estás dando instrucciones al editor, no a CSI.

Introduce:

```
CASE U
```

Así se selecciona el tratamiento en mayúsculas para el texto que vas a introducir.

Introduce:

```
INPUT MI PRIMER ARCHIVO
```

INPUT añade las palabras que lo siguen como una línea del texto del archivo. INPUT es una instrucción del editor y no forma parte de ese texto.

Introduce:

```
FILE
```

FILE guarda el texto y sale de EDITAR. Espera a que aparezca el indicador > de CSI.

3. Muestra lo que has guardado

Introduce:

```
LISTAR NOTA.TEXTO:S
```

LISTAR muestra el archivo. Su texto incluye `MI PRIMER ARCHIVO`; los encabezados dependen de las opciones de listado. Compáralo con LISDIR: LISTAR muestra el contenido, mientras que LISDIR muestra la entrada del catálogo.

4. Copia el archivo

Comprueba primero el destino propuesto:

```
LISDIR COPIA.TEXTO:S
```

Si ya existe, elige otro nombre de práctica que no esté en uso. Con el nuevo nombre previsto disponible, introduce:

```
COPIFI NOTA.TEXTO:S COPIA.TEXTO:S (VERIFI)
```

La primera especificación es el origen. La segunda es el destino. COPIFI crea una copia independiente; VERIFI solicita la verificación. No inviertas las dos especificaciones.

Examina el destino:

```
LISTAR COPIA.TEXTO:S
```

Su texto debería coincidir con el de NOTA.TEXTO. El original sigue disponible con su nombre original.

5. Cambia el nombre de la copia

Comprueba el nuevo nombre propuesto:

```
LISDIR OTRO.TEXTO:S
```

Si no está en uso, introduce:

```
ATRIB COPIA.TEXTO:S OTRO.TEXTO:S (NOCONSUL)
```

ATRIB cambia el nombre en el catálogo. NOCONSUL suprime la pregunta de confirmación en este ejemplo de un solo archivo. Los datos almacenados se conservan con el nombre OTRO.TEXTO; no se trata de otra copia.

Comprueba el nuevo nombre:

```
LISDIR OTRO.TEXTO:S
```

Ahora deberías encontrar la entrada con el nuevo nombre. Usa OTRO.TEXTO en el siguiente paso, porque COPIA.TEXTO ya no es su nombre.

6. Elimina la copia de práctica

Examina primero el nombre exacto que se ha seleccionado para borrar:

```
LISDIR OTRO.TEXTO:S
```

Comprueba que es la copia de práctica a la que cambiaste el nombre. Conserva NOTA.TEXTO para ejemplos posteriores.

> Precaución — borrado inmediato. El siguiente comando con nombre exacto borra el archivo seleccionado sin detenerse a pedir confirmación, aunque se indique CONSUL. Comprueba el nombre de archivo antes de introducirlo.

Borra la copia de práctica:

```
BORRAR OTRO.TEXTO:S
```

BORRAR elimina la entrada inmediatamente y libera el espacio asignado. El espacio borrado puede reutilizarse; el borrado no es una operación que pueda deshacerse de forma fiable. La confirmación funciona de otra manera cuando la selección usa comodines; consulta la referencia de comandos.

7. Comprueba el espacio libre

Introduce:

```
ESTADO DISCO
```

`Disponible` indica la capacidad libre total. `Mayor zona` indica la mayor región libre contigua. La distinción cobra importancia con los archivos directos, que pueden necesitar una única asignación continua. Has completado el ciclo básico de crear, examinar, copiar, renombrar y borrar, y has conservado el texto de práctica original.

Capítulo 7 — Edición de archivos y ejecución de programas

Un archivo de texto y un programa compilado sirven para cosas distintas. EDITAR te permite introducir y guardar texto normal; LISTAR muestra después ese texto. RUN pone en marcha un programa BASIC compilado. Este capítulo amplía los pasos de edición presentados en el capítulo 6 y luego explica cómo reconocer y ejecutar un programa que ya se ha compilado.

Prepara un archivo de texto

Usa BORRADOR.TEXTO para este ejercicio. En el indicador de CSI, comprueba primero el nombre:

```
LISDIR BORRADOR.TEXTO:S
```

Si ya existe, elige un nombre que no esté en uso y sustitúyelo en todo este ejemplo de edición. El ejemplo crea un nuevo archivo de práctica; no debe sustituir texto que quieras conservar.

Inicia el editor:

```
EDITAR BORRADOR.TEXTO:S
```

EDITAR abre una sesión de edición de texto. Su indicador de comandos es *, en lugar del > de CSI. Hasta que salgas del editor, las instrucciones que introduzcas son instrucciones del editor.

Introduce dos líneas

En el indicador del editor, introduce:

```
CASE U
```

Así se selecciona el tratamiento en mayúsculas para el texto que introduzcas. Añade la primera línea:

```
INPUT PRIMERA LINEA
```

INPUT pone en el archivo las palabras que lo siguen. El texto guardado es PRIMERA LINEA; INPUT no se guarda como parte de la línea.

Añade otra línea:

```
INPUT SEGUNDA LINEA
```

El archivo tiene ahora dos líneas de texto. Estas palabras son contenido del archivo, no comandos que se vayan a ejecutar.

Guarda y vuelve a CSI

Introduce:

```
FILE
```

Normalmente, FILE muestra `salvado y fin edicion`, guarda el texto y vuelve a CSI. Espera al indicador > antes de introducir otro comando del sistema.

Si se pulsó Esc seguida de Q en el indicador del editor, la petición de interrupción puede quedar pendiente. El siguiente FILE puede mostrar `salvado` y dejar EDITAR activo en su indicador *. En ese caso, introduce FILE de nuevo para guardar y terminar. Se trata de una respuesta al indicador del editor que sigue activo, no de un requisito de introducir FILE dos veces cada vez que se guarda. Esc seguida de Q no sustituye de forma fiable al comando de finalización del editor.

Cuando haya vuelto CSI, examina el archivo guardado:

```
LISTAR BORRADOR.TEXTO:S
```

El contenido debería ser PRIMERA LINEA seguido de SEGUNDA LINEA. LISTAR lee el texto para mostrarlo; no vuelve a abrir una sesión de edición. Comprueba el contenido guardado antes de confiar en el archivo en trabajos posteriores.

Protege el texto existente antes de editarlo

Antes de hacer cambios importantes, crea una copia explícita con el procedimiento de COPIFI del capítulo 6. La opción COPIA de EDITAR solicita una copia de seguridad y es el valor predeterminado documentado; NOCOPIA la suprime. Las reglas completas de nomenclatura y recuperación de la copia de seguridad no están especificadas, así que la copia automática no sustituye a una copia cuyo nombre y contenido hayas comprobado.

El flujo de trabajo fiable para crear texto que se usa aquí emplea CASE U, INPUT y FILE. El conjunto completo de comandos de desplazamiento, búsqueda, sustitución y recuperación no está suficientemente especificado para un curso de edición paso a paso. No tomes prestados comandos de otro editor. TEDITAR es un editor para archivos pequeños; CEDIT edita registros estructurados. Sus interfaces son independientes.

Reconoce un programa que RUN pueda ejecutar

RUN ejecuta archivos BASIC compilados, normalmente de tipo BASICCOM. El código fuente a partir del cual se generan lo maneja BASIC y se guarda como BASICOBJ. Son representaciones almacenadas distintas. Cambiar el nombre de BORRADOR.TEXTO para darle el tipo BASICCOM no convertiría su texto en un programa.

Una utilidad COMANDO se invoca normalmente por su nombre de comando, como se ha hecho antes con EDITAR y LISTAR. RUN es el punto de entrada para BASIC compilado; no es una forma general de abrir cualquier archivo.

El siguiente ejemplo usa HOLA.BASICCOM:S, que se genera con el ejemplo completo de programación del capítulo 15. Sigue estos pasos cuando tengas disponible ese archivo compilado. Si todavía no lo has generado, continúa con el capítulo 8 y vuelve a este ejemplo después de completar ese programa.

Ejecuta un programa compilado

En CSI, comprueba que existe el archivo compilado:

```
LISDIR HOLA.BASICCOM:S
```

Si no existe, completa los pasos de código fuente, guardado y compilación del capítulo 15. Un archivo de código fuente por sí solo no basta para este comando RUN.

Con el saludo compilado disponible, introduce:

```
RUN HOLA.BASICCOM:S
```

RUN carga el programa BASIC compilado y lo ejecuta con el entorno de ejecución de BASIC. Con el saludo del capítulo 15 sin modificar, el programa muestra HOLA MUNDO y termina. Cuando vuelva CSI, podrás introducir otro comando.

Editar o guardar el código fuente BASIC no actualiza automáticamente un BASICCOM anterior; compila el código fuente modificado antes de esperar que RUN ejecute la nueva versión. Para terminar una aplicación, usa su propia operación de salida. QUIT de BASIC sale del editor/intérprete de BASIC, mientras que END de BASIC finaliza un programa BASIC en ejecución. Una interrupción forzada puede dejar incompletos los archivos abiertos, aunque CSI vuelva a aparecer en pantalla.

El capítulo siguiente usa archivos de texto normales con otro fin: almacenar instrucciones que automatizan el trabajo en el nivel de comandos.

Para ver la lista completa de nombres internos que reconoce el editor y los límites de la descripción de sus argumentos, consulta la referencia de EDITAR del capítulo 35.

Capítulo 8 — Automatización del trabajo con EXEC

EXEC es el intérprete de archivos de comandos. Lee un archivo de texto guardado y ejecuta las instrucciones que contiene, para que no tengas que escribir la misma secuencia en el indicador del sistema cada vez. Un archivo de comandos puede contener comandos normales del sistema junto con instrucciones propias de EXEC para mostrar texto, usar variables, tomar decisiones y repetir trabajo. Su tipo de archivo habitual es EXEC.

Este texto se crea con EDITAR, siguiendo los pasos de edición del capítulo 7. Después sales del editor y pides a EXEC que lea el archivo guardado. No interviene ninguna compilación de BASIC: RUN ejecuta BASIC compilado, mientras que EXEC interpreta el texto de un archivo de comandos. Empieza con un archivo que muestre un saludo y termine, antes de añadir variables o bucles.

Tu primer archivo de comandos: HOLA MUNDO

Usa el nombre HOLA.EXEC. En CSI, comprueba que está disponible:

```
LISDIR HOLA.EXEC:S
```

Si ya existe, elige otro nombre que no esté en uso y sustitúyelo en todo el ejemplo. Este archivo de comandos es independiente de BORRADOR.TEXTO, el archivo de texto simple creado en el capítulo 7.

Inicia el editor:

```
EDITAR HOLA.EXEC:S
```

El indicador * del editor significa que está listo para recibir instrucciones de edición. Selecciona el tratamiento en mayúsculas:

```
CASE U
```

Ahora introduce la línea que mostrará el saludo:

```
INPUT &TYPE HOLA MUNDO
```

INPUT pertenece a EDITAR: pone el resto de la línea en el archivo. La instrucción guardada es `&TYPE HOLA MUNDO`. TYPE pertenece a EXEC y muestra los campos siguientes, aquí HOLA y MUNDO. El signo & introduce la instrucción de EXEC. Escribir este texto en el editor no ejecuta nada.

Añade la instrucción que finaliza el archivo de comandos:

```
INPUT &QUIT
```

Así se guarda `&QUIT` como segunda línea. QUIT indica a EXEC que finalice el archivo de comandos y vuelva a quien lo llamó. No sale de EDITAR; sigues redactando texto.

Guarda y sal del editor:

```
FILE
```

Espera al indicador > de CSI. Si FILE deja el editor activo tras una interrupción pendiente, sigue las indicaciones para guardar y salir del capítulo 7.

Examina las instrucciones guardadas

En CSI, introduce:

```
LISTAR HOLA.EXEC:S
```

El archivo debería contener estas dos líneas:

```
&TYPE HOLA MUNDO
&QUIT
```

Este bloque muestra el contenido guardado; no lo escribas como comandos en CSI. INPUT, CASE U y FILE eran instrucciones para el editor y no deben aparecer en este archivo de comandos. Comprueba el contenido antes de ejecutarlo. Que se haya guardado correctamente significa que el texto se ha escrito, no que las instrucciones sean válidas.

Ejecuta el saludo

En CSI, introduce:

```
EXEC HOLA
```

EXEC encuentra HOLA.EXEC mediante su búsqueda de archivos y lo lee. TYPE muestra HOLA MUNDO; QUIT finaliza el archivo de comandos. Cuando vuelve el indicador de CSI, la secuencia ha terminado. Usa el nombre de comando completo EXEC en estos ejemplos.

Puedes introducir de nuevo el mismo comando EXEC para repetir el saludo sin volver a abrir EDITAR. Esa es la finalidad de un archivo de comandos: las instrucciones guardadas pueden reutilizarse.

Las secciones restantes presentan instrucciones de EXEC que van dentro de los archivos de comandos. Para poner una instrucción en un archivo, usa la instrucción INPUT de EDITAR como antes; INPUT no forma parte de la instrucción guardada. Un bloque de programa completo muestra el contenido previsto del archivo. Los ejemplos descritos como comandos que se introducen en CSI, como EXEC CONTAR, se introducen después de guardar y salir del editor.

Muestra textos más largos

TYPE imprime campos y valores sustituidos. Los campos normales tienen un límite de ocho caracteres; HOLA y MUNDO caben cada uno por separado. Para una línea más larga que deba imprimirse literalmente, usa esta construcción de tres líneas en el archivo de comandos:

```
&BEGTYPE
ESTA ES UNA LINEA DE TEXTO MAS LARGA
&END
```

BEGTYPE inicia la salida literal; END la cierra. La línea central se imprime como texto, sin sustitución de variables. Los dos delimitadores son partes obligatorias de una misma construcción. Mantén un &QUIT final después de la construcción para terminar este archivo de saludo.

Almacena una variable

Escribe:

```
&I = 1
```

Así se crea la variable I, o se cambia, con el valor 1. El signo & marca los elementos del lenguaje de EXEC. Un marco tiene dieciséis celdas de variables normales; los nombres y los valores ocupan ocho caracteres cada uno.

A continuación, escribe:

```
&TYPE VALOR &I
```

TYPE sustituye el valor de I. Por tanto, la línea imprime la etiqueta VALOR seguida de 1.

Para cambiarlo, escribe:

```
&I = &I + 1
```

La expresión lee el valor anterior, le suma uno y almacena el resultado. La aritmética entera incluye la división con truncamiento: 7 / 2 da 3. La suma da la vuelta en el límite de 16 bits: 65 535 + 1 da 0.

Repite un grupo

Crea CONTAR.EXEC con este contenido completo:

```
&CONTROL OFF
&I = 1
&WHILE &I LE 3
&TYPE CONTADOR &I
&I = &I + 1
&REPEAT
&QUIT 0
```

CONTROL OFF suprime el eco de los comandos. La asignación pone I a 1. WHILE comprueba si I es menor o igual que 3. TYPE lo muestra; la asignación lo incrementa; REPEAT vuelve a la condición. Cuando I llega a 4, la condición deja de cumplirse y QUIT termina con el resultado 0.

En CSI, ejecútalo:

```
EXEC CONTAR
```

El bucle muestra CONTADOR con los valores 1, 2 y 3, y luego vuelve a CSI.

Toma una decisión

La forma de la bifurcación es:

```
&IF &I EQ 4 &GOTO -HECHO
```

IF comprueba la igualdad; GOTO transfiere el control solo si esa condición se cumple. El destino debe existir en el archivo. Un final con etiqueta puede escribirse así:

```
-HECHO &QUIT 0
```

La etiqueta marca el destino, mientras que QUIT es la instrucción que se ejecuta allí. Las etiquetas no son variables.

Pasa argumentos

Para un script PRUEBARG.EXEC cuyo contenido incluye `&TYPE &1` y `&QUIT 0`, introduce:

```
EXEC PRUEBARG ALFA
```

ALFA pasa a ser el argumento 1. `&2` selecciona el segundo argumento y `&INDEX` da el número de argumentos. EXEC almacena como máximo dieciséis campos de argumento. Si IDX contiene 2, `&&IDX` selecciona indirectamente el argumento 2.

Los operandos especiales `&*` y `&$` significan «cualquier argumento» y «todos los argumentos» en las comparaciones. Por ejemplo:

```
&IF &* EQ ALFA &GOTO -HALLADO
```

Esto bifurca a una etiqueta llamada HALLADO si al menos un argumento es igual a ALFA. No se expande en una cadena que contenga todos los argumentos.

Trabaja con cadenas cortas

Escribe:

```
&T = &SUB ABCDEFGH 3 5
```

SUB toma las posiciones 3 a 5, ambas incluidas, y produce CDE. Para unir dos campos, escribe:

```
&T = ABC | DEF
```

La barra vertical los concatena en ABCDEF, con sujeción a la anchura normal de los valores.

Trata los resultados anidados

Un script puede llamar a otro mediante el comando EXEC completo. Las variables normales pertenecen a su propio nivel de anidamiento. Cuando el hijo termina, `&RETCODE` contiene su resultado.

Registra una acción de error antes de un comando hijo:

```
&ERROR &GOTO -ERR
```

Un resultado distinto de cero que cumpla las condiciones transfiere el control a ERR. En esa etiqueta, esta instrucción propaga el resultado:

```
-ERR &QUIT &RETCODE
```

Sin una acción de error, un resultado distinto de cero del hijo no tiene por qué detener al padre. Las reglas completas de recursión y de clases de error siguen sin especificarse.

EXEC guarda el estado de los archivos de comandos anidados en un archivo de trabajo de la partición de ejecución actual. El nombre de archivo termina con el número de partición: la partición 1 usa SISTEMA.EXEC1, la partición 2 usa SISTEMA.EXEC2, y así sucesivamente. Incluso un script que solo muestra información puede actualizar este archivo de trabajo. El apéndice C describe estos archivos.

Ejecución implícita opcional

En CSI, introduce:

```
PONER IMPEXEC ON
```

Esto permite introducir el nombre de un script accesible sin el prefijo EXEC. Para volver al funcionamiento exclusivamente explícito, introduce:

```
PONER IMPEXEC OFF
```

La forma completa sigue siendo la más clara para las instrucciones. El capítulo 35 amplía esta introducción con la entrada en cola para programas, READ, SKIP, UNTIL, los controles de presentación y el índice completo de directivas. Identifica los límites de interfaz restantes por separado de los ejemplos.

Capítulo 9 — Cuentas, mensajes y hábitos cotidianos

Una cuenta da una identidad a tu sesión. Una partición aporta el contexto de ejecución y el terminal. Empieza cualquier trabajo delicado comprobando quién eres y qué se te permite hacer.

Comprueba la identidad y el privilegio

Introduce:

```
ESTADO QUIEN
```

QUIEN muestra la identidad actual. Ahora introduce:

```
ESTADO PRIVILEG
```

PRIVILEG muestra el privilegio actual. Un comando puede requerir un valor superior al de tu cuenta; eso es un fallo de autorización, no un error ortográfico.

Cambia de cuenta

Para conectarte a la cuenta de administración SISTEMA existente, si tienes autorización, introduce:

```
CONECTAR SISTEMA
```

CONECTAR pide la contraseña de la cuenta cuando es necesaria y cambia la conexión tras una autenticación correcta. Al cambiar a una cuenta disponible, no tienes que introducir FIN antes. Un número de cuenta que ya está conectado en otra partición no puede conectarse por segunda vez: el comando muestra `Esta cuenta esta siendo utilizada por la particion` seguido del número de partición, con CR = 10. Finaliza la conexión existente antes de volver a intentarlo con esa cuenta. Este rechazo se produce antes de la comprobación de la contraseña. Una contraseña incorrecta no autoriza la nueva identidad. Si tienes dudas, comprueba después ESTADO QUIEN.

Un archivo EXEC con el nombre de la cuenta puede ejecutarse automáticamente tras la conexión. Su contenido determina si llegas a un menú o al indicador de CSI. Esta función no implica que todas las cuentas tengan un archivo de inicio.

Envía un mensaje breve

Si el destinatario previsto está conectado en la partición 2, introduce:

```
MENSAJE 2 HOLA
```

MENSAJE se dirige al destinatario y le envía el texto breve HOLA. Usa un destinatario conocido; un número de partición no es un número de cuenta.

Para un texto más largo, introduce:

```
MENSAJE 2
```

El programa muestra el indicador ?. Introduce una línea cada vez y termina con una línea vacía. Lee el mensaje de resultado: CR 0 por sí solo no garantiza la entrega inmediata. Cuando se ofrezca el almacenamiento en el buzón, S lo acepta. Dirige los mensajes almacenados con el nombre completo de la cuenta.

Lee los mensajes almacenados

Introduce:

```
BUZON
```

BUZON muestra los mensajes de tu cuenta y elimina del buzón los mensajes recuperados. Anota lo que necesites antes de continuar; no es una presentación repetible que conserve los mensajes.

Para inhibir los mensajes entrantes normales de las utilidades, introduce:

```
PONER MENSAJE OFF
```

Para volver a recibirlos, introduce:

```
PONER MENSAJE ON
```

Los mensajes de difusión con privilegios pueden seguir llegando mientras los mensajes normales están inhibidos.

Una secuencia diaria fiable

Conéctate, comprueba tu cuenta, examina el archivo que piensas usar y, después, inicia su aplicación. Guarda antes de cambiar de tarea. Haz una copia explícita antes de una edición importante. Lee tanto los resultados de los comandos como los textos de diagnóstico. Termina cerrando la aplicación e introduciendo FIN en CSI.

Para la secuencia básica de trabajo con archivos, vuelve al capítulo 6. La referencia alfabética completa de comandos es el capítulo 31.

Parte III — Administración del sistema

Capítulo 10 — Administración de cuentas

Una cuenta identifica a un usuario y determina el privilegio que usan los comandos. También aporta el número de cuenta con el que se comprueba la propiedad de los archivos. Una partición es otra cosa: proporciona memoria y tiempo de ejecución. Crear una cuenta no crea una partición, y detener una partición no elimina la cuenta de su usuario.

SISTEMA es la cuenta de administración que se usa en los ejemplos. El mantenimiento de cuentas requiere el privilegio 5 y, con ese privilegio, el comando exige la identidad SISTEMA. Conéctate a SISTEMA como se describe en el capítulo 9 y, después, comprueba el privilegio actual:

```
ESTADO PRIVILEG
```

El informe debe mostrar el privilegio necesario para el mantenimiento. Son comprobaciones de software; la máquina no ofrece una frontera general de memoria protegida entre el código de las aplicaciones y el del ejecutivo.

Lista las cuentas y los usuarios conectados

Para ver las cuentas registradas, introduce:

```
CUENTA (PANT)
```

El listado contiene columnas con los encabezados `Cuenta`, `Pal.paso`, `Priv` e `Id`: nombre de cuenta, contraseña («palabra de paso»), privilegio y número de cuenta. Este listado administrativo puede mostrar las contraseñas almacenadas. Mantén en privado su pantalla o su copia impresa. Es una lista de registros de cuentas, no una lista de las personas que están usando los terminales.

Para examinar en cambio los usuarios conectados, introduce:

```
ESTADO USUARIOS
```

Consulta ese informe antes de cambiar o eliminar una cuenta. Pide al usuario afectado que termine su trabajo y se desconecte. Los nombres de cuenta pueden ser sinónimos del mismo número; no confundas borrar un nombre con borrar todas las identidades que comparten su número.

Comprende el diálogo de mantenimiento

Inicia el editor interactivo de cuentas:

```
CUENTA
```

El indicador de operación es:

```
Modo (Nueva/Camb/Borrar/Fin)?
```

En este indicador, las letras siguientes seleccionan una operación. Cada respuesta va seguida de Intro.

Respuesta	Significado	Qué sigue
N	Nueva: cuenta nueva	Nombre, sinónimo, contraseña y privilegio
C	Camb: cambiar una cuenta existente	Nombre, contraseña y privilegio
B	Borrar: borrar un nombre de cuenta	Nombre que se va a eliminar
F	Fin: terminar el mantenimiento	Escribe la lista de cuentas y vuelve a CSI

Haz un único cambio planificado cada vez, termina con F y vuelve a examinar la lista de cuentas. F es la operación de guardar y terminar; abandonar la máquina o interrumpir el programa a la fuerza no la sustituye.

En los ejemplos siguientes, `<cuenta-nueva>`, `<cuenta-a-eliminar>` y `<contraseña-nueva>` son marcadores de posición. Escribe el valor elegido sin los signos de menor y mayor. SISTEMA ya existe: no la uses como ejemplo de cuenta nueva ni de borrado.

Añade una cuenta de usuario

Usa primero el listado de cuentas para elegir un nombre que no esté en uso. Usa un nombre en mayúsculas de ocho caracteres como máximo, que empiece por una letra, y limita los demás caracteres a letras y dígitos. Inicia CUENTA como antes. En su indicador de operación, introduce:

```
N
```

En `Nombre cuenta`, introduce el nuevo nombre:

```
<cuenta-nueva>
```

En `Cta.sinonima?`, pulsa Intro sin escribir nada. Una respuesta vacía solicita un número de cuenta distinto. Si en su lugar indicas un nombre existente, se crea un sinónimo que comparte el número de esa cuenta; no es la opción habitual al añadir un usuario independiente.

En `Pal.paso?`, introduce la contraseña elegida para la cuenta:

```
<contraseña-nueva>
```

El diálogo de mantenimiento muestra en eco esta entrada. La contraseña se almacena en un campo de ocho caracteres y no se acepta la coma. Usa como máximo ocho caracteres y evita los espacios, que este diálogo elimina. Una respuesta vacía durante la creación deja la nueva cuenta sin contraseña. Al iniciar sesión la entrada se oculta, pero ni esa ocultación ni la contraseña hacen que el archivo de cuentas almacenado esté cifrado.

En `Privilegio?`, introduce:

```
1
```

El privilegio 1 permite el trabajo normal con archivos, pero no concede el mantenimiento de cuentas. Algunas herramientas, incluidas partes del entorno de programación, requieren más privilegio. Asigna el privilegio necesario para el trabajo previsto, en lugar de suponer que todas las cuentas nuevas deben igualar a SISTEMA.

Cuando vuelva el indicador de operación, introduce:

```
F
```

CUENTA escribe la lista de cuentas revisada. Compruébala:

```
CUENTA (PANT)
```

Comprueba el nuevo nombre, el privilegio y el número asignado. El programa asigna un número; este ejemplo no prescribe ninguno. Si es posible, prueba el nuevo inicio de sesión desde otro terminal, manteniendo abierta la conexión de administración. Esto es posible porque la nueva cuenta tiene un número de cuenta distinto; otro nombre de una cuenta que ya está en uso no puede probarse de esta forma.

Cambia una contraseña o un privilegio

Los cambios de contraseña se hacen con la operación C de CUENTA. No existe un comando independiente reconocido para que un usuario sin privilegios cambie su contraseña. El procedimiento siguiente cambia la contraseña de SISTEMA; usa otro nombre de cuenta existente para mantener esa cuenta. Antes de cambiar la contraseña de administración, asegúrate de disponer de un mecanismo de recuperación utilizable. La nueva contraseña no puede probarse mientras otra partición siga conectada con el mismo número de cuenta. No debe suponerse que una cuenta normal pueda restablecer el acceso de administración; los límites de las copias de seguridad y de la recuperación se describen en el capítulo 13.

Introduce:

```
CUENTA
```

En el indicador de operación, introduce:

```
C
```

En `Nombre cuenta`, introduce:

```
SISTEMA
```

En `Pal.paso?`, indica la nueva contraseña:

```
<contraseña-nueva>
```

Durante un cambio, una respuesta vacía conserva la contraseña existente. No la elimina. Para eliminar deliberadamente una contraseña, la respuesta especial es un único carácter almohadilla:

```
#
```

Son alternativas en el mismo indicador. Elige la nueva contraseña, una respuesta vacía para conservarla o # para borrarla; no escribas las tres. Borrar la contraseña permite conectarse sin que se pida esa contraseña.

A continuación, CUENTA pregunta `Privilegio?`, aunque solo se esté cambiando la contraseña. Para el privilegio de administración de SISTEMA, introduce:

```
5
```

Al mantener otra cuenta, introduce el privilegio previsto para ella. No copies el 5 automáticamente. Una respuesta de privilegio en blanco hace que se repita la pregunta, en lugar de conservar el valor anterior.

En el indicador de operación, termina:

```
F
```

Cuando F haya guardado el cambio, finaliza la conexión existente de SISTEMA en su terminal:

```
FIN
```

FIN desconecta esa cuenta. Después, en otro terminal disponible, introduce:

```
CONECTAR SISTEMA
```

Indica la nueva contraseña cuando se te pida. Si el comando muestra `Esta cuenta esta siendo utilizada por la particion 1` con CR = 10, la partición 1 sigue teniendo ese número de cuenta; en el mensaje puede aparecer otro número de partición. Ese rechazo no comprueba la contraseña. Finaliza la conexión existente antes de volver a intentarlo.

Cuando el nuevo inicio de sesión tenga éxito, comprueba su privilegio:

```
ESTADO PRIVILEG
```

Para SISTEMA, el valor esperado es 5. Los alias de cuenta tienen registros de nombre independientes, pero los nombres que comparten un número de cuenta no permiten conexiones simultáneas. No debe suponerse que cambiar un registro cambia la contraseña de todos los sinónimos.

Elimina una cuenta

Antes de eliminar una cuenta, pide a su usuario que cierre las aplicaciones e introduzca FIN. Anota su número de cuenta y decide qué debe ocurrir con sus archivos. Borrar una cuenta elimina el registro de la cuenta con ese nombre; no borra archivos ni transfiere su propiedad. Transfiere los archivos necesarios antes de retirar el inicio de sesión del usuario.

> Precaución — elimina un nombre de inicio de sesión. No borres SISTEMA como operación de práctica. Antes de continuar, busca sinónimos y archivos cuyo propietario sea ese número de cuenta.

Introduce:

```
CUENTA
```

En el indicador de operación, introduce:

```
B
```

En `Nombre cuenta`, indica la cuenta exacta que se ha seleccionado para eliminar:

```
<cuenta-a-eliminar>
```

El programa elimina el registro coincidente de su lista de trabajo y vuelve al indicador de operación. En este proceso no hay ninguna confirmación adicional de sí o no para cada cuenta. Termina y guarda:

```
F
```

Examina el resultado:

```
CUENTA (PANT)
```

El nombre eliminado no debería aparecer. Los demás nombres que comparten su número de cuenta son registros independientes y pueden permanecer.

No sustituyas este procedimiento por `CUENTA (ELIMINAR)`. ELIMINAR elimina el propio sistema de cuentas. `CUENTA (VACIAR)` borra el historial de cuentas; tampoco elimina una cuenta individual.

Propiedad y uso compartido

Las operaciones siguientes requieren la cuenta del propietario actual del archivo. SISTEMA no puede usar COMPART, y el privilegio de administración por sí solo no hace que CAMPRO seleccione archivos privados de otro propietario. Para estas operaciones, usa una cuenta distinta de SISTEMA con privilegio 3 o superior. La creación de cuentas y el mantenimiento de privilegios se describen más arriba; no asignes el privilegio 5 solo para seguir este ejemplo.

En CSI, conéctate a la cuenta elegida, sustituyendo su nombre real:

```
CONECTAR <cuenta-origen>
```

El número de cuenta no debe estar ya conectado en otro lugar. Indica su contraseña cuando se te pida y, después, comprueba la identidad:

```
ESTADO QUIEN
```

Comprueba que tiene privilegio suficiente:

```
ESTADO PRIVILEG
```

Los comandos siguientes requieren al menos 3. Crea NOTA.TEXTO con esta cuenta siguiendo los pasos del capítulo 6. Si ese nombre ya existe o se resuelve como un archivo público, elige un nombre de práctica que no esté en uso y sustitúyelo en todo este procedimiento. No reutilices el archivo de otra cuenta solo porque pueda listarse. Examina el archivo y su propiedad antes de continuar:

```
LISDIR NOTA.TEXTO:S
```

Para mostrar las cuentas que comparten este archivo de práctica, introduce:

```
COMPART NOTA.TEXTO:S (PANT)
```

PANT lista las concesiones. Si no hay ninguna, lo esperado es un resultado vacío y CR = 0. Desde SISTEMA, esta operación muestra en cambio `La cuenta sistema no puede compartir ficheros`, CR = 12. COMPART normal inicia un diálogo para conceder o revocar el acceso. Compartir crea un alias del archivo del propietario; no duplica el espacio asignado. El propietario conserva los datos y su política de protección.

Sin desconectarte como propietario actual, transfiere el archivo de práctica a SISTEMA:

```
CAMPRO NOTA.TEXTO:S (SISTEMA NOCONSUL)
```

Así se cambia la propiedad a SISTEMA. NOCONSUL suprime la pregunta de confirmación. Ejecutar el mismo comando desde SISTEMA no hace que el archivo privado de otra cuenta pase a ser de su propiedad; puede mostrar `No se ha cambiado ningun fichero`, CR = 12. Si ocurre, comprueba el propietario actual.

Finaliza la sesión de la cuenta de origen:

```
FIN
```

Cuando ninguna otra partición esté conectada con el número de cuenta de SISTEMA, conéctate a la cuenta de destino:

```
CONECTAR SISTEMA
```

Indica su contraseña si se te pide. Examina la entrada transferida:

```
LISDIR NOTA.TEXTO:S
```

Lee el texto de práctica para comprobar que su contenido sigue disponible:

```
LISTAR NOTA.TEXTO:S
```

La transferencia a SISTEMA no garantiza que el propietario anterior pierda el acceso por búsqueda: el número de cuenta 0 de SISTEMA participa en el mecanismo alternativo de archivos públicos. Usa COMPART cuando el propietario original deba conservar la propiedad y otra cuenta necesite acceso compartido.

Protege el servicio de cuentas

SISTEMA.CUENTA almacena los registros de cuentas y usa el byte de propiedad especial 0xFF. No es un archivo de texto público corriente que se pueda renombrar o reescribir a la ligera. CONECTAR.COMANDO realiza la conexión. Si cualquiera de los dos deja de ser accesible, CSI puede aparecer sin el proceso normal de autenticación. Por tanto, ver el indicador de comandos no demuestra por sí solo que el servicio de cuentas funcione correctamente.

CAMBIAR cambia el campo de privilegio mínimo de un comando; no cambia el privilegio de la cuenta actual. Las comprobaciones de las aplicaciones pueden imponer restricciones adicionales. El capítulo 31 explica ese comando y el apéndice C describe los componentes de las cuentas.

Capítulo 11 — Gestión del almacenamiento

Una asociación de disco indica al sistema qué volumen designa un nombre como S o A. Antes de modificar el almacenamiento, identifica la asociación y examina su capacidad. Nunca deduzcas el destino solo por una letra conocida.

Examina la capacidad

Introduce:

```
ASOCIAR
```

Sin argumentos, ASOCIAR lista las asociaciones actuales. Busca la letra de disco que piensas usar. Después, introduce:

```
ESTADO DISCO
```

Compara Disponible, el espacio libre total, con Mayor zona, la mayor región libre continua. Un archivo directo puede requerir una única asignación contigua. Por ejemplo, 38 KiB repartidos en tramos separados de 1 KiB y 37 KiB no pueden satisfacer una petición contigua de 38 KiB. Un archivo secuencial puede usar varios tramos.

ESTADO DISCO puede corregir un campo de recuento de espacio libre almacenado en caché cuando vuelve a calcular la capacidad. Es una operación de estado que puede actualizar los metadatos, no un examen de solo lectura byte a byte.

Asocia y lee un disquete

Distingue el nombre físico del dispositivo, la letra lógica del disco y la etiqueta del volumen. Cada uno identifica una parte distinta de la organización del almacenamiento. FLOPPY1 y FLOPPY2 identifican definiciones físicas de dispositivos. Las letras `A` a `G` designan discos lógicos a los que pueden acceder los programas. La etiqueta del volumen es el nombre asignado al soporte, independiente de su letra lógica de disco. Asociar A con FLOPPY1 no cambia el nombre del volumen de ese disquete, y A no es por naturaleza una unidad de disquete.

El procedimiento siguiente requiere una unidad de disquete compatible que funcione y un disquete con formato de sistema ya inicializado. FLOPPY1 y FLOPPY2 seleccionan interfaces de controlador distintas; usa la definición que corresponda a la controladora instalada. Una asociación por sí sola no demuestra que la unidad esté preparada.

Muestra primero las asociaciones:

```
ASOCIAR
```

Asegúrate de que A está disponible para este procedimiento. Si una aplicación está usando su asociación actual, cierra los archivos de la aplicación antes de cambiarla. Comprueba las definiciones físicas:

```
LISTAR SISTEMA.NOMBDISP:S
```

Para una unidad instalada que use FLOPPY1, asóciala con A:

```
ASOCIAR A FLOPPY1
```

Así se selecciona la primera definición de disquete. Si la unidad instalada usa en cambio FLOPPY2, sustituye ese nombre físico. No elijas una definición solo porque aparezca en el archivo.

Inserta el disquete preparado mientras la unidad está inactiva y cierra su mecanismo de carga. Notifícalo al sistema operativo:

```
MONTAR A
```

MONTAR indica al sistema que el disco o el disquete ha cambiado. Actualiza el estado del soporte del dispositivo; no monta un sistema de archivos bajo una ruta de directorio. No formatea un disquete virgen ni le asigna su letra lógica.

Lista los archivos visibles para la cuenta actual:

```
LISDIR *.*:A
```

El sufijo `:A` deja explícito el dispositivo. Que falte un archivo puede deberse a la propiedad de la cuenta y no a que el soporte esté vacío. Para leer el archivo de práctica cuando esté presente, introduce:

```
LISTAR NOTA.TEXTO:A
```

Así se muestra el archivo del disquete. No se lee NOTA.TEXTO de S.

Copia un archivo a un disquete y desde él

Usa soportes prescindibles con permiso de escritura, espacio libre suficiente y un nombre de destino que no esté en uso. Para copiar el texto de práctica cerrado de S a A, introduce:

```
COPIFI NOTA.TEXTO:S NOTA.TEXTO:A (VERIFI)
```

El primer nombre de archivo es el origen; el segundo es el destino. VERIFI solicita una comprobación por relectura. Espera a que termine y, después, examina el resultado:

```
LISTAR NOTA.TEXTO:A
```

Para volver a copiarlo con otro nombre que no esté en uso, introduce:

```
COPIFI NOTA.TEXTO:A VUELTA.TEXTO:S (VERIFI)
```

Al invertir los sufijos de unidad se invierte el sentido. Usar VUELTA.TEXTO evita sustituir el archivo de práctica original. Muestra la copia devuelta:

```
LISTAR VUELTA.TEXTO:S
```

Compara su texto con el original. Para un archivado más grande que ocupe varios soportes extraíbles, usa los procedimientos de ARCHIVAR y RECARGAR del capítulo 13.

Cambia o libera soportes extraíbles

Antes de cambiar un disquete, termina las transferencias y cierra todos los archivos que lo usen. Espera a que la unidad deje de tener actividad, sustituye el soporte y, después, introduce:

```
MONTAR A
```

Lista el nuevo soporte antes de usarlo. Un diagnóstico `no prep.` indica que el dispositivo no está preparado; comprueba que el soporte previsto está insertado y disponible. Un diagnóstico `prot. escritura` indica protección contra escritura. Restablece el soporte o el estado de protección previstos antes de volver a intentarlo; no elijas Ignorar para convertir una escritura fallida en un éxito aparente. El capítulo 27 explica las distintas opciones de reintentar, ignorar y anular. Cuando han cambiado varias unidades, la forma alternativa es:

```
MONTAR *
```

El asterisco pide al sistema que examine todas las unidades. No es un nombre de archivo con comodines.

Para liberar la asociación lógica cuando haya terminado el trabajo, introduce:

```
ASOCIAR A
```

Así se elimina la asociación de A. No expulsa el disquete ni cierra los archivos que una aplicación tenga abiertos en él. Aquí no se especifica ningún comando universal independiente para desmontar y vaciar búferes; primero hay que cerrar los archivos y completar las transferencias. No retires soportes activos ni desasocies S mientras el sistema operativo dependa de ella.

Formatea un disquete

El formateo prepara las pistas y los sectores físicos. INICIAR es la utilidad para disquetes; INICDISC es la utilidad para discos. Ambas ofrecen también operaciones que cambian las etiquetas, la capacidad del directorio o la protección contra escritura. Son peticiones distintas, no sinónimos de MONTAR.

> Precaución — el formateo destruye el contenido existente. Comprueba la asociación de A y el soporte físico antes de ejecutar FORMAT o VACIA. Usa un disquete prescindible con el tamaño, el número de caras y la densidad correctos para la unidad instalada. El privilegio mínimo del comando es 3.

Lee la ayuda de la utilidad de disquetes antes de seleccionar un formato:

```
AYUDA INICIAR
```

Para una unidad y un soporte compatibles con el formato de doble cara y doble densidad de la utilidad, su ejemplo de formato documentado puede escribirse completo así:

```
INICIAR A (FORMAT CARA 2)
```

FORMAT solicita el formateo físico; CARA 2 selecciona dos caras. En la ruta de FLOPPY2, esta forma usa 77 cilindros, dos cabezas y 26 identificadores de sector por pista. Las dos pistas del cilindro 0 usan grabación FM con sectores de 128 bytes; los cilindros 1 a 76 usan MFM con sectores de 256 bytes. Son las características del formato de software de la utilidad, no la identificación de un fabricante o modelo de unidad ni de un formato intercambiable con PC. La unidad y el soporte instalados deben admitirlas.

Lo siguiente es una plantilla de sintaxis, no un comando que deba escribirse literalmente:

```
INICIAR A (FORMAT CARA <caras> PISTA <pistas> SECTOR <sectores>)
```

La ayuda enumera estas opciones de geometría, pero no establece que cualquier valor de PISTA o SECTOR modifique todos los bucles físicos de formateo. El bucle de FLOPPY2 descrito antes llega hasta el cilindro 76. No uses la plantilla para dar por supuesta la compatibilidad con geometrías arbitrarias ni para deducir valores de un disco duro o de un formato de disquete moderno. Las opciones son:

Opción	Operación
FORMAT	Formatea físicamente el disquete; es obligatorio para un soporte virgen
CARA n	Selecciona el número de caras
PISTA n	Solicita el número de pistas por cara; el comportamiento del bucle físico depende de la ruta de la controladora
SECTOR n	Solicita el número de sectores por pista; no está establecido que se admitan valores arbitrarios de formato físico
DIRE n	Solicita capacidad de directorio para n archivos
ETIQU	Cambia la etiqueta del volumen
VACIA	Borra el contenido de los archivos descritos por el directorio del volumen
WP / NOPE	Activa o desactiva la protección contra escritura por software
BOOT=d	Copia la pista de arranque desde el disco especificado

No añadas BOOT solo para crear un disquete de datos normal. El cargador de arranque de origen y el hardware de destino deben ser compatibles. Aquí no se especifican el diálogo completo de inicialización ni sus mensajes de éxito, por lo que no se ofrece ninguna salida de ejemplo.

Cuando la utilidad termine correctamente, actualiza el dispositivo:

```
MONTAR A
```

Comprueba el formato:

```
VERFORM A
```

Lee los diagnósticos además del resultado de finalización. Esto no sustituye a la verificación de la copia de archivos ni certifica la compatibilidad con otra unidad. Después, examina su directorio:

```
LISDIR *.*:A
```

Después, usa el procedimiento anterior de copia y relectura con un archivo de práctica pequeño. Un listado correcto por sí solo no verifica todas las pistas del soporte.

Inicializa un volumen de disco duro

INICDISC formatea el volumen de disco seleccionado y crea sus estructuras de sistema de archivos. Actúa sobre el disco designado por una asociación lógica como A. No crea una partición de ejecución, y ASOCIAR no reserva ni divide espacio físico en el disco.

> Precaución — el formateo destruye el contenido del volumen seleccionado y puede cambiar los metadatos de correspondencia del disco. Haz una copia de seguridad de los datos que quieras conservar, detén las aplicaciones que usen el destino y verifica su extensión física. Nunca apliques el ejemplo a S ni a una región que se solape con el volumen del sistema. El privilegio mínimo es 3.

Selecciona el destino

Empieza examinando las asignaciones actuales:

```
ASOCIAR
```

A debe designar el volumen de disco duro prescindible previsto, no un disquete ni otro dispositivo activo. Si no hay ningún volumen adecuado configurado, sigue las indicaciones sobre volúmenes adicionales que aparecen a continuación antes de seguir.

Opcional: prepara un volumen adicional

Puede habilitarse un volumen de datos adicional cuando la configuración del disco ya ofrece una extensión adecuada sin usar y una definición de dispositivo. En una configuración en la que DISCO2 designe esa extensión prevista, introduce:

```
ASOCIAR A DISCO2
```

Así se conecta la letra lógica A con la definición DISCO2. Esto no demuestra que se haya reservado almacenamiento ni que exista allí un sistema de archivos utilizable. Continúa solo después de verificar que la extensión de destino está disponible y no se solapa con ningún volumen que quieras conservar. DISCO2 es un ejemplo condicional; no se afirma que todos los sistemas incluyan o tengan configurado un volumen adicional.

La ruta de formateo documentada para DISCO2 usa PISTA tanto para el número de cilindros como para el cilindro inicial. Con PISTA 397, su extensión física son los cilindros 397 a 793, ambos incluidos. Por tanto, cambiar PISTA puede desplazar el inicio de la región formateada además de cambiar su tamaño. No lo reduzcas para acortar el formateo sin volver a comprobar toda la distribución del disco. El número 397 es un ejemplo para una disposición compatible, no un parámetro de disco universal.

Los pasos restantes inicializan un sistema de archivos en esa extensión configurada. No hay establecido ningún procedimiento general para crear nuevos descriptores arbitrarios de volúmenes físicos ni para cambiar el tamaño de volúmenes con datos. Un nombre en SISTEMA.NOMBDISP no basta por sí solo para realizar esas operaciones de forma segura.

Formatea y revisa el diálogo

Para un destino prescindible cuya geometría configurada es de cinco cabezas, 397 cilindros y 32 sectores por pista, con sectores de 256 bytes, introduce:

```
INICDISC A (FORMAT CARA 5 PISTA 397 SECTOR 32 DIRE 64)
```

FORMAT solicita el formateo físico. CARA indica el número de cabezas; PISTA, el número de cilindros, con la salvedad sobre la ubicación indicada antes; y SECTOR, el número de sectores por pista. DIRE solicita la capacidad del directorio. La utilidad puede ajustar esa petición: esta forma indica 88 entradas de directorio, no exactamente 64. Revisa los valores mostrados antes de aceptarlos.

En `Usted quiere continuar (S/N)?`, introduce lo siguiente solo si la advertencia de destrucción menciona el destino previsto:

```
S
```

La utilidad pregunta `Etiqueta del disco:`. Indica una nueva etiqueta de volumen; para este ejemplo:

```
PRACTICA
```

La etiqueta identifica el volumen formateado. No cambia el nombre de la asociación A ni el de la definición física del dispositivo. A continuación, la utilidad muestra la capacidad del directorio, el entrelazado, el desplazamiento de pista y la geometría. En `Es correcta esta informacion (S/N)?`, acepta solo después de comprobar todo el resumen:

```
S
```

En la pregunta opcional de análisis de superficie, este ejemplo básico de formateo selecciona:

```
N
```

Así se omite el análisis de superficie independiente; el resultado no debe describirse como una prueba de superficie completa. En `Desea reintentos en lectura y escritura (S/N)?`, introduce:

```
S
```

Así se activan los reintentos de lectura y escritura solicitados. En `Quiere incluir ahora alguna pista (S/N)?`, el ejemplo no indica ninguna entrada adicional de pista defectuosa:

```
N
```

No reutilices esa respuesta para ocultar defectos conocidos del soporte. El tratamiento de las pistas defectuosas requiere información exacta sobre el soporte real. Espera a que termine el formateo y examina los diagnósticos que aparezcan; un resultado de comando por sí solo no certifica los datos.

Monta y comprueba el nuevo sistema de archivos

Notifica al sistema el volumen inicializado:

```
MONTAR A
```

MONTAR actualiza el estado del soporte. Comprueba el formato:

```
VERFORM A
```

Lee los errores indicados antes de continuar. Después, lista el catálogo:

```
LISDIR *.*:A
```

Así se listan las entradas visibles para la cuenta actual. Para comprobar el uso normal de archivos, empieza con el archivo de práctica cerrado NOTA.TEXTO del capítulo 6 y con nombres de destino que no estén en uso. Cópialo al nuevo volumen:

```
COPIFI NOTA.TEXTO:S NOTA.TEXTO:A (VERIFI)
```

VERIFI solicita la verificación por relectura. Vuelve a copiarlo con otro nombre:

```
COPIFI NOTA.TEXTO:A REVISION.TEXTO:S (VERIFI)
```

Así se conserva el origen original. Examina el texto devuelto:

```
LISTAR REVISION.TEXTO:S
```

Compáralo con NOTA.TEXTO. Estas comprobaciones ponen a prueba el montaje, el acceso al catálogo, la asignación y las transferencias de archivos; no establecen la compatibilidad de arranque ni sustituyen a un procedimiento de copia de seguridad.

Disquetes de intercambio compatibles con IBM

El intercambio IBM 3741 es una ruta de formato independiente. INICIBM escribe las cabeceras que usan las utilidades de intercambio; no es un sinónimo general de INICIAR y no establece por sí solo que un disquete virgen se haya formateado físicamente de forma correcta.

Para un disquete compatible, prescindible y correctamente preparado en A, introduce:

```
INICIBM A
```

Así se cambian las cabeceras del soporte. Para examinar un directorio con formato IBM, usa:

```
DIRIBM A
```

DIRIBM muestra los registros de volumen y de archivo en ese formato. Usa SECIBM para escribir datos secuenciales del sistema en un disquete IBM 3741 e IBMSEC para la conversión inversa. Estas utilidades se encargan de la conversión del formato de registro y del código de caracteres; el COPIFI normal no las sustituye.

Comprende la protección

ATRIB controla restricciones independientes de escritura, borrado y lectura. Estos bits afectan a rutas de acceso concretas; no crean un estado universal de «hacerlo todo seguro».

Para un archivo secuencial de práctica cuyo propietario sea tu cuenta, introduce:

```
ATRIB NOTA.TEXTO:S (E NOCONSUL)
```

E activa la protección contra escritura sin cambiar el nombre. Los cambios que solo afectan a los atributos usan un único nombre de archivo; la forma con dos nombres sirve para cambiar el nombre. Examina el resultado:

```
LISDIR NOTA.TEXTO:S
```

En el archivo secuencial de práctica, la indicación de formato cambia de S a SW y el texto se conserva. La protección contra escritura puede impedir que se amplíe el archivo y, aun así, permitir sustituirlo borrándolo y volviéndolo a crear. B protege contra el borrado; L es el bit interno de protección contra lectura; ATRIB acepta LE para esta operación. La palabra más larga LEER se rechaza. Para desactivarlo con NOL se necesita el privilegio 255.

Protección individual	Leer el contenido existente	Sustituir borrando y volviendo a crear	Añadir al contenido existente
E, escritura	Permitido	Permitido mediante borrado y nueva creación	Rechazado
B, borrado	Permitido	Rechazado	Permitido
L, lectura	Rechazado	Permitido mediante borrado y nueva creación	Rechazado

La tabla describe el acceso del propietario actual a un archivo secuencial con cada protección aplicada por separado. Sustituir un archivo puede implicar borrar la entrada antigua y crear una nueva; por eso la protección contra escritura por sí sola no impide todas las sustituciones. El acceso compartido y la

combinación de atributos pueden imponer restricciones adicionales. El capítulo 17 relaciona estas operaciones con los modos de apertura de BASIC. Cuando termines, desactiva el bit E del archivo de práctica:

```
ATRIB NOTA.TEXTO:S (NOE NOCONSUL)
```

NOE elimina la protección contra escritura. Vuelve a examinar el archivo:

```
LISDIR NOTA.TEXTO:S
```

La indicación de formato vuelve de SW a S. Ninguno de los dos comandos de atributos cambia el nombre del archivo.

Otras tareas de mantenimiento de discos

El procedimiento de inicialización del disco duro prepara un sistema de archivos en un volumen configurado. Sus requisitos de geometría y ubicación siguen siendo esenciales para el mantenimiento posterior. El capítulo 12 explica las particiones de memoria; el capítulo 13 explica la inicialización de cintas y las copias de seguridad.

ORGDIR reorganiza las entradas del directorio. ELIMINAR elimina una entrada sin liberar su espacio de almacenamiento. UNOSEC reconstruye los metadatos del volumen. Son herramientas de mantenimiento, no alternativas a la copia o el borrado normales. Haz una copia de seguridad utilizable antes de invocarlas.

No añadas datos a un archivo secuencial sin terminar solo para que vuelva a ser utilizable. Su información incompleta de fin de archivo puede hacer que la salida posterior sobrescriba los metadatos del volumen. Un archivo vacío cerrado correctamente es distinto de un archivo sin terminar. El capítulo 13 trata las precauciones de recuperación; el capítulo 17 expone la restricción para programadores y el capítulo 26 explica la ruta de escritura.

Capítulo 12 — Dispositivos, impresión y particiones

Los nombres de dispositivo separan la finalidad de un programa del hardware que la atiende. IMPR1 designa una impresora lógica; IMP designa una definición física de impresora. ASOCIAR conecta ambas. En este capítulo se configuran primero los dispositivos de terminal e impresora y, después, se tratan las particiones de memoria que los usan. Las asociaciones de discos y disquetes se tratan en el capítulo 11.

Examina las asociaciones y las definiciones de dispositivos

Lista primero las asociaciones actuales:

```
ASOCIAR
```

Así se ve qué nombres lógicos seleccionan ya dispositivos. Examina por separado sus definiciones físicas:

```
LISTAR SISTEMA.NOMBDISP:S
```

Este texto enumera nombres y definiciones. Que exista una definición para un hardware opcional no significa que ese hardware esté instalado. Compara la lista con la configuración real de la máquina.

Configura un terminal

El terminal y su controlador deben coincidir en los parámetros de comunicación. La consola puede funcionar con una conexión a 19 200 baudios, siete bits de datos, sin paridad y un bit de parada. Los baudios indican la velocidad de transmisión; la anchura de los datos y la paridad indican el formato de trama de los caracteres. Usa parámetros que admitan el terminal y el controlador reales.

Una clase de terminal traduce las operaciones de cursor y de presentación del sistema operativo a las secuencias que entiende ese terminal. La clase 1 usa secuencias de coordenadas ESC G; la clase 2 usa una secuencia similar a ANSI. Si la clase no coincide, puede que los comandos escritos funcionen, pero que el movimiento del cursor sea incorrecto. Las clases son componentes ejecutables de traducción; el capítulo 28 documenta las secuencias reales.

Para una consola en PAN1 que requiera la clase 2, introduce:

```
ASOCIAR CONSOLA PAN1 (C2)
```

C2 carga la traducción de presentación de la clase 2. El terminal debe entender esa familia de secuencias de control; un terminal que acepta texto normal no es necesariamente compatible con el posicionamiento del cursor y el borrado. Para seleccionar en cambio la clase 1, introduce:

```
ASOCIAR CONSOLA PAN1 (C1)
```

Elige la clase adecuada para el terminal; estos dos comandos son alternativos, no un recorrido de ida y vuelta obligatorio. Los cambios repetidos de clase pueden reducir el límite de memoria disponible de la partición. Las dimensiones normales son L79/P23. L se usa tanto para dar formato a la salida como para limitar la entrada de CSI; un valor de L muy pequeño puede enviar un comando incompleto antes de pulsar Intro. Mantén los valores normales salvo que haya un motivo concreto para cambiarlos.

Para la entrada serie, configura el terminal que envía para que respete el control de flujo de recepción del ordenador cuando la conexión lo admita. El controlador ALP solicita una pausa cuando se llena su anillo de entrada y reanuda la recepción cuando se vacía la entrada en cola. Esto no elimina el límite menor del búfer de recepción de la controladora serie: dosifica los flujos de entrada largos si se pierden caracteres. El cableado exacto de las señales de módem depende de la interfaz instalada.

PAN9 y PAN10 son nombres de dispositivos de terminal de SS2, con los alias IS9 e IS10. No designan los números de partición 9 y 10. Una partición puede usar una de estas consolas mediante ACTIVAR cuando el hardware correspondiente esté instalado; el capítulo 28 describe sus rutas serie independientes.

Imprime un archivo de texto

Si la impresora instalada usa la definición IMP y los parámetros serie indicados, asóciala:

```
ASOCIAR IMPR1 IMP (B4800 PE)
```

B4800 selecciona 4800 baudios nominales y PE selecciona paridad par. Esta forma selecciona siete bits de datos y un bit de parada; configura la impresora con el mismo formato de trama. Añadir W8 solicitaría una anchura distinta, de ocho bits. Usa los parámetros que requiera la impresora real. Ahora imprime el texto de práctica:

```
LISTAR NOTA.TEXTO:S (PRINT1 NOHEAD)
```

PRINT1 selecciona la impresora lógica 1; NOHEAD suprime el encabezado del listado. Confirma que realmente sale el papel impreso. Un resultado de comando correcto no certifica la impresión física.

IMP usa señales serie de disponibilidad. Una impresora fuera de línea o desconectada puede producir `Impresora no preparada (R/Q):`; restablece la conexión y la disponibilidad antes de pedir R para reintentar, o elige Q para abandonar la operación. CTS puede retener la transmisión hasta que la impresora la permita. Que una llamada de salida con búfer vuelva correctamente no demuestra que haya llegado la página completa.

Mantén IMPR1 asociada si vas a continuar con la impresión diferida que se describe a continuación.

Impresión diferida

SPOOLER recoge informes para imprimirlos más tarde. La inicialización puede recuperar el estado de la cola y eliminar determinados informes de respaldo, así que revisa primero el trabajo existente. Con la impresora configurada, introduce:

```
SPOOLER INIC
```

Así se inicializa el servicio. Examínalo:

```
SPOOLER ESTADO
```

Para pausar la salida normal de la cola, introduce:

```
SPOOLER PARA
```

PARA no anula todas las peticiones explícitas de impresión o de relanzamiento. Para examinar los informes en cola, introduce:

```
SPOOLER LIST
```

La recogida usa SISTEMA.SPOOLCOL y archivos de informe. El capítulo 29 explica su relación, y la referencia de comandos detalla por separado los demás controles de la cola.

Libera una asociación de impresora que no se use

La siguiente limpieza es opcional. Úsala solo cuando las aplicaciones y la impresión en cola ya no necesiten IMPR1. Pausar una cola no demuestra que su asociación de impresora ya no sea necesaria; no desasocies una impresora que siga atendiendo al SPOOLER.

Cuando la asociación ya no esté en uso, introduce:

```
ASOCIAR IMPR1
```

Así se elimina la asociación lógica. Es distinto de ASOCIAR sin argumentos, que lista las asociaciones, y no es un comando para detener el SPOOLER.

Las secciones restantes gestionan las particiones. La configuración de dispositivos selecciona el hardware; la asignación de particiones aporta la memoria y el estado de ejecución. Examina esos recursos por separado antes de hacer cambios.

Lista las particiones

Para mostrar las particiones de memoria asignadas, introduce:

```
ESTADO MEMORIA
```

Este es el listado de particiones. Su encabezado incluye `Puer`, `Modu`, `DIRBA`, `DIRMX`, `DIRAL`, `SP---`, `PC---`, `Tama|`, `Rutina` y `Usuario`. El número de partición y el número de módulo de memoria son columnas distintas. Las direcciones describen la memoria y el estado de ejecución de la partición; `Rutina` y `Usuario` identifican su rutina y la cuenta conectada, cuando están disponibles. No necesitas interpretar todas las direcciones para saber qué particiones están asignadas.

Una partición detenida puede seguir apareciendo en este informe porque PARAR conserva su memoria. Por tanto, un campo de usuario vacío no significa por sí solo que la partición pueda volver a asignarse. El informe también puede contener componentes residentes como NUCLEO y SPOOLER; esas filas no son cuentas de usuario adicionales.

Para ver los módulos de memoria disponibles, introduce:

```
ESTADO MODULO
```

Un módulo es un recurso de memoria a partir del cual puede asignarse una partición. Este informe ayuda a elegir el último argumento de PONER MEMORIA; no es una lista de volúmenes de disco.

Para ver los usuarios conectados, introduce:

```
ESTADO USUARIOS
```

Así se muestran las conexiones de los usuarios con su fecha y hora de conexión. Para listar las cuentas registradas, incluidas las que no están conectadas, usa CUENTA (PANT), como se describe en el capítulo 10. Comprueba estos informes independientes antes de asignar, detener o eliminar una partición.

Crea y activa una partición

Una partición es un área de ejecución en memoria que se planifica de forma independiente. No es una región de un disco y no tiene ningún sistema de archivos que montar. PONER MEMORIA la asigna; ACTIVAR pone en marcha su terminal o su entorno de comandos. El acceso a disco se configura por separado con ASOCIAR y MONTAR, como se explica en el capítulo 11.

Realiza la configuración de memoria desde SISTEMA con el privilegio 5. Usa el número de partición siguiente a los ya asignados: por ejemplo, 2 cuando solo existe la partición 1. El intervalo de numeración público es de 1 a 16, pero la memoria disponible y los terminales instalados determinan cuántas particiones útiles pueden ejecutarse.

Examina primero los módulos disponibles:

```
ESTADO MODULO
```

La configuración siguiente requiere que esté disponible el módulo 1 y que haya un terminal en PAN2. Es un ejemplo para esa configuración, no un comando que deba repetirse en una partición ya ocupada. Introduce:

```
PONER MEMORIA 2 48000 1
```

Así se solicita la partición mostrada como 2 en el módulo 1, con 48000 como parámetro de tamaño. El parámetro no garantiza una asignación exacta. En la configuración de demostración, la primera asignación de ese módulo ocupa 0x4C8B–0xFFFF, una región de 45 941 bytes; una petición de 40000 recibe también esa región. Este resultado no establece la regla de asignación para todas las configuraciones de módulo compartido. El 1 final es una selección de módulo, no un número de cuenta.

Examina la asignación real:

```
ESTADO MEMORIA
```

Usa los límites y el tamaño indicados en lugar de suponer que se asignó el número de bytes solicitado. Activa el terminal:

```
ACTIVAR 2 (PAN2)
```

Ahora el segundo terminal puede conectarse a una cuenta. La asignación y la activación son pasos distintos porque una partición necesita memoria antes de poder ejecutarse. Examina el resultado:

```
ESTADO MEMORIA
```

El comando muestra la memoria de las particiones. El intervalo de numeración admitido no garantiza que haya RAM suficiente para dieciséis usuarios activos útiles.

Detén una partición deliberadamente

En el terminal de destino, guarda y cierra la aplicación. En su indicador de CSI, introduce:

```
FIN
```

FIN desconecta la cuenta y deja ese terminal preparado para otro inicio de sesión. Para detener la propia partición, vuelve al terminal de administración e introduce:

```
PARAR 2
```

PARAR detiene la partición y conserva su asignación. Puede rechazar la operación si hay una cuenta conectada. FORZAR puede inyectar FIN en un destino, pero una finalización forzada puede dejar un registro de aplicación incompleto; úsalo solo para recuperar el sistema, no para cerrar sesiones de forma rutinaria.

Para volver a usar una partición detenida que sigue asignada, introduce:

```
ACTIVAR 2 (PAN2)
```

Así se reactiva su terminal. Se usa la asignación de memoria existente y no se crea otra cuenta.

Elimina una partición asignada

Detener y eliminar son operaciones distintas. PARAR libera el terminal activo, pero conserva la asignación de memoria de la partición. La forma de eliminación de PONER MEMORIA omite los argumentos de tamaño y de módulo.

Lo siguiente continúa el ejemplo anterior de módulo dedicado. La partición 2 debe ser la última asignada, debe estar detenida y no debe ser la partición desde la que ejecutas el comando. Si la reactivaste en el ejemplo anterior, finaliza su sesión y vuelve a detenerla antes de continuar.

Desde la partición de administración, introduce:

```
PONER MEMORIA 2
```

Sin argumento de tamaño, así se solicita la eliminación de la partición 2. No añadas `0` creyendo que es equivalente: el analizador del tamaño eleva los tamaños explícitos pequeños hasta su asignación mínima. Comprueba el resultado:

```
ESTADO MEMORIA
```

Después, examina los módulos disponibles:

```
ESTADO MODULO
```

La partición ya no debe aparecer como asignada. Elimina las particiones en orden inverso al de asignación; el comando rechaza la eliminación de una partición que no sea la última. Un destino activo, la propia partición de quien llama o una partición vecina incompatible también pueden impedir la eliminación. La rutina puede reorganizar el almacenamiento de contextos cuando las particiones comparten un módulo. Ese caso de módulo compartido queda fuera de este procedimiento básico; usa la disposición de módulo dedicado que se muestra aquí.

Estos comandos modifican la configuración en ejecución. No la guardan por sí solos como configuración para el próximo arranque. GENSIST escribe el núcleo del sistema y es una operación de mantenimiento independiente y de consecuencias importantes. Eliminar una partición de memoria no borra archivos del disco ni elimina cuentas.

Capítulo 13 — Copias de seguridad y recuperación

Una copia de seguridad debe servir para el tipo de recuperación que necesitas. Las copias de archivos protegen archivos concretos; las operaciones de archivado conservan archivos o cuentas seleccionados; la copia de un soporte completo tiene otra finalidad. Que exista un comando no garantiza que esté instalado el dispositivo de cinta o de disquete que necesita.

Haz una copia de seguridad de archivos

Para el texto de práctica, elige un destino que no esté en uso e introduce:

```
COPIFI NOTA.TEXTO:S RESPALDO.TEXTO:S (VERIFI)
```

COPIFI duplica el archivo y solicita la verificación. Examina la copia antes de confiar en ella:

```
LISTAR RESPALDO.TEXTO:S
```

Una copia en el mismo volumen ayuda a recuperar una edición accidental, pero no protege contra un fallo de ese volumen. Para una copia de seguridad independiente, selecciona un dispositivo de destino configurado por separado y verifica allí los archivos resultantes.

Archiva archivos en disquetes

ARCHIVAR guarda archivos seleccionados, los archivos de una cuenta o un volumen entero. RECARGAR lee los soportes escritos por ARCHIVAR. Estas operaciones son distintas de copiar un archivo normal con COPIFI. Prepara y asocia el disquete de destino como se describe en el capítulo 11. Los ejemplos siguientes muestran las formas de comando documentadas; para que la operación física funcione hacen falta una unidad y unos soportes compatibles.

Con un disquete de archivado con permiso de escritura en A, guarda solo el texto de práctica:

```
ARCHIVAR NOTA.TEXTO:S,A (PANT VERIFI CONSU)
```

NOTA.TEXTO:S selecciona el archivo de origen; A selecciona el destino del archivado. PANT muestra los nombres, VERIFI solicita la relectura después de escribir y CONSU pregunta antes de guardar cada archivo seleccionado. Lee cada pregunta antes de responder. Esta operación escribe en el soporte de destino; no uses un disquete que contenga información que deba conservarse.

Para seleccionar en cambio los archivos de la cuenta conectada, el comando alternativo es:

```
ARCHIVAR S,A (CUENTA PANT VERIFI CONSU)
```

CUENTA limita la selección a la cuenta actual. No significa «copiar la base de datos de cuentas». Es una copia de seguridad más amplia, no un segundo paso obligatorio después del ejemplo de un solo archivo. ARCHIVAR puede usar una sucesión de soportes, pero aquí no se especifica por completo el diálogo exacto de cambio de soporte; sigue las peticiones de la utilidad y conserva los soportes en el orden en que se escribieron.

Para hacer una restauración de prueba del archivo de práctica, conéctate a la cuenta de destino prevista y selecciona un disco de destino en el que esa cuenta no tenga ningún archivo NOTA.TEXTO. Con el archivado en A, introduce:

```
RECARGAR NOTA.TEXTO:A,S (CUENTA NOREEM PANT VERIFI CONSU)
```

RECARGAR selecciona ese nombre de archivo archivado para la cuenta conectada, escribe en S y se niega a sustituir un archivo existente porque está presente NOREEM. CONSU mantiene la pregunta para cada archivo. Si NOTA.TEXTO ya existe, esto no es una prueba de que la copia archivada pueda sustituirlo; para una prueba de restauración real, elige un entorno de destino independiente.

Tras una restauración correcta, examina el archivo:

```
LISTAR NOTA.TEXTO:S
```

Comprueba el contenido restaurado. La relectura al escribir un archivado y la posterior restauración de un archivo utilizable son comprobaciones distintas.

Prepara el acceso a la cinta

Los comandos de cinta usan un dispositivo lógico como TAPE1. TAP es la definición física del controlador de cinta. Un cartucho es un soporte secuencial, así que el sistema no lo presenta como un directorio de disco que pueda recorrerse con LISDIR.

El lenguaje de comandos y las operaciones de inicialización que se describen a continuación están disponibles, pero no está establecido un ciclo completo de copia de seguridad y restauración física en cinta. Los detalles de grabación física de la controladora también siguen siendo incompletos. Trata los ejemplos como instrucciones de uso para una instalación compatible y en funcionamiento, con esa limitación; no se garantiza ninguna velocidad de transferencia, capacidad de soporte ni salida de finalización de ejemplo.

Empieza examinando las asociaciones:

```
ASOCIAR
```

Si la controladora instalada usa TAP y TAPE1 está disponible, introduce:

```
ASOCIAR TAPE1 TAP
```

Así se carga e inicializa el controlador de cinta para esa asociación. El comando consulta a la controladora, así que una controladora ausente o incompatible puede dejarlo esperando. No pases a un comando de archivado salvo que la asociación se complete y el dispositivo esté preparado.

MONTAR es el comando de cambio de soporte para discos y disquetes. No existe una forma `MONTAR TAPE1` reconocida. La carga, la disponibilidad y el posicionamiento del cartucho corresponden a la utilidad de cinta y a la controladora. Carga el cartucho con los controles físicos de la unidad mientras esté inactiva y sigue las peticiones de la utilidad. Esta interfaz no especifica ningún comando universal independiente para montar, rebobinar o expulsar cintas.

Inicializa un cartucho

INITTAPE prepara un cartucho nuevo o borra un volumen de cinta anterior. Usa TAPE1; su línea de comandos no admite una letra de disco ni un nombre de archivo de cartucho.

> Precaución — destruye la información existente en la cinta. Comprueba el cartucho antes de inicializarlo. Nunca inicialices el cartucho de archivado que piensas restaurar.

Para una inicialización normal, introduce:

```
INITTAPE
```

Así se inicia la inicialización de la cinta y se escribe una etiqueta de volumen. La utilidad tiene pasos de confirmación y de introducción de la etiqueta; su diálogo completo no se reproduce aquí. Léelos en el terminal antes de dejar que la operación continúe.

Para un cartucho nuevo que requiera la pasada de tensado de la utilidad, usa esta alternativa:

```
INITTAPE (TENSION)
```

TENSION recorre y rebobina el cartucho antes de la inicialización. No es una petición de rebobinado inofensiva e independiente: sigue formando parte de INITTAPE y, por tanto, solo tiene cabida en el procedimiento de inicialización. La etiqueta de volumen registra un nombre de hasta ocho caracteres. En el apéndice A aparecen más detalles sobre las etiquetas de archivado.

Guarda archivos o un volumen en cinta

Con TAPE1 preparada y un cartucho inicializado y prescindible cargado, empieza con un único archivo de práctica cerrado:

```
ARCHIVAR NOTA.TEXTO:S,TAPE1 (PANT VERIFI CONSU)
```

El origen es NOTA.TEXTO en S y el destino es la cinta. Las opciones mantienen la selección visible de archivos, la verificación de escritura y la pregunta de confirmación. La operación escribe un archivado; LISTAR y la salida normal de caracteres a TAPE1 no producen un archivado en cinta equivalente.

Para archivar los archivos de la cuenta conectada, usa en su lugar esta selección más amplia:

```
ARCHIVAR S,TAPE1 (CUENTA PANT VERIFI CONSU)
```

CUENTA selecciona los archivos de esa cuenta. Para seleccionar todas las cuentas del volumen de origen, la alternativa es:

```
ARCHIVAR S,TAPE1 (VOLUM PANT VERIFI CONSU)
```

VOLUM amplía la selección al volumen. Pide que las aplicaciones cierren sus archivos antes de una copia de seguridad que deba representar un conjunto coherente de registros de gestión. Estos tres comandos son selecciones alternativas, no una secuencia que deba ejecutarse en el mismo cartucho sin tener en cuenta el archivado existente.

Restaura un archivado desde cinta

Usa el cartucho escrito por ARCHIVAR, con TAPE1 asociada a la unidad compatible. Conéctate a la cuenta de destino, elige el disco de destino y evita nombres de archivo existentes en la primera prueba de restauración. No ejecutes INITTAPE.

Para restaurar el archivo de práctica en S, introduce:

```
RECARGAR NOTA.TEXTO:TAPE1,S (CUENTA NOREEM PANT VERIFI CONSU)
```

La cinta aparece antes de la coma porque ahora es el origen. S es el destino. CUENTA selecciona la cuenta conectada y NOREEM impide sustituir un archivo existente. Tras una restauración correcta, examina su contenido:

```
LISTAR NOTA.TEXTO:S
```

Una restauración del volumen completo tiene un alcance mucho mayor. Su forma documentada, manteniendo la no sustitución y la pregunta de confirmación, es:

```
RECARGAR TAPE1,S (VOLUM NOREEM PANT VERIFI CONSU)
```

Úsala solo cuando se hayan planificado el volumen de destino y los requisitos de propiedad. VOLUM restaura todas las cuentas. REEMPLA permite sobrescribir archivos existentes; COPREEM limita la operación a sustituir archivos existentes; NOREEM carga solo los archivos que faltan. Son alternativas. Si se omite el nombre de archivo, puede seleccionarse de forma predeterminada la restauración del volumen completo, así que mantén la selección explícita.

RESCUE selecciona otra cuenta archivada para restaurarla en la cuenta actual; consulta RECARGAR antes de usar esa forma. No des por hecho que restaurar los archivos de una aplicación restaura también todas las cuentas o los ajustes de arranque.

Copia de soportes completos y finalización del trabajo con cinta

BACKUP realiza copias binarias entre discos o entre disco y cinta. No usa el formato de selección de archivos de ARCHIVAR. La forma explícita de disco a cinta es:

```
BACKUP S TAPE1
```

Así se selecciona todo el soporte de origen S para el destino en cinta. Requiere dispositivos compatibles y soportes de destino prescindibles. No ejecutes BACKUP sin argumentos mientras aprendes: sus valores predeterminados son S como origen y A como destino, lo que puede sobrescribir el disco equivocado. RECARGAR está documentado para soportes de ARCHIVAR; no des por hecho que restaura soportes de BACKUP solo porque ambos usen etiquetas de cinta.

Cuando haya terminado el trabajo con la cinta y la unidad esté inactiva, puede eliminarse la asociación lógica:

```
ASOCIAR TAPE1
```

Así se libera la asociación. No es una operación de rebobinado, de expulsión ni de espera a que termine la transferencia. Antes, completa la utilidad y sigue el procedimiento físico de descarga de la unidad. Los procedimientos exactos de recuperación de la controladora y de restauración completa de cintas binarias siguen sin especificarse.

Diagnostica antes de reparar

Síntoma	Pregunta probable que hay que resolver	Siguiente acción
Disco lleno a pesar de haber espacio libre	¿Está disponible el tramo contiguo necesario?	Compara Disponible con Mayor zona
No se encuentra el archivo exacto	¿Son correctas la cuenta, la unidad y la visibilidad en la búsqueda?	Comprueba la identidad, la asociación y el listado con comodines
Directorio lleno a pesar de haber huecos vacíos	¿Se ha llenado un ciclo de sondeo específico de ese nombre?	Deja de añadir nombres en conflicto; investiga el directorio

Síntoma	Pregunta probable que hay que resolver	Siguiente acción
OPEN se queda esperando	¿Tiene otra partición acceso exclusivo?	Pide a quien lo tiene que cierre el canal con CLOSE
READ se queda esperando	¿Hay sectores solapados bloqueados?	Identifica a quien tiene el bloqueo antes de interrumpirlo
La impresora no imprime	¿Es válida la asociación y está preparado el hardware?	Comprueba la configuración del dispositivo y la salida real
Aparece el indicador sin inicio de sesión	¿Son accesibles los componentes de cuentas?	Comprueba SISTEMA.CUENTA y CONECTAR.COMANDO

Ejecuta una comprobación del sistema de archivos

Después de hacer una copia de seguridad utilizable y de organizar el acceso para el mantenimiento, introduce:

```
REPARAR S
```

Así se examina el volumen. Lee todos los diagnósticos aunque el resultado final del comando sea 0. La comprobación puede actualizar un campo de recuento de espacio libre; no se garantiza que deje intactos todos los metadatos.

La opción F permite la reparación:

```
REPARAR S (F)
```

> Precaución — la reparación puede eliminar metadatos del directorio. Usa F solo después de evaluar el informe de comprobación. Puede conciliar el mapa de bits y eliminar entradas rechazadas o alterar las redirecciones de uso compartido. No reconstruye registros de aplicación perdidos de cualquier tipo.

Responde a los errores de disco

La ruta común de disco hace reintentos antes de preguntar. R vuelve a reintentar. M invoca la entrada de recuperación del controlador y reintenta. I ignora el sector fallido. Q anula la operación mediante el tratamiento de interrupciones. Un error ignorado puede parecer un éxito a quien llama aunque falten datos o sean incorrectos, así que I no es un comando de reparación.

Una escritura fallida puede dejar en el directorio un recuento superior a los registros útiles realmente almacenados. Un archivo estructuralmente coherente también puede contener un registro BASIC final incompleto tras una interrupción. Un CLOSE correcto, un recuento de asignación limpio y unos datos de aplicación completos son propiedades distintas.

Archivos borrados y sin terminar

BORRAR libera la asignación; las escrituras posteriores pueden reutilizarla. REVIVIR intenta recuperar la entrada del catálogo, pero su comportamiento completo en caso de colisión y de recuperación no está especificado. No sigas escribiendo con normalidad en un volumen del que necesites datos borrados.

Una salida secuencial sin terminar puede tener una cadena de sectores mientras sus recuentos en el directorio siguen a cero. No inicialices, reorganices ni añadas datos a un archivo así solo para que sus metadatos parezcan normales. Primero establece qué registros están completos. El capítulo 26 describe el orden de escritura y el capítulo 27 describe el tratamiento de los errores de disco.

Capítulo 14 — Mantenimiento del arranque y fin del trabajo

El arranque reconstruye parte del estado a partir de archivos del sistema y crea otra parte en memoria. Después, la conexión de la cuenta añade la identidad y la ejecución opcional de un archivo de comandos. Comprender estas etapas ayuda a distinguir un componente que falta de un menú normal de una aplicación.

Responsabilidades del arranque

El cargador de arranque carga SISTEMA.NUCLEO. El ejecutivo inicializa el hardware y la memoria, carga SISTEMA.STARTUP y vincula CSI, EXECLANG, el almacenamiento de trabajo de EXEC y MENSERR. SISTEMA.STARTUP es código ejecutable, no un procedimiento de comandos en texto.

CSI inicia el proceso de conexión cuando SISTEMA.CUENTA y CONECTAR.COMANDO son accesibles. Tras el inicio de sesión, CONECTAR busca un archivo EXEC con el nombre de la cuenta en S y después en las unidades A a G, teniendo en cuenta los candidatos privados y públicos de cada unidad. Por tanto, un perfil público en S puede seleccionarse antes que un perfil privado en A. Para el procedimiento automático debe haber capacidad de trabajo de EXEC disponible.

El IPL.EXEC de arranque en frío es un recurso alternativo independiente cuando los componentes de inicio de sesión no están disponibles. Un archivo de comandos no se ejecuta en cada arranque solo porque su nombre suene a procedimiento de inicio. El capítulo 21 detalla el orden.

Examina antes de cambiar la configuración

Introduce:

```
ESTADO USUARIOS
```

Así se muestran los usuarios conectados. Pídeles que terminen antes de cambiar recursos compartidos. Examina las asociaciones por separado:

```
ASOCIAR
```

Así sabrás qué nombres lógicos seleccionan actualmente cada dispositivo. Los cambios en las clases de terminal, los controladores y las particiones pueden afectar a la memoria además de al comportamiento del hardware.

GENSIST guarda en SISTEMA.NUCLEO determinados ajustes actuales:

```
GENSIST
```

> Precaución — modifica el archivo del núcleo. Úsalo solo cuando la configuración actual sea intencionada y exista una copia de seguridad utilizable. Las reglas completas de persistencia para cada combinación de ASOCIAR y PONER no están especificadas.

Una operación de estado puede actualizar contadores, EXEC puede escribir un marco de trabajo y BUZON elimina los mensajes recuperados. Planifica el mantenimiento según estos efectos reales, en lugar de tratar como de solo lectura todos los comandos orientados a mostrar información.

Termina el trabajo en orden

Pide a cada usuario que guarde desde la aplicación, cierre sus archivos y vuelva a CSI. Después, cada usuario introduce:

```
FIN
```

Un administrador autorizado puede detener con PARAR una partición sin sesión iniciada. Se trata de gestión de sesiones y particiones; no establece ninguna operación de sincronización global.

Aquí no se especifica ningún procedimiento completo de apagado de toda la máquina ni ningún comando universal de apagado. Esc seguida de I solicita un reinicio y puede descartar la salida pendiente. Rechazar ese reinicio también finaliza el comando interrumpido. Ninguna de las dos cosas sustituye al guardado y cierre desde la aplicación. No prometas a los usuarios que un reinicio completará sus archivos.

Parte IV — Programación

Capítulo 15 — Tu primer programa en BASIC

BASIC te permite desarrollar y ejecutar un programa de forma interactiva a partir de su código fuente y, después, compilar ese código fuente para ejecutarlo más adelante. En el uso interactivo, introduces líneas numeradas en el indicador de BASIC y pides al intérprete que las ejecute. Así resulta cómodo probar una idea, corregir una línea y volver a ejecutar el programa.

La compilación realiza la preparación antes de la ejecución y guarda el resultado en un archivo BASICCOM. El compilador convierte las expresiones y resuelve las referencias a líneas del código fuente en una forma destinada al entorno de ejecución de BASIC, lo que reduce el trabajo que queda para la ejecución y permite una ejecución más rápida cuando esa preparación importa. No se especifica ninguna mejora de velocidad fija; el beneficio depende del programa y de su entrada/salida. Es una compilación previa a la ejecución, pero no produce un programa Z80 autónomo enlazado estáticamente: el entorno de ejecución de BASIC sigue siendo necesario.

El ejemplo siguiente lleva el mismo saludo por ambas vías. Necesitas permiso para ejecutar BASIC, una asociación S con permiso de escritura y los nombres HOLA.BASICOBJ y HOLA.BASICCOM sin usar.

1. Entra en BASIC

En CSI, introduce:

```
BASIC
```

El indicador de guion pertenece a BASIC. Los comandos que se introducen aquí son comandos de BASIC, no de CSI.

2. Añade una instrucción

Introduce:

```
10 PRINT "HOLA MUNDO"
```

10 es el número de línea. PRINT muestra un valor. Las comillas hacen que HOLA MUNDO sea una cadena literal. Como la línea empieza por un número, BASIC la almacena como código fuente del programa en lugar de ejecutarla inmediatamente.

Introduce la línea final:

```
20 END
```

END finaliza la ejecución del programa. No cierra tu sesión ni sale del editor de BASIC.

3. Examina el código fuente

Introduce:

```
LIST
```

LIST muestra el código fuente almacenado, ordenado por número de línea. Comprueba ambas líneas. Para sustituir una línea, introduce ese número de línea con la instrucción corregida; no añadas una segunda línea con un número distinto salvo que quieras una segunda operación.

4. Ejecútalo de forma interactiva

Introduce:

```
RUN
```

El intérprete ejecuta el código fuente y muestra:

```
HOLA MUNDO
```

El control vuelve a BASIC después de END. La ejecución no ha guardado el programa.

5. Guarda el código fuente

Introduce:

```
SAVE HOLA:S
```

SAVE escribe HOLA.BASICOBJ:S. Aunque el nombre contenga OBJ, se trata de código fuente BASIC tokenizado y guardado, no de un objeto para el enlazador del Z80. Consérvalo para futuras modificaciones.

Sal de BASIC:

```
QUIT
```

QUIT vuelve a CSI. Más adelante puedes volver a entrar en BASIC y usar LOAD HOLA:S para recuperar el código fuente guardado.

6. Compila

En CSI, introduce:

```
BASIC HOLA.BASICOBJ:S
```

Con un nombre de archivo explícito de código fuente guardado, BASIC compila a HOLA.BASICCOM:S. Lee los diagnósticos del compilador y exige cero errores de compilación antes de continuar. Usa el tipo explícito BASICOBJ; no des por hecho que BASIC y RUN tienen los mismos tipos predeterminados.

7. Ejecuta el programa compilado

En CSI, introduce:

```
RUN HOLA.BASICCOM:S
```

RUN carga la representación compilada y la ejecuta con el entorno de ejecución de BASIC. Muestra el mismo saludo. BASICCOM es código para el entorno de ejecución, no un ejecutable Z80 nativo autónomo.

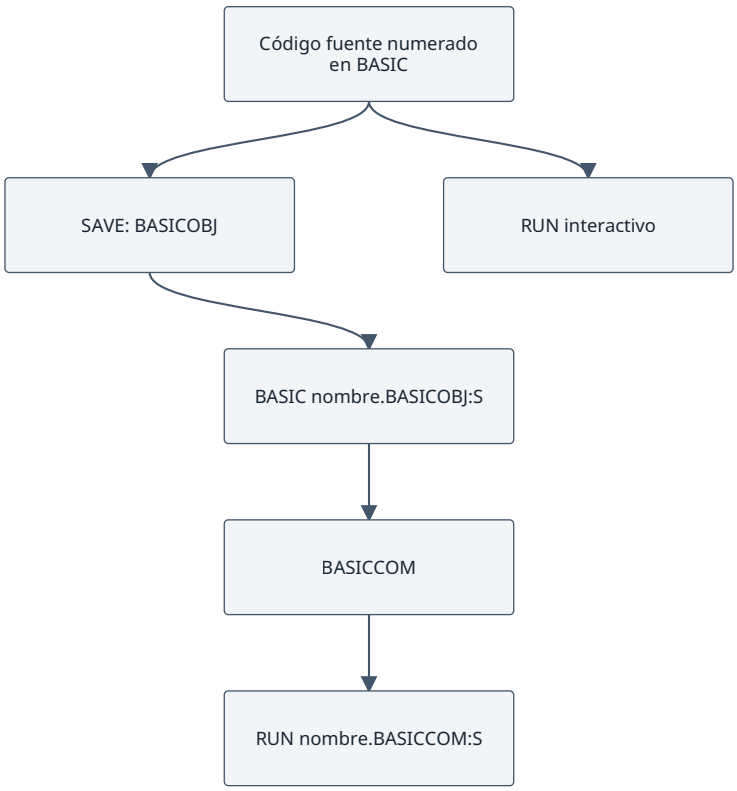


Figura 15.1. Las vías de desarrollo interpretada y compilada.

Modifica y vuelve a generar

Vuelve a entrar en BASIC. En su indicador, introduce:

```
LOAD HOLA:S
```

LOAD recupera el código fuente guardado. Cambia la línea 10 volviéndola a introducir con el texto nuevo, usa LIST para examinarla, RUN para probarla y SAVE para conservarla. Después de QUIT, vuelve a compilar para actualizar el BASICCOM. Editar el código fuente no actualiza automáticamente un archivo compilado anterior.

Opciones del compilador

Para mostrar un listado de compilación, usa:

```
BASIC HOLA.BASICOBJ:S (LIST)
```

LIST solicita un listado en la consola. XREF solicita referencias cruzadas; NOLIST suprime el listado; IMPRn lo envía a la impresora n; OBJ=unidad selecciona dónde se coloca la salida.

NOOBJ suprime la generación del BASICCOM. Por sí sola no solicita un listado del código fuente, y pueden seguir apareciendo las estadísticas del compilador. Para listar el código fuente sin producir salida compilada, introduce:

```
BASIC HOLA.BASICOBJ:S (LIST NOOBJ)
```

LIST controla la presentación del código fuente; NOOBJ controla la salida compilada. Puede quedar un BASICCOM anterior de una compilación previa. Ejecutar ese archivo anterior no demuestra que esta invocación haya generado uno nuevo.

Para mostrar las pasadas de compilación y los números de línea del código fuente, introduce:

```
BASIC HOLA.BASICOBJ:S (TRACE)
```

Así se produce una traza de compilación, que incluye de PASO # 1 a PASO # 8. Es distinta de la traza de ejecución de un programa BASIC. RUN con TRACE no produce ninguna traza visible de direcciones de ejecución para los ejemplos del saludo y numérico descritos en el apéndice D.

La ayuda suministrada anuncia una opción COMMA del compilador, pero esta versión rechaza tanto COMMA como COMA con `Opcion no valida`, CR = 8. En estos ejemplos, usa el punto decimal en el código fuente. La opción COMA de RUN anunciada también deja como 1.5 el valor mostrado por PRINT 1.5 en el programa numérico de demostración; ese ejemplo no establece ningún cambio en la presentación de decimales. PRINT 1,5 pasa dos elementos a PRINT, no un literal con coma decimal. El apéndice D ofrece ejemplos individuales y los límites de cada opción.

Capítulo 16 — Valores, decisiones y bucles en BASIC

Cuando un programa ya sabe imprimir un saludo, el siguiente paso es almacenar valores y decidir qué hacer con ellos. Introduce los ejemplos inmediatos en el indicador de guion de BASIC. Introduce los programas numerados completos línea a línea; guarda el trabajo anterior antes de que NEW borre el editor.

Asigna y muestra un valor

Introduce:

```
A=7
```

La asignación almacena un valor en A. Para verlo, introduce:

```
PRINT A
```

El programa usa el valor almacenado, no la letra A como texto. Un nombre terminado en $ contiene una cadena; un nombre terminado en % designa un entero. Las variables numéricas normales usan la representación real.

Introduce:

```
PRINT 7/2
```

Con operandos enteros se obtiene 3: la división trunca hacia cero. Ahora introduce:

```
PRINT 7.0/2
```

El operando decimal selecciona la aritmética real y da 3.5. Los valores reales usan decimal empaquetado con trece dígitos significativos, en lugar de coma flotante binaria IEEE.

Almacena y une texto

Una cadena es una secuencia de caracteres. Las comillas encierran texto literal, mientras que una variable terminada en $ contiene un valor de cadena. En el indicador de BASIC, introduce:

```
S$="UN"
```

Así se almacena UN en S$. Muestra su valor:

```
PRINT S$
```

PRINT muestra los caracteres almacenados. Para unir ese valor a otra cadena, introduce:

```
PRINT S$&"DOS"
```

El signo & une las dos cadenas y produce UNDOS sin insertar ningún separador. Más adelante en este capítulo, la misma operación coloca un separador especial entre campos.

Toma una decisión

Guarda el código fuente que quieras conservar y, después, borra el editor:

```
NEW
```

Así se empieza un programa nuevo. Introduce las líneas numeradas siguientes de una en una:

```
10 A=7
20 IF A=7 THEN PRINT "SIETE" ELSE PRINT "OTRO"
30 END
```

La línea 10 asigna A. IF lo compara con 7. THEN introduce la acción para el caso verdadero; ELSE introduce la alternativa. END termina. Cambia la línea 10 por otro valor y vuelve a ejecutar RUN para ver la otra vía.

Cuenta con FOR y NEXT

Guarda el ejemplo anterior si es necesario y, después, empieza un programa nuevo:

```
NEW
```

Introduce este bucle línea numerada a línea numerada:

```
10 FOR I%=1 TO 3
20 PRINT I%
30 NEXT I%
40 END
```

FOR establece el valor inicial y el límite. PRINT usa el contador actual. NEXT lo hace avanzar y repite mientras se siga dentro del intervalo. Los valores de salida son 1, 2 y 3; después de este bucle, I% vale 4. STEP puede indicar un incremento, incluido -1 para un bucle descendente. Un intervalo que se omite asigna igualmente su valor inicial.

Reutiliza una subrutina

Una subrutina realiza una tarea y vuelve. Guarda el ejemplo anterior, usa NEW para empezar un programa nuevo e introduce:

```
10 GOSUB 100
20 PRINT "BACK"
30 END
100 PRINT "SUB"
110 RETURN
```

GOSUB recuerda dónde debe reanudarse la ejecución y transfiere el control a la línea 100. RETURN reanuda en la línea 20. END impide que la ejecución vuelva a entrar en la subrutina por segunda vez. GOTO es distinto: transfiere el control sin crear ese punto de retorno.

ON calculado elige entre una lista de destinos numerada desde uno. Por ejemplo, `ON 2 GOTO 100,200,300` selecciona la línea 200 en un programa que contenga esos destinos. Un selector cero produce ON Error Rango. Una entrada intermedia omitida, como en `ON 2 GOTO 100,,300`, continúa después de la instrucción.

Lee datos incrustados

Empieza un programa nuevo después de guardar el ejemplo anterior. Introduce:

```
10 DATA 11,2.5,"ABC",-4
20 READ A,B,S$,I%
30 PRINT A,B,S$,I%
40 RESTORE
50 READ A
60 END
```

DATA proporciona literales en secuencia. READ los asigna a variables. RESTORE devuelve el cursor de datos al primer elemento de DATA; no deshace otras asignaciones. Leer más allá de los datos disponibles provoca No hay DATA.

Almacena una matriz

Una matriz contiene varios valores bajo un mismo nombre; un índice selecciona cada elemento. Antes de declarar una matriz, selecciona su índice mínimo permitido:

```
OPTION BASE 1
```

Después, decláralo:

```
DIM ARR(2)
```

Con BASE 1, los índices permitidos son 1 y 2. Almacena un elemento:

```
ARR(1)=7
```

Léelo:

```
PRINT ARR(1)
```

Con BASE 0, en cambio, DIM ARR(2) permite 0, 1 y 2. Establece la base antes de la declaración. Las reglas para cambiar matrices existentes y para trasladar este ajuste a otro programa mediante CHAIN no están especificadas por completo; quedan fuera de este ejemplo básico.

Extrae un campo de una cadena

Una cadena puede contener varios campos separados por un carácter especial. Por ejemplo, dos campos pueden contener UN y DOS. Un delimitador marca el límite; forma parte de la cadena almacenada, no es una instrucción para imprimir un signo de puntuación.

CHR$ construye una cadena de un carácter a partir de su código numérico de carácter. El delimitador de campos de este sistema tiene el código decimal 222, o hexadecimal 0xDE. La expresión siguiente une UN, ese delimitador y DOS, y después pide a EXT$ el segundo campo. Introduce:

```
PRINT EXT$("UN"&CHR$(222)&"DOS",2,0)
```

CHR$(222) aporta el separador y los signos & unen las partes. EXT$ toma la cadena completa, el número de campo 2 y el número de subcampo 0. Un cero en esta última posición solicita el campo completo, así que el resultado es DOS. Un acento circunflejo (^) literal no es el delimitador. Un campo también puede dividirse en subcampos mediante el código 0xDD; este ejemplo no necesita esa división adicional.

El capítulo siguiente aplica estos valores y cadenas a los archivos. BASIC también puede llamar a módulos de extensión suministrados por separado, pero eso introduce otra interfaz. El capítulo 19 explica qué son esos módulos antes de mostrar OPTION USR, y describe por separado las restricciones de privilegio y de número de serie de los programas compilados. El apéndice D ofrece la referencia completa de instrucciones y funciones.

Capítulo 17 — Programación con archivos

Un programa abre un archivo, transfiere valores y, después, lo cierra. Este capítulo sigue primero ese ciclo con un archivo secuencial, en el que los valores se leen en el orden en que se escribieron. Después presenta el acceso directo por número de registro, el acceso indexado por clave y la coordinación entre programas que usan los mismos archivos.

Usa una asociación S con permiso de escritura y los nombres de práctica LINEAS.DATOS, DIRECTO.DATOS e INDICE.DATOS, que no deben estar en uso. Necesitas permiso para usar BASIC y CREAR. Guarda el código fuente BASIC que quieras conservar antes de salir del intérprete. Las instrucciones de archivo siguientes se introducen en modo inmediato en el indicador de guion de BASIC; los comandos de CSI se identifican por separado.

Canales, registros y modos de apertura

OPEN conecta un archivo a un canal numerado. Las instrucciones READ, WRITE y CLOSE posteriores usan ese número en lugar de repetir el nombre de archivo. Un canal de BASIC no es una letra de disco ni un número de dispositivo físico. Estos ejemplos usan el canal 1 y lo cierran antes de reutilizarlo para otro archivo.

Un registro es un grupo de valores almacenados que un programa transfiere de una vez. WRITE de BASIC serializa los valores junto con la información necesaria para volver a leerlos. Por tanto, un archivo escrito por BASIC no es necesariamente texto normal apto para EDITAR, aunque el valor escrito sea una cadena.

El modo de OPEN indica qué acceso se solicita. Su palabra de organización indica cómo se seleccionan los registros. Los ejemplos siguientes usan estas combinaciones:

Forma de apertura	Finalidad en este capítulo
OUTPUT SEQUENTIAL	Crear el nuevo archivo secuencial de práctica y escribir valores en orden
INPUT SEQUENTIAL	Leer en orden los valores secuenciales existentes
OUTPUT SEQUENTIAL, EXTEND	Añadir salida al final de un archivo secuencial cerrado correctamente
UPDATE SEQUENTIAL	Sustituir el contenido secuencial existente; es destructivo
UPDATE DIRECT	Leer y escribir por número registros existentes de longitud fija
UPDATE INDEXED	Leer y escribir por clave registros indexados existentes

La palabra UPDATE no garantiza el mismo efecto para todas las organizaciones. En concreto, la sustitución secuencial debe mantenerse separada de las actualizaciones directas o indexadas.

Crea y escribe un archivo secuencial

Si ya estás en el indicador de guion de BASIC, continúa con el OPEN que aparece a continuación. Si no, entra en el intérprete desde CSI:

```
BASIC
```

En su indicador de guion, abre el nombre de archivo de práctica que no está en uso:

```
OPEN #1: "LINEAS.DATOS:S", OUTPUT SEQUENTIAL
```

OPEN conecta el archivo al canal 1. La almohadilla introduce el número de canal, los dos puntos lo separan de la especificación de archivo y la coma introduce el modo de apertura. Este ejemplo requiere un nombre que no esté en uso; no es una forma de conservar el contenido de un archivo existente.

Escribe un valor de cadena:

```
WRITE #1: "PRIMERO"
```

WRITE envía PRIMERO por el canal 1. Termina la salida:

```
CLOSE #1
```

CLOSE completa el tratamiento necesario de la salida y libera el canal. No sustituyas CLOSE por QUIT. Sigues en el indicador de BASIC, listo para volver a abrir el archivo para leerlo.

Lee el valor guardado

Abre el archivo existente para entrada secuencial:

```
OPEN #1: "LINEAS.DATOS:S", INPUT SEQUENTIAL
```

Lee su siguiente valor en la variable de cadena A$:

```
READ #1: A$
```

El canal selecciona el archivo; A$ recibe la cadena almacenada. Muestra ese valor:

```
PRINT A$
```

El resultado es PRIMERO. READ cambia la posición en el archivo, de modo que otro READ pide el valor siguiente en lugar de volver a leer el primero.

Reconoce el final de un archivo

Este archivo de práctica solo contiene el valor que acabas de leer. Pide otro:

```
READ #1: A$
```

No hay valor siguiente. Examina la condición de final del canal:

```
PRINT EOF(1)
```

EOF es la consulta de fin de archivo; su argumento es el número de canal. Aquí el resultado es -1, que indica la condición de final. Comprueba esta condición antes de tratar una variable como datos recién leídos. Un registro que falta puede activar EOF sin que se imprima un diagnóstico aparte.

Cierra el canal de entrada:

```
CLOSE #1
```

Añade datos a un archivo cerrado correctamente

La salida anterior se cerró con normalidad. Vuelve a abrirla para añadir datos:

```
OPEN #1: "LINEAS.DATOS:S", OUTPUT SEQUENTIAL, EXTEND
```

EXTEND es una opción final. Coloca la salida al final de un archivo secuencial válido y cerrado. Añade el valor siguiente:

```
WRITE #1: "FINAL"
```

Completa la salida:

```
CLOSE #1
```

En este flujo de adición, no hagas READ antes de WRITE: el cursor compartido puede avanzar más allá de una marca de EOF. No amplíes un archivo sin terminar cuya dirección de sector final sea cero. La ampliación confía en ese campo y puede dañar los metadatos del volumen si no es válido. Un archivo vacío cerrado correctamente tiene una dirección de sector final distinta de cero; no es lo mismo que una salida sin terminar. El capítulo 26 explica los detalles del almacenamiento.

Comprueba el contenido añadido

Ábrelo de nuevo para entrada:

```
OPEN #1: "LINEAS.DATOS:S", INPUT SEQUENTIAL
```

Lee el primer valor:

```
READ #1: A$
```

Muéstralo:

```
PRINT A$
```

Sigue siendo PRIMERO. Lee el valor siguiente:

```
READ #1: A$
```

Muestra ese valor por separado:

```
PRINT A$
```

Esta vez el resultado es FINAL. La adición ha conservado el valor original y ha añadido otro detrás. Cierra la entrada:

```
CLOSE #1
```

Sustituye deliberadamente el contenido secuencial

La sustitución es una operación distinta de la adición. El siguiente ejercicio opcional vacía el archivo de práctica que acabas de leer.

> Precaución — UPDATE SEQUENTIAL sustituye el contenido existente. OPEN puede borrar y volver a crear el archivo antiguo antes de que se ejecute WRITE. Úsalo solo con el archivo prescindible LINEAS.DATOS de este ejemplo.

Ábrelo en modo de sustitución:

```
OPEN #1: "LINEAS.DATOS:S", UPDATE SEQUENTIAL
```

Ciérralo sin escribir:

```
CLOSE #1
```

Ahora el archivo está lógicamente vacío; PRIMERO y FINAL no se han conservado. Si lo vuelves a abrir con INPUT SEQUENTIAL, intentas un READ y compruebas EOF(1), se alcanza inmediatamente la condición de final. La sustitución empieza en OPEN, no en el primer WRITE; por eso es importante seleccionar el modo correcto antes de cualquier transferencia de datos.

Crea un archivo directo

El acceso directo selecciona un registro de longitud fija por su número, en lugar de leer todos los registros anteriores. Primero debe crearse el archivo con el tamaño de registro y la capacidad necesarios. Con todos los canales cerrados, vuelve de BASIC a CSI:

```
QUIT
```

Con el nombre de práctica DIRECTO.DATOS, que no debe estar en uso, introduce:

```
CREAR DIRECTO.DATOS:S (DIRECTO REGISTRO 300 NUMREG 3)
```

CREAR crea tres registros de longitud fija de 300 bytes. DIRECTO selecciona la organización directa, REGISTRO 300 especifica los bytes por registro y NUMREG 3 solicita tres registros. La organización directa permite a un programa seleccionar un registro por su número. Vuelve a entrar en BASIC:

```
BASIC
```

En su indicador de guion, abre el archivo:

```
OPEN #1: "DIRECTO.DATOS:S", UPDATE DIRECT
```

#1 selecciona el canal. UPDATE DIRECT permite leer y escribir los registros existentes del archivo directo. Escribe el registro 2:

```
WRITE #1,2: "SEGUNDO"
```

El número que sigue al canal selecciona el registro; los dos puntos introducen sus datos. Lee el mismo registro:

```
READ #1,2: A$
```

READ coloca la cadena almacenada en A$. Muéstrala:

```
PRINT A$
```

El resultado es SEGUNDO. Termina la operación con el archivo:

```
CLOSE #1
```

CLOSE libera el canal abierto y completa el tratamiento necesario de su salida. No sustituyas CLOSE por QUIT. La numeración de registros empieza en 1; leer más allá del intervalo configurado alcanza la condición de final, disponible mediante EOF(1).

Crea y usa un archivo indexado

Un archivo indexado identifica cada registro mediante una clave, como un código de cuenta. El índice también permite leer los registros ordenados por clave. Esto es distinto de seleccionar un registro directo por su número.

Después de cerrar el archivo directo, vuelve a CSI:

```
QUIT
```

Crea el archivo de práctica INDICE.DATOS, que no debe estar en uso:

```
CREAR INDICE.DATOS:S (INDEXADO REGISTRO 32 NUMREG 16 LONCLA 4)
```

INDEXADO solicita la organización indexada, REGISTRO 32 selecciona la longitud del registro, NUMREG solicita la capacidad y LONCLA 4 selecciona una clave de cuatro caracteres. La capacidad de huecos asignada puede superar el número solicitado; en este caso, una petición de 16 produce 19 huecos.

Entra en el intérprete:

```
BASIC
```

En el indicador de BASIC, abre el archivo:

```
OPEN #1: "INDICE.DATOS:S", UPDATE INDEXED
```

Escribe un registro con clave:

```
WRITE #1,"AAAA": "PRIMERO"
```

La clave identifica el registro. Añade otro por separado:

```
WRITE #1,"MMMM": "MEDIO"
```

Lee por clave:

```
READ #1,"AAAA": A$
```

Muestra el valor recuperado:

```
PRINT A$
```

El resultado es PRIMERO. READNEXT permite recorrer en orden los archivos indexados. Una clave que no existe activa la condición EOF sin imprimir necesariamente un diagnóstico. Cierra el archivo después de usarlo:

```
CLOSE #1
```

La creación con CLAVE y UPDATE KEYED proporcionan acceso por clave sin el área de índice ordenado del formato I. No des por hecho que sus disposiciones en disco son idénticas.

Protección de archivos y modos de BASIC

El administrador puede establecer restricciones independientes de lectura, escritura y borrado, como se explica en el capítulo 11. La tabla siguiente relaciona esas restricciones con los modos de apertura que ya has usado. Describe al propietario actual con cada bit de protección por separado.

Protección individual	INPUT	Sustitución con UPDATE SEQUENTIAL	EXTEND
E, escritura	Permitido	Permitido mediante borrado y nueva creación	Rechazado
B, borrado	Permitido	Rechazado	Permitido
L, lectura	Rechazado	Permitido mediante borrado y nueva creación	Rechazado

La sustitución secuencial puede funcionar a pesar de la protección contra escritura porque su ruta borra y vuelve a crear la entrada. La combinación de atributos y el acceso compartido pueden imponer restricciones adicionales; no deduzcas su comportamiento completo de una fila que aplica una sola protección.

Coordina el acceso compartido

Una apertura exclusiva impide que otro programa que intente abrirlo entre en el mismo archivo hasta que quien lo tiene lo cierre. Los bloqueos de registros, en cambio, coordinan el acceso a partes de un archivo abierto. Para el archivo directo de práctica existente, esta forma solicita la apertura exclusiva:

```
OPEN #1: "DIRECTO.DATOS:S", UPDATE DIRECT, LOCK
```

Una apertura competidora puede esperar hasta que quien tiene el archivo lo cierre. UNLOCK es distinto:

```
UNLOCK #1
```

UNLOCK libera tramos de registros bloqueados, no la posesión exclusiva obtenida con OPEN. Para terminar el acceso exclusivo, introduce:

```
CLOSE #1
```

Los bloqueos de registros abarcan sectores, así que dos registros distintos de 300 bytes pueden solaparse en un sector.

La propiedad, las concesiones de uso compartido, la admisión de aperturas y los bloqueos de registros son reglas distintas. Un propietario puede borrar un archivo aunque otra partición lo tenga abierto en exclusiva. Evita borrar archivos compartidos activos.

Comprueba la condición de finalización adecuada

EOF, el resultado del comando y la finalización de la escritura física pertenecen a capas distintas. Un WRITE de BASIC puede requerir varias transferencias del núcleo, y una interrupción puede dejar incompleto el último registro BASIC. Un CLOSE explícito es esencial, pero los errores de disco ignorados pueden seguir produciendo datos almacenados incompletos. Trata los errores que se notifiquen y valida los registros que requiera tu aplicación.

Capítulo 18 — Escritura de programas en código máquina Z80

El ordenador Secoinsa Serie 20 usa un procesador Z80. Un programa nativo en código máquina consta de bytes de instrucción que ese procesador ejecuta directamente. La notación de ensamblador da a esos bytes nombres legibles, como LD para cargar un registro y JP para un salto. Este capítulo explica un pequeño programa de saludo, sus bytes exactos y el formato que espera el cargador del sistema operativo.

Aquí no se especifica ningún comando nativo de ensamblador o enlazador de uso general. El listado en ensamblador explica el código máquina necesario; no es un procedimiento completo e instalado de edición, ensamblado y enlazado que pueda escribirse en CSI. El capítulo 15 ofrece el flujo completo de desarrollo en BASIC. Aquí, el objetivo es entender cómo escribir las instrucciones nativas y representarlas como un programa cargable.

Registros, etiquetas y peticiones al sistema operativo

Los registros contienen valores que usa el procesador. Este ejemplo usa A para el resultado de un comando y el par de registros DE para la dirección de un texto. Una etiqueta da nombre a una posición del programa para que una instrucción pueda referirse a ella sin escribir su dirección numérica.

Para solicitar un servicio del sistema operativo, el programa ejecuta RST 08h seguido de un byte que selecciona la operación. El ejecutivo lee ese byte y atiende la petición. El servicio 0x02 muestra una cadena terminada en retorno de carro; el servicio 0x00 finaliza el comando con el resultado que hay en A. El capítulo 19 desarrolla esta convención de llamada con más ejemplos.

Escribe el saludo

El programa siguiente imprime «HOLA, Z80» y termina con el resultado de comando 0. No tiene archivos abiertos, así que no demuestra la finalización de una salida de archivo con búfer. Los nombres inicio y mensaje son etiquetas elegidas para este listado.

```
        jp      inicio
inicio: ld      de,mensaje
        rst     08h
        db      02h
        xor     a
        rst     08h
        db      00h
mensaje: db     "HOLA, Z80",13,0
```

Comprende cada instrucción

`JP inicio` transfiere la ejecución a la etiqueta inicio. Proporciona un salto de entrada convencional, aunque el formato de archivo no lo exige.

`LD DE,mensaje` coloca la dirección del texto en el par de registros DE. LD copia un valor; no imprime nada.

`RST 08h` entra en el despachador del ejecutivo. El `DB 02h` siguiente emite el byte selector del servicio. DB es una directiva de datos del ensamblador, no una instrucción del procesador. Juntos, estos bytes solicitan la salida de texto mediante DE.

`XOR A` pone A a cero. El par RST/DB siguiente selecciona el servicio 0x00, que finaliza el comando con ese resultado de finalización cero.

La directiva DB final emite los caracteres literales, un retorno de carro (13) y un byte cero. Una etiqueta identifica la dirección de los bytes siguientes; no reserva ningún byte adicional.

Lee los bytes del código máquina

El programa completo ocupa 22 bytes. Con sus direcciones expresadas inicialmente con respecto a cero, es:

```
C3 03 00 11 0B 00 CF 02 AF CF 00 48 4F 4C 41 2C 20 5A 38 30 0D 00
```

Los tres primeros bytes, C3 03 00, codifican el salto a inicio en el desplazamiento 3. Los tres siguientes, 11 0B 00, cargan DE con la dirección del mensaje, en el desplazamiento 11. Los dos bytes CF codifican RST 08h; cada uno va seguido de su selector de servicio. Los bytes de los caracteres y los terminadores siguen a las instrucciones.

El último servicio finaliza el comando; no vuelve a CSI mediante un RET normal de aplicación. Establece el resultado de forma explícita, en lugar de tratar una instrucción de retorno del procesador como la interfaz de salida del sistema operativo.

Por qué las direcciones necesitan reubicación

El cargador puede colocar el programa en distintas direcciones de memoria. Por tanto, una dirección escrita como desplazamiento desde cero debe ajustarse a la dirección de carga elegida. Este ajuste es la reubicación. Un ejecutable con formato R aporta una lista que indica al cargador qué palabras de dirección debe ajustar; los bytes de datos normales y los números de servicio no deben cambiarse.

En este ejemplo, el operando de JP y el de LD DE contienen direcciones. Sus palabras de dos bytes empiezan en los desplazamientos 0x0001 y 0x0004 del programa. Sumar la base de carga a estas palabras conserva la relación entre las instrucciones y sus destinos, se coloque donde se coloque el programa.

Empaqueta el programa para el cargador

El ejecutable almacena los bytes de su programa en sectores, seguidos de la información de reubicación. Rellena la imagen de 22 bytes hasta completar un sector de 256 bytes. Un segundo sector empieza así:

```
02 01 00 04 00
```

El primer byte indica que hay dos desplazamientos de reubicación. Cada desplazamiento es un valor de 16 bits little-endian (LE16): su byte bajo se almacena primero. Los desplazamientos 0x0001 y 0x0004 identifican las palabras de dirección de JP y LD DE. En cada una, el cargador suma la base de carga seleccionada módulo 65 536. El directorio registra el formato R, dos sectores almacenados, un bloque asignado de 1024 bytes y una longitud de imagen de 0x0016.

Por ejemplo, con una base de carga 0x5157, el prefijo reubicado pasa a ser `C3 5A 51 11 62 51`; el mensaje queda en 0x5162. El programa resultante imprime «HOLA, Z80» con CR 0. La base no es universal. Cada partición reserva antes del programa un área de contexto para su estado y sus áreas de trabajo. La carga normal coloca el código 0x04CC bytes después del inicio de ese contexto. El capítulo 20 explica esta organización de la memoria.

Una imagen R no tiene ninguna cabecera externa obligatoria ni ningún JP como número mágico. Muchas utilidades empiezan con un salto y un texto de versión, pero STARTUP empieza guardando registros. El directorio aporta la longitud y el formato. BASICOBJ no es un objeto para el enlazador y BASICCOM no tiene este formato ejecutable.

Responsabilidades en tiempo de ejecución

Usa el contrato concreto de registros e indicadores de cada servicio. El despachador conserva los registros de entrada mientras selecciona un destino, pero la rutina seleccionada puede modificarlos. No des por supuestos argumentos al estilo de Unix, un marco de pila universal, un éxito uniforme indicado en A ni el cierre automático de canales.

El límite superior de memoria de la partición es la frontera superior disponible para los comandos y sus pilas. El servicio 0x37 consulta esta frontera modificable, que CSI usa para establecer la pila; 0x38 la cambia. Los controladores y las clases pueden reducir ese límite. El valor de SP o de un registro de entrada en una ruta de carga concreta no constituye una ABI completa para todas las aplicaciones. El capítulo 30 sigue la colocación y la terminación conocidas del cargador sin inventar el contrato de registros iniciales, que sigue sin resolverse.

Capítulo 19 — Uso de los servicios del sistema

Un servicio del sistema realiza una operación del sistema operativo para un programa: enviar un carácter a la salida, obtener el privilegio actual, abrir un archivo o esperar un evento. El programa selecciona el servicio y aporta sus entradas; el servicio devuelve el resultado documentado o cambia la ejecución según lo especificado. Este capítulo empieza por la salida visible antes de presentar las peticiones de archivos, el estado de las particiones y las operaciones asíncronas.

Los programas nativos usan la forma de llamada del Z80 presentada en el capítulo 18: RST 08h seguido de un byte que selecciona el servicio. Los registros aportan muchos de los argumentos y reciben los resultados. Las extensiones de BASIC ofrecen otra vía para determinadas operaciones y se presentan por separado más adelante. Ni un fragmento en ensamblador ni una llamada a una extensión son comandos que se escriban en CSI.

Tres llamadas básicas

Los fragmentos siguientes suponen que la ejecución tiene lugar dentro de una partición válida, con su pila y su consola normales. Están en notación de ensamblador; no son comandos de CSI ni programas autónomos completos. El capítulo 18 explica las instrucciones y los bytes completos de un saludo nativo; ten en cuenta ese contexto al leer estos fragmentos.

Para mostrar un carácter, pon su código de carácter en C:

```
ld c,65
rst 08h
db 05h
```

LD prepara la A en ASCII, RST entra en el despachador y DB aporta el selector de la salida de caracteres por consola. Esta vía aplica la política normal de salida, así que una salida suprimida puede seguir sin verse.

Para empezar una línea nueva, usa:

```
rst 08h
db 12h
```

El servicio de nueva línea no necesita ningún búfer de caracteres explícito. Es distinto de imprimir como texto el valor numérico de un byte.

Para obtener el privilegio actual, usa:

```
rst 08h
db 5Ch
```

El valor de A devuelto contiene el byte de privilegio. La llamada no lo imprime ni cambia la identidad. Un programa que quiera mostrarlo en decimal debe dar formato al valor por separado y enviar los caracteres resultantes a la consola.

Interpreta el contrato de llamada de un servicio

Para cada servicio, identifica el número de selector, los registros de entrada, la memoria que debe preparar quien llama, los registros de resultado y la indicación de error. Un puntero es la dirección de unos datos en memoria; devolver un puntero no muestra los datos. Los registros que deban sobrevivir a una llamada necesitan una garantía explícita de conservación.

El Z80 también devuelve indicadores de condición, entre ellos el de acarreo y el de cero (Z). Su significado depende del servicio. Por ejemplo, una lectura de cola puede distinguir mediante el acarreo una cola vacía de un byte cero almacenado, aunque el valor de los datos sea cero en ambos casos. No des por supuesto un código de éxito universal en A ni el mismo significado para todos los indicadores.

El despachador lee el selector en línea y, por lo general, reanuda la ejecución después de ese byte cuando vuelve el servicio. La tabla instalada abarca 0x00–0x6E; no hay establecida ninguna comprobación de intervalo, así que usa solo las entradas documentadas. El capítulo 32 recoge el contrato de cada uno. Sus nombres de servicio son etiquetas descriptivas, mientras que los números y las direcciones de destino identifican las interfaces reales. El capítulo 24 explica el despacho y la distinción entre las implementaciones residentes y las de superposición.

Entrada de consola y salida a dispositivos

Una petición de salida puede atravesar varias capas. La política de la consola decide si la salida normal se suprime o se copia a una impresora. Una clase de terminal traduce las operaciones de cursor a secuencias específicas del terminal. Un controlador se ocupa del propio dispositivo. Elige la capa cuyo comportamiento necesite el programa; una llamada de salida en bruto no realiza automáticamente el trabajo de las capas superiores.

El servicio 0x05 aplica la política de la consola, incluida la supresión con Esc seguida de O y la copia a la impresora. El servicio 0x07 aplica los controles de cursor del núcleo y la traducción de la clase de terminal, pero omite esa supresión. El servicio 0x40 con la asociación de consola 9 llega al controlador en bruto y no actualiza las coordenadas de cursor registradas. Tres llamadas a 0x07 con `0x10`, columna y fila realizan la operación de coordenadas documentada.

Un búfer es memoria proporcionada por quien llama en la que puede almacenarse la entrada. El presupuesto de entrada limita el número de caracteres aceptados, pero también puede hacer falta espacio para los terminadores. La entrada de línea 0x01 toma DE como dirección del búfer y B como presupuesto. Reserva espacio para los terminadores: un presupuesto de cinco bytes agotado almacena cinco caracteres más CR; cuatro caracteres seguidos de un CR tecleado almacenan ese CR y un CR adicional. A indica el número de caracteres aceptados; DE y BC cambian. B=0 y las rutas de edición arbitrarias no están totalmente caracterizadas.

La familia ALP aporta los controladores de terminales serie. DMA es el acceso directo a memoria, que aquí se usa para las transferencias de salida; la disponibilidad del DMA y la existencia de entrada disponible son condiciones distintas. El estado del dispositivo es específico de cada familia. El estado de ALP devuelve A=0 tanto si no hay nada en cola como si lo hay; Z comprueba la ocupación de la entrada y el acarreo comprueba la disponibilidad del DMA. Su llamada de estado puede vaciar la salida pendiente. La entrada 0x3F puede devolver un NUL real con el mismo AF que una asociación no asignada, así que establece la asociación de forma independiente. La espera de eventos 0x6B puede despertar sin ningún carácter; vuelve a comprobar la condición. Vuelve con las interrupciones habilitadas en las rutas especificadas.

Canales de archivo y peticiones de almacenamiento

Las operaciones de canal de BASIC del capítulo 17 tienen servicios equivalentes de nivel inferior. Quien llama desde código nativo debe preparar la identidad del canal y proporcionar una petición que describa la operación y su área de trabajo. Un área de trabajo es memoria que se usa durante la operación; su puntero es la dirección de esa memoria. Estos objetos deben existir antes de llamar a OPEN.

Inicializa un nombre o clase de canal mediante 0x18 y, después, pasa una petición de cuatro bytes con permiso de escritura a OPEN 0x20. Contiene el número de canal, el byte del modo de apertura y un puntero de dos bytes little-endian al área de trabajo. Los modos de OPEN y el estado del canal son campos distintos; no debe sustituirse uno por otro. OPEN puede modificar bits del modo. Los servicios 0x21–0x23, 0x28–0x2D y 0x5E/0x5F proporcionan operaciones de cierre, de registros y de bytes contados. El apéndice E recoge las colas de canal específicas de cada organización; no interpretes la cola de un archivo indexado como un cursor secuencial.

La búsqueda en el directorio 0x14 normaliza en cada reintento una petición de unidad, nombre y tipo con permiso de escritura. A=0 con los indicadores establecidos significa que se ha encontrado; A=1 puede significar que no existe y que hay un hueco reutilizable. Un fallo con el acarreo activado puede dejar en HL un valor de dispersión o la dirección del último sondeo, no un puntero a una entrada. Clasifica el retorno antes de desreferenciarlo.

La asignación reserva bloques de disco para un archivo. Una asignación contigua es un único tramo de bloques sin interrupciones, en lugar del mismo total repartido entre varios huecos. El asignador distingue la asignación contigua exacta 0x1B de la asignación de tamaño mínimo 0x63. La liberación 0x1C cambia bits y contadores sin ninguna garantía demostrada frente a la doble liberación ni de comprobación de límites. El acceso directo a sectores omite la validación de registros en el nivel de aplicación y mantiene la política común de reintentar o ignorar.

Obtén el contexto de la partición actual

El ejecutivo mantiene un contexto para cada partición: una estructura de memoria que contiene su identidad, sus límites de memoria, sus canales y su estado de ejecución. El código que necesita esa estructura obtiene primero su dirección. Las llamadas normales de salida de caracteres no requieren que quien llama la descodifique.

El servicio 0x30 devuelve la dirección del contexto actual en IY, un registro índice del Z80:

```
rst 08h
db 30h
```

IY proporciona ahora una base para los campos documentados del contexto; la llamada por sí misma no imprime nada. Los desplazamientos de los campos se miden desde esa base. El capítulo 20 explica la organización de la memoria y el apéndice E define los campos. No des por supuesto que cada uso de IY en cada componente designa esta estructura.

Llama desde BASIC a una extensión relacionada con el sistema

Una extensión de BASIC es un módulo de código máquina suministrado por separado al que el entorno de ejecución de BASIC llama en nombre de un programa. Seleccionar un módulo y llamarlo son pasos distintos. El módulo tiene su propia convención de argumentos y resultados; no se selecciona mediante un número de servicio nativo del sistema.

Para este ejemplo, entra en BASIC desde CSI si no está ya activo:

```
BASIC
```

En el indicador de guion de BASIC, selecciona la función que obtiene el privilegio:

```
OPTION USR "GETPRIV"
```

Así se selecciona GETPRIV.BASICUSR. Llama a su entrada en el desplazamiento 0 con el argumento numérico 0:

```
PRINT USR(0,0)
```

El primer cero selecciona la entrada situada al principio de este módulo; el segundo es el argumento numérico que requiere esta función. El resultado es el privilegio actual, que PRINT muestra. Otros módulos requieren sus propias interfaces; estos argumentos no son una plantilla general para cualquier extensión.

De forma similar, GETID devuelve el número de cuenta. GETUSER usa un área de trabajo de cadena de ocho bytes para devolver el nombre de la cuenta; su ejemplo completo está en el apéndice C. Los módulos de extensión vuelven a través del envoltorio de BASIC en lugar de finalizar el comando de CSI. Esto es distinto del saludo nativo del capítulo 18, cuyo servicio 0x00 finaliza el comando.

Establece los requisitos de un programa BASIC compilado

Una aplicación puede exigir un privilegio mínimo o que coincida el número de serie de la instalación antes de continuar. Son restricciones a la ejecución, no formas de conceder autoridad a quien la llama.

OPTION PRIV establece el privilegio mínimo en tiempo de ejecución de un programa BASIC compilado. OPTION SERIAL impone una vinculación a un número de serie. El valor exacto del número de serie es específico de cada sistema y no debe copiarse de una configuración ajena. Sus formas concretas se dan en OPTION PRIV y OPTION SERIAL. Las comprobaciones corresponden al entorno de ejecución de BASIC, no al selector de servicios nativos de los fragmentos en ensamblador anteriores.

Programa una devolución de llamada

Una devolución de llamada (callback) es una rutina en la que el sistema entra cuando se produce un evento o transcurre un retardo solicitado. Su código puede ejecutarse más tarde que la petición que la registró. Por tanto, quien llama debe mantener válidos el código de la devolución de llamada y el almacenamiento que necesita, y debe entender qué partición aporta el contexto de ejecución.

El servicio de temporizador 0x35 usa seis bytes con permiso de escritura seguidos del código de la devolución de llamada, convierte el retardo indicado en pulsos de ocho milisegundos e invoca la devolución de llamada en la partición a la que pertenece. Está documentado el funcionamiento normal con A=0; las excepciones de propiedad con A distinto de cero no establecen una cancelación general entre bancos. El capítulo 25 explica la disposición de los nodos, la temporización y las rutas de entrada.

Coordina el acceso con bloqueos

Un bloqueo registra la propiedad para que otra partición pueda esperar en lugar de usar a la vez el recurso protegido. Un aviso de despertar indica a quien espera que vuelva a comprobarlo; no le transfiere automáticamente la propiedad.

Los bloqueos de byte 0x54/0x55 usan una marca de propietario de un byte. La liberación avisa a todos los que esperan, que deben volver a comprobar su propio bloqueo. Las adquisiciones repetidas por el mismo propietario no añaden ningún recuento de recursión. La admisión de aperturas de archivo y los tramos de registros son estructuras distintas con reglas de limpieza diferentes; el capítulo 17 explica la diferencia visible para el programador.

Intercambia bytes mediante una cola en memoria

Una cola almacena bytes hasta que un consumidor los retira. Los servicios de cola usan una cabecera y un área de datos proporcionadas por quien llama, en lugar de reservar ellos mismos ese almacenamiento. La capacidad, la ocupación y la posición de lectura son valores distintos de la cabecera.

Los servicios 0x47/0x48 usan este almacenamiento de quien llama. Comprueba el acarreo para detectar lecturas de una cola vacía: una lectura correcta de un dato cero también activa Z. No añadas datos a una cola llena; el bucle original con el estado de llena en caché no puede detectar a un consumidor

normal. Las capacidades de 1 a 128 derivadas del código evitan los defectos de indexación de ocho bits demostrados, pero no establecen una seguridad completa frente a la concurrencia. El capítulo 25 describe el algoritmo y sus límites; el apéndice E recoge la disposición.

Distingue la copia entre bancos de la entrada en una devolución de llamada

La conmutación de bancos selecciona qué memoria es visible para el procesador. Copiar bytes a través del banco de otra partición y ejecutar una rutina en esa partición son operaciones distintas. Estas interfaces avanzadas requieren el modelo de memoria y de planificador de los capítulos 20 y 22; sus contratos en el nivel de registros están en el capítulo 32.

Los servicios 0x68/0x69 copian a través del banco de un registro del planificador, no de un número de banco físico seleccionado por A. Hacen avanzar HL/DE, devuelven BC=0 y deshabilitan las interrupciones en los retornos especificados. Un recuento cero no es una convención para indicar una copia vacía. La devolución de llamada 0x56, en cambio, cambia el contexto de ejecución y puede borrar el estado de detenido o de espera; no equivale a una copia de memoria.

Interpreta los resultados de las funciones auxiliares numéricas

Las funciones auxiliares aritméticas usan registros para recibir los operandos y devolver los resultados. El tratamiento de errores importa tanto como el resultado normal: un resultado con desbordamiento puede ser un valor intermedio y no una respuesta truncada.

En la multiplicación sin signo 0x27 hay que tener en cuenta el acarreo. En caso de desbordamiento, HL puede contener un acumulador parcial temprano, no los dieciséis bits bajos del producto. `0x9000 × 3` devuelve 0x9000 con acarreo, mientras que con los operandos invertidos devuelve 0x3000 con acarreo. La división 0x26 entre cero devuelve HL=0, DE=el dividendo original y acarreo, sin provocar ninguna excepción.

Los analizadores numéricos pueden devolver el acarreo desactivado sin consumir ningún dígito. No omiten los espacios ni aceptan signos. Valida el texto consumido y el que queda detrás, además del acarreo. El desbordamiento consume el dígito que lo provoca y devuelve un resultado parcial intermedio.

Trata una devolución de llamada de error de disco

Una devolución de llamada de error de disco permite a un programa responder durante una operación de disco fallida. Se ejecuta dentro del contexto de bloqueo y de registros de esa operación, así que antes de instalar una hay que entender los significados normales de reintentar, ignorar y anular.

Una devolución de llamada de error de disco registrada mediante 0x4A se ejecuta mientras se mantiene el bloqueo del disco. Devolver A=0 da el fallo por tratado; un valor positivo en A aplica la política normal de error y reintento; un valor negativo en A anula la operación. Conserva los registros de trabajo de la operación. No está establecido que se pueda hacer E/S de disco reentrante dentro de esa devolución de llamada. Eliminar la causa y devolver cero no es un reintento.

El capítulo 27 explica la ruta de reintento y sus consecuencias para los datos almacenados. Usa conjuntamente la entrada exacta del servicio, la disposición de los datos y las salvedades sobre los fallos; los servicios no ofrecen una única convención uniforme de retorno o de limpieza.

Parte V — Fundamentos del sistema

Capítulo 20 — Organización y memoria

El sistema reparte su trabajo entre un ejecutivo residente común, la memoria de las particiones organizada en bancos, superposiciones (overlays) de servicios reemplazables y archivos normales que contienen comandos y controladores. Comprender estas fronteras explica por qué una utilidad puede desaparecer de la memoria tras ejecutarse mientras el estado de sus archivos abiertos sigue siendo visible para el ejecutivo, y por qué una dirección por sí sola no siempre identifica un byte de RAM.

Memoria común y memoria por bancos

El Z80 presenta un espacio de direcciones de 16 bits. Las direcciones 0x0000–0x3FFF son comunes a las particiones. El intervalo 0x4000–0xFFFF selecciona uno de cinco bancos de 48 KiB en una configuración de 256 KiB: 16 + 5 × 48 = 256 KiB. Los bits bajos de selección de banco del puerto 0xFD seleccionan el banco; el bit 7 controla la correspondencia de la ROM en 0x8000. Un emulador debe conservar el almacenamiento común mientras cambia la región superior seleccionada. Un modelo plano de 64 KiB no puede implementar esa configuración.

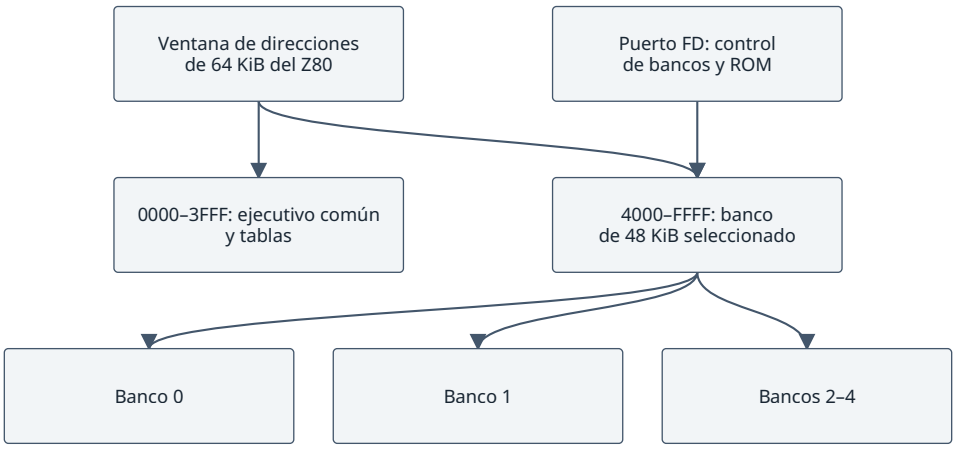


Figura 20.1. La ventana de la CPU y su almacenamiento subyacente. El diagrama describe la correspondencia, no la visibilidad simultánea de todos los bancos.

Componentes residentes y asignación de particiones

El cuerpo de NUCLEO ocupa 0x0000–0x39C1; el SPOOLER residente le sigue en 0x39C2–0x4C8A. Su extensión conjunta es de 19 595 bytes. El contexto de la primera partición empieza en 0x4C8B y su área de programa en 0x5157. El archivo de arranque incluye material adicional de inicialización y de superposiciones; la extensión residente nativa no es la misma medida que sus 113 sectores almacenados.

Un contexto de partición reserva 0x4CC bytes antes del código ejecutable. Sus primeros 0xAC bytes contienen el estado de identidad, de devoluciones de llamada, de consola y de comandos. A continuación vienen diecisiete canales de 32 bytes y, después, un área de trabajo de sector de 256 bytes y un búfer de comandos de 256 bytes. Por tanto, el inicio del ejecutable está en contexto + 0x4CC. Un límite superior de memoria en el contexto acota el extremo superior disponible para los comandos y sus pilas. Los controladores y las clases de terminal pueden cargarse hacia abajo desde ese límite.

Por tanto, las direcciones de carga dependen de la configuración. Sustituir una clase de terminal puede cambiar el límite; se observó que la sustitución repetida C1 → C2 → C1 lo reducía de 0xFB64 a 0xF919 y después a 0xF6CE. Trata una dirección de carga publicada como una dirección válida en una configuración concreta, no como una dirección de enlace universal.

Superposiciones y comandos normales

El despachador de RST 0x08 y las rutinas de uso frecuente permanecen en la zona común. A los servicios de archivo y de carga de uso menos frecuente se llega a través de rutinas puente (stubs) que seleccionan una superposición en 0x424A. La superposición 1 aporta las operaciones indexadas y por clave; la superposición 2, las operaciones de canal y las secuenciales y directas; y la superposición 3, la carga, la terminación, los mensajes y la inicialización. Antes de desensamblar una dirección de superposición hay que saber qué bytes están seleccionados en ella.

CSI, el procesador de comandos, es un ejecutable que se carga en la partición actual. La mayoría de los comandos con nombre son archivos `.COMANDO` independientes. BASIC tiene su propio archivo de carga, sus superposiciones y su entorno de ejecución. EXEC es un intérprete independiente con marcos de nivel persistentes. Son componentes que cooperan, no sintaxis integrada en un único procesador de comandos monolítico.

La pequeña tabla común de componentes con nombre situada en 0x04BC permite búsquedas exactas de ocho bytes. Su entrada con nombre del SPOOLER es `SPOOLER` → 0x39C2. No es un directorio, ni una tabla del planificador, ni una tabla general de símbolos. El capítulo 24 describe su disposición y las limitaciones de la búsqueda.

Consulta el apéndice B para ver los mapas canónicos de direcciones y de E/S, y el apéndice E para ver las disposiciones del contexto.

Capítulo 21 — Arquitectura de arranque

El arranque transforma un flujo de sectores físicos en un entorno multiusuario con memoria por bancos. El cargador de arranque físico y el sistema de archivos lógico usan reglas de direccionamiento distintas. Mantener separadas esas etapas es esencial al reconstruir un disco o al implementar la controladora.

El firmware y el primer cargador

El firmware ocupa 4096 bytes asignados en 0x8000, con un alias en cero durante el reinicio. Sus bytes de salto iniciales son `C3 03 80`, que transfieren el control a 0x8003. El firmware lee el cilindro físico 0, cabeza 0, sectores 0 a 3, y los carga en memoria a partir de 0x0100. Esos cuatro sectores forman un cargador de disco de 1024 bytes.

El cargador se reubica en 0xA000 y entra en 0xA023. Un SP anterior observado es 0x3FFD; después, el cargador reubicado usa SP=0xA600 e IY=0xA700. Estos valores sucesivos pertenecen a etapas distintas y no deben presentarse como especificaciones contradictorias de la pila de reinicio.

Los sectores físicos 4 y 5 contienen descriptores de volumen. El cargador usa la correspondencia del volumen del sistema para obtener el núcleo. El cargador obtiene el inicio del núcleo de su entrada de catálogo. Por ejemplo, un núcleo de 113 sectores ocupa 28 928 bytes (0x7100) y se lee primero en 0x0100. Su ubicación es un valor de asignación, no un sector fijo universal. La reubicación hacia abajo que sigue copia 0x9000 bytes a la dirección cero, incluida memoria situada más allá de los bytes recién leídos. Esa región adicional copiada no debe describirse como contenido inicializado del archivo del núcleo.

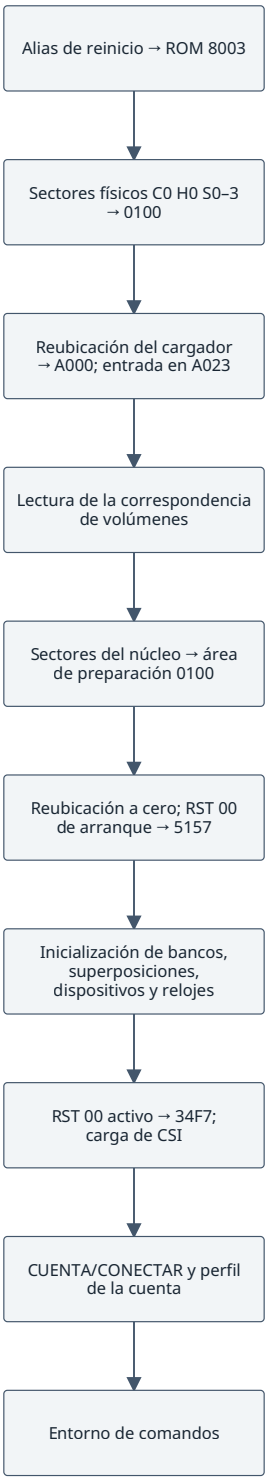


Figura 21.1. La ruta documentada de arranque desde disco. La autenticación se produce después de cargar CSI.

Inicialización del ejecutivo

Antes de la inicialización, el vector RST 0x00 del núcleo recién reubicado salta a 0x5157. La inicialización cambia el vector; en el estado de inicio de sesión observado, salta a 0x34F7. Se observa SP en 0xA5FE antes de que la inicialización establezca su propia pila en 0x5157. La inicialización reubica el material de superposición en el banco 4, instala los punteros comunes de servicio y construye los registros de particiones disponibles. El servicio 0x0C ilustra la distinción entre el estado de arranque y el estado en ejecución: la tabla en disco apunta inicialmente a 0x5157; la tabla activa apunta a 0x331B, una rutina que devuelve HL=0x5542. No se ha establecido el significado de esa firma.

El servicio 0x4B con B=0xFF realiza la inicialización en frío del volumen del sistema. Localiza NUCLEO, CSI, EXECLANG, EXEC1 y MENSERR, guarda en caché sus posiciones y escribe el estado del reloj en el archivo del núcleo. Por tanto, el arranque actualiza el estado del sistema en disco antes de que empiece el trabajo normal de los usuarios.

Arranque de cuentas y perfiles

CSI se carga antes de la autenticación de la cuenta. El arranque usa los componentes de mantenimiento de cuentas y de conexión con un valor de propietario temporal de 0xFF antes de restablecer la cuenta 0 para CONECTAR. Si faltan componentes de cuentas, el entorno puede quedarse en la cuenta 0, con privilegio 5 y un nombre mostrado sin resolver, y después intentar IPL.EXEC o el indicador. Esta ruta de fallo no ofrece un procedimiento fiable de recuperación de cuentas.

Una cuenta conectada puede tener un perfil EXEC con su nombre. La búsqueda recorre S y después las unidades A a G, teniendo en cuenta los candidatos privados y públicos de cada unidad asociada. Por tanto, un perfil público en S puede tener prioridad sobre un perfil privado en A. El primer candidato no secuencial que coincide detiene la búsqueda en lugar de continuar hasta un script válido posterior; los candidatos de capacidad cero se omiten. Un perfil debe tener el formato correcto además de la grafía correcta.

Un fallo explícito de EXEC puede mencionar `EXEC.COMANDO`, aunque intervenga el intérprete independiente EXECLANG. Diagnostica la búsqueda completa y el formato del archivo antes de concluir que ha desaparecido un ejecutable de comando.

Un reinicio es un arranque nuevo

La ruta aceptada de Esc seguida de I llega a 0x2D01, escribe 0x80 en el puerto 0xFD y salta a la ROM en 0x8000. Vuelve a ejecutar los diagnósticos del firmware, el cargador de arranque físico y la inicialización en frío. Rechazarlo con N llama al servicio 0x00 y finaliza el comando interrumpido; no lo reanuda. Ninguna de las dos rutas aporta un CLOSE implícito para la salida con búfer de las aplicaciones. Las consecuencias operativas se describen en el capítulo 14.

Capítulo 22 — Particiones, identidad y protección

La partición es el entorno de ejecución planificado del sistema. Una cuenta aporta la identidad y la autoridad dentro de ese entorno. Estos conceptos se relacionan durante el inicio de sesión, pero no son intercambiables: cambiar de cuenta no asigna un nuevo banco de memoria, y el propietario de un bloqueo no es un número de cuenta.

Tres identificadores

El planificador común usa un índice de registro que empieza en cero. Cada registro de tres bytes contiene un puntero al contexto y un byte de banco y estado. Los comandos de usuario muestran los números de partición con su propia convención, que normalmente empieza en 1. El banco es el nibble bajo del tercer byte del registro. Puede ser distinto del índice del registro: el registro 1 puede seleccionar el banco 2.

El número de cuenta es el byte +0x1A del contexto. El nombre de la cuenta ocupa +0x12–+0x19, y el privilegio está en +0x11. Las estructuras de bloqueo almacenan en cambio el índice del planificador + 1, para que el cero pueda significar «sin propietario». Un ingeniero que descodifique el byte de propietario 2 como la cuenta 2 atribuirá mal el estado de sincronización.

Planificación y espera

Las particiones se reparten el procesador por apropiación mediante el temporizador; un cálculo que no contenga llamadas al sistema operativo no monopoliza el procesador. Los programas también pueden ceder el control mediante el servicio 0x4F. La selección normal recorre circularmente los registros en busca del primero configurado cuyo nibble alto de estado sea cero. En esa ruta no hay especificada ninguna comparación de prioridades por registro.

Bit de estado	Significado en las rutas establecidas
4	Esperando un bloqueo de byte
5	Esperando un evento de dispositivo
6	Estado de espera relacionado con el disco
7	Partición detenida

Son bits, no un conjunto de números de estado mutuamente excluyentes. Las devoluciones de llamada de recepción serie y la entrada de devoluciones de llamada entre particiones pueden borrar todo el nibble alto. Así, una devolución de llamada dirigida a una partición detenida y asignada puede reanudar su bucle inactivo de cesión del control sin restaurar su consola. El capítulo 25 detalla la selección, los marcos guardados y el comportamiento del temporizador.

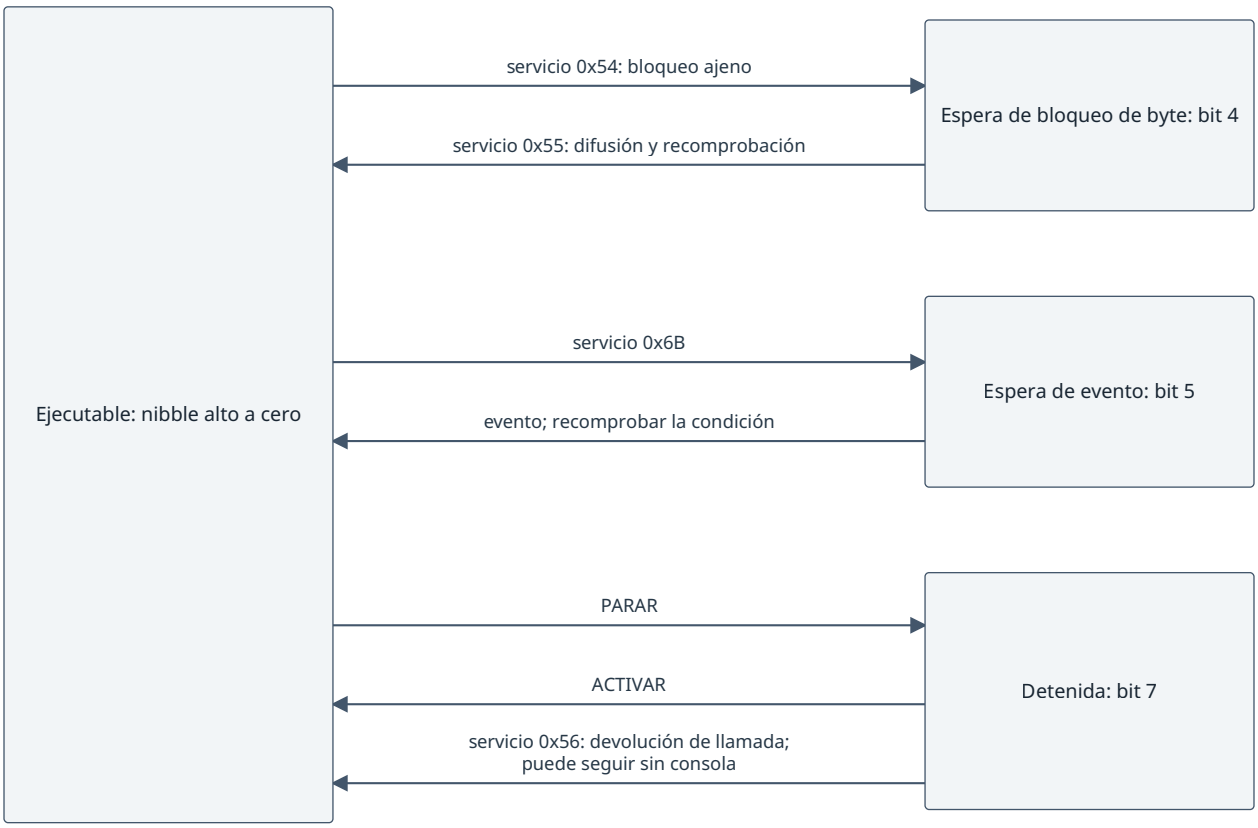


Figura 22.1. Algunas transiciones documentadas. El diagrama no prescribe una máquina de estados universal para todas las combinaciones de bits.

Fronteras de protección

El sistema combina la propiedad por cuenta, la búsqueda de archivos públicos, los alias de uso compartido, los bits de protección de archivos, los privilegios mínimos de los comandos y las vinculaciones por software a números de serie. Cada uno protege operaciones distintas. El prólogo estándar de los comandos comprueba el privilegio y el número de serie de la instalación antes del cuerpo de la utilidad; la utilidad puede imponer comprobaciones adicionales. BASIC implementa sus propias protecciones equivalentes en tiempo de ejecución.

La protección de archivos usa bits de lectura, borrado y escritura en el byte de formato del directorio. La operación de sustitución secuencial observada borra primero el archivo antiguo. Por tanto, la protección contra escritura por sí sola puede permitir la sustitución aunque impida añadir datos. La protección contra borrado por sí sola impide la sustitución, pero permite añadir datos. La protección contra lectura impide INPUT y la ruta de adición establecida; no impide la sustitución por el mismo propietario. El capítulo 11 explica las operaciones administrativas; el capítulo 17 ofrece la matriz de protección de los modos de BASIC y sus límites con atributos combinados.

Una apertura exclusiva controla la admisión en una tabla de archivos abiertos. No impide de forma fiable que BORRAR nativo borre el archivo: la ruta de borrado llama a la función auxiliar de admisión exclusiva, pero no comprueba su acarreo antes de continuar. El canal existente puede sobrevivir mientras sus bloques ya se han liberado. Este defecto hace que una descripción exacta del comportamiento sea especialmente importante para las implementaciones compatibles.

No se ha establecido ninguna frontera de protección entre núcleo y usuario impuesta por el hardware. Los programas contienen instrucciones Z80 normales y pueden llamar a rutinas comunes; varios servicios de bajo nivel no validan índices ni punteros. El diseño histórico se basa en gran medida en programas de confianza y en la disciplina en el nivel de comandos. Esta observación describe la arquitectura especificada, no una certificación de seguridad moderna.

Capítulo 23 — Conceptos de almacenamiento y dispositivos

Los archivos son entradas de catálogo que se interpretan según un byte de formato. El catálogo identifica la propiedad y la asignación, mientras que el formato de registro de una aplicación da sentido a los bytes contenidos en esa asignación. Estas capas explican por qué un archivo estructuralmente legible puede contener registros BASIC no válidos, o por qué una comprobación correcta del sistema de archivos puede dejar inutilizable una aplicación.

Asignación, sectores y registros

Un sector lógico tiene 256 bytes. El mapa de bits de asignación gestiona bloques de cuatro sectores, es decir, de 1024 bytes. Por tanto, un archivo puede poseer más almacenamiento del que consumen sus datos actuales. Los archivos secuenciales reservan los dos últimos bytes de cada sector para un enlace, lo que deja 254 bytes de datos. Los archivos directos, en cambio, exponen registros numerados de longitud fija sobre un almacenamiento de sectores completos; un registro de 300 bytes cruza necesariamente los límites de sector.

Los formatos ejecutables R y A usan almacenamiento contiguo. Un archivo R lleva registros de reubicación después de su imagen; un archivo A lleva una dirección de carga absoluta en su entrada de directorio. Los archivos I combinan un índice de huecos ordenado con huecos de clave, datos y enlace siguiente. Los archivos K usan huecos fijos por dispersión, sin ese prefijo. Ninguna de esas diferencias se codifica únicamente mediante el tipo del nombre de archivo.

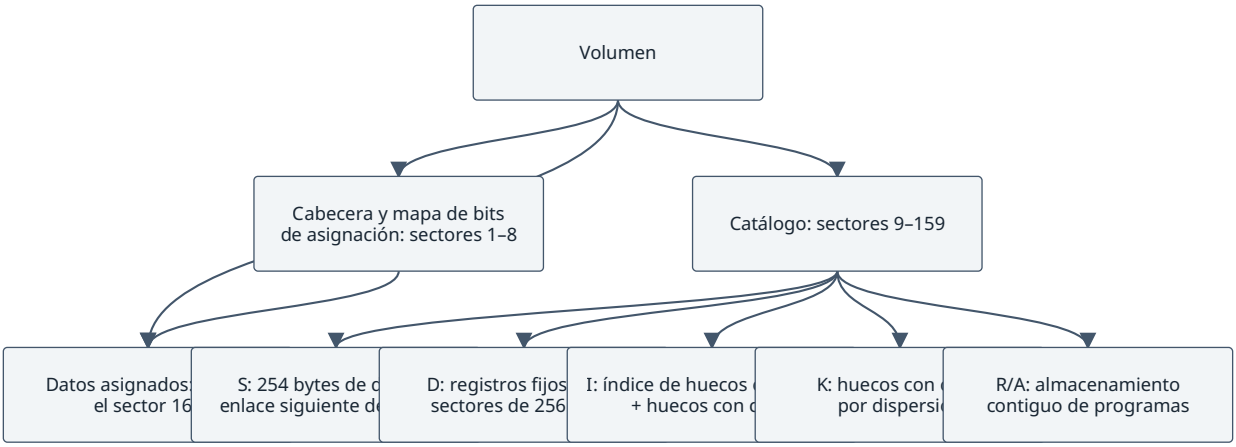


Figura 23.1. Organización del sistema de archivos de un volumen con 151 sectores de directorio que empiezan en el sector 9. Son valores de configuración, no valores predeterminados universales.

En la geometría del ejemplo, el directorio contiene 151 sectores × 8 entradas = 1208 registros. Su búsqueda por nombre exacto es un algoritmo de sondeo, no un recorrido de todos los registros. Un defecto puede agotar una ruta de sondeo corta y repetitiva mientras muchas entradas del catálogo siguen sin usarse. El listado con comodines recorre todos los sectores del directorio y puede mostrar entradas que la búsqueda exacta no encuentra. El capítulo 26 distingue la presencia física, la accesibilidad desde el catálogo y la legibilidad para la aplicación.

Dispositivos lógicos y físicos

Cada partición tiene un mapa de asociaciones de 24 bytes. Una asociación designa un uso, como S, la entrada de consola o IMPR1; su byte selecciona un descriptor de dispositivo físico. A su vez, un descriptor identifica un controlador y unos parámetros específicos del dispositivo. Estos tres niveles permiten que dos particiones usen el mismo nombre lógico para hardware distinto sin cambiar el código de la aplicación.

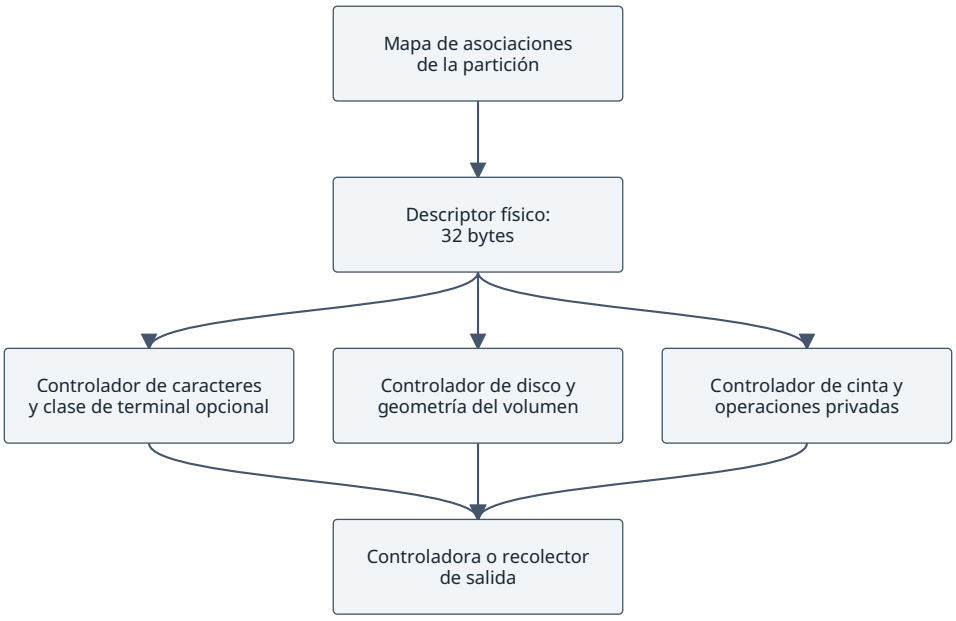


Figura 23.2. Resolución de dispositivos. Los campos del descriptor forman una unión que depende de la familia; los desplazamientos de entrada de caracteres y de disco no implican funciones idénticas.

Los controladores de caracteres proporcionan normalmente estado, entrada, salida, inicialización y terminación en los desplazamientos +0, +3, +6, +9 y +0x0C. Los controladores de disquete tienen cuatro entradas con significados distintos: selección y configuración, petición de recalibrado, lectura y escritura. Su byte en +0x0C es un dato, no un quinto salto. Las rutinas puente públicas de caracteres de DIS10 no realizan registros de cinta; las utilidades llaman a una interfaz privada en el controlador +0x0F.

La clase de terminal traduce los controles lógicos del sistema operativo a las secuencias de salida de un terminal. Un servicio de salida en bruto al dispositivo omite esa traducción y el cursor registrado por el ejecutivo. El SPOOLER aporta otra capa al sustituir la salida de impresora por un recolector residente en disco. Que una de estas capas acepte la salida no demuestra que una impresora física o el equipo conectado por la línea serie la hayan recibido.

Parte VI — Funcionamiento interno del sistema operativo

Capítulo 24 — Entrada al núcleo y estructuras compartidas

Las aplicaciones entran en el ejecutivo mediante una convención de reinicio con un número en línea. El despachador común es pequeño; la mayor parte de la complejidad está en el código específico de cada servicio y en el contexto seleccionado cuando se ejecuta ese código. Este capítulo relaciona la convención de llamada con los mapas residente y de superposiciones.

La ruta de RST 0x08

Una llamada consta del código de operación 0xCF seguido inmediatamente de un número de servicio de un byte. RST apila la dirección de ese byte en línea. El vector salta a 0x08BD, donde el despachador guarda los registros de trabajo HL, DE y AF, obtiene el número de servicio y hace avanzar la dirección de retorno más allá de él. Una tabla de palabras en 0x08E0–0x09BD selecciona una de 111 entradas, numeradas de 0x00 a 0x6E.

No hay establecida ninguna comprobación de límites para un número mayor. Que el selector tenga el tamaño de un byte no implica que haya 256 servicios. Los nombres simbólicos del capítulo 32 son nombres editoriales descriptivos, salvo que se identifique explícitamente un nombre original.

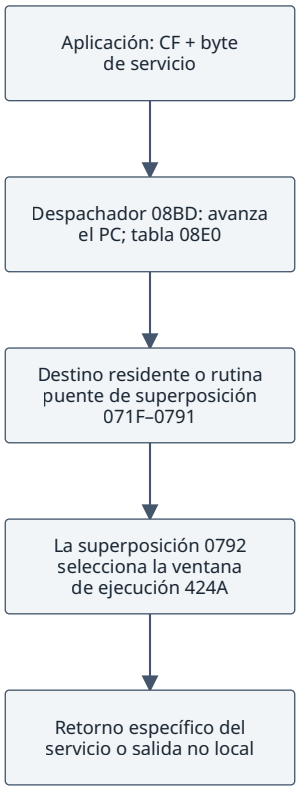


Figura 24.1. Despacho de servicios. Un servicio que finaliza o sustituye un comando no sigue la flecha de retorno normal.

Las rutinas puente de 0x071F–0x0791 llaman a 0x0792 y llevan información en línea para seleccionar la superposición. Los descriptores de superposición de 0x0713, 0x0717 y 0x071B se modifican durante el arranque. Así, el código del núcleo almacenado y la memoria inicializada pueden mostrar legítimamente contenidos distintos en esta zona. Las tablas de direcciones del apéndice B indican su alcance.

Almacenamiento de contextos y canales

El puntero al contexto actual es la palabra little-endian situada en 0x0033. El servicio 0x30 lo devuelve en IY. El servicio 0x50, en cambio, devuelve IY=0, lo que permite acceder a las variables globales comunes. No descodifiques cada instrucción relativa a IY como un campo de partición: el código de los controladores, BASIC y los envoltorios temporales usan registros índice para otras estructuras.

Dentro de un contexto, +0x00 es el SP guardado, +0x02 es la base del contexto, +0x04 es el límite superior de memoria actual y +0x06 es el extremo asignado, donde cero representa 0x10000. El área de trabajo de sector empieza en +0x2CC y el búfer de comandos en +0x3CC. El atlas completo de bytes del apéndice E marca explícitamente como desconocidos los campos desconocidos.

El servicio 0x18 inicializa uno de los diecisiete registros de canal copiando dieciocho bytes de unidad, nombre de archivo, tipo y clase, y después pone a cero catorce bytes. OPEN usa una petición independiente de cuatro bytes de quien llama, que contiene el canal, el modo y un puntero little-endian al área de trabajo. La petición es modificable: tanto el ejecutivo como el BASIC que llama pueden cambiar su modo. El byte de estado de un canal interno no debe sustituir a ese byte de la petición.

Tablas compartidas

Las filas comunes del planificador empiezan en 0x048C. Las filas de bloqueo de registros empiezan en 0x1B15; las de archivos abiertos, en 0x1DA8. Todos sus propietarios están relacionados con el estado de ejecución, pero sus longitudes y claves son distintas. Las 65 filas iniciales de la tabla de bloqueos de registros incluyen su fila de terminación, lo que deja 64 filas utilizables según el código y la disposición. La región de archivos abiertos contiene 129 filas de cuatro bytes; no se ha establecido mediante una prueba de saturación un máximo práctico admitido.

La tabla de componentes con nombre situada en 0x04BC tiene filas de diez bytes: ocho bytes de clave y una dirección little-endian. El servicio 0x6C compara exactamente los ocho bytes y se detiene cuando el primer byte de una fila es cero o un espacio. La imagen inicializada tiene ocho huecos seguidos de un cero en 0x050C. La rutina no impone ningún límite de recuento independiente, no almacena banco ni propietario, ni normaliza mayúsculas y minúsculas. `SPOOLER` exacto devuelve 0x39C2; una grafía rellenada con NUL o en minúsculas falla. En la interfaz establecida, el éxito es AF=0x0044 y HL=dirección; la ausencia es AF=0x0101 con el HL de quien llama conservado.

La entrada por interrupción es una interfaz independiente

El hardware reconstruido usa, en modo IM 0, códigos de operación de reinicio suministrados por el hardware. El temporizador entra por RST 0x10, la interrupción de disco por RST 0x20 y la ruta serie de ALP por RST 0x28. La ruta relacionada con la cinta usa RST 0x18; RST 0x38 puede invocar un gancho en línea por contexto que empieza en +0x39. No son números de servicio adicionales de RST 0x08.

Un marco suspendido del planificador contiene diez palabras de registros y un PC de continuación, pero no es una descripción completa de la pila inmediata de cada interrupción. El apéndice E documenta este marco y el apéndice B, los vectores observados. El arbitraje físico de interrupciones más allá de las rutas implementadas y observadas sigue estando incompleto.

Capítulo 25 — Planificación, devoluciones de llamada y sincronización

El planificador conmuta un contexto guardado, no simplemente un número de banco. Su comportamiento depende también de los bits de espera compartidos y del estado de las devoluciones de llamada. Esas dependencias importan tanto a los programas que coordinan trabajo como a los emuladores que deben reproducir la siguiente instrucción de una aplicación bloqueada.

Selección normal y cuanto

El servicio 0x4F entra en 0x0558. Con una sola partición configurada, conserva AF y vuelve tras habilitar las interrupciones. Con varias particiones, guarda los registros principales y alternativos, almacena SP en el contexto +0x00, cambia a la pila común y recorre las filas de tres bytes. Selecciona la primera fila elegible después del origen del recorrido, volviendo al principio al llegar al número configurado. Si no hay ninguna elegible, restaura el estado guardado anterior, habilita las interrupciones y ejecuta HALT antes de volver a intentarlo.

La fila seleccionada actualiza el índice actual 0x000C, el puntero a la fila actual 0x048A y el puntero al contexto 0x0033. La correspondencia se establece a través de 0x21DE; el nibble alto de estado seleccionado se borra. La restauración en 0x062F–0x063E desapila HL′, DE′, BC′, AF′, IY, IX, HL, DE, BC y AF, habilita las interrupciones y vuelve a través de la continuación guardada.

El temporizador resta 8 del byte de cuanto restante situado en 0x0554 y bifurca según el signo o el cero. Esta aritmética de bytes tiene consecuencias visibles. Los valores medidos de `TIEMPO` 10, 128, 135 y 136 producen, respectivamente, 2, 16, 17 y 1 decrementos del temporizador: aproximadamente 16, 128, 136 y 8 milisegundos en la configuración de temporizador reconstruida. Un valor de 136 no significa un intervalo más largo que 135. El periodo de 8 ms se deriva de la configuración instalada del temporizador; las variaciones arbitrarias del reloj del hardware son otra cuestión.

La entrada de una devolución de llamada guarda el cuanto restante en 0x0556 y activa la indicación 0x0555. Si la cantidad guardada es al menos 4, la siguiente preparación especial retrocede una fila el origen normal y reutiliza ese resto; si no, vuelve a cargar el valor predeterminado de 0x000B. Por tanto, el origen en 0x0551/0x0553 puede ser distinto de la partición que se está ejecutando. Estas variables globales no son prioridades por partición.

Contexto de las devoluciones de llamada

El servicio 0x56 toma un índice de registro de destino en los siete bits bajos de A y una dirección de devolución de llamada en HL. El bit 7 suprime la instrucción final de habilitación de interrupciones. La entrada entre particiones guarda al emisor, selecciona el banco de destino, restaura el marco de registros del destino y salta a la devolución de llamada con su continuación todavía en la pila. Un RET en la devolución de llamada reanuda el cálculo del destino. Los registros preparados por el emisor no se pasan como argumentos de la devolución de llamada.

Las ramas para la misma partición y para una sola partición siguen rutas más cortas. Los destinos no válidos no constituyen una interfaz segura de retorno de error. La rutina enmascara el estado del destino con 0x0F, incluido el bit de detenida. Por tanto, una devolución de llamada puede hacer ejecutable un contexto detenido con PARAR y todavía asignado, mientras su consola sigue sin asignar. Usa ACTIVAR para restaurar el ciclo de vida observado de la consola; no equipares la inyección de una devolución de llamada con la creación de una nueva partición.

Las devoluciones de llamada temporizadas usan el servicio 0x35. Un nodo de seis bytes con permiso de escritura precede inmediatamente al código de la devolución de llamada. El retardo DE se convierte en floor(DE/8) pulsos; cero elimina un nodo coincidente. La lista almacena tiempos diferenciales y la partición propietaria. Al vencer, el nodo se desenlaza y sus seis bytes se ponen a cero antes de entrar en el nodo +6 mediante el servicio 0x56. A=0 es el modo normal establecido. Una excepción con el bit 0 omite la comprobación de propiedad solo para una cabeza de lista numéricamente coincidente y no ofrece una operación documentada de cancelación general entre bancos.

Bloqueos de byte

Un bloqueo de byte contiene cero cuando está libre, o el índice actual del planificador + 1 cuando tiene propietario. El servicio 0x54 lo adquiere, vuelve inmediatamente si el propietario es el mismo, o activa el bit de espera 4 y cede el control si el propietario es otro. No hay recuento de adquisiciones recursivas. Una sola liberación basta tras dos adquisiciones por el mismo propietario.

El servicio 0x55 ignora el bloqueo de otro propietario. Para el propietario actual, borra el bit 4 en todas las filas configuradas y después pone el byte a cero. Es un despertar por difusión: una partición que espera un bloqueo no relacionado se despierta, comprueba su propia condición y puede volver a esperar. Ni la liberación ni el despertar garantizan la equidad ni transfieren directamente la propiedad.

La limpieza genérica de comandos libera una lista fija de bloqueos en 0x29A7, 0x2462, 0x21DD, 0x046C, 0x2108, 0x2109 y 0x0057, seguida de la transacción de disco. No está establecido que libere bytes de bloqueo arbitrarios definidos por las aplicaciones.

Admisión de archivos y tramos de registros

Una fila de archivo abierto usa como clave el dispositivo físico y el primer sector lógico; su cuarto byte contiene el propietario y un bit de exclusividad. Las aperturas compartidas pueden coexistir. Una apertura exclusiva entra en conflicto con cualquier apertura con la misma clave, mientras que una apertura compartida entra en conflicto con una exclusiva existente. Las llamadas OPEN pendientes ceden el control y vuelven a intentarlo sin usar el bit de espera 4 de los bloqueos de byte. Dos canales compartidos de un mismo propietario crean filas idénticas; CLOSE elimina una fila, y el último cierre o la limpieza del propietario admiten a quien esperaba una apertura exclusiva.

Los bloqueos de registros, en cambio, usan como clave el dispositivo físico, el canal y un intervalo de sectores con extremos incluidos. Con varias particiones, el modo UPDATE directo 0x58 pasa a ser 0x5A, lo que habilita el bloqueo de registros. Una lectura directa puede liberar los tramos antiguos del canal,

adquiere su tramo y lo conserva tras una E/S correcta. Una reescritura libera los tramos cuando el bit 1 está activado y el bit 2 desactivado. UNLOCK/servicio 0x53 libera los tramos de registros, pero deja intacta la admisión exclusiva del archivo.

La implementación de los intervalos es imperfecta. Con registros de 300 bytes, la lectura retenida 1 puede poseer los sectores 0xDA48–0xDA49. La lectura 2 del mismo propietario solicita 0xDA49–0xDA4A, pero vuelve sin ampliar esa fila. Otra partición puede entonces adquirir el registro 3 en 0xDA4A–0xDA4B. La lectura 2, aunque correcta, no ha adquirido toda la cobertura que quien llama podría esperar.

La ruta inferior de lectura de disco comprueba por su cuenta los tramos de registros existentes. Una petición directa que omite la adquisición puede seguir esperando a los sectores que ya tiene otro propietario. Esta segunda comprobación es la razón de que desactivar el bit de adquisición superior no evite toda la contención de registros.

Colas en memoria

Los servicios 0x47 y 0x48 actúan sobre almacenamiento de quien llama que contiene la capacidad, la ocupación, el índice de lectura y los datos. No deben describirse como una abstracción fiable de cola bloqueante. La rutina de adición guarda en caché la ocupación antes de comprobar si la cola está llena y no la vuelve a leer mientras espera; una devolución de llamada del temporizador puede retirar un byte y el escritor puede seguir en un bucle indefinido. El bucle no cede el control.

La aritmética de índices da la vuelta a los ocho bits antes de todas las reducciones. Las capacidades superiores a 128 pueden seleccionar datos equivocados o sobrescribir bytes de la cabecera; con las capacidades 254 y 255 se ha demostrado la corrupción de la cabecera. Las capacidades serializadas de 1 a 128 evitan estos defectos aritméticos concretos gracias a límites definidos por la implementación, pero no es un límite anunciado originalmente ni una garantía completa frente a la concurrencia. Los consumidores de la cola deben distinguir la cola vacía (A=0 con acarreo activado) de un byte cero leído correctamente con el acarreo desactivado.

Capítulo 26 — Implementación del sistema de archivos

Una operación de archivo pasa por varias decisiones independientes: normalización del nombre, búsqueda por cuenta y unidad, sondeo del catálogo, preparación del canal según el formato, asignación y E/S física. Un fallo o una incoherencia en una capa no implica un fallo en todas las demás. Este capítulo ofrece los algoritmos necesarios para reproducir esa distinción.

Búsqueda y normalización

El servicio 0x14 recibe una petición de 17 bytes con permiso de escritura: la unidad seguida de ocho bytes de nombre y ocho bytes de tipo. Convierte las minúsculas en mayúsculas, conserva espacios, signos de dólar, dígitos y letras, y sustituye los demás bytes por `$`. Un carácter inicial inferior a `A`, salvo `$`, recibe 0x11. Por tanto, un espacio inicial puede convertirse en `1` y, en una pasada de normalización posterior, en `B`.

La normalización se repite durante los reintentos por cuenta y por unidad. Si la cuenta 0 busca un nombre explícitamente en blanco, puede buscar `1` y fallar; la cuenta 2 puede fallar primero en privado y después volver a normalizar hasta encontrar el archivo público `B`. Este comportamiento no es una transferencia de propiedad. CSI, por su parte, trata el guion bajo como borrado cuando el bit 6 del contexto +0x0A está activado, mientras que los nombres de BASIC entre comillas conservan el guion bajo hasta que la normalización del sistema de archivos lo convierte en `$`.

La dispersión del nombre suma ocho palabras big-endian formadas con los dieciséis bytes normalizados de nombre y tipo, módulo 65 536. Para un volumen de ejemplo, sea N=151 sectores de directorio y F=9 el primer sector del directorio. El LBA inicial es `hash % N + F`. Cada sector contiene ocho entradas. Un cero termina la búsqueda; 0xFF es un candidato borrado que se recuerda para reutilizarlo. Una entrada privada coincidente tiene prioridad sobre la alternativa de la cuenta 0, salvo que esa alternativa esté inhibida por el bit 6 del contexto +0x0C.

La regla del sector siguiente está especificada con exactitud: si L<N, suma F; si L=N, usa 1+F; en caso contrario, usa F. Con el valor inicial L=9, solo recorre cíclicamente diecisiete sectores. Llenar esos diecisiete sectores solo ocupa 136 de los 1208 huecos nominales. Por tanto, la búsqueda acotada puede agotar su ciclo mientras siguen disponibles huecos en otros lugares. OUTPUT asigna un bloque antes de detectar esta situación y lo libera en la ruta normal de fallo establecida.

El LISDIR con comodines, en cambio, recorre los 151 sectores. Puede listar una entrada posterior intacta que la búsqueda exacta no ve por culpa de una marca de cero anterior. OUTPUT puede crear entonces un duplicado normal con el mismo nombre, tipo y propietario en la ubicación reutilizable anterior. La búsqueda exacta selecciona la entrada nueva; al borrarla, queda a la vista la antigua. REPARAR acepta estos duplicados. Ni una creación correcta ni un resultado limpio del verificador demuestran que los nombres sean únicos en todo el volumen.

Asignación y borrado

El mapa de bits empieza en el byte 0x20 del sector lógico 1 y continúa hasta el sector 8. El bloque b=LBA//4 usa el byte `0x100 + 0x20 + b//8` del flujo lógico del volumen y la máscara `0x80 >> (b%8)`. Los bits activados representan asignación. La cabecera y el descriptor en caché también llevan la cuenta de los bloques libres.

El servicio 0x1B requiere un número exacto de bloques contiguos. El servicio 0x63 acepta un número deseado y un número mínimo: toma el primer tramo que alcanza la longitud deseada o, en su defecto, el mayor tramo más pequeño que cumpla los requisitos, y en caso de empate prefiere el tramo posterior. Por tanto, el espacio libre total no basta para predecir si una asignación contigua tendrá éxito. Una petición de 38 bloques falló con 38 libres, pero con un tramo máximo de 37.

El borrado marca la entrada de directorio con 0xFF antes de liberar sus bloques. Las cadenas secuenciales pueden estar fragmentadas, así que sus tramos se liberan por separado. No hay establecida ninguna reversión transaccional de todas las escrituras. El conflicto con la apertura exclusiva que ignora la ruta de borrado nativa se describe en el capítulo 22; un archivo abierto no está protegido solo por estar abierto.

Apertura, transferencia y cierre secuenciales

El modo OUTPUT secuencial 0x88 sustituye un archivo existente. UPDATE SEQUENTIAL de BASIC también selecciona 0x88 salvo que se especifique EXTEND. La implementación borra la entrada antigua, asigna un bloque, crea metadatos de recuento y final a cero e inicializa los datos con 0x1A antes de volver. Por tanto, un fallo del sistema después de OPEN puede dejar una asignación válida con una entrada de directorio sin terminar.

EXTEND empieza con el modo 0x84 y devuelve 0x8C. Carga directamente la palabra del sector final del directorio, restaura los recuentos y recorre hacia atrás los 0x1A finales para determinar la posición de adición. Los datos 0x1A finales no se distinguen del relleno y pueden sobrescribirse en una adición posterior. Un archivo vacío cerrado correctamente tiene un sector final distinto de cero. Una entrada sin terminar con sector final cero es distinta: EXTEND puede cargar el sector lógico cero, y CLOSE puede sobrescribirlo y poner a cero los sectores 1 a 3. Este caso de archivo mal formado destruye metadatos reservados en esa ruta.

La entrada de registros 0x22 copia hasta el CR incluido, pero trata 0x1A como fin de archivo sin copiar la marca ni añadir un terminador. Un registro parcial anterior al 0x1A permanece en el destino. En el bucle de disco especificado no hay ningún argumento de quien llama para la capacidad del destino. La entrada contada 0x5E, en cambio, copia los bytes de datos en bruto, incluido 0x1A, y sigue los enlaces. B=0 solicita 256 bytes. BASIC da a esos bytes devueltos una interpretación adicional de campo y tipo.

Las lecturas contadas siguen de forma anticipada un enlace en el límite de sector. Leer exactamente 254 bytes del sector final puede copiar todos los bytes solicitados y, aun así, devolver A=1. Una petición mayor puede copiar solo el prefijo disponible y dejar intacto el resto del destino. El EOF queda registrado en el estado del canal; las lecturas posteriores no copian nada. BC no es un recuento universal de bytes copiados.

La salida contada 0x5F incrementa una vez por llamada el recuento de registros del canal y puede dejar un sector parcial únicamente en el búfer de quien llama. El WRITE de aplicación de BASIC puede abarcar varias de estas llamadas y no es atómico. Un READ seguido de WRITE con UPDATE secuencial comparte el cursor; leer una marca 0x1A puede situar la salida posterior detrás de una marca de final de aplicación incrustada, lo que produce registros contados pero normalmente inaccesibles.

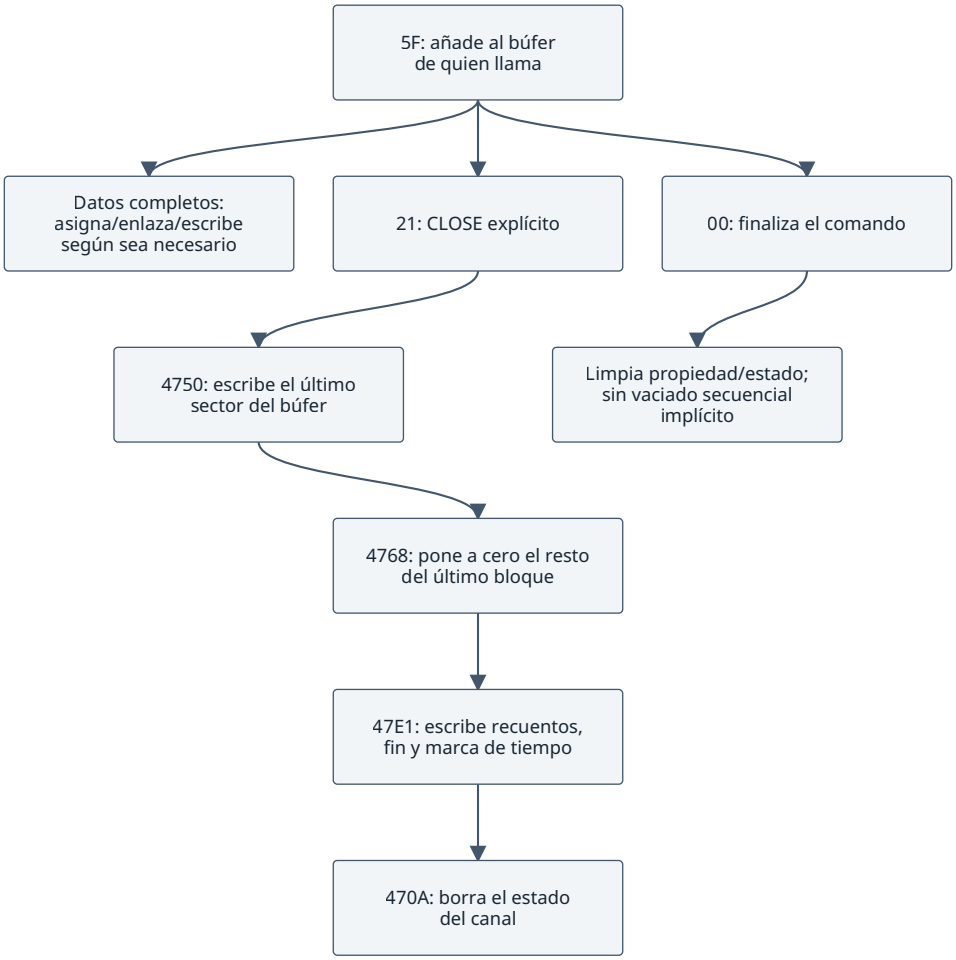


Figura 26.1. Límites de durabilidad secuencial. Ignorar una escritura de disco fallida puede hacer que una etapa posterior se ejecute sin que los bytes anteriores hayan llegado al disco.

Archivos directos y con clave

El registro directo n empieza en `(n-1) × record_length`. La lectura y la escritura pueden abarcar varios sectores; la escritura conserva los bytes de alrededor mediante operaciones de lectura, modificación y escritura. El registro cero y las peticiones fuera de intervalo devuelven A=0x80 en el código. Las operaciones seleccionadas que terminan correctamente conservan BC, DE, HL, IX e IY; la protección contra escritura puede aceptarse en OPEN y rechazarse solo en WRITE.

Un archivo indexado I empieza con N números de hueco little-endian, redondeados hasta un límite de 1024 bytes. Sus primeras C entradas identifican los huecos activos ordenados por clave. Cada hueco contiene una clave fija, datos fijos y un enlace de dos bytes al hueco siguiente; las entradas de índice restantes son cero. Los enlaces siguen la misma secuencia ordenada. El prefijo de índice contiene referencias ordenadas a huecos, no un conjunto de cabezas independientes de cadenas de dispersión.

Un archivo K tiene huecos fijos de clave y datos, sin prefijo de índice ni palabra de enlace siguiente. La creación redondea la capacidad solicitada mediante una regla de búsqueda de primos: empieza con la capacidad solicitada OR 3 y suma 4 hasta obtener un primo. Una petición de 16 da 19. La dispersión aplica `h=(10*h+byte)%N` a lo largo de la clave hasta el NUL. Empezando en h+1, los pasos de colisión usan un estado inicializado a -N e incrementado en 2, y suman su valor absoluto módulo N. El cero marca un hueco sin usar; 0xFF es una lápida reutilizable. El apéndice E ofrece las disposiciones de bytes y las ecuaciones de ambos formatos.

La reparación es una interpretación independiente

REPARAR recorre los 1208 registros y omite las entradas cero, las borradas y las que valen exactamente 0x80. Su máscara de formatos admitidos incluye R, A, S, D, I y K, pero su aceptación de nombres de archivo es más estricta que la creación por el núcleo. Puede rechazar y, con F, borrar un nombre con un espacio interior que BASIC creó y leyó correctamente. No valida los recuentos de registros de las aplicaciones ni repara todas las incoherencias semánticas.

La ruta F puede borrar entradas incorrectas, eliminar redirecciones de uso compartido y conciliar el mapa de bits. Incluso una invocación de solo comprobación puede actualizar el contador de espacio libre. Comprobar la asignación del directorio y validar los registros de las aplicaciones responden a preguntas distintas. Un mapa de bits coherente no certifica que los datos de las aplicaciones estén completos.

Capítulo 27 — E/S de disco y traducción física

La ruta común de disco resuelve una asociación de partición, comprueba la sincronización y la protección, convierte una petición lógica en coordenadas del controlador y aplica la política de errores. La transacción final con la controladora es solo una parte del servicio público. Un retorno correcto puede producirse tras un fallo de la controladora ignorado deliberadamente.

Traducción de direcciones

En un disco configurado con 830 cilindros, cinco cabezas y 32 sectores por pista, con sectores de 256 bytes, la capacidad es de 132 800 sectores, es decir, 33 996 800 bytes. En esta interfaz de disco duro, los identificadores de sector empiezan en cero. El índice lineal de sector es `(cylinder*5 + head)*32 + sector`.

La correspondencia de un volumen puede seguir un sentido descendente de cilindros. Divide el LBA lógico entre 32 para obtener la pista t y el sector s; después divide t entre cinco para obtener q y r. En un volumen de cinco cabezas que desciende desde el cilindro 396, la correspondencia es cilindro `396-q`, cabeza `4-r`. El orden lógico del sistema de archivos no tiene por qué coincidir con el orden ascendente de los cilindros físicos.

Un descriptor para esta geometría de ejemplo empieza con cinco cabezas y un número de cilindros de 397 en little-endian. Cuarenta y cinco pares de pista de origen y de sustitución ocupan los desplazamientos 0x03–0xB6. El sistema en ejecución sustituye una vez una pista coincidente y se detiene en cero o en un origen mayor. El recorrido del cargador de arranque es distinto: carece de esa misma regla de parada en cero, así que el origen cero puede coincidir. En la correspondencia descendente, sustituir la pista lógica 169 por la 1939 redirige C363/H0 a C9/H0. No está especificado el comportamiento completo de las firmas especiales de descriptor 0xFD/0xFE ni de las variantes de correspondencia 0x55/0x99.

La extensión en cilindros del descriptor no es por sí misma la capacidad utilizable del volumen. Un sistema de archivos que declara 1939 pistas lógicas de 32 sectores tiene 62 048 sectores, que corresponden a 15 512 bloques de asignación de cuatro sectores. Usa los límites declarados del sistema de archivos para la asignación.

Ruta común de las peticiones

Los servicios de un solo sector 0x0A y 0x0B toman B=asociación, DE=LBA y HL=búfer; ambos fuerzan C=1. Los servicios de varios sectores 0x32 y 0x33 usan C como recuento. Determinadas llamadas correctas de ocho sectores conservan BC, DE, HL, IX e IY y devuelven AF=0x0044. Las formas de un solo sector conservan DE, HL, IX e IY según lo especificado, pero devuelven C=1 en lugar del C original de quien llama.

La ruta común adquiere el bloqueo de transacción de disco y comprueba los tramos de registros. Un descriptor invalidado puede provocar una lectura preliminar de la cabecera antes de la transferencia solicitada. Así, un solo sector solicitado puede generar dos lecturas físicas sin que haya ningún reintento. El servicio 0x09 invalida los bytes +0x14/+0x15 del descriptor en caché; no es una operación general de vaciado.

En las escrituras, el bit 7 del descriptor +0x13 rechaza la petición con el error de controlador 2 antes de enviar ningún comando a la controladora. La ruta WD usa el comando 0x20 para leer y 0x30 para escribir, y transfiere a través del puerto 0xB8 antes de examinar el estado. El estado de fallo especificado en los registros WD correspondientes se traduce al error 7. No se garantiza que los bytes devueltos tras un fallo físico sean válidos.

Formateo de pistas y tamaño de sector

INICDISC envía el comando WD 0x50 para formatear una pista. Con sectores de 256 bytes, envía un búfer completo de 256 bytes por el puerto 0xB8. El principio de ese búfer aporta pares formados por un indicador de bloque defectuoso y un identificador de sector; con 32 sectores, 64 bytes contienen los pares y el resto de los bytes completan el búfer de transferencia. El registro Sector Count determina cuántas entradas se usan. El formateador construye la permutación de identificadores de sector; su selección de entrelazado no debe deducirse del orden lógico en el que se leen los archivos.

El registro SDH en 0xBE separa los bits de tamaño 6:5, los bits de unidad 4:3 y los bits de cabeza 2:0. La codificación de tamaño 00 significa 256 bytes; 01, 512 bytes; y 11, 128 bytes. La codificación 10 no define aquí ningún tamaño de sector. Así, activar solo el bit 5 selecciona 512 bytes, no 256. Las peticiones de 256 bytes del sistema usan los bits de tamaño 00 y establecen la cabeza y la unidad de forma independiente.

Una transferencia de la controladora puede completar su secuencia normal de datos y finalización y, aun así, notificar un error. El controlador debe examinar el estado; la finalización no sustituye a un estado correcto. El capítulo 11 trata el diálogo de formateo para el operador y el efecto de PISTA en la ubicación del volumen.

Reintentar, ignorar y anular

El número de reintentos instalado es cuatro. R reintenta; M llama a la entrada +3 del controlador y reintenta; I da por tratado el sector fallido; Q abandona la operación mediante una salida no local. En el controlador WD residente, +3 es un RET, así que no está demostrado que M reinicie la controladora.

Una lectura fallida ignorada puede devolver un éxito normal con un contenido de búfer no válido. No hay ninguna garantía general de que las lecturas fallidas devuelvan ceros. Una escritura fallida ignorada devuelve un éxito normal sin escribir un sector válido. El software no debe interpretar A=0 como prueba independiente de la integridad del soporte tras una intervención del operador.

Una devolución de llamada 0x4A registrada se ejecuta antes de que se decremente el número de reintentos, mientras se mantiene el bloqueo del disco. A=0 ignora el error; un valor positivo en A sustituye la política normal por un código de error; un valor negativo en A anula la operación. Si en la

devolución de llamada se elimina la protección contra escritura y se devuelve cero, se omite esa escritura fallida en lugar de reintentarla. CLOSE puede entonces confirmar los metadatos de un registro cuyos datos nunca llegaron al disco.

Normalmente, Q por sí sola produce el resultado 254 de CSI a través de una devolución de llamada de interrupción activa. Durante un CLOSE protegido, puede que esa devolución de llamada ya se haya borrado; el manejador de Q devuelve A=0 y la finalización genérica posterior indica el resultado 0 a pesar de haberse perdido datos del búfer. Los códigos de resultado describen el flujo de control, no la durabilidad de una transacción.

Interfaz de disquete

DIS1 y DIS2 usan una ABI distinta de cuatro entradas. Las entradas privadas de lectura y escritura son A=número de sectores, B=cabeza, C=sector empezando en cero, DE=pista (se usa E), HL=búfer e IY=descriptor. El paquete usa el código de operación de lectura 0x06 o el de escritura 0x05, sumándoles 0x40 para el modo establecido de 256 bytes. El primer sector físico es C+1 con sectores de 256 bytes, o 2*C+1 con sectores de 128 bytes. El fin de pista y el recuento se duplican en consecuencia en la traducción a 128 bytes.

DIS1 sondea los puertos 0xCC–0xCE y puede quedarse en la inicialización cuando el dispositivo esperado no responde. DIS2 usa 0xC0–0xC2 y puede notificar el error 1 de no preparado si no hay ninguna unidad disponible. La ruta de transferencia de DIS2 escribe en el puerto 0xC2 el byte de banco seguido de los bytes alto y bajo de la dirección del búfer y, después, sondea el bit 7 para detectar la finalización. Los bytes de resultado de la controladora llegan a través de 0xC1. El cableado físico del adaptador, la temporización del bus y el acarreo de la dirección al cruzar un límite de 64 KiB siguen sin especificarse por completo.

El formato FLOPPY2 que selecciona INICIAR con CARA 2 tiene 77 cilindros y 26 identificadores de sector por pista: el cilindro 0 usa sectores FM de 128 bytes en ambas cabezas, y los demás cilindros usan sectores MFM de 256 bytes. El orden de los sectores físicos es 1,14,2,15 hasta 13,26. Esto describe el formato de software, no un fabricante o modelo concreto de unidad. El capítulo 11 explica el formateo, el montaje y las comprobaciones por relectura. Las transferencias de archivos por sí solas no establecen el arranque desde disquete, el intercambio IBM ni un flujo completo de archivado en soportes extraíbles. El apéndice B recoge los paquetes y las correspondencias de errores.

Capítulo 28 — Controladores de terminal, serie, impresora y cinta

La aparente sencillez de la consola oculta varias capas: la política de la aplicación, los controles lógicos, la traducción de la clase de terminal, las colas de los dispositivos y las interrupciones de la controladora. Los controladores de otros dispositivos reutilizan algunos desplazamientos de entrada, pero mantienen sus propios contratos. Este capítulo sigue la ruta del software desde la petición de un carácter hasta su controlador y explica por qué una salida aceptada no demuestra la entrega física.

Política de la consola y traducción de la clase

El servicio 0x05 aplica la supresión de salida del contexto y la política de copia a la impresora antes del servicio 0x07. El servicio 0x07 interpreta los controles lógicos del cursor y llama a la entrada +3 de la clase de terminal, con una alternativa en bruto cuando la clase devuelve acarreo. El servicio 0x40 escribe directamente en un dispositivo asociado y no realiza la traducción de clase ni actualiza el cursor registrado.

Esc seguida de O conmuta el bit 1 del contexto +0x0A. Suprime el servicio 0x05 y el eco generado por el servicio 0x04, pero la salida directa de 0x07 y 0x40 continúa. Esc seguida de S ejecuta un bucle local de pausa con cesiones al planificador; las demás particiones continúan. La reanudación vacía la entrada en cola, así que un carácter tecleado durante la pausa no tiene por qué llegar a la aplicación. Esc seguida de Q interrumpe el bucle no cooperativo establecido y devuelve el resultado 254 de CSI.

Los archivos de clase tienen la rutina de entrada en +0 y la de salida en +3. La tabla de punteros de control de CLASE1 empieza en +0x8C, y la de CLASE2 en +0xBE; cada una contiene 32 punteros little-endian. Las secuencias terminan en 0xFF. Los controles 0x00, 0x10, 0x15 y 0x19 reciben un tratamiento del núcleo en lugar de una emisión normal desde la tabla. El control 0x10 consume los bytes de columna y fila. CLASE1 los codifica como ESC G seguido de coordenada+0x20; CLASE2 emite ESC `[` fila+1 `;` columna+1 `H`. La aritmética da la vuelta como bytes; no es una operación de recorte.

Los valores L y P del descriptor tienen usos que dependen de quien llama. CSI pasa L directamente como su presupuesto de entrada de línea, y la salida normal comprueba el umbral horizontal antes de avanzar. LF incrementa la fila y la limita a P. Por tanto, los valores predeterminados L79/P23 no deben reinterpretarse categóricamente como un tamaño de pantalla de 80 × 24.

En la configuración de consola reconstruida, el transporte es de siete bits. Por tanto, un control emitido con el bit alto activado puede aparecer de otra forma en el terminal. El estado diferido de salto de página puede consumir el siguiente eco y dejar válido el carácter de entrada. Diagnostica por separado los bytes de entrada, el eco y el seguimiento del cursor.

Controladores serie ALP

DIS18–DIS21 implementan PAN1–PAN4 en los pares de puertos 0x32/0x33, 0x30/0x31, 0x36/0x37 y 0x34/0x35. DIS23–DIS26 aportan los pares del segundo grupo opcional: 0x3A/0x3B, 0x38/0x39, 0x3E/0x3F y 0x3C/0x3D. El controlador de 887 bytes contiene un anillo de entrada de 80 bytes y dos búferes de salida de 83 bytes.

El anillo situado en el controlador +0x240 almacena la capacidad, el recuento y el índice de lectura, seguidos de los datos en +0x243. La entrada que llega con el anillo lleno se descarta. Las ramas de control de flujo establecidas actúan cuando el recuento anterior es 75 al recibir y 5 al consumir. Las ocupaciones resultantes son 76 y 4, respectivamente; confundir los valores anteriores y posteriores a la actualización lleva a describir mal los umbrales de pausa y reanudación del controlador. Los puertos 0x24–0x27 proporcionan salidas independientes de recepción preparada, mientras que el WR5 del SCC controla RTS. Su combinación física exacta sigue sin rastrearse por completo. La contrapresión del anillo de software y la FIFO de recepción más pequeña del SCC son límites distintos; detener al emisor no retira un carácter que ya está en camino.

Los búferes de salida empiezan en +0x293 y +0x2E6. La salida se vacía en torno a un carácter ESC y después de CR, FF, ETX o de un recuento superior a 82; el sondeo de estado también puede vaciarla y esperar al DMA. Por tanto, el retorno de una llamada de salida de carácter puede indicar solo que se ha almacenado en el búfer. La disponibilidad del DMA se guarda en los bytes comunes fijos 0x02A6/0x02A7, no en un indicador de ocupado local del descriptor.

El servicio 0x67 registra una fila de devolución de llamada de tres bytes en `0x23FF + 3*A`, para los índices 0x02–0x20. La fila contiene el índice del propietario y el puntero a la devolución de llamada. Los índices de recepción y DMA de PAN1 son 0x11/0x15; los de PAN2, 0x12/0x16; y los canales posteriores siguen el mapa instalado del apéndice B. El despacho de recepción entra en el propietario registrado mediante la ruta de devolución de llamada entre particiones y borra el nibble alto de estado. La finalización del DMA borra solo el bit de espera 5.

Una rutina de entrada bloqueante vacía la salida, deshabilita las interrupciones, intenta extraer un byte del anillo y llama a 0x6B si está vacío. Con una sola partición, 0x6B habilita las interrupciones y ejecuta HALT; con varias, marca el bit de espera de eventos 5 y planifica. Despertar no demuestra que haya entrada en cola. La finalización del DMA, la entrada de escape consumida y otras devoluciones de llamada pueden despertar a un lector, que debe volver a comprobarlo.

Controladores serie SS2

DIS16 y DIS17 son controladores de 650 bytes para PAN9/IS9 y PAN10/IS10 en 0x7E/0x7F y 0x7C/0x7D. Sus anillos de entrada tienen 128 bytes; la salida se hace por sondeo, sin el búfer de ALP. El acarreo del estado se deriva del bit 2 de la controladora, mientras que los demás indicadores reflejan la ocupación del anillo.

La inicialización registra devoluciones de llamada en los índices 0x0B y 0x10. La terminación espera al bit 0 del registro 1 de la controladora, reinicia el canal, anula el registro de la devolución de llamada y habilita las interrupciones. El despacho de RST 0x10 lee el puerto 0x60 enmascarado con 0xE0. El origen 0xE0 sigue la ruta del SCC a través de 0x2386, que lee RR3 en 0x7E: los bits 3 a 5 seleccionan la

devolución de llamada 0x0B; en caso contrario, la ruta seleccionada es la devolución de llamada 0x10. El origen 0xC0 sigue la ruta del temporizador y se reconoce escribiendo 0xA0 en el puerto 0x60. El arbitraje físico y el cableado completo de la máscara de orígenes siguen sin especificarse. Los controladores serie requieren su propia controladora instalada y una conexión de terminal con el formato de trama correcto; son independientes de los puertos ALP.

El controlador sustituye temporalmente la devolución de llamada +0x28 del contexto durante el estado y la salida. En un reintento controlado por ocupado, sobrescribió su puntero guardado y volvió con su propio manejador todavía instalado. La salida con el dispositivo ya preparado restauró el manejador anterior. Se trata de un defecto de ciclo de vida del controlador, no de un contrato distinto para el servicio 0x39.

Impresoras

IP/DIS11 usa los puertos 0xA4/0xA5 y notifica la desconexión en la configuración sin correspondencia especificada, de modo que ASOCIAR puede eliminar su asignación. CDC/DIS12 puede modificar sus puertos a 0x50/0x51. Convierte las minúsculas en mayúsculas, sustituye por ? los caracteres especiales no admitidos observados, complementa los bytes y los escribe. IMP/DIS22 usa los puertos 0x78–0x7B con la disposición de registros del SCN2651: datos, estado, registros de modo alternos y comando. La asignación de puertos está respaldada por el controlador y por la identificación de componentes de la placa; el cableado completo de señales y relojes sigue sin especificarse. Sus comprobaciones de estado requieren transmisión preparada, DCD y DSR, mientras que CTS controla la transmisión serie. La inicialización normal solicita 9600 baudios nominales con ocho bits de datos, paridad impar y un bit de parada. La forma B4800 PE selecciona siete bits de datos, paridad par y un bit de parada.

IMP convierte 0x80 en 0x1F antes de la salida. También puede confundir con disponibilidad un estado 0xFF con todos los bits a uno procedente de una interfaz no disponible, así que la aceptación del estado por sí sola no demuestra la entrega. Una impresora fuera de línea puede provocar la pregunta nativa R/Q. Son contratos de transferencia de bytes y de disponibilidad; el modelo físico de la impresora y su comportamiento de descripción de página siguen sin especificarse.

La política común de impresoras para las asociaciones 0x0C–0x0F sondea la disponibilidad y gestiona los controles, el estado de la página y los retardos. En la ruta predeterminada establecida, 0x1F y FF se convierten en CR, FF antes de llegar al controlador. Una impresora sin asignar puede devolver éxito sin indicar nada. Los puntos de entrada de lectura de las impresoras son rutinas puente RET; sus valores accidentales de A no son caracteres de entrada. Estos retornos de software no certifican que se haya impreso una página.

Cinta

DIS10 es un controlador de 2980 bytes. Sus entradas públicas de estado, entrada y salida son rutinas puente XOR A/RET; las operaciones reales con registros usan el controlador +0x0F con los selectores privados 0x80–0x8C. Las utilidades obtienen esa dirección a través del descriptor físico. Estos selectores no son llamadas al sistema por encima de 0x6E.

La inicialización registra el índice de devolución de llamada 0x03 y espera en los puertos 0xA0–0xA3. La inicialización puede esperar indefinidamente si no llega la respuesta esperada de la controladora. Los procedimientos completos de lectura, escritura, rebobinado y recuperación de archivados no están especificados.

Algunas rutas privadas son especialmente inciertas: la operación 0x8B tiene una rama que vuelve antes de restaurar un DE apilado, la operación 0x8C almacena 0xDF donde una interpretación de reinicio sugeriría cero, y 0x83 puede devolver cero sin salida en una rama concreta. Estos contratos incompletos obligan a actuar con cautela al implementar un comportamiento compatible. Las tablas de selectores y de etiquetas de 80 bytes del apéndice A bastan para identificar el protocolo especificado sin presentarlo como un servicio de copia de seguridad especificado.

Capítulo 29 — Impresión diferida y mensajes

Tanto la impresión diferida como la mensajería cruzan la frontera entre una petición inmediata y una actividad posterior. Usan mecanismos de almacenamiento y de ejecución distintos. El SPOOLER residente intercepta la salida a la impresora, los mensajes en directo inyectan una devolución de llamada en otra partición y las utilidades de buzón mantienen un archivo secuencial normal.

SPOOLER residente

La entrada residente con nombre situada en 0x39C2 lleva al código de administración de 0x3FB2 y a una tabla de selectores en 0x3FC4–0x3FE9. Los selectores 0x00–0x12 no tienen especificada ninguna comprobación de límites independiente. La recogida entra en 0x39C8 y continúa en 0x3B9C. La palabra global 0x0044 identifica el SPOOLER activo.

SPOOLER.COMANDO consulta el nombre residente SPOOLER mediante el servicio 0x6C en el desplazamiento de archivo 0x0160. La rama para cuando falta el residente hace referencia a `CARGAR SPOOLER`, almacenado en 0x0D47; en 0x0D1D hay otra cadena, `VACIAR SPOOLER`. Faltan ambos ejecutables independientes. Como la entrada residente SPOOLER ya existe, la falta de estos componentes asociados no impide la inicialización documentada ni la recogida en pausa. Las referencias sugieren un ciclo de vida independiente para el componente residente, pero no establecen su interfaz completa ni convierten DESCARGAR en sinónimo de VACIAR.

Antes de la inicialización, SPOOLER LIST puede indicar una cola vacía con el resultado 0, mientras que ESTADO indica su situación sin inicializar con el resultado 12. Después de INIC, la configuración en pausa establecida sustituye el destino del vector de salida del controlador de impresora por 0x39C8. La ruta común de impresora detecta un valor distinto de cero en 0x0044 y omite su formato normal, lo que permite al recolector retener el flujo de entrada.

Un informe puede recogerse mientras la impresión normal está en pausa. Los controles de salto de página y de fin de informe afectan a su recuento de páginas y a su estado de finalización. Los datos en cola describen la salida recogida; no certifican que una impresora física la haya producido.

Las dieciséis ranuras de recogida situadas en 0x39DD tienen un paso de 16 bytes. Contienen indicadores, el número de búferes libres, el puntero al búfer, el sector actual, las páginas, los bloques, el número de cola, la selección de cola, indicadores/copias y el estado de línea; el último byte sigue sin resolverse. SPOOLCOL es un archivo directo de 64 registros de dieciséis bytes en un solo bloque. Un registro de cola almacena una etiqueta de informe de ocho bytes, el índice + 1 de la partición que lo envía, una marca de tiempo de tres bytes, indicadores, la cola y el número de páginas. La vuelta al principio por igualdad del productor parece usar los registros 1 a 63; esa capacidad efectiva sigue estando definida por la implementación y no se ha saturado de forma nativa.

Los indicadores del informe distinguen 0x80 abierto, 0x40 cerrado, 0xC0 imprimiéndose y cero impreso, con el bit de conservación 0x20, el número de copias en los bits 2 a 4 más uno y la impresora en los bits 0 a 1 más uno. El recuento del directorio del archivo de respaldo son páginas, no escrituras de la aplicación. El recolector escribe sus propios metadatos en lugar de depender del CLOSE secuencial genérico.

Codificación de informes y reglas de reinicio

Los datos del informe usan 0x06 seguido de un byte para representar espacios; las longitudes normales se codifican como longitud+0x20. El descodificador exacto da la vuelta, así que los recuentos mal formados 0x21 y 0x20 producen 257 y 256 espacios. El byte 0x0D significa CR/LF, 0x8D significa solo CR, 0x1A termina el informe y 0x1F salta el resto de un sector antes de seguir su cadena. Los demás bytes, incluido 0xFF, son literales en la ruta especificada. Es una codificación propia de los informes, sin relación con el bit 0x20 de protección contra lectura del directorio.

En la ruta de reinicio de INIC definida por la implementación, se borran las entradas de cola incompletas que valen exactamente 0x80. Los casos conservados 0xA0 y de varias copias 0x84 pueden sobrevivir; los estados de impresión 0xC0–0xFF pasan a ser 0x40–0x7F. Si falla la búsqueda de un informe, se borran los indicadores. La limpieza hacia atrás de las entradas finales inactivas borra el primer byte de la etiqueta, no todos los campos. Las cadenas no válidas pueden encontrarse con enlaces a cero; no hay establecida ninguna protección contra ciclos de sectores ya visitados. La recogida nativa está documentada, mientras que el ciclo de vida completo de reinicio y de salida física sigue sin resolverse.

Transferencia de mensajes en directo

El servicio 0x58 serializa un mensaje con el bloqueo de propietario 0x21DD y un byte de transferencia marcado en 0x21DC. Entra en la devolución de llamada 0x21BE del destinatario mediante el servicio 0x56. La devolución de llamada usa la política de salida de consola del destinatario. El DE del emisor avanza más allá del NUL o el CR; BC, HL, IX e IY se conservan en las llamadas seleccionadas.

Los bytes del mensaje se enmascaran a siete bits. NUL termina; CR emite CR/LF y termina; LF emite CR/LF y continúa; y DEL se omite. El control 0x10 consume dos coordenadas, las enmascara a siete bits y cambia 0x7F por 0x7E. Un destino que sea el propio emisor o esté fuera de intervalo provoca un retorno anticipado, pero sigue llamando a la rutina de liberación del bloqueo; por tanto, un 0x21DD que pertenezca a quien llama puede liberarse sin que haya entrega.

El propio servicio no comprueba ni la cuenta ni la política de mensajes desactivados. MENSAJE impone sus propias reglas de usuario destinatario, y la ruta con comodín para el privilegio 5 especificada omite la desactivación. El resultado 0 puede significar que no ha habido entrega en directo. No trates esta interfaz como entrada de consola ni como una cola de mensajes persistente.

Buzón persistente

Los depósitos de MENSAJE y la recuperación con BUZON usan `SISTEMA.BUZON`, un archivo secuencial cuyo propietario es la cuenta temporal 0xFF. Cada mensaje empieza con una cabecera de 33 bytes: destinatario, ocho bytes; emisor, ocho; fecha, ocho; hora, ocho; y CR. Las líneas del cuerpo terminan en CR. El byte 0x7F rellena hasta un límite de datos de 254 bytes y termina el mensaje; el último sector del archivo usa relleno 0x1A.

BUZON muestra y elimina los mensajes del nombre de cuenta exacto, compacta los mensajes restantes y actualiza los bloques y el estado de final mediante operaciones con sectores en bruto. Su recuento en el directorio puede quedar desactualizado. Un prefijo de destinatario aceptado durante el depósito no tiene por qué coincidir con el nombre exacto usado al recuperar. El buzón forma parte del estado del servicio y no tiene por qué existir antes de usar la mensajería almacenada. El apéndice E registra la cabecera por separado del envoltorio de sectores secuenciales.

Capítulo 30 — Procesamiento de comandos, carga y terminación

CSI resuelve un comando en la invocación de un ejecutable o de un intérprete. Después, el cargador interpreta el formato del archivo y sus reubicaciones, mientras que el ejecutable aporta su propia política y su comportamiento en tiempo de ejecución. El resultado de un comando corresponde a la ruta de control final; no sustituye al examen de los resultados del cierre de archivos y de los dispositivos.

Análisis y selección de comandos

CSI usa la geometría del terminal para establecer el presupuesto de entrada de línea. El servicio 0x01 puede enviar la línea automáticamente cuando se agota ese presupuesto, así que un valor de L demasiado pequeño por accidente puede partir un comando tecleado antes del CR. El texto de comando preparado que se pasa mediante el servicio 0x36 no está sujeto a este presupuesto interactivo. El servicio 0x36 acepta bytes imprimibles, convierte las minúsculas en mayúsculas, termina en CR y vuelve a CSI pasando por la limpieza; en el bucle de copia no se aprecia ninguna protección de longitud.

La tabla de abreviaturas original tiene 45 entradas. Define la grafía de los comandos y sus prefijos mínimos, no un inventario de los programas instalados. CLASIF y CARGAR no tienen ningún ejecutable ni archivo EXEC correspondiente. DESCARGAR se almacena con nueve letras, pero la búsqueda usa el campo de ocho caracteres DESCARGA, para el que no hay instalado ningún archivo. Ninguna de estas entradas aporta una implementación integrada. EXEC es un caso especial: EX y EXE seleccionan su ruta, pero transfieren a EXECLANG una posición de token original equivocada, lo que permite que se ejecute otro script. Escribe EXEC completo.

CSI expande las abreviaturas de la tabla en los desplazamientos de archivo 0x053A–0x0589. Su ruta final de archivo no encontrado, en 0x06E1–0x06F2, restaura el nombre de ocho bytes introducido antes de mostrar el error. Así, un error que menciona CAR.COMANDO después de introducir CAR no significa que haya fallado la expansión a CARGAR. La grafía que aparece en ese error no es un registro completo de los nombres buscados.

Los sinónimos personalizados se tienen en cuenta antes que la tabla de abreviaturas estándar. Dentro de cada asociación de disco en la que busca, CSI comprueba el archivo COMANDO del token original y después su archivo EXEC, si IMPEXEC está habilitado, antes de comprobar los nombres expandidos. A continuación pasa a la asociación siguiente. Este orden hace que un archivo real pueda tener prioridad sobre la abreviatura estándar esperada. El capítulo 35 recoge los ajustes visibles para el operador y los prefijos exactos aceptados.

La ejecución implícita de EXEC depende del ajuste correspondiente y de la búsqueda de archivos. Las palabras clave de los lenguajes BASIC y EXEC las interpretan sus propios componentes, no la tabla de comandos de CSI. Esta separación explica por qué no se puede pegar un comando del editor o una instrucción BASIC en el indicador del sistema operativo y esperar que tenga el significado que tiene en su lenguaje.

Carga de archivos R

Un archivo R no tiene ninguna cabecera fija obligatoria. Su imagen de carga empieza en el desplazamiento de archivo cero y puede contener tanto instrucciones como datos inicializados. La interfaz de ejecución de comandos entra en un comando por su base de carga; una superposición o un controlador cargados por separado siguen su propia convención de entrada. Las superposiciones de BASIC empiezan con un byte de componente antes de su hueco JP, mientras que las superposiciones de SORT empiezan con un área de control y datos. El apéndice C recoge estas disposiciones. Cargar un componente así es distinto de transferir el control a su entrada. El directorio aporta la longitud en bytes de la imagen, el número de sectores almacenados y el LBA inicial. Cuando existen, los registros de reubicación empiezan después de la imagen, redondeada hasta un límite de sector de 256 bytes. Algunos módulos R pequeños no tienen sector de reubicación. Cada sector de reubicación empieza con un recuento de 1 a 127 seguido de desplazamientos little-endian. En cada desplazamiento, el cargador suma la base de carga a una palabra little-endian módulo 65 536.

La imagen de carga de CSI, incluidos el código y los datos inicializados, ocupa 3104 bytes (0x0C20), y el almacenamiento de reubicación empieza en el desplazamiento de archivo 0x0D00. Sus quince sectores almacenados incluyen dos sectores de reubicación con 250 correcciones. Con una base de carga de 0x5157, esas correcciones ajustan en consecuencia las direcciones incrustadas. Un analizador debe distinguir la longitud en bytes de la imagen, el número de sectores de la imagen y el total de sectores almacenados.

El servicio 0x1E carga un comando y entra en él en el contexto +0x4CC para el formato R; el formato A usa su dirección de carga absoluta del directorio +0x1E. El servicio 0x16 carga un módulo con nombre sin entrar en él. El servicio 0x62 construye metadatos temporales a partir de un descriptor compacto de siete bytes para los submódulos R. Estas interfaces de carga comparten mecanismos, pero no son convenciones de llamada intercambiables.

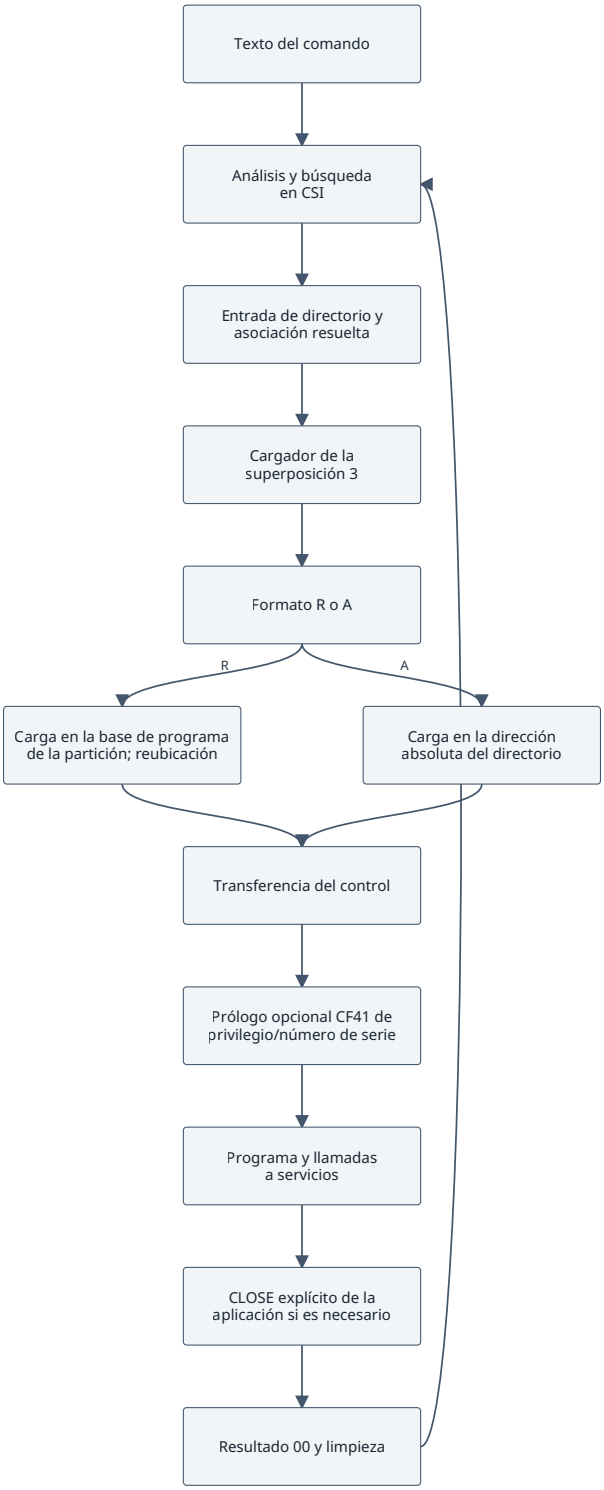


Figura 30.1. Ejecución de comandos. El paso CLOSE es responsabilidad de la aplicación, no una operación automática del cargador ni de la terminación.

Política de comandos y retorno en tiempo de ejecución

El prólogo estándar opcional es `CF 41`. Los cuatro bytes anteriores contienen el privilegio mínimo y una vinculación de tres bytes al número de serie de la instalación. Si el primer byte del número de serie es cero, se omite la comparación. Un fallo muestra el mensaje 19 o 20 y termina con el resultado 252. Los prólogos examinados usan los bytes de número de serie `16 00 00` y valores de privilegio de 0 a 5. Las protecciones OPTION PRIV/SERIAL de BASIC usan su propia representación compilada y su propia limpieza; sus fallos devuelven en cambio 253.

El contrato completo de registros iniciales y de trampolín de retorno del cargador sigue incompleto. El pequeño ejemplo Z80 documentado en el capítulo 18 usa solo servicios establecidos y termina mediante 0x00; no presupone un entorno de ejecución de C, un vector de argumentos, un montón automático ni una convención de RET a CSI. Los módulos BASICUSR pequeños son distintos: su envoltorio definido los llama y restaura IX/IY en torno a un RET normal.

Limpieza y continuación

El servicio 0x00 almacena A en el contexto +0x68, elimina el estado de bloqueos y aperturas del propietario actual, borra el estado de ejecución y vuelve a cargar CSI. No vacía implícitamente un búfer parcial de salida secuencial. Para la ruta de durabilidad secuencial establecida es necesario un CLOSE explícito antes de la terminación. Un manejador de errores de BASIC puede llamar por sí mismo a CLOSE; esta limpieza de la aplicación no debe generalizarse al ejecutivo.

El servicio 0x46 añade un límite superior de continuación a la sustitución de comandos. Establece el límite superior de memoria y SP a partir de HL, activa el bit 5 del contexto +0x38 y usa la ruta del servicio 0x36. Más tarde, CSI borra ese bit y salta al límite superior. El código establece el mecanismo, pero siguen sin especificarse un ejemplo completo de llamada de uso general y el contrato de registros.

FIN finaliza la sesión de la cuenta y puede descargar y volver a cargar la consola para el inicio de sesión. PARAR detiene una partición y conserva su RAM asignada. Un IPL aceptado reinicia la máquina. Estas operaciones tienen alcances distintos y ninguna aporta pruebas de que exista un comando universal para apagar todo el sistema y vaciar los búferes.

Parte VII — Referencia

Capítulo 31 — Referencia de comandos

Consulta aquí el nombre completo de un comando después de aprender las operaciones básicas con archivos. Cada entrada explica la finalidad, la sintaxis, las opciones y un ejemplo básico. Los ejemplos son operaciones individuales, no una secuencia para ejecutar de principio a fin. Los archivos de práctica como NOTA.TEXTO proceden del capítulo 6; los demás nombres de archivo son ilustraciones explícitas que requieren que existan antes archivos compatibles.

Antes de usar la letra de dispositivo de un ejemplo de mantenimiento, debe comprobarse con ASOCIAR. Los ejemplos destructivos explican la sintaxis y no son instrucciones para modificar el disco del sistema. Algunas funciones opcionales solo tienen una interfaz documentada; sus entradas indican cuándo no se dispone de un procedimiento de uso completo. Los nombres sin una interfaz invocable especificada tienen un ejemplo de inspección o de ayuda en lugar de una invocación inventada.

En todos los ejemplos se usan los nombres completos de los comandos. El capítulo 35 recoge sus abreviaturas permitidas, y sus tablas de opciones distinguen las palabras clave completas de sus formas abreviadas aceptadas.

El privilegio indicado es la comprobación mínima estándar, cuando se conoce. Un comando puede realizar comprobaciones adicionales. Consulta el capítulo 33 para ver los mensajes exactos; un código de finalización no es lo mismo que un número de mensaje del sistema.

Comando	Finalidad
ACTIVAR	Activar una partición o un comando en segundo plano
ARCHDIR	Listar un disquete de archivado
ARCHIVAR	Archivar archivos, cuentas o volúmenes
ASIGNAR	Asignar canales
ASOCIAR	Asociar dispositivos lógicos y físicos
ATRIB	Renombrar o proteger archivos
AYUDA	Mostrar la ayuda de los comandos
BACKUP	Copiar un soporte completo de disco o de cinta
BASIC	Editar, interpretar o compilar BASIC
BORRAR	Borrar archivos y liberar almacenamiento
BSC	Configurar las comunicaciones 2780/3780
BUSCAR	Comprobar si existe un archivo
BUZON	Leer y eliminar los mensajes almacenados
CAMBIAR	Cambiar el privilegio mínimo de un comando

Comando	Finalidad
CAMCU	Cambiar el número de una cuenta existente
CAMPRO	Transferir la propiedad de archivos privados
CARGAR	Ejecutable ausente; entrada conservada en la tabla de CSI
CEDIT	Editar registros directos, indexados o con clave
CIFRA	Calcular un valor de comprobación de un archivo
CLASIF	Ejecutable ausente; no es un alias de SORT
CLAVIN	Función auxiliar CLAVE.CUENTA instalada; al iniciarse devuelve CR 254
COMPART	Conceder, revocar o listar el acceso compartido
CONECTAR	Conectarse a otra cuenta
CONSULTA	Entrar en el lenguaje de consulta en español
COPFICH	Copiar registros con clave entre archivos
COPIFI	Copiar archivos
CORLUZ	Volver a contar los registros indexados o con clave
CREAR	Crear archivos estructurados
CUENTA	Mantener las cuentas de usuario
DESCARGAR	Ejecutable ausente; nombre de búsqueda DESCARGA
DIRIBM	Listar un disquete IBM-3741
DUPLICAR	Duplicar disquetes a través del almacenamiento en disco
EDITAR	Editar texto secuencial
ELIMINAR	Eliminar una entrada de catálogo dañada
ENVIAR	Enviar un archivo de forma síncrona
ENVIARA	Enviar un archivo de forma asíncrona
ESPIAR	Mostrar la actividad de la consola de otro usuario
ESTADO	Examinar los ajustes y el estado del sistema
EXEC	Ejecutar un archivo de comandos
FIN	Desconectar la cuenta
FISDEF	Mostrar las pistas físicas defectuosas
FORZAR	Inyectar un comando en otra partición
GENSIST	Guardar la configuración actual del sistema

Comando	Finalidad
HASH	Examinar la distribución de claves
IBMSEC	Importar registros IBM-3741
INICDISC	Inicializar un disco o cambiar su etiqueta
INICIAR	Inicializar un disquete
INICIBM	Escribir cabeceras de disquete compatibles con IBM
INITTAPE	Inicializar o borrar un cartucho
LCOMPART	No disponible: tema de ayuda sin ejecutable
LISDIR	Listar las entradas del catálogo
LISTAR	Mostrar o imprimir un archivo
LOGDEF	Mostrar las pistas lógicas defectuosas
MENSAJE	Enviar mensajes en directo o almacenados
MODIMPR	Cambiar las dimensiones de la impresora
MONTAR	Notificar al sistema un cambio de soporte
O8PRECOM	Utilidad de precompensación de disco y migración
ORGDIR	Reorganizar el catálogo
PANTALLA	Diseñar pantallas de entrada de datos
PARAR	Detener una partición
PONER	Cambiar los ajustes del sistema o de la sesión
RECARGAR	Restaurar archivos, cuentas o volúmenes archivados
RECIBIR	Recibir archivos síncronos
RECIBIRA	Recibir archivos asíncronos
REPARAR	Comprobar y conciliar los metadatos del sistema de archivos
REVIVIR	Recuperar una entrada de catálogo ASCII borrada
RUN	Ejecutar BASIC compilado
SECIBM	Exportar registros secuenciales a IBM-3741
SECTOR	Listar los sectores ocupados por un archivo
SERIAL	Editar la vinculación de un ejecutable a un número de serie
SORT	Ordenar registros por claves
SPOOLER	Gestionar la impresión diferida

Comando	Finalidad
TECLAS	Emitir secuencias de programación del terminal
TEDITAR	Editar archivos pequeños
TERMINAL	Usar una consola como terminal remoto
TITULO	Imprimir rótulos en letras grandes
UNOSEC	Reconstruir el sector lógico 1
VERFORM	Comprobar el formato de los soportes
VOLCAR	Volcar los bytes de un archivo o de un sector lógico

ACTIVAR — activar una partición o un comando en segundo plano

Activar una partición o un comando en segundo plano.

Sintaxis

```
ACTIVAR partición (PANn)
```

Opciones. Consolas de entrada y salida y opciones de dispositivo de ASOCIAR; la ayuda del comando también describe un comando en la primera partición libre.

Privilegio mínimo. 2.

Ejemplo

```
ACTIVAR 2 (PAN2)
```

Después de asignar la partición 2 como se describe en el capítulo 12, este comando la activa en el terminal PAN2. La asignación debe preceder a la activación.

Escribe PANn entre paréntesis. Los números de partición mostrados y los números de módulo de memoria son distintos.

ARCHDIR — listar un disquete de archivado

Listar un disquete de archivado.

Sintaxis

```
ARCHDIR unidad [(opciones)]
```

Opciones. IMP imprime; VOL n empieza en el volumen de archivado seleccionado (el predeterminado es 1).

Privilegio mínimo. 1.

Ejemplo

```
ARCHDIR A
```

Con un soporte de archivado en el dispositivo asociado con A, este comando lista el contenido del archivado. Un catálogo de archivado es distinto de un listado normal de LISDIR.

ARCHIVAR — archivar archivos, cuentas o volúmenes

Archivar archivos, cuentas o volúmenes.

Sintaxis

```
ARCHIVAR [nombre.tipo:]origen,destino [(opciones)]
```

Opciones. Dos fechas acotan la selección; CUENTA selecciona la cuenta actual y VOLUM, todas las cuentas; PANT/NOPANT, VERIFI/NOVERIFI y CONSU/NOCONSU controlan el listado, la verificación y las preguntas.

Privilegio mínimo. 2.

> **Precaución — escribe en el soporte de destino del archivado.**

Ejemplo

```
ARCHIVAR NOTA.TEXTO:S,A (VERIFI CONSU)
```

Este ejemplo de sintaxis selecciona NOTA.TEXTO en S para un archivado en A, solicita la verificación y mantiene la pregunta de confirmación. Requiere un dispositivo de archivado compatible y un soporte de destino prescindible; no está especificado un procedimiento completo para el dispositivo físico.

ASIGNAR — asignar canales

Asignar canales.

Sintaxis

```
ASIGNAR [canal dispositivo-o-archivo] [DESASI]
```

Opciones. Canales 1 a 16; CONSOLA o nombre.tipo:unidad; DESASI elimina las asignaciones.

Privilegio mínimo. 0.

Ejemplo

```
ASIGNAR 1 CONSOLA
```

Este comando asigna el canal de aplicación 1 a CONSOLA. No cambia qué terminal físico designa CONSOLA; de esa relación se encarga ASOCIAR.

ASOCIAR — asociar dispositivos lógicos y físicos

Asociar dispositivos lógicos y físicos.

Sintaxis

```
ASOCIAR [lógico [físico [(opciones)]]]
```

Opciones. L longitud de línea; P longitud de página; B baudios; C clase; retardos LF/FF; paridad PE/PO; protocolo ENAB; PP paridad de página; O líneas en blanco; ALF LF automático; H carga de cabezas; R reintentos; STP búsqueda; STS latencia; PUBLIC dispositivo público.

Privilegio mínimo. 0.

Ejemplo

```
ASOCIAR
```

Sin argumentos, el comando lista las asociaciones actuales entre dispositivos lógicos y físicos. Examina esta lista antes de cambiar el destino de un disco o de una impresora.

ATRIB — renombrar o proteger archivos

Renombrar o proteger archivos.

Sintaxis

```
ATRIB antiguo [nuevo] [(opciones)]
```

Opciones. PANT/NOPANT; CONSUL/NOCONSUL; E/NOE protección contra escritura; B/NOB protección contra borrado; LE protección contra lectura (también se acepta L; LEER no).

Privilegio mínimo. 1.

Ejemplo

```
ATRIB COPIA.TEXTO:S OTRO.TEXTO:S (NOCONSUL)
```

Este comando cambia el nombre de la copia de práctica a OTRO.TEXTO y suprime la pregunta de confirmación. Comprueba que el destino no esté en uso. La asignación se conserva; no se crea una segunda copia de los datos.

E, B y la protección contra lectura se aplican a rutas de acceso distintas. La sustitución secuencial puede eludir E borrando y volviendo a crear la entrada; véase el capítulo 11.

Para cambiar atributos sin cambiar el nombre, omite el segundo nombre de archivo. Para un archivo de práctica cuyo propietario sea la cuenta actual:

```
ATRIB NOTA.TEXTO:S (E NOCONSUL)
```

E activa la protección contra escritura; la indicación de formato del archivo secuencial de práctica cambia de S a SW. Para desactivar esa protección:

```
ATRIB NOTA.TEXTO:S (NOE NOCONSUL)
```

NOE devuelve la indicación a S. Examina cada resultado con LISDIR. Repetir el mismo nombre de archivo en los dos argumentos no es la forma de solo atributos y produce Error sintactico, CR = 8.

AYUDA — mostrar la ayuda de los comandos

Mostrar la ayuda de los comandos.

Sintaxis

```
AYUDA [comando]
```

Opciones. Sin tema, muestra el resumen; con un tema coincidente, normalmente muestra la ayuda de ese comando. Si la búsqueda falla, puede invocar la utilidad indicada con AYUD.

Privilegio mínimo. 0.

Ejemplo

```
AYUDA LISDIR
```

Este comando muestra el tema de ayuda de LISDIR. Describe el comando, pero no realiza el listado del directorio.

Si la búsqueda falla, AYUDA puede invocar la utilidad indicada con AYUD. AYUDA BORRAR sigue esta ruta y muestra Falta tipo del fichero, CR = 8; no muestra el tema esperado. Consulta la entrada de BORRAR para obtener instrucciones de borrado. El ejemplo de LISDIR, que funciona, no establece que todas las peticiones de tema sean pasivas.

BACKUP — copiar un soporte completo de disco o de cinta

Copiar un soporte completo de disco o de cinta.

Sintaxis

```
BACKUP [origen destino]
```

Opciones. Los valores predeterminados originales son S y A.

Privilegio mínimo. 3.

> **Precaución — sustituye el contenido del destino.**

Ejemplo

```
BACKUP S A
```

Este ejemplo de sintaxis copia el soporte de origen completo S en el destino A. Solo es adecuado cuando A es un destino de copia de seguridad compatible, configurado correctamente y cuyo contenido puede sustituirse. No confíes en los dispositivos predeterminados cuando se omiten.

BASIC — editar, interpretar o compilar BASIC

Editar, interpretar o compilar BASIC.

Sintaxis

```
BASIC [nombre.tipo:unidad] [(opciones)]
```

Opciones. LIST/NOLIST listado; IMPRn impresora; XREF referencias; NOOBJ suprime la generación del BASICCOM; NOPANT suprime la consola; OBJ=unidad ubicación de la salida; TRACE pasadas de compilación y números de línea. COMMA se anuncia en la ayuda, pero esta versión la rechaza.

Privilegio mínimo. 2.

Ejemplo

```
BASIC HOLA.BASICOBJ:S
```

Este comando compila el código fuente BASIC guardado en HOLA.BASICCOM:S. Primero crea y guarda el código fuente como se explica en el capítulo 15. BASIC sin nombre de archivo entra en el editor/intérprete.

NOOBJ controla la salida compilada independientemente de LIST. Para mostrar el código fuente sin generar el BASICCOM:

```
BASIC HOLA.BASICOBJ:S (LIST NOOBJ)
```

NOOBJ por sí sola puede mostrar estadísticas de compilación sin listar el código fuente. Puede quedar un BASICCOM anterior; no debe confundirse con la salida de esta invocación.

En esta versión, tanto COMMA como la grafía alternativa COMA producen Opcion no valida, CR = 8, como opciones del compilador. Usa el punto decimal en el código fuente; no se ha establecido que exista una opción del compilador para la coma decimal que funcione. El apéndice D mantiene la forma anunciada con esta restricción.

BORRAR — borrar archivos y liberar almacenamiento

Borrar archivos y liberar almacenamiento.

Sintaxis

```
BORRAR nombre.tipo[:unidad] [(opciones)]
```

Opciones. PANT/NOPANT nombres; CONSUL/NOCONSUL confirmación en la selección con comodines. El borrado por nombre exacto es inmediato incluso con CONSUL.

Privilegio mínimo. 1.

> Precaución — borrado inmediato. La selección por nombre exacto no se detiene a pedir confirmación.

Ejemplo

Examina el nombre de archivo exacto con LISDIR antes de continuar. Este ejemplo borra de inmediato, aunque se indique CONSUL.

```
BORRAR OTRO.TEXTO:S
```

Este comando borra inmediatamente el archivo de práctica indicado y libera su almacenamiento. Esta forma con nombre exacto no se detiene a pedir confirmación, ni siquiera con CONSUL. Examina el nombre de archivo exacto con LISDIR antes de introducir el comando de borrado.

CONSUL puede solicitar confirmación en la ruta de selección con comodines: N rechaza un archivo seleccionado y S permite borrarlo. Ese comportamiento no se aplica al ejemplo anterior con nombre exacto. No introduzcas un comodín solo para obtener una pregunta de confirmación; amplía la selección.

BSC — configurar las comunicaciones 2780/3780

Configurar las comunicaciones 2780/3780.

Sintaxis

```
BSC [archivo-de-parámetros] [AYUDA] [(opciones)]
```

Opciones. COMU1 selecciona el canal lógico; MODIF edita o crea los parámetros de protocolo guardados.

Privilegio mínimo. 1.

Ejemplo

```
AYUDA BSC
```

Este comando muestra el tema de ayuda central de BSC sin establecer ninguna conexión de comunicaciones. BSC AYUDA solicita la presentación de las opciones propias de la utilidad, pero requiere que COMU1 esté asociado incluso para esa petición. Si no lo está, el comando muestra COMU1 no esta asociado, CR = 16. El funcionamiento del protocolo requiere además un controlador compatible y un equipo remoto; la dependencia de DIS27 no es un componente que se garantice instalado.

BUSCAR — comprobar si existe un archivo

Comprobar si existe un archivo.

Sintaxis

```
BUSCAR [nombre.tipo[:unidad]] [(opción)]
```

Opciones. PANT muestra el resultado (valor predeterminado); NOPANT lo suprime; PUBLIC incluye los archivos públicos.

Privilegio mínimo. 0.

Ejemplo

```
BUSCAR NOTA.TEXTO:S (PANT)
```

Este comando busca el archivo de práctica y muestra el resultado. Un resultado negativo puede deberse a la cuenta o a la asociación actuales, no a que el archivo no exista en ningún almacenamiento.

BUZON — leer y eliminar los mensajes almacenados

Leer y eliminar los mensajes almacenados.

Sintaxis

```
BUZON
```

Opciones. No hay opciones establecidas.

Privilegio mínimo. 0.

Ejemplo

```
BUZON
```

Este comando recupera los mensajes de la cuenta conectada. La recuperación elimina los mensajes almacenados seleccionados, así que anota lo que necesites antes de terminar.

CAMBIAR — cambiar el privilegio mínimo de un comando

Cambiar el privilegio mínimo de un comando.

Sintaxis

```
CAMBIAR nombre.tipo:unidad (PRIV n)
```

Opciones. PRIV indica el nuevo umbral.

Privilegio mínimo. 3.

> **Precaución — cambia la política del ejecutable.**

Ejemplo

```
CAMBIAR DEMO.COMANDO:S (PRIV 4)
```

Esta ilustración de mantenimiento eleva a 4 el privilegio mínimo estándar del ejecutable de práctica seleccionado. Úsala solo con una copia desechable de un comando, no con una utilidad esencial del sistema. No eleva tu propio privilegio.

CAMCU — cambiar el número de una cuenta existente

Cambiar el número de una cuenta existente.

Sintaxis

```
CAMCU
```

Opciones. Interfaz interactiva.

Privilegio mínimo. 5.

Ejemplo

```
CAMCU
```

Este comando entra en el mantenimiento interactivo de números de cuenta. Quedan excluidos la cuenta 0/SISTEMA y sus sinónimos, así como los números superiores a 254. El diálogo completo y sus efectos en todos los archivos con propietario no están especificados, así que no se prescribe ninguna secuencia de renumeración.

CAMPRO — transferir la propiedad de archivos privados

Transferir la propiedad de archivos privados.

Sintaxis

```
CAMPRO nombre.tipo[:unidad] (cuenta [opciones])
```

Opciones. PANT/NOPANT y CONSUL/NOCONSUL.

Privilegio mínimo. 3.

> **Precaución — transfiere la propiedad de archivos.**

Ejemplo

Antes de este ejemplo, conéctate como el propietario de NOTA.TEXTO distinto de SISTEMA y comprueba que su privilegio sea al menos 3. La cuenta de destino SISTEMA no es la cuenta desde la que se selecciona el archivo privado de origen.

```
CAMPRO NOTA.TEXTO:S (SISTEMA NOCONSUL)
```

Ejecútalo mientras estés conectado como el propietario actual de NOTA.TEXTO distinto de SISTEMA, con privilegio 3 o superior. Transfiere el archivo a SISTEMA. La identidad de administración por sí sola no selecciona los archivos privados de otro propietario: desde la cuenta equivocada, el comando puede mostrar No se ha cambiado ningun fichero, CR = 12. Después de la transferencia, examina la entrada y el contenido en la cuenta de destino. La cuenta 0 participa en el mecanismo alternativo de archivos públicos, así que la transferencia a SISTEMA no garantiza que el propietario anterior pierda el acceso por búsqueda.

CARGAR — entrada de la tabla de comandos; ejecutable ausente

CSI reconoce CARGAR y expande su abreviatura mínima CAR, pero el ejecutable necesario no está instalado. Tanto CARGAR solo como CARGAR SPOOLER fallan con un error de archivo no encontrado. La entrada de la tabla aporta una grafía, no una implementación.

SPOOLER.COMANDO contiene `CARGAR SPOOLER` en la ruta que se usa cuando no se encuentra el SPOOLER residente. Esto indica una función prevista de carga de componentes residentes; no establece el tipo del archivo de entrada, las reglas de asignación ni una interfaz general de carga. CARGAR no debe tratarse como un enlazador documentado.

Sintaxis y opciones. No se ha establecido ninguna interfaz utilizable.

Privilegio mínimo. Desconocido.

Ejemplo: consulta la ayuda disponible

```
AYUDA
```

AYUDA muestra los temas de ayuda disponibles. Es un ejemplo de ayuda, no una invocación de la utilidad CARGAR, que falta. Para la función de impresión instalada, consulta SPOOLER. Su componente residente puede estar ya presente aunque falte CARGAR.

CEDIT — editar registros directos, indexados o con clave

Editar registros directos, indexados o con clave.

Sintaxis

```
CEDIT
```

Opciones. Editor interactivo de registros descrito en la ayuda del comando.

Privilegio mínimo. 2.

Ejemplo

```
CEDIT
```

Este comando entra en el editor de registros estructurados. Puede modificar registros de archivos directos, indexados o con clave. Su diálogo completo de selección, guardado y salida no está especificado; no le apliques comandos de EDITAR.

CIFRA — calcular un valor de comprobación de un archivo

Calcular un valor de comprobación de un archivo.

Sintaxis

```
CIFRA nombre.tipo[:unidad] [(opción)]
```

Opciones. P consola (valor predeterminado); I impresora.

Privilegio mínimo. 4.

Ejemplo

```
CIFRA NOTA.TEXTO:S (P)
```

Este comando pide en la consola el valor de comprobación del archivo secuencial de práctica. Anota el resultado si vas a comparar copias repetidas. El algoritmo no está especificado como un hash moderno ni como un CRC.

CLASIF — entrada de la tabla de comandos; ejecutable ausente

CSI reconoce CLASIF y expande su abreviatura mínima CL, pero no hay instalado ningún ejecutable ni archivo EXEC correspondiente. Invocar cualquiera de las dos grafías falla; CSI no lo redirige a SORT. Se desconoce si CLASIF era un nombre anterior, una utilidad opcional o una entrada obsoleta de la tabla.

Sintaxis y opciones. No se ha establecido ninguna interfaz utilizable. Las opciones de SORT no pueden atribuirse a CLASIF.

Privilegio mínimo. Desconocido.

Ejemplo: consulta la utilidad de ordenación instalada

```
AYUDA SORT
```

Este comando muestra la ayuda de SORT, cuyo ejecutable y sus dos superposiciones están instalados. Continúa con la referencia de SORT para ver su propia sintaxis y sus ejemplos. CLASIF no es un alias de SORT.

Las palabras CLASIFICAR y ORDENAR, y las referencias a un archivo CLASIF, pertenecen a un contexto independiente dentro de CONSULTA. No hacen de CLASIF un comando de CSI disponible.

CLAVIN — función auxiliar CLAVE.CUENTA instalada; fallo al iniciarse

CLAVIN.COMANDO está instalado y se carga, pero las invocaciones tanto sin argumentos como con un argumento de prueba devuelven `CR = 254` durante el inicio, incluso con SISTEMA y privilegio 5. Este fallo se produce antes de la operación principal aparente; ninguna de las dos formas establece una interfaz utilizable.

El ejecutable se identifica como `CLAVIN ver 1.0 01JUN83`. Su código contiene rutas independientes que parecen destinadas a escribir un argumento en CLAVE.CUENTA y a leer y mostrar un valor cuando no se indica ningún argumento. Junto con CLAVOUT.BASICUSR, parece intercambiar una cadena con recuento o un valor a través de ese archivo. Siguen siendo desconocidos el significado del valor para las aplicaciones y el formato previsto del argumento.

Sintaxis y opciones. No hay establecida ninguna sintaxis operativa que funcione; no hay ningún tema de ayuda central disponible.

Privilegio mínimo. 5, según especifica el prólogo del ejecutable. Cumplir este requisito no evita el fallo al iniciarse.

Ejemplo: examina el ejecutable instalado

```
LISDIR CLAVIN.COMANDO:S
```

Este comando lista la entrada de catálogo de la utilidad en el disco del sistema. Establece que el archivo está presente; no establece que la utilidad funcione.

Invocación y fallo

```
CLAVIN
```

El ejecutable muestra su encabezado de versión y devuelve `CR = 254`. Esto documenta la invocación que falla, no una operación correcta de lectura de un valor. El mismo resultado con un argumento no demuestra que funcione una operación de escritura.

CLAVE.CUENTA no está presente en la instalación documentada. Crear un archivo vacío no corrige las incoherencias del código de inicio y de la reubicación. No se ha establecido ninguna reparación fiable; el apéndice C describe el ejecutable y su componente asociado de BASIC.

CLAVE.CUENTA no es la base de datos de contraseñas de inicio de sesión. Las credenciales de las cuentas están en SISTEMA.CUENTA; usa CUENTA para las operaciones documentadas con cuentas y contraseñas.

COMPART — conceder, revocar o listar el acceso compartido

Conceder, revocar o listar el acceso compartido.

Sintaxis

```
COMPART nombre.tipo[:unidad] [(PANT)]
```

Opciones. PANT muestra las cuentas con acceso compartido; el diálogo normal concede o revoca el acceso.

Privilegio mínimo. 3.

Ejemplo

Antes de este ejemplo, conéctate con una cuenta distinta de SISTEMA con privilegio 3 o superior y crea el archivo de práctica con esa identidad.

```
COMPART NOTA.TEXTO:S (PANT)
```

Conéctate como el propietario del archivo distinto de SISTEMA, con privilegio 3 o superior, antes de usar este ejemplo. Lista las cuentas a las que se ha concedido acceso; si no hay concesiones, lo esperado es un resultado vacío y CR = 0. SISTEMA queda excluida y recibe La cuenta sistema no puede compartir ficheros, CR = 12. Omite PANT solo cuando quieras iniciar el diálogo independiente para conceder o revocar el acceso. Las concesiones mantienen al propietario original.

CONECTAR — conectarse a otra cuenta

Conectarse a otra cuenta.

Sintaxis

```
CONECTAR cuenta
```

Opciones. La petición de contraseña y la ejecución del perfil de la cuenta dependen de la cuenta.

Privilegio mínimo. 0.

Ejemplo

```
CONECTAR SISTEMA
```

Este comando solicita la conexión a SISTEMA cuando su número de cuenta no lo está usando otra partición. Introduce su contraseña cuando se te pida. Una conexión duplicada se rechaza antes de comprobar la contraseña, con Esta cuenta esta siendo utilizada por la particion seguido del número de partición y CR = 10. Finaliza la conexión existente antes de volver a intentarlo; un alias con el mismo número de cuenta no evita esta restricción. Una conexión correcta puede ejecutar el perfil EXEC de esa cuenta.

CONSULTA — entrar en el lenguaje de consulta en español

Entrar en el lenguaje de consulta en español.

Sintaxis

```
CONSULTA [archivo-de-instrucciones]
```

Opciones. El archivo secuencial opcional aporta una consulta; si no se indica, la entrada es interactiva.

Privilegio mínimo. 1.

Ejemplo

```
CONSULTA
```

Este comando inicia la función interactiva de consulta en español. Su gramática de consulta completa no está especificada; introducir instrucciones SQL no es un sustituto admitido.

Referencia del lenguaje. Consulta el capítulo 35 para ver el vocabulario de este componente.

COPFICH — copiar registros con clave entre archivos

Copiar registros con clave entre archivos.

Sintaxis

```
COPFICH origen.nombre:unidad destino.nombre:unidad
```

Opciones. La ayuda del comando no incluye especificaciones con comodines.

Privilegio mínimo. 1.

Ejemplo

```
COPFICH INDICE.DATOS:S DESTINO.DATOS:S
```

Este ejemplo de sintaxis copia registros entre archivos indexados o con clave compatibles. Prepara primero el destino con la organización necesaria. La interfaz documentada no acepta comodines; el comportamiento ante colisiones no está especificado por completo.

COPIFI — copiar archivos

Copiar archivos.

Sintaxis

```
COPIFI origen destino [(opciones)]
```

Opciones. PANT/NOPANT; SISFECH/ANTFECH marcas de tiempo; DESD n, DSTRING texto, NUM n, HSTRING texto selección; TRUNC/NOTRUNC; PACK/UNPACK; LMINUSC/MAYUSC; NOREEMP/REEMPLA; A@ADIR adición; MONTA; VERIFI/NOVERIFI; CONSU/NOCONSU; PUBLIC; TABLA conversión.

Privilegio mínimo. 1.

Ejemplo

```
COPIFI NOTA.TEXTO:S COPIA.TEXTO:S (VERIFI)
```

Este comando copia NOTA.TEXTO en un nuevo COPIA.TEXTO y solicita la verificación. La especificación de origen va primero. Examina después el destino.

CORLUZ — volver a contar los registros indexados o con clave

Volver a contar los registros indexados o con clave.

Sintaxis

```
CORLUZ nombre.tipo[:unidad]
```

Opciones. No hay opciones establecidas.

Privilegio mínimo. 0.

> **Precaución — actualiza los metadatos del directorio.**

Ejemplo

```
CORLUZ INDICE.DATOS:S
```

Este comando vuelve a contar los registros de un archivo con clave o indexado y actualiza su recuento en el directorio. Haz antes una copia de seguridad: es una corrección de metadatos, no un simple listado.

CREAR — crear archivos estructurados

Crear archivos estructurados.

Sintaxis

```
CREAR nombre.tipo:unidad (opciones)
```

Opciones. DIRECTO, INDEXADO o CLAVE; NUMREG número; REGISTRO longitud; LONCLA longitud de la clave; VACIAR vacía un archivo existente.

Privilegio mínimo. 1.

Ejemplo

```
CREAR DIRECTO.DATOS:S (DIRECTO REGISTRO 32 NUMREG 10)
```

Este comando crea un nuevo archivo directo con diez registros de 32 bytes. DIRECTO selecciona la organización, REGISTRO indica la longitud del registro y NUMREG indica el número solicitado.

VACIAR vacía un archivo existente y destruye sus registros. Las capacidades y las fórmulas de asignación de I y K son distintas.

CUENTA — mantener las cuentas de usuario

Mantener las cuentas de usuario.

Sintaxis

```
CUENTA [(opción)]
```

Opciones. PANT; IMPRIMIRn; VACIAR historial; ELIMINAR sistema de cuentas; COPIAR=unidad; HISTORI.

Privilegio mínimo. 5.

Ejemplo

```
CUENTA (PANT)
```

Este comando muestra las cuentas registradas, incluida la columna de contraseñas. Mantén el listado en privado. CUENTA sin opciones entra en el mantenimiento: N añade una cuenta, C cambia su contraseña y su privilegio, B elimina una cuenta con nombre y F escribe la lista revisada y termina. Con el privilegio 5, usa la cuenta SISTEMA. El capítulo 10 ofrece procedimientos paso a paso independientes para la creación, los cambios de contraseña y la eliminación.

ELIMINAR elimina el sistema de cuentas; VACIAR borra el historial de cuentas. Ninguno de los dos es un comando normal para eliminar una sola cuenta.

DESCARGAR — entrada de la tabla de comandos; ejecutable ausente

DESCARGAR es el nombre original de nueve letras en la tabla de abreviaturas de CSI. El campo de búsqueda de comandos solo admite ocho caracteres, así que el nombre de búsqueda expandido es DESCARGA. No hay instalado ni un ejecutable COMANDO coincidente ni un archivo EXEC, y la invocación falla.

La abreviatura mínima es DESCAR; DESCARG y DESCARGA también son prefijos aceptados. El reconocimiento de estas grafías no aporta el programa que falta. Un error que menciona DESCARGA.COMANDO después de introducir DESCARGAR refleja la longitud del campo de búsqueda.

Sintaxis, finalidad y opciones. Desconocidas. En concreto, el nombre no establece una operación de descarga ni hace que DESCARGAR equivalga a VACIAR.

Privilegio mínimo. Desconocido.

Ejemplo: consulta la ayuda disponible

```
AYUDA
```

Este comando muestra el resumen de ayuda disponible. No es un ejemplo de descarga de un componente. SPOOLER.COMANDO contiene por separado `VACIAR SPOOLER`, pero el ejecutable independiente VACIAR también falta. Las opciones VACIAR de CUENTA, CREAR e INICIAR pertenecen a esos comandos concretos y son independientes de estos programas no disponibles.

DIRIBM — listar un disquete IBM-3741

Listar un disquete IBM-3741.

Sintaxis

```
DIRIBM unidad
```

Opciones. No hay opciones establecidas.

Privilegio mínimo. 0.

Ejemplo

```
DIRIBM A
```

Con un disquete IBM-3741 adecuado en A, este comando lista la información de su volumen y de sus archivos. Los campos de pista y sector usan la notación PPSSS; no es un listado del sistema de archivos nativo.

DUPLICAR — duplicar disquetes a través del almacenamiento en disco

Duplicar disquetes a través del almacenamiento en disco.

Sintaxis

```
DUPLICAR
```

Opciones. Requiere un archivo directo SISTEMA.DUPLICAR preparado.

Privilegio mínimo. Mínimo estándar desconocido.

> **Precaución — escribe en el disquete de destino.**

Ejemplo

```
DUPLICAR
```

Este comando entra en la duplicación de disquetes usando un área de trabajo en disco. Requiere un archivo directo SISTEMA.DUPLICAR preparado; el ejemplo documentado de área de trabajo es de 4000 registros de 256 bytes. El diálogo completo de intercambio de soportes no está especificado.

EDITAR — editar texto secuencial

Editar texto secuencial.

Sintaxis

```
EDITAR nombre.tipo[:unidad] [(COPIA o NOCOPIA)]
```

Opciones. COPIA crea una copia de seguridad de forma predeterminada; NOCOPIA la suprime.

Privilegio mínimo. 2.

Ejemplo

```
EDITAR NOTA.TEXTO:S
```

Este comando abre el editor de texto secuencial para el archivo indicado. En su indicador de asterisco, sigue los pasos de creación de archivos del capítulo 6; FILE guarda y sale.

Referencia del lenguaje. Consulta el capítulo 35 para ver el vocabulario de este componente.

ELIMINAR — eliminar una entrada de catálogo dañada

Eliminar una entrada de catálogo dañada.

Sintaxis

```
ELIMINAR nombre.tipo:unidad
```

Opciones. No hay opciones establecidas.

Privilegio mínimo. 3.

> **Precaución — elimina la entrada sin liberar el almacenamiento.**

Ejemplo

```
ELIMINAR ROTO.TEXTO:S
```

Esta ilustración de mantenimiento elimina una entrada dañada llamada ROTO.TEXTO sin liberar su almacenamiento. No es un borrado normal. Úsala solo cuando se hayan examinado la entrada y sus bloques asignados y exista una copia de seguridad.

ENVIAR — enviar un archivo de forma síncrona

Enviar un archivo de forma síncrona.

Sintaxis

```
ENVIAR archivo [(opciones)]
```

Opciones. NOEOT suprime el EOT final; COMUn selecciona el canal (predeterminado: COMU1).

Privilegio mínimo. 1.

Ejemplo

```
ENVIAR NOTA.TEXTO:S (COMU1)
```

Este comando selecciona el archivo para su transmisión síncrona por COMU1. Requiere un enlace de comunicaciones configurado y un extremo receptor compatible; no es una copia local.

ENVIARA — enviar un archivo de forma asíncrona

Enviar un archivo de forma asíncrona.

Sintaxis

```
ENVIARA archivo [(opciones)]
```

Opciones. La ayuda del comando lo identifica como equivalente asíncrono de ENVIAR; la equivalencia exacta de las opciones no está documentada de forma independiente.

Privilegio mínimo. 1.

Ejemplo

```
ENVIARA NOTA.TEXTO:S
```

Este comando selecciona el archivo para su transmisión asíncrona. La equivalencia exacta con el conjunto de opciones de ENVIAR no está especificada, así que este ejemplo no usa opciones adicionales.

ESPIAR — mostrar la actividad de la consola de otro usuario

Mostrar la actividad de la consola de otro usuario.

Sintaxis

```
ESPIAR cuenta-o-partición
```

Opciones. No hay opciones establecidas.

Privilegio mínimo. 5.

Ejemplo

```
ESPIAR 2
```

Este comando solicita observar la actividad de la consola de la partición 2. Úsalo solo para tareas de administración autorizadas. El comportamiento completo de terminación y de política no está especificado.

ESTADO — examinar los ajustes y el estado del sistema

Examinar los ajustes y el estado del sistema.

Sintaxis

```
ESTADO opción
```

Opciones. ALL, QUIEN, HORA, FECHA, SISTEMA, FORFECHA, PRIVILEG, RELOJ, DIA, MENSAJE, TIEMPO, USUARIOS, MEMORIA, MODULO, DISCO, ERRNUM, ERRTEXT, INCOM, ABREV, IMPEXEC, HISTOR, RTCAVAIL, SISFECHA, INIREL, QET, ALEMAN, ESCAPE, SUBIR, BAJAR, IZQU, DERECHA, RUBOUT, CANCEL, SINONIMO, VERSION. Cada una es una consulta de información; las formas abreviadas aceptadas se enumeran en el capítulo 35.

Privilegio mínimo. 0.

Ejemplo

```
ESTADO DISCO
```

Este comando informa del estado del disco, incluidos el espacio libre total y la mayor región libre contigua. Puede actualizar un recuento de espacio libre en caché. ESTADO requiere indicar qué se consulta; el comando solo es un error de sintaxis.

Lista las particiones

```
ESTADO MEMORIA
```

Este comando informa de las particiones de memoria asignadas, incluidas las asignaciones detenidas. Consulta el procedimiento de listado de particiones.

Lista los módulos de memoria

```
ESTADO MODULO
```

Muestra los módulos de memoria utilizables, que aportan el almacenamiento de las particiones.

Lista los usuarios conectados

```
ESTADO USUARIOS
```

Informa de las conexiones actuales de los usuarios. CUENTA (PANT), en cambio, lista las cuentas registradas.

EXEC — ejecutar un archivo de comandos

Ejecutar un archivo de comandos.

Sintaxis

```
EXEC nombre [argumentos]
```

Opciones. Las directivas del lenguaje y la sustitución se documentan en el capítulo 8.

Privilegio mínimo. Mínimo estándar desconocido.

Ejemplo

```
EXEC HOLA
```

Este comando ejecuta el archivo de comandos de saludo HOLA.EXEC creado en el capítulo 8. Usa el nombre EXEC completo para evitar el defecto de transferencia de argumentos de las abreviaturas.

Los campos normales tienen ocho caracteres; los argumentos se limitan a dieciséis campos. Usa BEGTYPE/END para textos literales largos.

Referencia del lenguaje. Consulta el capítulo 35 para ver el vocabulario de este componente.

FIN — desconectar la cuenta

Desconectar la cuenta.

Sintaxis

```
FIN
```

Opciones. No hay opciones establecidas.

Privilegio mínimo. 0.

Ejemplo

```
FIN
```

Después de guardar y cerrar los archivos de las aplicaciones, introduce esto en CSI para desconectar la cuenta. No apaga la máquina ni garantiza un vaciado global.

FISDEF — mostrar las pistas físicas defectuosas

Mostrar las pistas físicas defectuosas.

Sintaxis

```
FISDEF:número-de-unidad
```

Opciones. Se mantiene la forma con dos puntos de la ayuda a la espera de su verificación nativa.

Privilegio mínimo. Mínimo estándar desconocido.

Ejemplo

```
FISDEF:3
```

Esta es la ilustración documentada con la forma de dos puntos para la unidad física número 3. Confirma la numeración de las unidades antes de usarla. Solicita información sobre las pistas físicas defectuosas; la forma con dos puntos no es una regla general de sintaxis de dispositivos.

FORZAR — inyectar un comando en otra partición

Inyectar un comando en otra partición.

Sintaxis

```
FORZAR partición comando
```

Opciones. No hay opciones establecidas.

Privilegio mínimo. 5.

> **Precaución — puede interrumpir trabajo sin guardar.**

Ejemplo

```
FORZAR 2 FIN
```

Este comando fuerza el comando FIN en la partición 2. Puede finalizar el trabajo antes de que se guarde un registro completo de la aplicación. Resérvalo para la recuperación cuando no sea posible cerrar la sesión normalmente.

GENSIST — guardar la configuración actual del sistema

Guardar la configuración actual del sistema.

Sintaxis

```
GENSIST
```

Opciones. La ayuda menciona ASOCIAR y determinados ajustes de PONER.

Privilegio mínimo. 5.

> **Precaución — escribe en SISTEMA.NUCLEO.**

Ejemplo

```
GENSIST
```

Este comando guarda en SISTEMA.NUCLEO determinados ajustes actuales de asociaciones y del sistema. Haz antes una copia de seguridad y usa una configuración intencionada; las reglas completas de persistencia no están especificadas.

HASH — examinar la distribución de claves

Examinar la distribución de claves.

Sintaxis

```
HASH [I]
```

Opciones. I imprime estadísticas; el diálogo pregunta la capacidad, el archivo o las claves manuales.

Privilegio mínimo. Mínimo estándar desconocido.

Ejemplo

```
HASH
```

Este comando entra en el diálogo para examinar la distribución y la colocación de las claves. Pide la capacidad y la entrada desde archivo o de claves manuales. Su nombre no designa una suma de comprobación criptográfica de archivos; usa CIFRA para la operación independiente de valor de comprobación de archivos.

IBMSEC — importar registros IBM-3741

Importar registros IBM-3741.

Sintaxis

```
IBMSEC destino.nombre:unidad unidad-origen [(opciones)]
```

Opciones. Nombre de origen opcional; REEM sustituye el destino.

Privilegio mínimo. 3.

Ejemplo

```
IBMSEC IMPORTA.TEXTO:S A
```

Este ejemplo de sintaxis importa registros de un disquete IBM-3741 en A a IMPORTA.TEXTO en S, convirtiendo EBCDIC en ASCII. Prepara un soporte compatible y elige un destino que no esté en uso.

INICDISC — inicializar un disco o cambiar su etiqueta

Inicializar un disco o cambiar su etiqueta.

Sintaxis

```
INICDISC unidad (opciones)
```

Opciones. ETIQU solicita un cambio de etiqueta; VACIAR borra los archivos; DIRE solicita entradas de directorio; WP/NOWP cambia la protección contra escritura por software; FORMAT solicita el formateo físico; CARA especifica las cabezas; PISTA especifica los cilindros/pistas por cabeza y puede afectar a la ubicación del volumen; SECTOR especifica los sectores por pista; BOOT=unidad aporta un origen para el cargador de arranque; INCR selecciona el parámetro de incremento/entrelazado del formato; su efecto completo, específico del hardware, no está especificado.

Privilegio mínimo. 3.

> **Precaución — la inicialización o el formateo pueden destruir el contenido.**

Ejemplo

```
INICDISC A (FORMAT CARA 5 PISTA 397 SECTOR 32 DIRE 64)
```

Este comando formatea un volumen prescindible y configurado correctamente en A con cinco cabezas, 397 cilindros y 32 sectores por pista, y solicita capacidad de directorio. No es una geometría universal. En DISCO2, PISTA establece también el cilindro inicial: 397 da los cilindros 397 a 793. DIRE 64 puede ajustarse a 88 entradas. El procedimiento del capítulo 11 explica la asociación, las comprobaciones de geometría, cada respuesta del diálogo, el montaje y la verificación de archivos.

INICIAR — inicializar un disquete

Inicializar un disquete.

Sintaxis

```
INICIAR unidad (opciones)
```

Opciones. ETIQU, VACIA, DIRE, WP/NOPE, FORMAT, CARA, PISTA, SECTOR, BOOT=unidad.

Privilegio mínimo. 3.

> **Precaución — la inicialización o el formateo pueden destruir el contenido.**

Ejemplo

```
INICIAR A (FORMAT CARA 2)
```

Este comando solicita el formato de disquete de doble cara y doble densidad de la utilidad, con el resto de los valores de geometría predeterminados. A debe designar el disquete prescindible previsto, y la unidad y el soporte deben admitir ese formato. Destruye el contenido existente. El capítulo 11 explica las opciones de geometría y las comprobaciones de acceso posteriores.

INICIBM — escribir cabeceras de disquete compatibles con IBM

Escribir cabeceras de disquete compatibles con IBM.

Sintaxis

```
INICIBM unidad
```

Opciones. No hay opciones establecidas.

Privilegio mínimo. 3.

> **Precaución — cambia las cabeceras del soporte.**

Ejemplo

```
INICIBM A
```

Este comando escribe en A cabeceras de disquete compatibles con IBM para operaciones compatibles posteriores. La información existente puede alterarse; el ejemplo requiere un soporte prescindible.

INITTAPE — inicializar o borrar un cartucho

Inicializar o borrar un cartucho.

Sintaxis

```
INITTAPE [(TENSION)]
```

Opciones. TENSION recorre y rebobina el soporte.

Privilegio mínimo. 1.

> **Precaución — destruye la información existente en la cinta.**

Ejemplo

```
INITTAPE (TENSION)
```

Este comando inicializa la unidad de cinta seleccionada y solicita el tensado, que recorre y rebobina el cartucho. Destruye la información existente en la cinta y requiere una controladora compatible.

LCOMPART — utilidad de uso compartido y cuentas no disponible

Disponibilidad. No hay ningún ejecutable COMANDO coincidente ni ninguna entrada en la tabla de abreviaturas estándar. El tema de ayuda describe cómo examinar o reconstruir el uso compartido y las cuentas; no establece ningún procedimiento de uso disponible.

Sintaxis

```
LCOMPART unidad [(opciones)]
```

Opciones. BORRA elimina el uso compartido; IMP imprime; CUENTA crea las cuentas que faltan con privilegio 0; PRIVn selecciona de 1 a 5.

Privilegio mínimo. Mínimo estándar desconocido.

Ejemplo

```
LCOMPART S (IMP)
```

Esta ilustración de sintaxis histórica describe un informe impreso de uso compartido en S. No puede ejecutarse con los comandos disponibles: la utilidad correspondiente falta, así que no es un procedimiento de recuperación. Las formas con BORRA/CUENTA modificarían además el estado del uso compartido o de las cuentas.

LISDIR — listar las entradas del catálogo

Listar las entradas del catálogo.

Sintaxis

```
LISDIR [nombre.tipo][:unidad] [(opciones)]
```

Opciones. PRINTn; EXEC/FICH archivos de selección; AP adición; FN/FT/FD/DIA/ALOC/FORMAT/ LETIQUETA detalle; SORT/NOSORT; OCUENTA; HCAB/NOHCAB; PUBLIC; límite inferior de fecha; * todo el detalle.

Privilegio mínimo. 0.

Ejemplo

```
LISDIR *.*:S
```

Este comando lista las entradas de catálogo visibles en S. Cada asterisco selecciona todos los valores de su campo. No muestra el contenido de los archivos ni garantiza la visibilidad de los archivos privados de otras cuentas.

OCUENTA es la selección privilegiada de todas las cuentas. NOHCAB puede anular OCUENTA, así que no los combines cuando necesites un listado completo de todas las cuentas.

LISTAR — mostrar o imprimir un archivo

Mostrar o imprimir un archivo.

Sintaxis

```
LISTAR nombre.tipo[:unidad] [(opciones)]
```

Opciones. PRINTn; primer/último registro; HEAD/NOHEAD; FORMAT; INSER margen; TRUNC longitud; PAG primera página; TITULO encabezado.

Privilegio mínimo. 0.

Ejemplo

```
LISTAR NOTA.TEXTO:S
```

Este comando muestra el archivo de texto de práctica. Usa LISDIR cuando necesites sus metadatos de catálogo en lugar de su contenido.

LOGDEF — mostrar las pistas lógicas defectuosas

Mostrar las pistas lógicas defectuosas.

Sintaxis

```
LOGDEF número-de-unidad
```

Opciones. No hay opciones establecidas.

Privilegio mínimo. Mínimo estándar desconocido.

Ejemplo

```
LOGDEF 3
```

Esta ilustración muestra el informe de pistas lógicas defectuosas de la unidad 3. Confirma antes la identidad de la unidad. La información sobre pistas lógicas de sustitución es distinta del informe físico de FISDEF.

MENSAJE — enviar mensajes en directo o almacenados

Enviar mensajes en directo o almacenados.

Sintaxis

```
MENSAJE cuenta-o-partición-o-* [texto]
```

Opciones. Si se omite el texto, se entra en el modo de varias líneas.

Privilegio mínimo. 0.

Ejemplo

```
MENSAJE 2 HOLA
```

Este comando envía el texto breve HOLA a la partición 2. Omite el texto para entrar en un diálogo de varias líneas. Lee el diagnóstico, porque un resultado de finalización cero no garantiza la entrega en directo.

MODIMPR — cambiar las dimensiones de la impresora

Cambiar las dimensiones de la impresora.

Sintaxis

```
MODIMPR columnas líneas
```

Opciones. La ayuda limita ambas dimensiones a valores estrictamente comprendidos entre 0 y 255.

Privilegio mínimo. 1.

Ejemplo

```
MODIMPR 80 60
```

Este comando solicita unas dimensiones de impresora de 80 columnas y 60 líneas. Ambas deben ser mayores que cero y menores que 255. La interfaz está documentada, pero los efectos completos específicos de cada controlador no están especificados.

MONTAR — notificar al sistema un cambio de soporte

Notificar al sistema un cambio de soporte.

Sintaxis

```
MONTAR unidad-o-*
```

Opciones. * solicita todas las unidades.

Privilegio mínimo. 0.

Ejemplo

```
MONTAR A
```

Después de sustituir el soporte en el dispositivo asociado, este comando actualiza la visión que tiene el sistema de A. No es una petición para crear un punto de montaje de directorio.

O8PRECOM — utilidad de precompensación de disco y migración

O8PRECOM cambia el estado de precompensación de la controladora de disco. CAMBVERS.EXEC lo invoca durante una migración de 6.1/6.1A a 6.2. No es un precompilador de lenguaje. Su ruta de controladora escribe datos en el disco; los parámetros completos y el comportamiento de recuperación siguen sin especificarse. Solo se ofrece un ejemplo de inspección.

Sintaxis

```
No hay sintaxis operativa documentada
```

Opciones. No hay tema de ayuda central.

Privilegio mínimo. Mínimo estándar desconocido.

Ejemplo

```
LISDIR O8PRECOM.COMANDO:S
```

Este comando examina la entrada de catálogo de la utilidad sin iniciar su ruta de escritura en la controladora. Su función histórica en la migración no ofrece una receta de mantenimiento general segura; no se prescribe ningún flujo de ejecución ni de compilación.

Límite de la interfaz. El nombre de archivo por sí solo no aporta el contrato de uso que falta.

ORGDIR — reorganizar el catálogo

Reorganizar el catálogo.

Sintaxis

```
ORGDIR unidad [(RELAN)]
```

Opciones. Según la ayuda, RELAN reanuda tras una reorganización interrumpida.

Privilegio mínimo. 5.

> **Precaución — reescribe las entradas del directorio.**

Ejemplo

```
ORGDIR A
```

Esta ilustración de mantenimiento reorganiza el directorio de A. Reescribe las entradas y puede descartar las entradas borradas y las entradas secuenciales sin terminar. Haz una copia de seguridad y revisa los archivos dañados antes de usarla.

PANTALLA — diseñar pantallas de entrada de datos

Diseñar pantallas de entrada de datos.

Sintaxis

```
PANTALLA [nombre-de-pantalla]
```

Opciones. Controles incrustados: ;I incluir, ;M modificar, ;B borrar campo, ;S guardar, ;F terminar, ;G/;R guardar/recuperar campo, ;D/;H mover, ;P cuadrícula, ;C centrar, ;N número, ;L cursor, ;A ayuda.

Privilegio mínimo. 1.

Ejemplo

```
PANTALLA AYUDA
```

Este comando solicita la rama de ayuda del diseñador de pantallas. El editor crea una salida .PANT, pero su comportamiento completo de guardado y sustitución no está especificado. No des por supuesto que los archivos con otros tipos relacionados con pantallas sean intercambiables.

Los controles del editor incluyen ;I incluir, ;M modificar, ;B borrar campo, ;S guardar, ;F terminar, ;G/;R guardar/recuperar campo, ;D/;H mover, ;P cuadrícula, ;C centrar, ;N número, ;L cursor y ;A ayuda. Las formas en mayúsculas y en minúsculas son distintas; son controles individuales dentro del diseñador, no comandos de CSI. Las minúsculas ;i/;j, ;s/;t y ;p/;q seleccionan los controles de vídeo inverso, subrayado y parpadeo. El capítulo 35 recoge las formas literales, la sintaxis de movimiento de ventanas y los límites restantes.

PARAR — detener una partición

Detener una partición.

Sintaxis

```
PARAR partición
```

Opciones. No hay opciones establecidas.

Privilegio mínimo. 5.

Ejemplo

```
PARAR 2
```

Cuando el usuario de la partición 2 haya guardado su trabajo y se haya desconectado, este comando detiene esa partición y conserva su memoria asignada. No apaga todo el sistema.

PONER — cambiar los ajustes del sistema o de la sesión

Cambiar los ajustes del sistema o de la sesión.

Sintaxis

```
PONER opción valor
```

Opciones. FECHA; ABREV, INCOM, IMPEXEC, MENSAJE, VERSION, ERRNUM, ERRTEXT, HISTORIA, SISFECHA ON/OFF; FORFECHA 1/2/3; definiciones de los controles ESCAPE, SUBIR, BAJAR, IZQ, DER y CANCEL.

Privilegio mínimo. 1.

Ejemplo

```
PONER IMPEXEC ON
```

Este comando permite invocar los archivos EXEC accesibles escribiendo solo el nombre del script. PONER IMPEXEC OFF desactiva esa comodidad; el EXEC completo sigue pudiendo usarse.

Formas para particiones

La configuración de memoria requiere el privilegio 5. Para asignar la partición mostrada como 2 a partir del módulo 1 disponible, cuando sea la siguiente partición sin asignar, introduce:

```
PONER MEMORIA 2 48000 1
```

Así se solicita la partición 2 en el módulo 1, con 48000 como parámetro de tamaño; no garantiza ese número exacto de bytes. Examina ESTADO MODULO antes de la asignación y ESTADO MEMORIA después. En la configuración de demostración, la primera asignación ocupa 0x4C8B–0xFFFF, es decir, 45 941 bytes. La activación es una operación independiente. Para eliminar la partición 2 después de detenerla, cuando sea la última partición asignada y no sea la de quien llama, usa la forma distinta:

```
PONER MEMORIA 2
```

Si se omite el tamaño, se selecciona la eliminación; añadir un tamaño cero no es equivalente. El capítulo 12 enseña a listar, asignar, activar y eliminar particiones, incluidas las restricciones de los módulos compartidos. Las formas generales son `PONER MEMORIA partición bytes módulo` para la asignación y `PONER MEMORIA partición` para la eliminación.

La forma de temporización requiere el privilegio 5 y se explica en el capítulo 25. La aritmética de TIEMPO tiene una discontinuidad entre 135 y 136.

RECARGAR — restaurar archivos, cuentas o volúmenes archivados

Restaurar archivos, cuentas o volúmenes archivados.

Sintaxis

```
RECARGAR [nombre.tipo:]origen,destino [(opciones)]
```

Opciones. CUENTA; RESCUE cuenta; REEMPLA/NOREEM/COPREEM; CONSU/NOCONSU; PANT/NOPANT; VERIFI/NOVERIFI; VOLUM.

Privilegio mínimo. 3.

> **Precaución — la restauración puede sustituir archivos si no se restringe.**

Ejemplo

```
RECARGAR NOTA.TEXTO:A,S (NOREEM CONSU)
```

Esta ilustración de sintaxis selecciona NOTA.TEXTO del origen de archivado A para restaurarlo en S, rechaza la sustitución y mantiene la pregunta de confirmación. Requiere un archivado compatible; el procedimiento completo con los soportes depende del dispositivo.

RECIBIR — recibir archivos síncronos

Recibir archivos síncronos.

Sintaxis

```
RECIBIR [nombre[tipo]]unidad [(COMUn)]
```

Opciones. COMU1 de forma predeterminada.

Privilegio mínimo. 1.

Ejemplo

```
RECIBIR S (COMU1)
```

Este comando usa la forma documentada con solo la unidad para recibir por COMU1 en S. La puntuación opcional de nombre y tipo y el comportamiento de sustitución no están especificados por completo. Úsalo solo con un entorno de recepción preparado y un protocolo acordado con el equipo remoto.

RECIBIRA — recibir archivos asíncronos

Recibir archivos asíncronos.

Sintaxis

```
RECIBIRA [nombre[tipo]]unidad [(COMUn)]
```

Opciones. La ayuda lo identifica como equivalente asíncrono de RECIBIR; la equivalencia exacta de las opciones sigue sin probarse.

Privilegio mínimo. 1.

Ejemplo

```
RECIBIRA S (COMU1)
```

Esta es la forma documentada análoga de recepción asíncrona. Configura antes el enlace y el destino de recepción; la equivalencia completa de las opciones y el comportamiento de sustitución no están especificados.

REPARAR — comprobar y conciliar los metadatos del sistema de archivos

Comprobar y conciliar los metadatos del sistema de archivos.

Sintaxis

```
REPARAR unidad [(F)]
```

Opciones. F actualiza la asignación y puede eliminar entradas o redirecciones rechazadas.

Privilegio mínimo. 3.

Ejemplo

```
REPARAR S
```

Este comando comprueba los metadatos del sistema de archivos de S. Puede informar de errores y aun así devolver el resultado de comando 0; lee el informe completo. La opción F autoriza reparaciones adicionales y la posible eliminación de entradas.

Incluso la comprobación puede actualizar el contador de espacio libre. F puede eliminar entradas rechazadas o redirecciones de uso compartido; no puede reconstruir datos de aplicación arbitrarios.

REVIVIR — recuperar una entrada de catálogo ASCII borrada

Recuperar una entrada de catálogo ASCII borrada.

Sintaxis

```
REVIVIR nombre.tipo:unidad (sector-inicial)
```

Opciones. Inicio numérico de búsqueda opcional, descrito en la ayuda.

Privilegio mínimo. 3.

> **Precaución — escribe metadatos de recuperación.**

Ejemplo

```
REVIVIR PERDIDO.TEXTO:S (160)
```

Esta ilustración de sintaxis inicia la búsqueda de recuperación en el sector 160 para la entrada ASCII borrada indicada. El número es ilustrativo, no una ubicación prescrita. La recuperación escribe metadatos; el éxito y el tratamiento de colisiones no están garantizados.

RUN — ejecutar BASIC compilado

Ejecutar BASIC compilado.

Sintaxis

```
RUN nombre[.tipo[:unidad]] [(opción)]
```

Opciones. Tipo predeterminado BASICCOM. La ayuda anuncia TRACE para las direcciones de ejecución y COMA para la coma decimal; los programas del saludo y numérico de demostración no muestran esos efectos. Véanse las salvedades que siguen.

Privilegio mínimo. 0.

Ejemplo

```
RUN HOLA.BASICCOM:S
```

Este comando ejecuta el programa BASIC compilado del capítulo 15. Se necesita el entorno de ejecución de BASIC; el archivo no es un comando Z80 nativo.

Un resultado correcto de RUN no demuestra que se haya reconocido una palabra opcional. RUN con TRACE no produce ninguna traza visible de direcciones de ejecución para los ejemplos del saludo y numérico, y con COMA, PRINT 1.5 sigue mostrándose como 1.5 en el ejemplo numérico. Una opción arbitraria también termina con la misma salida numérica. Estas observaciones no establecen el comportamiento de la conversión de entrada ni excluyen otros efectos internos. Consulta TRACE en tiempo de ejecución y COMA en tiempo de ejecución.

BASIC con TRACE traza la compilación, una operación distinta que se describe en el capítulo 15. No lo uses como sustituto de un depurador en tiempo de ejecución.

SECIBM — exportar registros secuenciales a IBM-3741

Exportar registros secuenciales a IBM-3741.

Sintaxis

```
SECIBM origen.nombre:unidad unidad-destino [(opciones)]
```

Opciones. Nombre de salida y longitud de registro opcionales, con un máximo de 128 según la ayuda; el valor predeterminado es el registro de origen más largo.

Privilegio mínimo. 3.

> **Precaución — escribe en el soporte de destino.**

Ejemplo

```
SECIBM NOTA.TEXTO:S A
```

Este ejemplo de sintaxis exporta registros secuenciales de S a un disquete compatible con IBM en A, convirtiendo ASCII en EBCDIC. El soporte debe tener las cabeceras adecuadas; los registros demasiado largos pueden truncarse.

SECTOR — listar los sectores ocupados por un archivo

Listar los sectores ocupados por un archivo.

Sintaxis

```
SECTOR nombre.tipo[:unidad]
```

Opciones. No hay opciones establecidas.

Privilegio mínimo. 3.

Ejemplo

```
SECTOR NOTA.TEXTO:S
```

Este comando lista los sectores lógicos ocupados por el archivo. Describe la asignación, no el texto del archivo. Los números de sector lógico requieren la traducción del dispositivo antes de convertirse en direcciones de la controladora.

SERIAL — editar la vinculación de un ejecutable a un número de serie

Editar la vinculación de un ejecutable a un número de serie.

Sintaxis

```
SERIAL nombre.tipo:unidad (opciones)
```

Opciones. SERIAL nnnnn, PANT, NOPANT en la ayuda incrustada.

Privilegio mínimo. 5.

> **Precaución — cambia la política del ejecutable.**

Ejemplo

```
SERIAL DEMO.COMANDO:S (SERIAL 22 PANT)
```

Esta ilustración de sintaxis vincula una copia desechable de un ejecutable al número de serie 22 y solicita su presentación. Sustituye 22 solo por un valor de vinculación intencionado. Esto cambia la política del ejecutable; no configura un puerto serie.

SORT — ordenar registros por claves

Ordenar registros por claves.

Sintaxis

```
SORT [entrada [salida [(opciones)]]]
```

Opciones. FICH archivo de parámetros; CLAVE posición longitud con A/D y N/U/I; REGSAL, CLAVESAL, NUMSAL, DIRSAL; NOCLAVE; NOSORT; MONTAR/MONTARE/MONTARS/MONTARW; WORK unidad; PANT; ESTADIS.

Privilegio mínimo. 1.

Ejemplo

```
SORT ENTRADA.TEXTO:S ORDENADO.TEXTO:S (CLAVE 1 4 A U REGSAL)
```

Esta ilustración de la forma documentada ordena por una clave en mayúsculas de cuatro caracteres que empieza en la posición 1, en orden ascendente, y escribe los registros ordenados. Prepara una entrada compatible y una salida que no esté en uso; las interacciones completas entre el formato de registro y las opciones no están especificadas.

Con un solo nombre de archivo, SORT lo trata como archivo de parámetros. Dos nombres de archivo identifican la entrada y la salida.

SPOOLER — gestionar la impresión diferida

Gestionar la impresión diferida.

Sintaxis

```
SPOOLER opción
```

Opciones. INIC, SAL, ACTIV, ESTADO, PARA, CAMBIA, ELIM, LIST, FINAL, ALINE, SALVAR, FORM código, IMPR n, RELANZA n página, COPIAS n, RETE/NORETE, COLA A–Z.

Privilegio mínimo. 0.

Ejemplo

```
SPOOLER ESTADO
```

Este comando informa del estado del servicio de impresión diferida. Si no está inicializado, diagnostica esa situación antes de enviar salida; el capítulo 12 recoge la secuencia básica de inicialización.

PARA pausa la salida normal, pero las peticiones explícitas de impresión o de relanzamiento pueden tener prioridad. La finalización en la impresora física debe comprobarse por separado.

El comando comprueba primero si existe el componente residente SPOOLER. El sistema documentado ya lo incluye, así que la inicialización normal no requiere CARGAR. Si falta ese componente residente, la alternativa hace referencia al comando no disponible `CARGAR SPOOLER`. Otra referencia, `VACIAR SPOOLER`, también carece de su ejecutable independiente. Estas referencias no ofrecen un procedimiento utilizable para cargar o descargar el componente residente; consulta el capítulo 29.

TECLAS — emitir secuencias de programación del terminal

Emitir secuencias de programación del terminal.

Sintaxis

```
TECLAS
```

Opciones. No se ha establecido ningún analizador de argumentos.

Privilegio mínimo. Mínimo estándar desconocido.

Ejemplo

```
TECLAS
```

Este comando invoca el emisor de secuencias de programación del terminal. Se desconocen la familia de terminales compatible y los efectos exactos, así que trátalo como la identificación de la invocación, no como un paso de configuración recomendado.

TEDITAR — editar archivos pequeños

Editar archivos pequeños.

Sintaxis

```
TEDITAR nombre.tipo[:unidad]
```

Opciones. No hay opciones establecidas.

Privilegio mínimo. 1.

Ejemplo

```
TEDITAR NOTA.TEXTO:S
```

Este comando abre el editor de archivos pequeños con el texto de práctica. Su diálogo completo de edición y salida no está especificado; usa el flujo de trabajo de EDITAR del capítulo 6 cuando necesites un procedimiento de creación fiable.

TERMINAL — usar una consola como terminal remoto

Usar una consola como terminal remoto.

Sintaxis

```
TERMINAL [(opciones)]
```

Opciones. COMUn (predeterminado: COMU1); modo de sistema remoto IBER o HAZELTIN.

Privilegio mínimo. 1.

Ejemplo

```
TERMINAL (COMU1)
```

Este comando selecciona COMU1 para usar la consola como terminal remoto. Establece el equipo remoto compatible y el método de terminación antes de entrar; no está especificado un procedimiento completo de escape o salida.

TITULO — imprimir rótulos en letras grandes

Imprimir rótulos en letras grandes.

Sintaxis

```
TITULO canal título1 [título2 ... título12]
```

Opciones. P consola; In impresora n; hasta doce títulos, de hasta ocho caracteres cada uno según la ayuda.

Privilegio mínimo. Mínimo estándar desconocido.

Ejemplo

```
TITULO P HOLA
```

Este comando solicita la palabra HOLA en letras grandes en la consola seleccionada por P. En la interfaz documentada, cada título se limita a ocho caracteres. La utilidad hace referencia a MAESTRO.TITULO, cuya probable función como patrón de caracteres y cuyos detalles de almacenamiento se describen en el apéndice C.

UNOSEC — reconstruir el sector lógico 1

Reconstruir el sector lógico 1.

Sintaxis

```
UNOSEC unidad [(opciones)]
```

Opciones. C=cabezas (predeterminado: 4), P=pistas (243), S=sectores (42), E=entradas de directorio; si no se indica, se deduce el límite del directorio.

Privilegio mínimo. 5.

> **Precaución — sobrescribe los metadatos del volumen.**

Ejemplo

```
UNOSEC A (C=5 P=397 S=32 E=1208)
```

Esta es una ilustración de lectura de parámetros, no una receta de reparación: especifica cabezas, pistas, sectores y entradas de directorio. Todos los valores deben proceder del formato real del volumen de destino. El comando sobrescribe los metadatos del volumen.

VERFORM — comprobar el formato de los soportes

Comprobar el formato de los soportes.

Sintaxis

```
VERFORM unidad
```

Opciones. No hay opciones establecidas.

Privilegio mínimo. 3.

Ejemplo

```
VERFORM A
```

Este comando comprueba el formato del volumen asociado con A. Úsalo después de la inicialización y de MONTAR y, a continuación, verifica una copia normal de archivo con COPIFI (VERIFI), como se explica en el capítulo 11. Lee los diagnósticos además del resultado de finalización. Su contrato completo de efectos secundarios no está especificado; no lo uses como sustituto de una copia de seguridad ni des por hecho que sea un diagnóstico de solo lectura garantizado.

VOLCAR — volcar los bytes de un archivo o de un sector lógico

Volcar los bytes de un archivo o de un sector lógico. Interfaz sin especificar; solo se ofrece un ejemplo de inspección.

Sintaxis

```
Sintaxis completa no establecida
```

Opciones. El código tiene rutas para archivos y para un intervalo de sectores inicial y final, ambos incluidos; opciones de consola que empiezan por P y de impresora que empiezan por I.

Privilegio mínimo. 2.

Ejemplo

```
LISDIR VOLCAR.COMANDO:S
```

Este comando examina la entrada de la utilidad. VOLCAR muestra bytes en hexadecimal y como caracteres, pero su gramática de invocación completa no está especificada y algunas rutas de salida escriben archivos o manejan la cinta. No se ofrece ningún comando de volcado inventado.

Límite de la interfaz. El nombre de archivo por sí solo no aporta el contrato de uso que falta.

Capítulo 32 — Referencia de servicios del sistema

A continuación aparecen, en orden numérico, las 111 entradas de RST 0x08. Los nombres describen funciones; no son identificadores simbólicos oficiales. Cada entrada especifica su función, los destinos almacenado e inicializado, las entradas, las salidas, los efectos conocidos sobre los registros y las restricciones. Si se omite la garantía sobre un registro, es que se desconoce; no debe deducirse de otro servicio. Los indicadores de error son específicos de cada servicio.

La convención de entrada común es `RST 08H` seguido de `DB número`; la dirección de retorno salta ese byte. La escritura de la instrucción es notación Z80 ilustrativa, no una prueba de que haya un ensamblador instalado. Por ejemplo, el servicio 0x02 se codifica como `CF 02`. El capítulo 18 ofrece un ejemplo ejecutable exacto byte a byte que usa 0x02 y 0x00; el capítulo 17 aporta llamadas documentadas a los servicios de archivo desde BASIC. Cada entrada incluye un ejemplo de secuencia de llamada. Etiquetas como peticion, bufer y dev_llamada designan almacenamiento o código preparados por quien llama; no son símbolos integrados. Resérvalos e inicialízalos según el contrato de entrada y el apéndice E antes de hacer la llamada. Los fragmentos muestran la colocación y la selección de los argumentos, no un programa ensamblado autónomo. Las llamadas de mantenimiento mantienen todas sus condiciones previas y sus riesgos.

Las estructuras de datos se definen en el apéndice E; los formatos de ejecutables y de tiempo de ejecución, en el apéndice A. El capítulo 24 explica el despacho y las superposiciones; el capítulo 25, la sincronización; y el capítulo 27, la política de errores. La tabla siguiente es el índice canónico de destinos.

Número	Función descriptiva	Destino almacenado	Destino activo
0x00	Finalizar el comando / restaurar CSI	0x075B	0x075B
0x01	Entrada de línea por consola	0x1139	0x1139
0x02	Mostrar una cadena	0x1290	0x1290
0x03	Sondear la entrada de consola	0x134E	0x134E
0x04	Leer un carácter de la consola	0x1360	0x1360
0x05	Salida de caracteres por consola	0x1461	0x1461
0x06	Leer un carácter de la consola con eco directo	0x144E	0x144E
0x07	Salida de caracteres mediante la clase de terminal	0x149B	0x149B
0x08	Salida de caracteres a la asociación de impresora	0x1698	0x1698
0x09	Invalidar los campos del descriptor de disco en caché	0x31E3	0x31E3
0x0A	Leer un sector lógico	0x09BF	0x09BF
0x0B	Escribir un sector lógico	0x09C5	0x09C5

Número	Función descriptiva	Destino almacenado	Destino activo
0x0C	Devolver la firma del núcleo inicializado	0x5157	0x331B
0x0D	Crear/escribir una entrada de directorio	0x27CA	0x27CA
0x0E	Analizar un hexadecimal sin signo	0x1899	0x1899
0x0F	Analizar un decimal sin signo	0x18CC	0x18CC
0x10	Dar formato hexadecimal a un byte	0x1916	0x1916
0x11	Dar formato decimal a una palabra sin signo	0x192C	0x192C
0x12	Nueva línea en la consola	0x1317	0x1317
0x13	Retardo activo calibrado	0x2C83	0x2C83
0x14	Buscar una entrada de directorio	0x2463	0x2463
0x15	Obtener el descriptor del dispositivo asociado	0x3145	0x3145
0x16	Cargar un ejecutable con nombre sin entrar en él	0x0765	0x0765
0x17	Salida de cadenas a la asociación de impresora	0x1673	0x1673
0x18	Inicializar el nombre/clase de un canal	0x31CA	0x31CA
0x19	Convertir un carácter de unidad en índice de asociación	0x327A	0x327A
0x1A	Convertir una asociación de disco en carácter	0x325C	0x325C
0x1B	Asignar un número exacto de bloques contiguos	0x2847	0x2847
0x1C	Liberar un intervalo de bloques contiguos	0x29BB	0x29BB
0x1D	Borrar un archivo con nombre	0x262B	0x262B
0x1E	Cargar y ejecutar un comando	0x0760	0x0760
0x1F	Renombrar un archivo con nombre	0x2741	0x2741
0x20	Abrir un canal	0x0733	0x0733
0x21	Cerrar un canal	0x0738	0x0738
0x22	Leer un registro de un canal	0x3865	0x3865
0x23	Escribir un registro en un canal	0x38D6	0x38D6
0x24	Dar formato a la fecha actual	0x2B34	0x2B34
0x25	Dar formato a la hora actual	0x2BC7	0x2BC7
0x26	División sin signo	0x1973	0x1973
0x27	Multiplicación sin signo	0x1998	0x1998
0x28	Leer un registro directo numerado	0x0751	0x0751

Número	Función descriptiva	Destino almacenado	Destino activo
0x29	Escribir un registro directo numerado	0x0756	0x0756
0x2A	Analizar un decimal o un hexadecimal con sufijo H	0x18FA	0x18FA
0x2B	Leer un registro indexado/con clave	0x071F	0x071F
0x2C	Leer el siguiente registro indexado/con clave	0x0724	0x0724
0x2D	Escribir un registro indexado/con clave	0x0729	0x0729
0x2E	Empaquetar la fecha/hora actuales	0x2BE8	0x2BE8
0x2F	Resolver una etiqueta de volumen o una unidad de un carácter	0x31F9	0x31F9
0x30	Obtener el contexto de la partición actual	0x310E	0x310E
0x31	Pausa de paginación	0x32D5	0x32D5
0x32	Leer varios sectores lógicos	0x09C1	0x09C1
0x33	Escribir varios sectores lógicos	0x09C7	0x09C7
0x34	Mostrar una cadena con determinados indicadores de modo suprimidos	0x1326	0x1326
0x35	Programar, sustituir o cancelar una devolución de llamada temporizada	0x2EA4	0x2EA4
0x36	Sustituir la entrada de comandos y volver a CSI	0x0779	0x0779
0x37	Obtener el límite superior de memoria de la partición	0x3172	0x3172
0x38	Establecer el límite superior de memoria de la partición	0x317F	0x317F
0x39	Establecer la devolución de llamada de la palabra 28 del contexto	0x318C	0x318C
0x3A	Resolver una asociación	0x31AB	0x31AB
0x3B	Obtener los campos de geometría del terminal asociado	0x32A0	0x32A0
0x3C	Borrar un registro indexado/con clave	0x072E	0x072E
0x3D	Despacho de inicialización de dispositivos	0x16AE	0x16AE
0x3E	Despacho de estado de dispositivos	0x169F	0x169F
0x3F	Despacho de entrada de dispositivos	0x16A4	0x16A4
0x40	Despacho de salida de dispositivos	0x16A9	0x16A9
0x41	Prólogo estándar de privilegio/número de serie de los comandos	0x0774	0x0774
0x42	Copiar la etiqueta del volumen	0x324E	0x324E
0x43	Establecer la entrada del controlador del dispositivo físico	0x170D	0x170D
0x44	Terminar/descargar el controlador asociado	0x16B3	0x16B3

Número	Función descriptiva	Destino almacenado	Destino activo
0x45	Sondear y consumir el indicador de evento de consola	0x35C2	0x35C2
0x46	Sustituir la entrada de comandos con límite superior de continuación	0x077E	0x077E
0x47	Leer un byte de una cola en memoria	0x3320	0x3320
0x48	Añadir un byte a una cola en memoria	0x3356	0x3356
0x49	Cambiar los bits de modo de la consola	0x32B1	0x32B1
0x4A	Establecer la devolución de llamada de error de disco	0x3199	0x3199
0x4B	Inicializar o cambiar el volumen del sistema	0x078D	0x078D
0x4C	Retardo activo escalado	0x2C90	0x2C90
0x4D	Obtener una entrada de la tabla de canales	0x312B	0x312B
0x4E	Despachar un comando de control del terminal	0x3619	0x3619
0x4F	Ceder el control / planificar	0x0558	0x0558
0x50	Obtener la base global	0x31A6	0x31A6
0x51	Leer el byte global 003B	0x315A	0x315A
0x52	Obtener el índice actual del planificador	0x3113	0x3113
0x53	Liberar los bloqueos de registros del canal	0x19C1	0x19C1
0x54	Adquirir un bloqueo de byte	0x208A	0x208A
0x55	Liberar un bloqueo de byte	0x20AC	0x20AC
0x56	Entrar en una devolución de llamada en la partición seleccionada	0x063F	0x063F
0x57	Obtener la base del mapa de asociaciones	0x3121	0x3121
0x58	Enviar texto a la consola de otra partición	0x2141	0x2141
0x59	Adquirir el bloqueo global fijo	0x20C9	0x20C9
0x5A	Liberar el bloqueo global fijo	0x20D2	0x20D2
0x5B	Obtener el área de trabajo de sector	0x3117	0x3117
0x5C	Obtener el byte de privilegio	0x315E	0x315E
0x5D	Comparar memoria	0x3559	0x3559
0x5E	Leer bytes secuenciales contados	0x0747	0x0747
0x5F	Escribir bytes secuenciales contados	0x074C	0x074C
0x60	Mostrar un mensaje del sistema numerado	0x0783	0x0783
0x61	Mostrar un mensaje numerado y finalizar el comando	0x0788	0x0788

Número	Función descriptiva	Destino almacenado	Destino activo
0x62	Cargar un submódulo R a partir de un descriptor compacto	0x076F	0x076F
0x63	Asignar un tramo de bloques con tamaño mínimo	0x2849	0x2849
0x64	Buscar en una tabla de palabras clave abreviables	0x3564	0x3564
0x65	Obtener el número de cuenta	0x3168	0x3168
0x66	Filtrar la entrada del dispositivo para detectar controles de escape	0x35D8	0x35D8
0x67	Registrar una devolución de llamada indexada	0x21E4	0x21E4
0x68	Copiar desde el banco del registro seleccionado	0x1FAC	0x1FAC
0x69	Copiar al banco del registro seleccionado	0x1FF0	0x1FF0
0x6A	Dar formato a una fecha BCD proporcionada	0x2B4B	0x2B4B
0x6B	Esperar un evento de dispositivo o una interrupción	0x06EA	0x06EA
0x6C	Buscar un componente residente con nombre	0x20DB	0x20DB
0x6D	Escribir la hora global actual mediante la devolución de llamada del RTC	0x2C61	0x2C61
0x6E	Escribir la fecha global actual mediante la devolución de llamada del RTC	0x2C6B	0x2C6B

0x00 — Finalizar el comando / restaurar CSI

Entrada y parámetros. A=resultado del comando.

Salida, registros y errores. No vuelve; almacena el resultado en contexto+68 y vuelve a cargar CSI.

Estructuras, restricciones y notas. Libera los bloqueos y las filas de apertura del propietario y los bloqueos de byte fijos del ejecutivo, pero no hace CLOSE de la salida secuencial con búfer. El resultado cero no es una garantía de durabilidad. Véanse los capítulos 26 y 30.

Ejemplo de llamada

```
xor a
rst 08h
db 00h
```

El cero en A solicita la finalización correcta del comando. Esta llamada no vuelve; cierra antes con CLOSE los canales de salida.

0x01 — Entrada de línea por consola

Entrada y parámetros. DE=destino; B=presupuesto de entrada.

Salida, registros y errores. A=número de caracteres aceptados; HL/IY restaurados; BC/DE cambian.

Estructuras, restricciones y notas. Al agotarse el presupuesto, la línea se envía sin que se teclee CR. Un CR tecleado se almacena y va seguido de otro CR; la salida por presupuesto agotado añade uno. El borrado con guion bajo de CSI depende del bit 6 de contexto+0A. B=0 y todas las combinaciones de edición no están especificados.

Ejemplo de llamada

```
ld de,bufer
ld b,20
rst 08h
db 01h
```

Quien llama aporta un búfer mayor que el presupuesto de 20 caracteres para dejar sitio a los bytes de terminación. El número de caracteres devuelto está en A; no esperes que DE o BC permanezcan sin cambios.

0x02 — Mostrar una cadena

Entrada y parámetros. DE=cadena.

Salida, registros y errores. DE avanza; BC/HL/IY se guardan.

Estructuras, restricciones y notas. NUL termina; CR emite un salto de línea y termina; LF emite un salto de línea y continúa. El comportamiento de las tabulaciones y los controles depende del contexto de la consola. Quien llama no indica ninguna longitud.

Ejemplo de llamada

```
ld de,mensaje
rst 08h
db 02h
```

Define mensaje como los bytes de HOLA seguidos de CR y cero, como en el capítulo 18. DE identifica el primer byte; la llamada muestra la cadena.

0x03 — Sondear la entrada de consola

Entrada y parámetros. Ningún argumento explícito; asociación de consola actual 8.

Salida, registros y errores. Estado que depende del controlador y del estado de la entrada; BC/IY se conservan en la ruta de estado físico.

Estructuras, restricciones y notas. El estado de entrada inyectada puede devolver NZ directamente. No interpretes el acarreo como un error de entrada universal; véase 3E.

Ejemplo de llamada

```
rst 08h
db 03h
```

El fragmento selecciona el servicio «Sondear la entrada de consola».

0x04 — Leer un carácter de la consola

Entrada y parámetros. Entrada de consola actual.

Salida, registros y errores. A=carácter traducido; los demás efectos completos sobre los registros no están establecidos de forma uniforme.

Estructuras, restricciones y notas. Puede bloquearse, consumir entrada inyectada, traducir controles y hacer eco mediante 05. Esc seguida de O suprime el eco sin impedir la recepción de caracteres. Los modos de mayúsculas/minúsculas y de guion bajo de CSI y de BASIC son distintos.

Ejemplo de llamada

```
rst 08h
db 04h
```

El fragmento selecciona el servicio «Leer un carácter de la consola».

0x05 — Salida de caracteres por consola

Entrada y parámetros. C=carácter.

Salida, registros y errores. Las llamadas seleccionadas conservan BC/DE/HL/IX/IY; AF no es fijo.

Estructuras, restricciones y notas. Aplica la política de supresión y de copia a la impresora antes de 07. El bit 1 de contexto+0A suprime la salida. Que la llamada vuelva no demuestra la transmisión física.

Ejemplo de llamada

```
ld c,65
rst 08h
db 05h
```

C contiene el código de carácter 65, la letra A. Esta ruta aplica la política normal de la consola, incluida la supresión de la salida.

0x06 — Leer un carácter de la consola con eco directo

Entrada y parámetros. Entrada de consola actual.

Salida, registros y errores. A=carácter; el envoltorio conserva BC/IY.

Estructuras, restricciones y notas. Usa la entrada traducida y hace eco mediante 07 de los valores iguales o superiores a 20h, omitiendo para ese eco la política de supresión de 05.

Ejemplo de llamada

```
rst 08h
db 06h
```

El fragmento selecciona el servicio «Leer un carácter de la consola con eco directo».

0x07 — Salida de caracteres mediante la clase de terminal

Entrada y parámetros. C=carácter o control lógico del terminal.

Salida, registros y errores. Las llamadas seleccionadas conservan BC/DE/HL/IX/IY; AF cambia.

Estructuras, restricciones y notas. Se aplican la traducción de la clase, el seguimiento del cursor y el consumo de coordenadas del control 10. La llamada directa a 07 omite Esc seguida de O. Consulta el capítulo 28 para ver las capas de salida.

Ejemplo de llamada

```
ld c,65
rst 08h
db 07h
```

C contiene la letra A. Esta capa de salida inferior aplica la traducción de la clase de terminal y es distinta del servicio 0x05, controlado por la política.

0x08 — Salida de caracteres a la asociación de impresora

Entrada y parámetros. C=carácter para la impresora; asociación 0C.

Salida, registros y errores. Las llamadas seleccionadas conservan BC/DE/HL/IX/IY; AF varía.

Estructuras, restricciones y notas. Una impresora sin asignar puede devolver 0044 sin indicar nada. Un SPOOLER activo recoge la salida en lugar de imprimirla. La expansión de controles común de las impresoras depende del controlador y de la ruta.

Ejemplo de llamada

```
ld c,65
rst 08h
db 08h
```

El fragmento selecciona el servicio «Salida de caracteres a la asociación de impresora».

0x09 — Invalidar los campos del descriptor de disco en caché

Entrada y parámetros. B=asociación de disco 0..7.

Salida, registros y errores. BC/DE/HL se conservan.

Estructuras, restricciones y notas. Pone descriptor+14/+15 a FF. Los demás valores de B no modifican nada. La actualización puede añadir una lectura preliminar de la cabecera; no es un vaciado general.

Ejemplo de llamada

```
ld b,0
rst 08h
db 09h
```

El fragmento selecciona el servicio «Invalidar los campos del descriptor de disco en caché».

0x0A — Leer un sector lógico

Entrada y parámetros. B=asociación de unidad; DE=sector lógico; HL=destino de 256 bytes.

Salida, registros y errores. Los retornos normales seleccionados y los de error ignorado devuelven AF0044; C=1; DE/HL/IX/IY se conservan.

Estructuras, restricciones y notas. Comprueba los tramos de registros existentes. Ignorar puede devolver como éxito datos no válidos; Q sale de forma no local. Las asociaciones no válidas pueden anular la operación antes de la E/S. Véase el capítulo 27.

Ejemplo de llamada

```
ld b,0
ld de,1
ld hl,bufer
```

```
rst 08h
db 0Ah
```

La asociación 0 selecciona S. La petición lee el sector lógico 1 en un búfer de 256 bytes. No imprime ese búfer; usa la disposición de la cabecera del apéndice E para interpretarlo.

0x0B — Escribir un sector lógico

Entrada y parámetros. B=unidad; DE=sector lógico; HL=origen de 256 bytes.

Salida, registros y errores. Los retornos seleccionados devuelven AF0044; C=1; DE/HL/IX/IY y el origen se conservan.

Estructuras, restricciones y notas. El bit 7 de descriptor+13 rechaza la operación antes de llegar al hardware. Ignorar da por bueno un sector no escrito. Una devolución de llamada que elimina la protección no reintenta una escritura ignorada.

Ejemplo de llamada

```
ld b,0
ld de,sector_destino
ld hl,bufer
rst 08h
db 0Bh
```

sector_destino debe identificar un almacenamiento asignado deliberadamente para esta operación. Escribe 256 bytes y puede destruir metadatos si apunta a un sector de cabecera o de directorio.

0x0C — Devolver la firma del núcleo inicializado

Entrada y parámetros. Solo con el núcleo inicializado.

Salida, registros y errores. La rutina activa devuelve HL=5542h.

Estructuras, restricciones y notas. La entrada en disco apunta inicialmente a la inicialización 5157; el destino activo 331B es distinto. Se desconocen el significado de la firma y la conservación completa de los registros.

Ejemplo de llamada

```
rst 08h
db 0Ch
```

El fragmento selecciona el servicio «Devolver la firma del núcleo inicializado».

0x0D — Crear/escribir una entrada de directorio

Entrada y parámetros. B=unidad; DE=entrada candidata de 32 bytes.

Salida, registros y errores. Las creaciones seleccionadas devuelven AF0044; BC/DE/HL/IX/IY y los bytes de quien llama se conservan.

Estructuras, restricciones y notas. Normaliza mediante la búsqueda con la alternativa pública inhibida. La rama de entrada existente o de error devuelve AFF/acarreo según el código. No establece la atomicidad de la asignación ni la unicidad global; las entradas ocultas pueden permitir duplicados.

Ejemplo de llamada

```
ld b,0
ld de,candidata
rst 08h
db 0Dh
```

El fragmento selecciona el servicio «Crear/escribir una entrada de directorio». Es una operación interna o de configuración, no un atajo normal para preparar aplicaciones.

0x0E — Analizar un hexadecimal sin signo

Entrada y parámetros. DE=texto hexadecimal sin signo en mayúsculas.

Salida, registros y errores. HL=valor o resultado parcial con desbordamiento; DE avanza; BC se restaura.

Estructuras, restricciones y notas. Dígitos 0–9/A–F y H opcional; sin signo, espacios ni minúsculas. Una entrada sin dígitos puede tener éxito. El desbordamiento consume su dígito y puede devolver un HL desplazado parcialmente. Valida la sintaxis y el acarreo de forma independiente.

Ejemplo de llamada

```
ld de,texto_hex
rst 08h
db 0Eh
```

Como entrada básica, texto_hex puede contener el ASCII 2A seguido de NUL. Comprueba tanto el acarreo como el puntero DE avanzado; con otras entradas, es posible que tenga éxito sin consumir dígitos.

0x0F — Analizar un decimal sin signo

Entrada y parámetros. DE=texto decimal sin signo.

Salida, registros y errores. HL=valor o resultado parcial con desbordamiento; DE avanza; BC se restaura.

Estructuras, restricciones y notas. Sufijo D opcional, sin signo ni espacios. Una entrada vacía o con solo D puede tener éxito. El desbordamiento en las etapas de multiplicación o suma devuelve un resultado intermedio, no necesariamente módulo 65 536.

Ejemplo de llamada

```
ld de,texto_decimal
rst 08h
db 0Fh
```

texto_decimal puede contener el ASCII 123 seguido de NUL. Valida el texto restante además del valor numérico devuelto.

0x10 — Dar formato hexadecimal a un byte

Entrada y parámetros. B=byte; DE=destino.

Salida, registros y errores. Dos dígitos hexadecimales en mayúsculas; DE+=2; el destino no modifica BC/HL; AF cambia.

Estructuras, restricciones y notas. Sin terminador. Aporta dos bytes con permiso de escritura.

Ejemplo de llamada

```
ld b,42
ld de,bufer
rst 08h
db 10h
```

42 decimal es el byte al que se va a dar formato. Quien llama aporta el almacenamiento de salida con permiso de escritura; la llamada da formato al valor, no lo imprime.

0x11 — Dar formato decimal a una palabra sin signo

Entrada y parámetros. HL=palabra sin signo; DE=destino.

Salida, registros y errores. Escribe dígitos decimales; DE avanza; IX se restaura; AF, el AF alternativo, BC y HL no se conservan en general.

Estructuras, restricciones y notas. Sin ceros iniciales, salvo un único 0 para el valor cero; sin terminador. Aporta espacio para cinco dígitos.

Ejemplo de llamada

```
ld hl,123
ld de,bufer
rst 08h
db 11h
```

123 es el valor sin signo al que se va a dar formato. bufer debe tener sitio para los caracteres decimales producidos y para la convención de terminación que se necesite.

0x12 — Nueva línea en la consola

Entrada y parámetros. Ningún argumento explícito.

Salida, registros y errores. El envoltorio conserva AF/BC.

Estructuras, restricciones y notas. Envía LF y después CR mediante 05. Siguen aplicándose la política de supresión y la de copia a la impresora; la nueva línea de las cadenas para la impresora usa otro orden.

Ejemplo de llamada

```
rst 08h
db 12h
```

Así se solicita una nueva línea en la consola actual. No requiere ningún búfer de caracteres aparte.

0x13 — Retardo activo calibrado

Entrada y parámetros. A=número de iteraciones.

Salida, registros y errores. BC se conserva; se desconocen los demás efectos completos.

Estructuras, restricciones y notas. Bucle activo anidado calibrado que usa 0046. A=0 realiza 256 iteraciones. La unidad de tiempo absoluta sigue sin resolverse; no es una espera del planificador.

Ejemplo de llamada

```
ld a,1
rst 08h
db 13h
```

El fragmento selecciona el servicio «Retardo activo calibrado».

0x14 — Buscar una entrada de directorio

Entrada y parámetros. DE=unidad+nombre de 8+tipo de 8 con permiso de escritura; HL=búfer de directorio de 256 bytes.

Salida, registros y errores. Encontrado: AF0044, DE=LBA, HL=entrada; ausente con hueco reutilizable: AF0100; B=unidad resuelta. IX/IY se conservan según la matriz especificada; BC cambia.

Estructuras, restricciones y notas. Los retornos con el acarreo activado no identifican un hueco utilizable. La petición puede normalizarse varias veces. La alternativa pública, el recorrido de unidades FF, el 00 temprano, la memoria de huecos borrados y los ciclos de sondeo cortos se especifican en el capítulo 26.

Ejemplo de llamada

```
ld de,nombre_archivo
ld hl,bufer
rst 08h
db 14h
```

nombre_archivo debe tener permiso de escritura y contener la unidad y dos campos de ocho bytes. El búfer de 256 bytes recibe los datos del directorio. Comprueba el resultado antes de tratar HL como puntero a una entrada.

0x15 — Obtener el descriptor del dispositivo asociado

Entrada y parámetros. B=asociación.

Salida, registros y errores. A/C=índice físico; HL=0069h+32*índice; DE se conserva; acarreo si la asociación no se resuelve.

Estructuras, restricciones y notas. Los campos del descriptor son una unión que depende de la familia del dispositivo. Las utilidades de cinta suman 0F a su puntero de controlador para acceder a una interfaz privada independiente.

Ejemplo de llamada

```
ld b,9
rst 08h
db 15h
```

El fragmento selecciona el servicio «Obtener el descriptor del dispositivo asociado».

0x16 — Cargar un ejecutable con nombre sin entrar en él

Entrada y parámetros. DE=unidad+nombre/tipo; HL=dirección de carga solicitada en las llamadas establecidas.

Salida, registros y errores. Carga A/R sin entrar en él; el acarreo indica errores. Determinadas cargas de BASICUSR devuelven AF0100 y conservan HL.

Estructuras, restricciones y notas. La matriz completa de errores y registros está incompleta. No exijas A=0 en cada llamada correcta al cargador. Véanse el capítulo 18 y el apéndice A.

Ejemplo de llamada

```
ld de,nombre_archivo
ld hl,dir_carga
rst 08h
db 16h
```

El fragmento selecciona el servicio «Cargar un ejecutable con nombre sin entrar en él». Es una operación interna o de configuración, no un atajo normal para preparar aplicaciones.

0x17 — Salida de cadenas a la asociación de impresora

Entrada y parámetros. DE=cadena para la impresora.

Salida, registros y errores. DE avanza; BC se conserva; las llamadas seleccionadas conservan también HL/IX/IY.

Estructuras, restricciones y notas. NUL termina; CR emite CR/LF y termina; LF emite CR/LF y continúa. Una impresora sin asignar puede tener éxito sin producir salida.

Ejemplo de llamada

```
ld de,mensaje
rst 08h
db 17h
```

El fragmento selecciona el servicio «Salida de cadenas a la asociación de impresora».

0x18 — Inicializar el nombre/clase de un canal

Entrada y parámetros. B=canal 0..16; DE=18 bytes: unidad, nombre de 8, tipo de 8 y clase.

Salida, registros y errores. Copia 18 bytes y pone a cero 14; BC/DE se conservan; HL=uno más allá del canal. Canal no válido: acarreo.

Estructuras, restricciones y notas. No normaliza el nombre de archivo. Determinadas llamadas de BASIC devuelven AF0040. La entrada interna empieza en contexto+AC+32*B.

Ejemplo de llamada

```
ld b,1
ld de,nombre_canal
rst 08h
db 18h
```

nombre_canal contiene la unidad, el nombre rellenado, el tipo rellenado y la clase, con un total de 18 bytes. Así se inicializa el canal 1; el archivo no se abre hasta que se llama al servicio 0x20.

0x19 — Convertir un carácter de unidad en índice de asociación

Entrada y parámetros. B=carácter de unidad en mayúsculas.

Salida, registros y errores. A=asociación; acarreo si no es válida o no está asignada.

Estructuras, restricciones y notas. S→0; A–G→1–7 tras la validación; *→FF. No monta soportes.

Ejemplo de llamada

```
ld b,53h
rst 08h
db 19h
```

0x53 es la S en ASCII. La operación convierte ese carácter de unidad en su índice de asociación de disco; no monta ningún dispositivo.

0x1A — Convertir una asociación de disco en carácter

Entrada y parámetros. B=asociación de disco.

Salida, registros y errores. A=S o A–G; si no es válida o no está asignada, da * y acarreo; BC/IY se conservan.

Estructuras, restricciones y notas. Conversión inversa para la presentación tras resolver la asociación.

Ejemplo de llamada

```
ld b,0
rst 08h
db 1Ah
```

El fragmento selecciona el servicio «Convertir una asociación de disco en carácter».

0x1B — Asignar un número exacto de bloques contiguos

Entrada y parámetros. B=unidad; DE=número exacto de bloques distinto de cero.

Salida, registros y errores. Éxito: A0/Z/acarreo 0, HL=primer LBA, DE=bloques asignados; fallo: AFF/NZ/acarreo 0; fallo de asociación: acarreo.

Estructuras, restricciones y notas. Requiere bloques contiguos de 1 KiB. C puede cambiar; IX/IY se conservan en las pruebas seleccionadas. La asignación de cero bloques y la asignación interrumpida no están documentadas.

Ejemplo de llamada

```
ld b,0
ld de,1
rst 08h
db 1Bh
```

Así se solicita un bloque de asignación contiguo en S. Guarda la dirección inicial devuelta y libera solo el almacenamiento que te pertenezca; la asignación cambia el estado del sistema de archivos.

0x1C — Liberar un intervalo de bloques contiguos

Entrada y parámetros. B=unidad; HL=primer LBA; DE=número de bloques.

Salida, registros y errores. Éxito: A0/Z/acarreo 0, HL=LBA alineado con el bloque, DE=número de bloques.

Estructuras, restricciones y notas. Borra el mapa de bits y actualiza los recuentos de espacio libre con el bloqueo 29A7. No hay garantía frente a la doble liberación ni comprobación de límites. Una cabecera protegida contra escritura puede devolver AFF. Está documentada la reversión normal tras un OPEN fallido.

Ejemplo de llamada

```
ld b,0
ld hl,inicio_asignado
ld de,1
rst 08h
db 1Ch
```

inicio_asignado debe ser el inicio de un tramo de un bloque perteneciente a quien llama. Una dirección supuesta o una liberación repetida pueden corromper la contabilidad de la asignación.

0x1D — Borrar un archivo con nombre

Entrada y parámetros. DE=unidad+nombre/tipo.

Salida, registros y errores. Un archivo que no existe puede devolver A0/Z; si está protegido o el propietario no es el correcto, devuelve AFF/acarreo; DE se conserva en las rutas normales.

Estructuras, restricciones y notas. Marca la entrada con FF antes de liberar. Borra las cadenas S por tramos. Ignora el acarreo de admisión exclusiva en la ruta de varias particiones medida; una apertura exclusiva no protege contra el borrado.

Ejemplo de llamada

```
ld de,nombre_archivo
rst 08h
db 1Dh
```

El fragmento selecciona el servicio «Borrar un archivo con nombre». Es una operación interna o de configuración, no un atajo normal para preparar aplicaciones.

0x1E — Cargar y ejecutar un comando

Entrada y parámetros. B=unidad resuelta; DE=entrada de directorio; HL=contexto de argumentos de CSI.

Salida, registros y errores. Carga y transfiere el control; el contrato completo de registros iniciales y trampolín está incompleto.

Estructuras, restricciones y notas. R entra en partición+4CC; A usa la dirección de carga absoluta del directorio. EX/EXE tienen documentado un comportamiento de argumentos con el token equivocado. En el ejemplo nativo documentado se termina mediante 00.

Ejemplo de llamada

```
ld b,0
ld de,entrada
ld hl,argumentos
rst 08h
db 1Eh
```

El fragmento selecciona el servicio «Cargar y ejecutar un comando». Es una operación interna o de configuración, no un atajo normal para preparar aplicaciones.

0x1F — Renombrar un archivo con nombre

Entrada y parámetros. DE=descriptor antiguo; HL=descriptor nuevo con permiso de escritura.

Salida, registros y errores. A08: el destino existe; A04: el origen no existe; A0A: rechazo por protección, propietario o dispositivo; los indicadores completos están incompletos.

Estructuras, restricciones y notas. Copia la unidad antigua en el descriptor nuevo, marca la entrada antigua con FF y crea los metadatos con el nuevo nombre. No mueve datos; se desconoce su atomicidad ante fallos del sistema.

Ejemplo de llamada

```
ld de,nombre_antiguo
ld hl,nombre_nuevo
rst 08h
db 1Fh
```

El fragmento selecciona el servicio «Renombrar un archivo con nombre». Es una operación interna o de configuración, no un atajo normal para preparar aplicaciones.

0x20 — Abrir un canal

Entrada y parámetros. DE=petición de cuatro bytes: canal, modo y área de trabajo LE16; canal inicializado con 18.

Salida, registros y errores. Éxito seleccionado: A0/Z; BC/HL pueden cambiar; el modo de la petición es modificable.

Estructuras, restricciones y notas. Secuencial: 88 sustituye, 84→8C amplía, 90 lee; directo: 58→5A y 59→5B con varias particiones. Ya abierto: A1; ausente: 40h; sondeo completo: 20h; protección: 0Ah en las rutas seleccionadas. Los capítulos 17 y 26 especifican los fallos y las obligaciones sobre los búferes.

Ejemplo de llamada

```
ld de,peticion
rst 08h
db 20h
```

peticion debe ser una estructura de cuatro bytes con permiso de escritura (canal, modo y área de trabajo), tras inicializar el nombre del canal. Selecciona el modo de forma intencionada: la sustitución secuencial puede borrar el contenido existente durante OPEN.

0x21 — Cerrar un canal

Entrada y parámetros. DE=petición, como en 20.

Salida, registros y errores. Éxito seleccionado: A0/Z; BC/HL se sobrescriben; borra el estado del canal.

Estructuras, restricciones y notas. Vacía en orden la cola secuencial, el espacio sobrante y los metadatos; borra el bit 3 del modo en las rutas de adición establecidas. Las escrituras ignoradas pueden seguir devolviendo éxito. Libera los tramos de registros directos y una fila de archivo abierto. Véase el capítulo 26.

Ejemplo de llamada

```
ld de,peticion
rst 08h
db 21h
```

Usa la petición que corresponde al canal abierto. CLOSE completa el tratamiento de la salida con búfer; no es intercambiable con la terminación genérica de comandos.

0x22 — Leer un registro de un canal

Entrada y parámetros. DE=petición; HL=destino.

Salida, registros y errores. Éxito de disco: A0; EOF por marca o por enlace: A1, posiblemente tras una copia parcial; DE/HL/IX/IY se conservan en las llamadas seleccionadas; BC no es un recuento de bytes.

Estructuras, restricciones y notas. Copia el registro terminado en CR, se detiene antes de 1A y no añade NUL automáticamente. No hay límite visible del destino. Registra el EOF; repetir no copia nada. Las clases de dispositivos se despachan de forma distinta.

Ejemplo de llamada

```
ld de,peticion
ld hl,bufer
rst 08h
db 22h
```

El fragmento selecciona el servicio «Leer un registro de un canal».

0x23 — Escribir un registro en un canal

Entrada y parámetros. DE=petición; HL=origen del registro.

Salida, registros y errores. El contrato completo de longitud del origen, resultado y registros sigue incompleto.

Estructuras, restricciones y notas. Valida el estado de apertura y escritura y despacha a la clase de disco o de dispositivo. Es preferible usar los ejemplos documentados de salida contada o de BASIC en lugar de inventar un contrato de cadenas terminadas.

Ejemplo de llamada

```
ld de,peticion
ld hl,registro
rst 08h
db 23h
```

El fragmento selecciona el servicio «Escribir un registro en un canal».

0x24 — Dar formato a la fecha actual

Entrada y parámetros. DE=destino.

Salida, registros y errores. Da formato a la fecha actual; DE avanza; BC/HL se conservan tras la actualización.

Estructuras, restricciones y notas. Usa el estado BCD 0013–0015 y 6A; sin NUL. La actualización desde el hardware no actualiza el año de forma independiente.

Ejemplo de llamada

```
ld de,bufer
rst 08h
db 24h
```

El fragmento selecciona el servicio «Dar formato a la fecha actual».

0x25 — Dar formato a la hora actual

Entrada y parámetros. DE=destino.

Salida, registros y errores. Escribe exactamente ocho bytes HH:MM:SS; DE avanza 8; BC/HL se guardan durante el formateo.

Estructuras, restricciones y notas. Actualiza la hora y la fecha mediante 2C4D/2C57 a partir de las variables globales BCD 001B–001D; sin NUL.

Ejemplo de llamada

```
ld de,bufer
rst 08h
db 25h
```

El fragmento selecciona el servicio «Dar formato a la hora actual».

0x26 — División sin signo

Entrada y parámetros. HL=dividendo; DE=divisor.

Salida, registros y errores. Cociente sin signo en HL y resto en DE; BC se restaura; A=byte alto del cociente, así que Z comprueba ese byte, no todo el cociente; acarreo si el divisor es cero.

Estructuras, restricciones y notas. Un divisor cero devuelve HL0 y el dividendo original en DE sin provocar ninguna excepción. Los indicadores completos dependen de cada caso; BC/IX/IY preparados se conservan en los controles aritméticos nativos.

Ejemplo de llamada

```
ld hl,7
ld de,2
rst 08h
db 26h
```

Las entradas representan 7 dividido entre 2. Consulta el contrato de cociente y resto anterior; los divisores cero requieren un tratamiento de errores explícito.

0x27 — Multiplicación sin signo

Entrada y parámetros. HL y DE=factores sin signo.

Salida, registros y errores. HL=producto si no hay desbordamiento; BC/DE se restauran; el acarreo indica desbordamiento. A/Z reflejan el estado del multiplicador de trabajo, no si el producto es cero.

Estructuras, restricciones y notas. El desbordamiento puede devolver un acumulador parcial temprano. 9000h×3 devuelve 9000h/acarreo; con los factores invertidos devuelve 3000h/acarreo. No des por supuestos los 16 bits bajos del producto en caso de fallo.

Ejemplo de llamada

```
ld hl,3
ld de,4
rst 08h
db 27h
```

Los factores son 3 y 4. Comprueba el acarreo antes de usar HL como producto válido; un resultado con desbordamiento no es en general la palabra baja del producto matemático.

0x28 — Leer un registro directo numerado

Entrada y parámetros. DE=petición; BC=registro directo numerado desde uno; HL=destino de la longitud del registro.

Salida, registros y errores. Éxito: A0/Z/acarreo 0; fuera de intervalo: A80; BC/DE/HL/IX/IY seleccionados se conservan.

Estructuras, restricciones y notas. Los bloqueos opcionales de sectores con extremos incluidos se mantienen después de la lectura. Un solapamiento retenido del mismo propietario puede omitir la nueva cobertura. Un intervalo fallido deja el destino sin cambios en el control seleccionado.

Ejemplo de llamada

```
ld de,peticion
ld bc,2
ld hl,bufer
rst 08h
db 28h
```

El archivo directo ya debe estar abierto. BC selecciona el registro 2, numerado desde uno; bufer debe tener capacidad para la longitud de registro configurada.

0x29 — Escribir un registro directo numerado

Entrada y parámetros. DE=petición; BC=registro directo numerado desde uno; HL=origen de la longitud del registro.

Salida, registros y errores. Éxito seleccionado: A0/Z/acarreo 0 y BC/DE/HL/IX/IY se conservan; cerrado: AFF; protegido: 0A; fuera de intervalo: 80, según el código y los controles seleccionados.

Estructuras, restricciones y notas. La lectura, modificación y escritura conserva los bytes de alrededor del sector. La política de liberación sigue los bits de la petición. En la ruta de reescritura medida no hay ninguna adquisición explícita nueva de tramos.

Ejemplo de llamada

```
ld de,peticion
ld bc,2
ld hl,registro
rst 08h
db 29h
```

El archivo directo ya debe estar abierto para escritura. BC selecciona el registro 2 y registro aporta sus datos; la protección puede rechazarse en WRITE en lugar de en OPEN.

0x2A — Analizar un decimal o un hexadecimal con sufijo H

Entrada y parámetros. DE=texto numérico.

Salida, registros y errores. HL=valor analizado, DE avanzado; comparte la conservación de registros y los errores de 0E/0F.

Estructuras, restricciones y notas. Mira por adelantado los caracteres hexadecimales y solo selecciona hexadecimal si hay una H mayúscula. FF solo tiene éxito como cero decimal sin consumir nada; FFH→255; 12AB consume 12; 12DH es el hexadecimal 012D.

Ejemplo de llamada

```
ld de,texto_numerico
rst 08h
db 2Ah
```

El fragmento selecciona el servicio «Analizar un decimal o un hexadecimal con sufijo H».

0x2B — Leer un registro indexado/con clave

Entrada y parámetros. DE=petición; BC=búfer de clave fija; HL=destino de los datos.

Salida, registros y errores. A0: éxito; A1/NZ/acarreo 0: no existe, con el destino sin cambios en la ruta indicada; BC/DE/HL/IX/IY se conservan.

Estructuras, restricciones y notas. Puede rellenar con NUL la clave con permiso de escritura y modificar el canal o el área de trabajo. Las disposiciones I y K son distintas; véase el apéndice E.

Ejemplo de llamada

```
ld de,peticion
ld bc,clave
ld hl,bufer
rst 08h
db 2Bh
```

El fragmento selecciona el servicio «Leer un registro indexado/con clave».

0x2C — Leer el siguiente registro indexado/con clave

Entrada y parámetros. DE=petición; BC=destino de la clave con permiso de escritura; HL=destino de los datos.

Salida, registros y errores. Devuelve la clave y los datos siguientes; en las llamadas seleccionadas, A0 y BC/DE/HL se conservan.

Estructuras, restricciones y notas. El orden de I sigue los enlaces ordenados de los huecos; el recorrido de K es distinto. La correspondencia completa de EOF, errores y registros está incompleta.

Ejemplo de llamada

```
ld de,peticion
ld bc,clave
ld hl,bufer
rst 08h
db 2Ch
```

El fragmento selecciona el servicio «Leer el siguiente registro indexado/con clave».

0x2D — Escribir un registro indexado/con clave

Entrada y parámetros. DE=petición; BC=búfer de clave; HL=origen del registro fijo.

Salida, registros y errores. Éxito seleccionado: A0/Z; BC/DE/HL/IX/IY se conservan.

Estructuras, restricciones y notas. Crea o sustituye registros y actualiza el índice, los enlaces, el recuento y el área de trabajo. Si el primer byte de la clave es cero, vuelve inmediatamente según el código. Las rutas de capacidad y de error siguen siendo parciales.

Ejemplo de llamada

```
ld de,peticion
ld bc,clave
ld hl,registro
rst 08h
db 2Dh
```

El fragmento selecciona el servicio «Escribir un registro indexado/con clave».

0x2E — Empaquetar la fecha/hora actuales

Entrada y parámetros. DE=destino de tres bytes.

Salida, registros y errores. Escribe la fecha y la hora actuales empaquetadas.

Estructuras, restricciones y notas. La disposición es la marca de tiempo de tres bytes del apéndice E. El comportamiento completo de los registros y ante horas mal formadas sigue incompleto.

Ejemplo de llamada

```
ld de,fecha_empaquetada
rst 08h
db 2Eh
```

El fragmento selecciona el servicio «Empaquetar la fecha/hora actuales».

0x2F — Resolver una etiqueta de volumen o una unidad de un carácter

Entrada y parámetros. DE=etiqueta rellenada de ocho bytes.

Salida, registros y errores. Etiqueta encontrada: A=índice/Z/acarreo 0; no existe: A1/NZ/acarreo 1; DE/HL/IY se conservan durante el recorrido; B se sustituye antes de guardar BC.

Estructuras, restricciones y notas. Si el byte 1 es un espacio, el byte 0 se interpreta como carácter de unidad. Si no, recorre las asociaciones 0..7 y actualiza los descriptores no válidos.

Ejemplo de llamada

```
ld de,etiqueta
rst 08h
db 2Fh
```

El fragmento selecciona el servicio «Resolver una etiqueta de volumen o una unidad de un carácter».

0x30 — Obtener el contexto de la partición actual

Entrada y parámetros. Ninguna entrada explícita.

Salida, registros y errores. IY=palabra 0033; los demás registros del destino no cambian.

Estructuras, restricciones y notas. Devuelve el contexto actual, no la base de las variables globales comunes. GETUSER usa su campo de nombre.

Ejemplo de llamada

```
rst 08h
db 30h
```

IY recibe el contexto de la partición actual. La disposición de los campos está en el apéndice E; no es una estructura de C proporcionada por un entorno de ejecución externo.

0x31 — Pausa de paginación

Entrada y parámetros. Terminal y contexto actuales.

Salida, registros y errores. El envoltorio conserva AF/BC/IY.

Estructuras, restricciones y notas. Marca de paginación y espera de entrada; acepta espacio, CR, LF o Ctrl + X. Los indicadores del contexto pueden suprimir la pausa.

Ejemplo de llamada

```
rst 08h
db 31h
```

El fragmento selecciona el servicio «Pausa de paginación».

0x32 — Leer varios sectores lógicos

Entrada y parámetros. B=unidad; C=número de sectores; DE=LBA inicial; HL=destino.

Salida, registros y errores. Determinadas llamadas con C8 devuelven AF0044 y conservan BC/DE/HL/IX/IY.

Estructuras, restricciones y notas. Comparte la política de errores de 0A. C=0, los fallos parciales y la atomicidad siguen sin resolverse. EXEC usa C2 para marcos de 512 bytes.

Ejemplo de llamada

```
ld b,0
ld c,2
ld de,1
ld hl,bufer
rst 08h
db 32h
```

Así se leen dos sectores a partir del sector lógico 1 de S en al menos 512 bytes de almacenamiento con permiso de escritura. Las peticiones físicas pueden incluir una actualización preliminar del descriptor.

0x33 — Escribir varios sectores lógicos

Entrada y parámetros. B=unidad; C=número de sectores; DE=LBA inicial; HL=origen.

Salida, registros y errores. Determinadas llamadas con C8 devuelven AF0044 y conservan BC/DE/HL/IX/IY y el origen.

Estructuras, restricciones y notas. Comparte la política de 0B; las escrituras pueden incluir sectores sin cambios. Se desconocen el comportamiento con recuento 0 y la atomicidad ante fallos parciales.

Ejemplo de llamada

```
ld b,0
ld c,2
ld de,sector_destino
ld hl,bufer
rst 08h
db 33h
```

Así se escriben dos sectores a partir de al menos 512 bytes de datos de quien llama. sector_destino debe ser un almacenamiento válido, con permiso de escritura y seleccionado deliberadamente; el fragmento no es una prueba de disco genérica y segura.

0x34 — Mostrar una cadena con determinados indicadores de modo suprimidos

Entrada y parámetros. DE=cadena, como en 02.

Salida, registros y errores. Restaura contexto+0A/+0B e IY.

Estructuras, restricciones y notas. Borra temporalmente los bits 1/2 de 0A y el bit 7 de 0B antes de la presentación normal de la cadena. Los demás efectos sobre la cadena son los de 02.

Ejemplo de llamada

```
ld de,mensaje
rst 08h
db 34h
```

El fragmento selecciona el servicio «Mostrar una cadena con determinados indicadores de modo suprimidos».

0x35 — Programar, sustituir o cancelar una devolución de llamada temporizada

Entrada y parámetros. HL=seis bytes con permiso de escritura seguidos de la devolución de llamada; DE=retardo; A0 normal.

Salida, registros y errores. Las llamadas seleccionadas conservan AF/BC/DE/HL/IX/IY; vuelve con las interrupciones habilitadas.

Estructuras, restricciones y notas. DE/8 pulsos diferenciales; DE0 cancela; al vencer, borra el nodo e invoca +6 en el contexto propietario. La excepción de cabeza con el bit 0 de A no es una cancelación general entre bancos. Véase el capítulo 25.

Ejemplo de llamada

```
xor a
ld hl,nodo_temporizador
ld de,80
rst 08h
db 35h
```

nodo_temporizador consta de seis bytes con permiso de escritura seguidos inmediatamente del código de la devolución de llamada. El retardo 80 se convierte en diez pulsos de ocho milisegundos. La devolución de llamada se ejecuta en el contexto al que pertenece.

0x36 — Sustituir la entrada de comandos y volver a CSI

Entrada y parámetros. DE=comando de sustitución terminado en CR.

Salida, registros y errores. No vuelve; limpieza y después CSI con el resultado 0.

Estructuras, restricciones y notas. Copia los imprimibles 20–7E, convierte las minúsculas, descarta los demás y activa el bit 1 de contexto+38. No hay protección de longitud visible. No está sujeto al presupuesto de entrada interactivo.

Ejemplo de llamada

```
ld de,comando
rst 08h
db 36h
```

El fragmento selecciona el servicio «Sustituir la entrada de comandos y volver a CSI». Es una operación interna o de configuración, no un atajo normal para preparar aplicaciones.

0x37 — Obtener el límite superior de memoria de la partición

Entrada y parámetros. Ninguna entrada explícita.

Salida, registros y errores. HL=límite superior de memoria en contexto+04; IY se conserva.

Estructuras, restricciones y notas. El límite cambia a medida que los controladores y las clases ocupan la memoria superior.

Ejemplo de llamada

```
rst 08h
db 37h
```

HL recibe el límite superior de memoria actual. Los controladores y las clases cargados pueden reducirlo; no lo sustituyas por una dirección máxima fija.

0x38 — Establecer el límite superior de memoria de la partición

Entrada y parámetros. HL=nuevo límite superior de memoria.

Salida, registros y errores. Lo almacena en contexto+04; IY se conserva.

Estructuras, restricciones y notas. No hay ningún contrato general de asignación ni de validación. Cambiar solo este puntero no asigna memoria de forma segura.

Ejemplo de llamada

```
ld hl,nuevo_limite
rst 08h
db 38h
```

nuevo_limite debe ajustarse a la organización válida de la memoria de la partición y al código que se conserva. Un valor mayor arbitrario no asigna más memoria.

0x39 — Establecer la devolución de llamada de la palabra 28 del contexto

Entrada y parámetros. HL=devolución de llamada o cero.

Salida, registros y errores. Lo almacena en contexto+28; IY se conserva.

Estructuras, restricciones y notas. Devolución de llamada de interrupción y control que usan CSI y los controladores. El defecto de restauración de SS2 corresponde a quien la llama, no a esta rutina de almacenamiento.

Ejemplo de llamada

```
ld hl,dev_llamada
rst 08h
db 39h
```

El fragmento selecciona el servicio «Establecer la devolución de llamada de la palabra 28 del contexto». Es una operación interna o de configuración, no un atajo normal para preparar aplicaciones.

0x3A — Resolver una asociación

Entrada y parámetros. B=asociación 0..23.

Salida, registros y errores. A=índice físico; válida: acarreo 0/Z0; no válida o sin asignar: AFF/Z1/acarreo 1; BC/IY se conservan.

Estructuras, restricciones y notas. El mapa empieza en contexto+7D. La asociación lógica, el descriptor físico y el índice de partición son valores distintos.

Ejemplo de llamada

```
ld b,9
rst 08h
db 3Ah
```

El fragmento selecciona el servicio «Resolver una asociación».

0x3B — Obtener los campos de geometría del terminal asociado

Entrada y parámetros. B=asociación, normalmente 8/9.

Salida, registros y errores. B = byte en el descriptor del dispositivo físico +0x02; C = byte en el descriptor +0x03; A = byte en el descriptor +0x04 (número de clase de terminal). HL/DE se conservan; asociación sin resolver: BC=0x4F17/acarreo.

Estructuras, restricciones y notas. L es el presupuesto de entrada de CSI y un umbral de salida; P es el límite de filas. No afirmes un 80 × 24 genérico a partir de 79/23.

Ejemplo de llamada

```
ld b,9
rst 08h
db 3Bh
```

El fragmento selecciona el servicio «Obtener los campos de geometría del terminal asociado».

Los desplazamientos anteriores corresponden a campos del descriptor del dispositivo físico, no a bytes del ejecutable de la clase de terminal. La disposición del descriptor se recoge en el apéndice E.

0x3C — Borrar un registro indexado/con clave

Entrada y parámetros. DE=petición; BC=clave.

Salida, registros y errores. Borra el registro I/K encontrado; si no existe, devuelve éxito según el código; las garantías completas sobre los registros están incompletas.

Estructuras, restricciones y notas. Marca una lápida con FF y ajusta las referencias ordenadas y el recuento en I. No es un borrado de archivo.

Ejemplo de llamada

```
ld de,peticion
ld bc,clave
rst 08h
db 3Ch
```

El fragmento selecciona el servicio «Borrar un registro indexado/con clave». Es una operación interna o de configuración, no un atajo normal para preparar aplicaciones.

0x3D — Despacho de inicialización de dispositivos

Entrada y parámetros. B=dispositivo de caracteres asociado.

Salida, registros y errores. Resultado de controlador+09, no un estado de éxito uniforme; BC/DE/HL/IX/IY seleccionados se conservan.

Estructuras, restricciones y notas. Puede modificar código, registrar devoluciones de llamada o salir de forma no local. En los controladores de disquete, +09 significa escritura; no lo llames como inicialización.

Ejemplo de llamada

```
ld b,asociacion_disp
rst 08h
db 3Dh
```

El fragmento selecciona el servicio «Despacho de inicialización de dispositivos». Es una operación interna o de configuración, no un atajo normal para preparar aplicaciones.

0x3E — Despacho de estado de dispositivos

Entrada y parámetros. B=asociación.

Salida, registros y errores. Resultado de controlador+00; las llamadas serie seleccionadas conservan BC/DE/HL/IX/IY.

Estructuras, restricciones y notas. En ALP, Z describe la ocupación de la entrada y el acarreo la disponibilidad del DMA; el sondeo puede vaciar la salida. SS2, las impresoras y la cinta son distintos. Una asociación sin asignar puede devolver 0044, no un error diferenciado.

Ejemplo de llamada

```
ld b,8
rst 08h
db 3Eh
```

El fragmento selecciona el servicio «Despacho de estado de dispositivos».

0x3F — Despacho de entrada de dispositivos

Entrada y parámetros. B=asociación.

Salida, registros y errores. +03 del controlador de caracteres; en las llamadas serie seleccionadas, A=byte/acarreo 0, con BC/DE/HL/IX/IY conservados.

Estructuras, restricciones y notas. Puede bloquearse y volver a comprobar tras los eventos. Una asociación sin asignar devuelve cero en el caso especificado; los valores de los RET de impresora son accidentales; la rutina puente de cinta no lee ningún registro; en los disquetes, +03 significa petición de recalibrado.

Ejemplo de llamada

```
ld b,8
rst 08h
db 3Fh
```

El fragmento selecciona el servicio «Despacho de entrada de dispositivos».

0x40 — Despacho de salida de dispositivos

Entrada y parámetros. B=asociación; C=carácter para la línea serie o la impresora.

Salida, registros y errores. AF depende del controlador y de la ruta; BC/DE/HL/IX/IY seleccionados se conservan.

Estructuras, restricciones y notas. La consola en bruto omite la clase y el cursor. La política de impresoras y el gancho del SPOOLER se aplican según la asociación. Que vuelva una salida serie con búfer no significa que la transmisión haya terminado. No es la ABI de registros del controlador de disco.

Ejemplo de llamada

```
ld b,9
ld c,65
rst 08h
db 40h
```

El fragmento selecciona el servicio «Despacho de salida de dispositivos».

0x41 — Prólogo estándar de privilegio/número de serie de los comandos

Entrada y parámetros. Bytes anteriores a CF41: privilegio mínimo en P-4; número de serie en P-3..P-1.

Salida, registros y errores. Si se supera, continúa; si falla, muestra 19/20 y termina con CR 252.

Estructuras, restricciones y notas. Si el primer byte del número de serie es 0, se omite la vinculación. El bit 5 de contexto+0C puede solicitar que se muestre la versión. Las comprobaciones adicionales de cada utilidad son independientes.

Ejemplo de llamada

```
db 1,0,0,0
entrada: rst 08h
db 41h
```

La ejecución debe entrar por el RST, no por los datos que lo preceden. Este ejemplo requiere el privilegio 1 y usa un campo de número de serie todo a ceros. El prólogo es una protección por software; no eleva el privilegio.

0x42 — Copiar la etiqueta del volumen

Entrada y parámetros. B=unidad; DE=ocho bytes con permiso de escritura.

Salida, registros y errores. Copia la etiqueta; DE+=8; BC/HL se conservan.

Estructuras, restricciones y notas. El destino carece de una rama explícita con acarreo para asociaciones sin resolver. Valida la asociación antes de confiar en los bytes copiados.

Ejemplo de llamada

```
ld b,0
ld de,etiqueta
rst 08h
db 42h
```

El fragmento selecciona el servicio «Copiar la etiqueta del volumen».

0x43 — Establecer la entrada del controlador del dispositivo físico

Entrada y parámetros. B=índice físico 8..31; HL=entrada del controlador.

Salida, registros y errores. Almacena la palabra del descriptor; DE se conserva; HL termina en descriptor+1.

Estructuras, restricciones y notas. Con B fuera de intervalo, vuelve sin almacenar nada. Es la instalación de un descriptor físico, no una asociación lógica.

Ejemplo de llamada

```
ld b,indice_fisico
ld hl,entrada_controlador
rst 08h
db 43h
```

El fragmento selecciona el servicio «Establecer la entrada del controlador del dispositivo físico». Es una operación interna o de configuración, no un atajo normal para preparar aplicaciones.

0x44 — Terminar/descargar el controlador asociado

Entrada y parámetros. B=asociación de dispositivo de caracteres.

Salida, registros y errores. Controlador+0C y descarga condicional; AF no es uniforme.

Estructuras, restricciones y notas. Borra los campos de clase; la rama del límite superior coincidente recupera el código cargado. No hace CLOSE de los archivos de las aplicaciones. DIS1/2 no tienen quinta entrada; la retirada de discos usa otra ruta.

Ejemplo de llamada

```
ld b,asociacion_disp
rst 08h
db 44h
```

El fragmento selecciona el servicio «Terminar/descargar el controlador asociado». Es una operación interna o de configuración, no un atajo normal para preparar aplicaciones.

0x45 — Sondear y consumir el indicador de evento de consola

Entrada y parámetros. Consola actual.

Salida, registros y errores. A1/NZ si estaba activado el bit 5 de contexto+0B; si no, A0/Z; borra el bit; IY se conserva.

Estructuras, restricciones y notas. Llama primero a 03. La interpretación exacta de los eventos fuera de las rutas de cancelación observadas sigue incompleta.

Ejemplo de llamada

```
rst 08h
db 45h
```

El fragmento selecciona el servicio «Sondear y consumir el indicador de evento de consola».

0x46 — Sustituir la entrada de comandos con límite superior de continuación

Entrada y parámetros. DE=comando terminado en CR; HL=límite superior de continuación.

Salida, registros y errores. Normalmente no vuelve; establece el límite superior y SP y el bit 5 de contexto+38, y después 36.

Estructuras, restricciones y notas. Más tarde, CSI borra el bit 5 y salta al límite superior. No está especificada una receta completa y segura de pila y registros para quien llama.

Ejemplo de llamada

```
ld de,comando
ld hl,continuacion
rst 08h
db 46h
```

El fragmento selecciona el servicio «Sustituir la entrada de comandos con límite superior de continuación». Es una operación interna o de configuración, no un atajo normal para preparar aplicaciones.

0x47 — Leer un byte de una cola en memoria

Entrada y parámetros. HL=cabecera y datos de la cola.

Salida, registros y errores. Vacía: A0/acarreo 1, sin cambios; con datos: A=dato/acarreo 0; BC/DE/HL/IX/IY seleccionados se conservan.

Estructuras, restricciones y notas. La ruta con datos habilita las interrupciones; al vaciarse, se mantiene el índice de lectura anterior. Con capacidades 254/255, la aritmética de ocho bits puede alcanzar la cabecera. Véase el capítulo 25.

Ejemplo de llamada

```
ld hl,cola
rst 08h
db 47h
```

cola debe tener la cabecera y el almacenamiento descritos en el apéndice E. Comprueba el acarreo para detectar la cola vacía; un byte cero devuelto puede ser un dato válido.

0x48 — Añadir un byte a una cola en memoria

Entrada y parámetros. HL=cola; A=byte.

Salida, registros y errores. Las llamadas normales seleccionadas conservan A/BC/DE/HL/IX/IY; los indicadores cambian; habilita las interrupciones.

Estructuras, restricciones y notas. El bucle de cola llena guarda en caché la ocupación y puede seguir girando incluso después de que una devolución de llamada retire datos. Las capacidades superiores a 128 tienen defectos de indexación demostrados. No hay ningún retorno de error seguro para la cola llena.

Ejemplo de llamada

```
ld hl,cola
ld a,65
rst 08h
db 48h
```

La cola debe tener capacidad libre antes de añadir el código de carácter 65. Una cola llena puede entrar en un bucle que no detecta el avance normal del consumidor.

0x49 — Cambiar los bits de modo de la consola

Entrada y parámetros. B=máscara de activación; C=máscara de borrado.

Salida, registros y errores. A=nuevo contexto+0A; HL/IY se restauran; B/C se enmascaran en el sitio.

Estructuras, restricciones y notas. Solo se aplican los bits 2 a 7; conserva los bits 0/1. Usa DI/EI. MENSAJE OFF es, en cambio, el bit 7 de contexto+0C.

Ejemplo de llamada

```
ld b,mascara_activar
ld c,mascara_borrar
rst 08h
db 49h
```

El fragmento selecciona el servicio «Cambiar los bits de modo de la consola». Es una operación interna o de configuración, no un atajo normal para preparar aplicaciones.

0x4A — Establecer la devolución de llamada de error de disco

Entrada y parámetros. HL=devolución de llamada de error de disco o cero.

Salida, registros y errores. El registro seleccionado conserva AF/BC/DE/HL/IX/IY.

Estructuras, restricciones y notas. La devolución de llamada se ejecuta con el bloqueo activo antes de decrementar los reintentos: A0 ignora, un valor positivo sustituye el error, un valor negativo anula. Conserva los registros de trabajo. Las llamadas de disco reentrantes no están especificadas.

Ejemplo de llamada

```
ld hl,dev_llamada_error
rst 08h
db 4Ah
```

El fragmento selecciona el servicio «Establecer la devolución de llamada de error de disco». Es una operación interna o de configuración, no un atajo normal para preparar aplicaciones.

0x4B — Inicializar o cambiar el volumen del sistema

Entrada y parámetros. B=FF inicialización en frío; en otro caso, nueva asociación del sistema.

Salida, registros y errores. Entra en la ruta de inicialización y carga de CSI; no es una consulta que vuelva.

Estructuras, restricciones y notas. Localiza los componentes del sistema y escribe el estado del reloj en el núcleo. Véase el capítulo 21.

Ejemplo de llamada

```
ld b,nueva_asociacion_sistema
rst 08h
db 4Bh
```

El fragmento selecciona el servicio «Inicializar o cambiar el volumen del sistema». Es una operación interna o de configuración, no un atajo normal para preparar aplicaciones.

0x4C — Retardo activo escalado

Entrada y parámetros. A: 6 bits bajos=recuento; 2 bits altos=escala 1/10/100/1000.

Salida, registros y errores. Si los 6 bits bajos son cero, vuelve inmediatamente; BC se restaura; A0/Z en el retorno normal.

Estructuras, restricciones y notas. Retardo activo escalado basado en 13. La unidad absoluta sigue sin resolverse.

Ejemplo de llamada

```
ld a,1
rst 08h
db 4Ch
```

El fragmento selecciona el servicio «Retardo activo escalado».

0x4D — Obtener una entrada de la tabla de canales

Entrada y parámetros. B=canal 0..16.

Salida, registros y errores. HL=contexto+AC+32*B; no válido: acarreo.

Estructuras, restricciones y notas. Devuelve un canal interno de 32 bytes, no la petición de cuatro bytes de quien llama.

Ejemplo de llamada

```
ld b,1
rst 08h
db 4Dh
```

El fragmento selecciona el servicio «Obtener una entrada de la tabla de canales».

0x4E — Despachar un comando de control del terminal

Entrada y parámetros. A=letra de control.

Salida, registros y errores. Depende de la ruta; puede cambiar la pila o el flujo de control, o no volver.

Estructuras, restricciones y notas. Normaliza mayúsculas y minúsculas; reconoce Q, D, O, P, I, W, A, B, S,], ^ y C. El comportamiento establecido de Q/S/O/I/C se recoge en los capítulos 4, 14 y 28; las demás combinaciones siguen siendo parciales.

Ejemplo de llamada

```
ld a,letra_control
rst 08h
db 4Eh
```

El fragmento selecciona el servicio «Despachar un comando de control del terminal». Es una operación interna o de configuración, no un atajo normal para preparar aplicaciones.

0x4F — Ceder el control / planificar

Entrada y parámetros. Partición actual.

Salida, registros y errores. Guarda y restaura el marco de registros y selecciona un contexto ejecutable; el estado de habilitación de interrupciones cambia.

Estructuras, restricciones y notas. Primer elegible en orden circular, preferencia por el resto de las devoluciones de llamada y anomalías del cuanto de un byte. La ruta de una sola partición habilita las interrupciones. Véase el capítulo 25.

Ejemplo de llamada

```
rst 08h
db 4Fh
```

El fragmento selecciona el servicio «Ceder el control / planificar».

0x50 — Obtener la base global

Entrada y parámetros. Ninguna entrada explícita.

Salida, registros y errores. IY=0.

Estructuras, restricciones y notas. Base de las variables globales comunes, usada por la política de números de serie de BASIC. No implica que sea seguro cambiar los campos con permiso de escritura.

Ejemplo de llamada

```
rst 08h
db 50h
```

El fragmento selecciona el servicio «Obtener la base global».

0x51 — Leer el byte global 003B

Entrada y parámetros. Ninguna entrada explícita.

Salida, registros y errores. A=byte 003B.

Estructuras, restricciones y notas. Existe un consumidor relacionado con el número de serie, pero no se ha establecido un nombre semántico independiente completo. Los demás efectos sobre los registros son solo los del breve destino.

Ejemplo de llamada

```
rst 08h
db 51h
```

El fragmento selecciona el servicio «Leer el byte global 003B».

0x52 — Obtener el índice actual del planificador

Entrada y parámetros. Ninguna entrada explícita.

Salida, registros y errores. A=índice actual del planificador, empezando en cero (000C).

Estructuras, restricciones y notas. No es el número de cuenta, ni el número de banco, ni el número de partición mostrado.

Ejemplo de llamada

```
rst 08h
db 52h
```

El fragmento selecciona el servicio «Obtener el índice actual del planificador».

0x53 — Liberar los bloqueos de registros del canal

Entrada y parámetros. DE=petición con el canal 0..16 en el byte 0.

Salida, registros y errores. Válido: A0/Z; BC/DE/HL/IX/IY seleccionados se conservan; no válido: A04/NZ según el código.

Estructuras, restricciones y notas. Libera los tramos de registros del propietario actual y del canal. No libera la admisión exclusiva de apertura de archivo. La ruta de una sola partición omite la operación con la tabla.

Ejemplo de llamada

```
ld de,peticion
rst 08h
db 53h
```

El fragmento selecciona el servicio «Liberar los bloqueos de registros del canal».

0x54 — Adquirir un bloqueo de byte

Entrada y parámetros. HL=bloqueo de un byte.

Salida, registros y errores. Las llamadas inmediatas y diferidas seleccionadas conservan AF/BC/DE/HL/IX/IY.

Estructuras, restricciones y notas. Cero→propietario=índice+1; el mismo propietario vuelve sin recuento de recursión; otro propietario activa el bit de espera 4, cede el control y vuelve a comprobar. Se desconocen la conservación de IFF, la equidad y los tiempos de espera.

Ejemplo de llamada

```
ld hl,byte_bloqueo
rst 08h
db 54h
```

byte_bloqueo debe ser un byte compartido inicializado a cero antes de usarse. La adquisición registra la propiedad y puede ceder el control mientras espera. La adquisición repetida por el propietario no añade ningún recuento de recursión.

0x55 — Liberar un bloqueo de byte

Entrada y parámetros. HL=bloqueo de un byte.

Salida, registros y errores. Las llamadas seleccionadas conservan AF/BC/DE/HL/IX/IY.

Estructuras, restricciones y notas. Si el propietario es otro, se ignora sin indicador de error. La liberación propia borra el bit 4 en todas las filas y después el bloqueo; el despertar es por difusión y requiere volver a comprobar.

Ejemplo de llamada

```
ld hl,byte_bloqueo
rst 08h
db 55h
```

Libera solo según el protocolo de propiedad del bloqueo. Las particiones despertadas vuelven a comprobar sus propias condiciones; el despertar no concede por sí mismo un bloqueo.

0x56 — Entrar en una devolución de llamada en la partición seleccionada

Entrada y parámetros. A: 7 bits bajos=índice de destino, bit 7=suprimir EI; HL=devolución de llamada en la correspondencia del destino.

Salida, registros y errores. Restaura el marco del destino y salta a la devolución de llamada; el RET de la devolución de llamada reanuda el destino.

Estructuras, restricciones y notas. Enmascara el nibble alto de estado, incluido STOP. Los argumentos del emisor no se restauran como registros del destino. Los destinos no válidos o anidados no tienen un contrato general seguro. Véase el capítulo 25.

Ejemplo de llamada

```
ld a,indice_destino
ld hl,dev_llamada
```

```
rst 08h
db 56h
```

El fragmento selecciona el servicio «Entrar en una devolución de llamada en la partición seleccionada». Es una operación interna o de configuración, no un atajo normal para preparar aplicaciones.

0x57 — Obtener la base del mapa de asociaciones

Entrada y parámetros. Ninguna entrada explícita.

Salida, registros y errores. IY=contexto+7D; DE se conserva.

Estructuras, restricciones y notas. Apunta a 24 bytes de asociación. Sin asignar es FF.

Ejemplo de llamada

```
rst 08h
db 57h
```

El fragmento selecciona el servicio «Obtener la base del mapa de asociaciones».

0x58 — Enviar texto a la consola de otra partición

Entrada y parámetros. A=índice de destino; DE=cadena del emisor.

Salida, registros y errores. BC/HL/IX/IY seleccionados se conservan; DE avanza al enviar y no cambia al omitir; A no es un estado genérico.

Estructuras, restricciones y notas. Protocolo de texto de siete bits mediante la devolución de llamada del destinatario y el bloqueo 21DD. Las omisiones por destino propio o fuera de intervalo pueden liberar un bloqueo que ya pertenecía a quien llama; aquí no se comprueba la política OFF. Véase el capítulo 29.

Ejemplo de llamada

```
ld a,indice_destino
ld de,mensaje
rst 08h
db 58h
```

El fragmento selecciona el servicio «Enviar texto a la consola de otra partición».

0x59 — Adquirir el bloqueo global fijo

Entrada y parámetros. Ninguna entrada explícita.

Salida, registros y errores. Adquiere el bloqueo 0006 mediante 54; restaura el HL de quien llama.

Estructuras, restricciones y notas. Misma semántica de propiedad y espera, y mismas limitaciones, que 54.

Ejemplo de llamada

```
rst 08h
db 59h
```

El fragmento selecciona el servicio «Adquirir el bloqueo global fijo».

0x5A — Liberar el bloqueo global fijo

Entrada y parámetros. Ninguna entrada explícita.

Salida, registros y errores. Libera el bloqueo 0006 mediante 55; restaura el HL de quien llama.

Estructuras, restricciones y notas. Misma semántica de difusión y de no propietario, y mismas limitaciones, que 55.

Ejemplo de llamada

```
rst 08h
db 5Ah
```

El fragmento selecciona el servicio «Liberar el bloqueo global fijo».

0x5B — Obtener el área de trabajo de sector

Entrada y parámetros. Ninguna entrada explícita.

Salida, registros y errores. HL=contexto+2CC; DE se conserva.

Estructuras, restricciones y notas. Devuelve el área de trabajo de sector de 256 bytes. Otros servicios pueden usarla internamente; evita suposiciones no admitidas sobre su vida útil.

Ejemplo de llamada

```
rst 08h
db 5Bh
```

El fragmento selecciona el servicio «Obtener el área de trabajo de sector».

0x5C — Obtener el byte de privilegio

Entrada y parámetros. Ninguna entrada explícita.

Salida, registros y errores. A=privilegio en contexto+11; IY se conserva; BC/DE/HL/IX/IY seleccionados se conservan.

Estructuras, restricciones y notas. GETPRIV lo amplía con ceros a HL. La función de consulta no eleva la autoridad.

Ejemplo de llamada

```
rst 08h
db 5Ch
```

El resultado en A es el privilegio actual. Leerlo no cambia la cuenta ni concede un privilegio superior.

0x5D — Comparar memoria

Entrada y parámetros. BC=recuento; DE/HL=intervalos de memoria.

Salida, registros y errores. Compara hasta encontrar una diferencia o agotar el recuento; los punteros y el recuento avanzan.

Estructuras, restricciones y notas. No está establecido que un BC0 inicial sea una comparación vacía segura. El contrato completo de indicadores y registros fuera del uso de comparación sigue incompleto.

Ejemplo de llamada

```
ld bc,4
ld de,primeros_bytes
ld hl,segundos_bytes
rst 08h
db 5Dh
```

El fragmento selecciona el servicio «Comparar memoria».

0x5E — Leer bytes secuenciales contados

Entrada y parámetros. DE=petición; HL=destino; B=recuento, 0=256.

Salida, registros y errores. A0 normal, A1 EOF, posiblemente después de copiar; DE/HL/IX/IY seleccionados se conservan, BC cambia.

Estructuras, restricciones y notas. Flujo de datos en bruto de 254 bytes, lectura anticipada del sector siguiente, sin terminador. Una lectura que termina exactamente en el límite final puede copiar todos los bytes y devolver EOF. Un EOF registrado impide copias posteriores; véase el capítulo 26.

Ejemplo de llamada

```
ld de,peticion
ld hl,bufer
ld b,4
rst 08h
db 5Eh
```

La petición secuencial abierta aporta cuatro bytes a bufer. Un retorno en el límite puede indicar EOF incluso después de copiar todos los bytes solicitados; examina el estado documentado.

0x5F — Escribir bytes secuenciales contados

Entrada y parámetros. DE=petición; HL=origen; B=recuento.

Salida, registros y errores. Selección: A0/Z/acarreo 0; DE/HL/IX/IY se restauran; BC puede cambiar.

Estructuras, restricciones y notas. Un incremento del recuento del canal por llamada. El búfer parcial no es duradero hasta CLOSE; el comportamiento del bucle de 256 con B0 está definido por la implementación. EXTEND puede recortar más tarde los 1A finales. BASIC puede cambiar antes la petición a A8/AC.

Ejemplo de llamada

```
ld de,peticion
ld hl,registro
ld b,4
rst 08h
db 5Fh
```

La petición de salida abierta acepta cuatro bytes de registro. Pueden quedar en el búfer hasta un CLOSE explícito; esta llamada no garantiza la persistencia.

0x60 — Mostrar un mensaje del sistema numerado

Entrada y parámetros. HL=registro de mensaje numerado desde uno; B=unidad del sistema; DE=campos de sustitución de ocho bytes.

Salida, registros y errores. BC/DE/HL/IY se guardan.

Estructuras, restricciones y notas. Lee registros de 64 bytes en el sector 25h+(HL-1)//4, desplazamiento 64*((HL-1)%4). Los bits 6/7 de la variable global 000E afectan a la presentación. Aquí no se comprueban los límites de los números no válidos.

Ejemplo de llamada

```
ld hl,1
ld b,0
ld de,sustituciones
rst 08h
db 60h
```

El registro 1 es el mensaje de error de sintaxis. sustituciones apunta al área de sustitución de anchura fija de quien llama, aunque el texto elegido no necesite sustituciones.

0x61 — Mostrar un mensaje numerado y finalizar el comando

Entrada y parámetros. Como en 60, más A=resultado del comando.

Salida, registros y errores. Muestra el mensaje y termina mediante 00; no vuelve.

Estructuras, restricciones y notas. El número de mensaje y el resultado del comando son números distintos.

Ejemplo de llamada

```
ld hl,1
ld b,0
ld de,sustituciones
ld a,8
rst 08h
db 61h
```

Así se muestra el mensaje 1 y se termina con el resultado 8. No vuelve a la instrucción situada después del selector.

0x62 — Cargar un submódulo R a partir de un descriptor compacto

Entrada y parámetros. A=formato; B=unidad; DE=LBA base; HL=descriptor de módulo de siete bytes.

Salida, registros y errores. Carga el submódulo R mediante metadatos construidos; la matriz completa de retornos y errores está incompleta.

Estructuras, restricciones y notas. Descriptor: carga LE16, bytes de imagen LE16, byte de sectores almacenados, LBA relativo LE16. Lo usa BASIC; no inventes un comportamiento general de enlazador de objetos.

Ejemplo de llamada

```
ld a,formato_modulo
ld b,0
```

```
ld de,sector_base
ld hl,descriptor_modulo
rst 08h
db 62h
```

El fragmento selecciona el servicio «Cargar un submódulo R a partir de un descriptor compacto». Es una operación interna o de configuración, no un atajo normal para preparar aplicaciones.

0x63 — Asignar un tramo de bloques con tamaño mínimo

Entrada y parámetros. B=unidad; DE=bloques deseados; HL=mínimo, ambos distintos de cero.

Salida, registros y errores. Éxito: A0/Z/acarreo 0, HL=primer LBA, DE=recuento; sin tramo: AFF/NZ/acarreo 0; asociación sin resolver: acarreo.

Estructuras, restricciones y notas. Primer tramo del tamaño deseado o, si no, el mayor tramo que cumpla los requisitos, con preferencia por el posterior en caso de empate. BC cambia; IX/IY seleccionados se conservan. Comparte con 1B/1C la rutina de actualización del mapa de bits con bloqueo.

Ejemplo de llamada

```
ld b,0
ld de,4
ld hl,1
rst 08h
db 63h
```

El tramo deseado es de cuatro bloques y el mínimo aceptable, de uno. El asignador puede devolver un tramo más pequeño que cumpla los requisitos; usa el recuento devuelto, no el solicitado.

0x64 — Buscar en una tabla de palabras clave abreviables

Entrada y parámetros. DE=token; HL=tabla de palabras clave.

Salida, registros y errores. Coincidencia: A0/Z, HL=resultado; ausente: A1/NZ; DE se restaura; BC se sobrescribe.

Estructuras, restricciones y notas. Registro: byte de prefijo mínimo, palabra clave con el bit 7 activado en el último carácter, resultado LE16; un mínimo cero termina la tabla. Se aceptan A–Z/0–9/$; quien llama convierte las mayúsculas y minúsculas. Gana la primera coincidencia que cumpla los requisitos.

Ejemplo de llamada

```
ld de,token
ld hl,palabras_clave
```

```
rst 08h
db 64h
```

El fragmento selecciona el servicio «Buscar en una tabla de palabras clave abreviables».

0x65 — Obtener el número de cuenta

Entrada y parámetros. Ninguna entrada explícita.

Salida, registros y errores. A=número de cuenta en contexto+1A; IY se conserva.

Estructuras, restricciones y notas. GETID lo amplía con ceros a HL. La cuenta pública 0, la temporal FF y la propiedad de bloqueos por partición son conceptos distintos.

Ejemplo de llamada

```
rst 08h
db 65h
```

A recibe el número de cuenta. Es distinto del servicio 0x52, que devuelve el índice del planificador.

0x66 — Filtrar la entrada del dispositivo para detectar controles de escape

Entrada y parámetros. A=carácter de entrada; IY=descriptor del dispositivo.

Salida, registros y errores. La ruta habilitada restaura BC/HL; la devolución de llamada puede cambiar otros estados; en las rutas establecidas, el acarreo significa que se ha consumido.

Estructuras, restricciones y notas. El bit 7 de descriptor+05 lo habilita y el bit 6 registra el escape; devolución de llamada opcional en +16. Llama a 4E tras el tratamiento del escape y de mayúsculas y minúsculas. No es un filtro universal de caracteres en bruto.

Ejemplo de llamada

```
ld a,byte_entrada
ld iy,descriptor_disp
rst 08h
db 66h
```

El fragmento selecciona el servicio «Filtrar la entrada del dispositivo para detectar controles de escape». Es una operación interna o de configuración, no un atajo normal para preparar aplicaciones.

0x67 — Registrar una devolución de llamada indexada

Entrada y parámetros. A=índice 02..20h; DE=devolución de llamada.

Salida, registros y errores. Almacena el propietario y el puntero en 23FF+3*A; AF/BC/DE/HL/IX/IY seleccionados se conservan.

Estructuras, restricciones y notas. Según el código, un intervalo no válido vuelve sin almacenar nada. No ejecuta DI por sí mismo; se desconoce la atomicidad si quien llama tiene las interrupciones habilitadas. Las filas de propietarios de DMA no tienen por qué invocar la devolución de llamada almacenada.

Ejemplo de llamada

```
ld a,indice_dev_llamada
ld de,dev_llamada
rst 08h
db 67h
```

El fragmento selecciona el servicio «Registrar una devolución de llamada indexada». Es una operación interna o de configuración, no un atajo normal para preparar aplicaciones.

0x68 — Copiar desde el banco del registro seleccionado

Entrada y parámetros. A=índice de registro; HL=origen en ese registro; DE=destino en quien llama; BC=recuento positivo.

Salida, registros y errores. BC0; HL/DE avanzan; A/IX/IY se conservan; vuelve con DI.

Estructuras, restricciones y notas. Entre bancos conserva AF y usa la pila común; en el mismo banco, LDIR cambia F y los solapamientos se propagan hacia delante. No valida punteros, índices ni recuentos cero. Los punteros de identidad actuales no cambian.

Ejemplo de llamada

```
ld a,indice_destino
ld hl,dir_origen
ld de,destino
ld bc,16
rst 08h
db 68h
```

indice_destino selecciona un registro del planificador, no un número de banco físico. Se copian dieciséis bytes en el destino de quien llama. Ten en cuenta que vuelve con las interrupciones deshabilitadas.

0x69 — Copiar al banco del registro seleccionado

Entrada y parámetros. A=índice de registro; HL=origen en quien llama; DE=destino en ese registro; BC=recuento positivo.

Salida, registros y errores. Como en 68.

Estructuras, restricciones y notas. El sentido es el inverso de 68. El índice de registro no es el banco. Con BC0 podría iterar 65 536 veces; esto y el solapamiento arbitrario con estructuras de control no son recetas admitidas.

Ejemplo de llamada

```
ld a,indice_destino
ld hl,origen
ld de,dir_destino
ld bc,16
rst 08h
db 69h
```

El origen pertenece a quien llama y el destino, a la correspondencia del registro seleccionado. El recuento es dieciséis; cero no es una convención para indicar una copia vacía.

0x6A — Dar formato a una fecha BCD proporcionada

Entrada y parámetros. C=mes BCD, H=día BCD, L=año BCD; DE=salida.

Salida, registros y errores. DE avanza normalmente 8, o 9 con ambos bits de formato; BC/HL/IY se conservan.

Estructuras, restricciones y notas. El bit 4 de contexto+0C elige MM/DD/AA; el valor predeterminado es DD-MM-AA; el bit 2 solo da DDMMMAA más un espacio. Meses en español. Sin NUL; no se comprueban los BCD ni los meses no válidos.

Ejemplo de llamada

```
ld c,01h
ld h,01h
ld l,90h
ld de,bufer
rst 08h
db 6Ah
```

La fecha BCD proporcionada es 01/01/90. bufer debe tener permiso de escritura y tamaño suficiente para el formato de fecha seleccionado; esta llamada da formato a datos, no cambia el reloj.

0x6B — Esperar un evento de dispositivo o una interrupción

Entrada y parámetros. Condición actual del dispositivo o del evento, gestionada por quien llama.

Salida, registros y errores. AF/BC/DE/HL/IX/IY seleccionados se conservan; vuelve con las interrupciones habilitadas.

Estructuras, restricciones y notas. Con una sola partición, EI/HALT; con varias, activa el bit de espera 5 y planifica. El despertar puede no traer ninguna entrada. Vuelve a comprobar siempre la condición.

Ejemplo de llamada

```
rst 08h
db 6Bh
```

Espera solo dentro de un bucle que vuelva a comprobar la condición correspondiente. Un despertar no significa necesariamente que haya un carácter disponible.

0x6C — Buscar un componente residente con nombre

Entrada y parámetros. DE=ocho bytes de clave exacta legibles.

Salida, registros y errores. Encontrado: AF0044/HL=dirección; ausente: AF0101/HL sin cambios; BC/DE/IX/IY se conservan en las rutas indicadas.

Estructuras, restricciones y notas. Filas de diez bytes desde 04BC; se detiene en el primer 00 o espacio; sin límite numérico. No hay normalización ni campos de tamaño, banco o propietario. SPOOLER seguido exactamente de un espacio se resuelve como 39C2.

Ejemplo de llamada

```
ld de,clave_componente
rst 08h
db 6Ch
```

clave_componente debe contener exactamente ocho bytes legibles; por ejemplo, SPOOLER seguido de un espacio. Comprueba el éxito antes de usar HL como dirección devuelta.

0x6D — Escribir la hora global actual mediante la devolución de llamada del RTC

Entrada y parámetros. Hora global BCD actual en 001B–001D; devolución de llamada 0062 instalada.

Salida, registros y errores. El envoltorio conserva AF/BC/DE/HL.

Estructuras, restricciones y notas. ESCRIBE en el RTC a través de 2E88 en 84/83/82 y habilita las interrupciones. No es una consulta de solo lectura del reloj.

Ejemplo de llamada

```
rst 08h
db 6Dh
```

El fragmento selecciona el servicio «Escribir la hora global actual mediante la devolución de llamada del RTC». Es una operación interna o de configuración, no un atajo normal para preparar aplicaciones.

0x6E — Escribir la fecha global actual mediante la devolución de llamada del RTC

Entrada y parámetros. Mes y día globales actuales en 0013/0014; devolución de llamada 0064 instalada.

Salida, registros y errores. El envoltorio conserva AF/BC/DE/HL.

Estructuras, restricciones y notas. ESCRIBE en el RTC a través de 2E98 en 87/86; no escribe el año 0015. La configuración de la devolución de llamada puede ser distinta en otra máquina.

Ejemplo de llamada

```
rst 08h
db 6Eh
```

El fragmento selecciona el servicio «Escribir la fecha global actual mediante la devolución de llamada del RTC». Es una operación interna o de configuración, no un atajo normal para preparar aplicaciones.

Capítulo 33 — Diagnósticos y mensajes

El número de registro de un mensaje, un número entre corchetes dentro de su texto, un error de ejecución de BASIC y el resultado de un comando de CSI pertenecen a espacios de nombres distintos. El propio registro 30 contiene el número entre corchetes 29. No lo renumeres para que la tabla quede regular. Algunos registros son preguntas o mensajes de estado de éxito, no errores.

SISTEMA.MENSERR contiene 200 registros fijos de 64 bytes: 141 tienen texto no vacío, cuatro contienen solo CR y 55 están rellenos de ceros. La tabla conserva los 145 registros distintos de cero; los registros omitidos están rellenos de ceros. `{1}` y campos similares son sustituciones que aporta quien llama. Una barra vertical literal se conserva como texto. Los bytes de control se muestran como `\xNN`. Ningún byte final se ha reinterpretado como un signo de puntuación inglés.

La columna de subsistema es una ayuda editorial basada en el texto y en las llamadas identificadas. En los registros cuya llamada no está especificada, el texto da un significado sin prometer una acción correctora universal. El capítulo 13 recoge el flujo de trabajo de diagnóstico.

Diagnostica los fallos habituales

La tabla siguiente relaciona los diagnósticos mostrados con operaciones concretas. Su columna CR indica el resultado del comando de CSI cuando se ha observado; no es el número de registro del mensaje en el catálogo siguiente. Algunos comandos solo muestran los números de mensaje cuando están configurados para ello.

Texto mostrado	Operación y resultado	Acción correctora
Esta cuenta esta siendo utilizada por la particion 1	CONECTAR; CR = 10; el número de partición varía	Finaliza la conexión existente con ese número de cuenta antes de volver a intentarlo. La contraseña todavía no se ha comprobado. Consulta los cambios de contraseña.
No se ha cambiado ningun fichero	CAMPRO; CR = 12	Comprueba la selección y conéctate como el propietario actual del archivo con privilegio suficiente. SISTEMA no selecciona automáticamente los archivos privados de otro propietario.
La cuenta sistema no puede compartir ficheros	COMPART; CR = 12	Usa una cuenta propietaria de archivos distinta de SISTEMA con privilegio 3 o superior.
Error sintactico	Ejemplo de ATRIB solo de atributos; CR = 8	Indica un solo nombre de archivo para cambiar atributos; reserva la forma con dos nombres para renombrar.
Falta tipo del fichero	Alternativa de AYUDA BORRAR; CR = 8	Consulta la referencia de BORRAR. AYUDA ha invocado la utilidad con AYUD en lugar de mostrar su tema.
COMU1 no esta asociado	BSC AYUDA; CR = 16	Usa AYUDA BSC para la ayuda central. La propia utilidad requiere una configuración compatible de COMU1.
Opcion no valida	BASIC con COMMA o COMA; CR = 8	Esta versión rechaza estas opciones del compilador. Usa el punto decimal en el código fuente.

Texto mostrado	Operación y resultado	Acción correctora
FALTAN PARENTESIS	Ejemplo de DEL$ con cuatro argumentos; edición de corrección de BASIC	Usa la forma de tres argumentos demostrada en el apéndice D. No des por hecho que a este diagnóstico de BASIC le siga un indicador de CSI.
NECESITA EXPRESION STRING	NBR con argumento numérico; edición de corrección de BASIC	Indica una expresión de cadena como en NBR. El significado del valor devuelto sigue sin especificarse.
[27] Fich. Cerrado	LINE(1), PAGE(1) o POS(1) sin ningún canal abierto	El ejemplo aislado carece de un contexto con un canal abierto. Que se acepte su sintaxis no establece que la consulta dé un resultado útil.

Catálogo de mensajes almacenados

Registro	Texto almacenado exacto (controles con escape)	Significado	Subsistema	Causa o acción correctora
1	`[1] Error sintactico\x0D`	Error de sintaxis	Comandos/ archivos	Comprueba los argumentos obligatorios; ESTADO sin argumentos es una causa observada.
2	`[2] Falta nombre del fichero\x0D`	Falta el nombre de archivo	Comandos/ archivos	Indica el nombre de archivo necesario; consulta la sintaxis del comando.
3	`[3] Falta tipo del fichero\x0D`	Falta el tipo de archivo	Comandos/ archivos	Incluye el tipo después de un punto, como en NOTA.TEXTO.
4	`[4] Falta codigo de unidad\x0D`	Falta el código de unidad	Comandos/ archivos	Incluye la unidad después de dos puntos y comprueba ASOCIAR.
5	`[5] Disco no asociado\x0D`	Disco no asociado	Comandos/ archivos	Examina ASOCIAR y la unidad solicitada.
6	`[6] Disco no montado\x0D`	Disco no montado o no reconocido como el soporte esperado	Comandos/ archivos	Comprueba la identidad del soporte y la asociación; MONTAR es la interfaz de notificación.
7	`[7] Opcion no valida\x0D`	Opción no válida	Comandos/ archivos	Comprueba la grafía de la opción y la lista de opciones específica del comando.

Registro	Texto almacenado exacto (controles con escape)	Significado	Subsistema	Causa o acción correctora
8	`[8] Fichero no encontrado\x0D`	Archivo no encontrado	Comandos/ archivos	Comprueba la grafía, la asociación de unidad y la cuenta actual antes de crear un sustituto.
9	`[9] Disco lleno\x0D`	Disco lleno o sin un tramo de asignación adecuado	Comandos/ archivos	Compara con ESTADO DISCO tanto el total de bloques libres como el mayor tramo contiguo.
10	`[10] Directorio lleno\x0D`	Directorio lleno según la ruta de búsqueda	Comandos/ archivos	Conserva el catálogo; un ciclo de sondeo corto puede agotar la búsqueda antes que la capacidad física.
11	`[11] Nombre de fichero no valido\x0D`	Nombre de archivo no válido	Comandos/ archivos	Comprueba el nombre y el tipo explícito; la compilación de BASIC requiere la forma completa documentada.
12	`[12] Codigo de unidad no valido\x0D`	Código de unidad no válido	Comandos/ archivos	Usa una letra de disco asignada que el comando acepte.
13	`[13] Fichero "{1}.{2}" no encontrado\x0D`	No se encuentra el archivo indicado	Comandos/ archivos	Comprueba los dos campos del archivo indicado y la unidad seleccionada.
14	`[14] "{1}" no es reconocible\x0D`	Token no reconocido	Comandos/ archivos	Compara el token mostrado con el vocabulario de comandos.
15	`[15] "{1}" es un numero no valido\x0D`	Valor numérico no válido	Comandos/ archivos	Comprueba el argumento numérico frente al intervalo y la notación permitidos.
16	`[16] Esperando "{1}", encontrado "{2}"\x0D`	Se esperaba un token y se ha encontrado otro	Comandos/ archivos	Indica el token esperado en la posición documentada.
17	`[17] Esperando "{1}", encontrado fin de linea\x0D`	Se esperaba un token, pero se ha llegado al final de la línea	Comandos/ archivos	Completa el comando con el argumento esperado.
18	`[18] Fichero protegido\x0D`	Archivo protegido	Comandos/ archivos	Comprueba el propietario y la protección correspondiente a esta operación.
19	`[19] Privilegio insuficiente\x0D`	Privilegio insuficiente	Comandos/ archivos	Comprueba ESTADO PRIVILEG y la protección del comando o del entorno de ejecución; no alteres la política solo para ocultar el diagnóstico.

Registro	Texto almacenado exacto (controles con escape)	Significado	Subsistema	Causa o acción correctora
20	`[20] Numero de serie incorrecto\x0D`	Número de serie de instalación incorrecto	Comandos/ archivos	Comprueba la vinculación del ejecutable con la identidad de la instalación; no es un fallo del puerto serie.
21	`[21] Esperando fin de linea, encontrado "{1}"\x0D`	Se esperaba el final de la línea, pero se ha encontrado otro token	Comandos/ archivos	Depende de quien llama; no se especifica ninguna operación correctora universal.
22	`[22] Falta parametro necesario\x0D`	Falta un parámetro obligatorio	Comandos/ archivos	Depende de quien llama; no se especifica ninguna operación correctora universal.
23	`[23] Debe ser "ON" u "OFF"\x0D`	El valor debe ser ON u OFF	Comandos/ archivos	Depende de quien llama; no se especifica ninguna operación correctora universal.
24	`[24] Formato incorrecto, deberia ser "{1}"\x0D`	Formato incorrecto; se sustituye el formato requerido	Comandos/ archivos	Depende de quien llama; no se especifica ninguna operación correctora universal.
25	`[25] No se puede desasociar la CONSOLA\x0D`	La consola no puede desasociarse	Dispositivos/ soportes	Usa ASOCIAR sin argumentos para listar; ASOCIAR CONSOLA intenta una eliminación no permitida.
26	`[26] Registro no valido en "{1}. {2}"\x0D`	Registro no válido en el archivo indicado	Dispositivos/ soportes	Depende de quien llama; no se especifica ninguna operación correctora universal.
27	`[27] No se puede asociar a un dispositivo de solo salida\x0D`	No se puede asociar un dispositivo solo de salida para este uso	Dispositivos/ soportes	Depende de quien llama; no se especifica ninguna operación correctora universal.
28	`[28] No se puede asociar a un dispositivo de solo entrada\x0D`	No se puede asociar un dispositivo solo de entrada para este uso	Dispositivos/ soportes	Depende de quien llama; no se especifica ninguna operación correctora universal.
29	`[29] Solo se puede asociar a un dispositivo de acceso aleatorio\x0D`	Se requiere un dispositivo de acceso aleatorio	Dispositivos/ soportes	Depende de quien llama; no se especifica ninguna operación correctora universal.
30	`[29] No se puede asociar a un dispositivo de acceso aleatorio\x0D`	No se permite un dispositivo de acceso aleatorio para este uso	Dispositivos/ soportes	Depende de quien llama; no se especifica ninguna operación correctora universal.

Registro	Texto almacenado exacto (controles con escape)	Significado	Subsistema	Causa o acción correctora
31	`[31] No se puede desasociar el disco del sistema\x0D`	El disco del sistema no puede desasociarse	Dispositivos/ soportes	Depende de quien llama; no se especifica ninguna operación correctora universal.
32	`[32] Dispositivo logico no definido\x0D`	Dispositivo lógico no definido	Dispositivos/ soportes	Depende de quien llama; no se especifica ninguna operación correctora universal.
33	`[33] Dispositivo fisico no definido\x0D`	Dispositivo físico no definido	Dispositivos/ soportes	Depende de quien llama; no se especifica ninguna operación correctora universal.
34	`[34] Debe ser organizacion secuencial\x0D`	Se requiere organización secuencial	Programa/ ejecución	Depende de quien llama; no se especifica ninguna operación correctora universal.
35	`[35] El fichero ya existe\x0D`	El archivo ya existe	Comandos/ archivos	Depende de quien llama; no se especifica ninguna operación correctora universal.
36	`[36] Falta el codigo de organizacion\x0D`	Falta el código de organización	Comandos/ archivos	Depende de quien llama; no se especifica ninguna operación correctora universal.
37	`[37] Falta opcion: "{1}"\x0D`	Falta una opción obligatoria	Comandos/ archivos	Depende de quien llama; no se especifica ninguna operación correctora universal.
38	`[38] {1} no esta asociado\x0D`	El dispositivo indicado no está asociado	Dispositivos/ soportes	Depende de quien llama; no se especifica ninguna operación correctora universal.
39	`[39] El dispositivo no esta asociado\x0D`	Dispositivo no asociado	Dispositivos/ soportes	Depende de quien llama; no se especifica ninguna operación correctora universal.
40	`[40] El canal del fichero no esta asignado\x0D`	Canal de archivo no asignado	Programa/ ejecución	Depende de quien llama; no se especifica ninguna operación correctora universal.
41	`[41] Ningun fichero encontrado\x0D`	No se ha encontrado ningún archivo coincidente	Comandos/ archivos	Comprueba la cuenta, la protección y si la búsqueda es exacta o con comodines antes de declarar que los bytes no existen.
42	`[42] {1} fichero(s), {2} bloque(s)\x0D`	Resumen de número de archivos y bloques	Comandos/ archivos	Depende de quien llama; no se especifica ninguna operación correctora universal.

Registro	Texto almacenado exacto (controles con escape)	Significado	Subsistema	Causa o acción correctora
43	`[43] Debe haber al menos dos discos asociados\x0D`	Debe haber al menos dos discos asociados	Dispositivos/soportes	Depende de quien llama; no se especifica ninguna operación correctora universal.
44	`[44] Las unidades fuente y destino no pueden ser las mismas\x0D`	El origen y el destino no pueden ser la misma unidad	Dispositivos/soportes	Depende de quien llama; no se especifica ninguna operación correctora universal.
45	`[45] Ningun fichero borrado\x0D`	No se ha borrado ningún archivo	Comandos/archivos	Depende de quien llama; no se especifica ninguna operación correctora universal.
46	`[46] {1} fichero(s) borrado(s), {2} bloque(s) recuperado(s)\x0D`	Resumen de archivos borrados y bloques especificados	Comandos/archivos	Depende de quien llama; no se especifica ninguna operación correctora universal.
47	`[47] Fuente y destino no tienen la misma capacidad\x0D`	Las capacidades de origen y destino son distintas	Dispositivos/soportes	Depende de quien llama; no se especifica ninguna operación correctora universal.
48	`[48] Receptor desconectado \x0D`	Receptor desconectado	Dispositivos/soportes	Depende de quien llama; no se especifica ninguna operación correctora universal.
49	`[49] Receptor timed out\x0D`	Tiempo de espera del receptor agotado	Dispositivos/soportes	Depende de quien llama; no se especifica ninguna operación correctora universal.
50	`[50] Debe ser fichero privado \x0D`	Se requiere un archivo privado	Comandos/archivos	Depende de quien llama; no se especifica ninguna operación correctora universal.
51	`[51] Modulo de carga con referencias externas no resueltas\x0D`	El módulo de carga tiene referencias externas sin resolver	Programa/ejecución	Depende de quien llama; no se especifica ninguna operación correctora universal.
52	`[52] No se pueden mezclar modulos ABS con REL o COM\x0D`	Los módulos ABS no pueden mezclarse con REL ni con COM	Programa/ejecución	Depende de quien llama; no se especifica ninguna operación correctora universal.
53	`[53] No puede tener varios modulos ABS\x0D`	No se permite más de un módulo ABS	Programa/ejecución	Depende de quien llama; no se especifica ninguna operación correctora universal.
54	`[54] Overflow en tabla de simbolos\x0D`	Desbordamiento de la tabla de símbolos	Programa/ejecución	Depende de quien llama; no se especifica ninguna operación correctora universal.

Registro	Texto almacenado exacto (controles con escape)	Significado	Subsistema	Causa o acción correctora
55	`[55] Fin de linea no esperado \x0D`	Final de línea inesperado	Programa/ejecución	Depende de quien llama; no se especifica ninguna operación correctora universal.
56	`[56] El programa no tiene PABs!\x0D`	El programa no tiene PAB; el significado exacto de la estructura no se ha resuelto	Programa/ejecución	Depende de quien llama; no se especifica ninguna operación correctora universal.
57	`[57] Solo los ficheros privados pueden ser compartidos\x0D`	Solo pueden compartirse los archivos privados	Cuenta/sesión	Depende de quien llama; no se especifica ninguna operación correctora universal.
58	`[58] Falta el fichero SISTEMA.CUENTA\x0D`	Falta SISTEMA.CUENTA	Cuenta/sesión	Restablece la accesibilidad y la identidad del archivo de cuentas a partir de una copia conservada.
59	`[59] La cuenta sistema no puede compartir ficheros\x0D`	La cuenta del sistema no puede compartir archivos mediante esta interfaz	Cuenta/sesión	Depende de quien llama; no se especifica ninguna operación correctora universal.
60	`[60] Palabra de paso incorrecta\x0D`	Contraseña incorrecta	Cuenta/sesión	Vuelve a introducir la contraseña correcta de la cuenta; el rechazo por contraseña errónea está documentado.
61	`[61] Cuenta no encontrada\x0D`	Cuenta no encontrada	Cuenta/sesión	Depende de quien llama; no se especifica ninguna operación correctora universal.
62	`[62] El fichero "{1}.{2}:{3}" ya existe en la cuenta "{4}"\x0D`	El archivo indicado ya existe en la cuenta especificada	Cuenta/sesión	Depende de quien llama; no se especifica ninguna operación correctora universal.
63	`[63] El spooler no esta inicializado\x0D`	SPOOLER no inicializado	SPOOLER	La ruta inactiva de ESTADO devuelve el resultado 12; no deduzcas que falta el código residente.
64	`[64] El spooler esta imprimiendo el fichero #{1}\x0D`	El SPOOLER está imprimiendo el informe numerado	SPOOLER	Depende de quien llama; no se especifica ninguna operación correctora universal.
65	`[65] El spooler esta esperando trabajo\x0D`	SPOOLER esperando trabajo	SPOOLER	Depende de quien llama; no se especifica ninguna operación correctora universal.
66	`[66] Spooler imprimiendo fichero #{1}, impresora desconectada!\x0D`	SPOOLER imprimiendo, pero con la impresora desconectada	SPOOLER	Depende de quien llama; no se especifica ninguna operación correctora universal.
67	`[67] Spooler suspendido \x0D`	SPOOLER suspendido	SPOOLER	Depende de quien llama; no se especifica ninguna operación correctora universal.
68	`[68] No hay suficiente memoria\x0D`	Memoria insuficiente	Partición/mensajes	Examina el límite superior actual y los controladores y clases cargados antes de cambiar la asignación.

Registro	Texto almacenado exacto (controles con escape)	Significado	Subsistema	Causa o acción correctora
69	`[69] El dispositivo esta asociado a la particion {1}\x0D`	Dispositivo asociado con otra partición	Partición/ mensajes	Depende de quien llama; no se especifica ninguna operación correctora universal.
70	`[70] Numero de particion no valido \x0D`	Número de partición no válido	Partición/ mensajes	Depende de quien llama; no se especifica ninguna operación correctora universal.
71	`[71] La particion no esta ubicada\x0D`	Partición no asignada	Partición/ mensajes	Depende de quien llama; no se especifica ninguna operación correctora universal.
72	`[72] La particion aun esta activa\x0D`	La partición sigue activa	Partición/ mensajes	Depende de quien llama; no se especifica ninguna operación correctora universal.
73	`[73] La particion no esta activa\x0D`	Partición no activa	Partición/ mensajes	Depende de quien llama; no se especifica ninguna operación correctora universal.
74	`[74] No se puede enviar un mensaje a su propia particion\x0D`	No se puede enviar un mensaje a la propia partición	Partición/ mensajes	Elige otra partición; la utilidad rechaza la entrega a uno mismo.
75	`[75] El usuario no esta conectado \x0D`	Usuario no conectado	Partición/ mensajes	Verifica la sesión del destinatario; la entrega almacenada es una ruta distinta.
76	`[76] El usuario no esta recibiendo mensajes\x0D`	El usuario no acepta mensajes	Partición/ mensajes	La inhibición de mensajes del usuario destinatario puede provocar el rechazo; la política con comodines o mediante la API es distinta.
77	`[77] Debe ser modo unico usuario o volumen de disco privado \x0D`	Se requiere el modo de un solo usuario o un volumen de disco privado	Partición/ mensajes	Depende de quien llama; no se especifica ninguna operación correctora universal.
78	`[78] Debe ser modo unico usuario\x0D`	Se requiere el modo de un solo usuario	Partición/ mensajes	Depende de quien llama; no se especifica ninguna operación correctora universal.
79	`[79] El spooler debe ser parado\x0D`	El SPOOLER debe estar detenido	SPOOLER	Depende de quien llama; no se especifica ninguna operación correctora universal.
80	`[80] No se ha cambiado ningun fichero\x0D`	No se ha cambiado ningún archivo	Comandos/ archivos	Depende de quien llama; no se especifica ninguna operación correctora universal.

Registro	Texto almacenado exacto (controles con escape)	Significado	Subsistema	Causa o acción correctora
81	`[81] El fichero "{1}.{2}:{3}" pertenece ahora a la cuenta "{4}"\x0D`	Propiedad del archivo transferida a la cuenta indicada	Comandos/ archivos	Depende de quien llama; no se especifica ninguna operación correctora universal.
82	`[82] Particion demasiado pequena para activar\x0D`	Partición demasiado pequeña para activarla	Partición/ mensajes	Depende de quien llama; no se especifica ninguna operación correctora universal.
83	`[83] Particion demasiado grande\x0D`	Partición demasiado grande	Partición/ mensajes	Depende de quien llama; no se especifica ninguna operación correctora universal.
84	`[84] El numero de particion debe ser mayor que el actual\x0D`	El número de partición debe ser mayor que el actual	Partición/ mensajes	Depende de quien llama; no se especifica ninguna operación correctora universal.
85	`[85] Solo se puede modificar la particion mas alta\x0D`	Solo puede modificarse la partición más alta	Partición/ mensajes	Depende de quien llama; no se especifica ninguna operación correctora universal.
86	`[86] La particion anterior todavia esta activa\x0D`	La partición anterior sigue activa	Partición/ mensajes	Depende de quien llama; no se especifica ninguna operación correctora universal.
87	`[87] El modulo {1} no esta cargado \x0D`	El módulo indicado no está cargado	Partición/ mensajes	Depende de quien llama; no se especifica ninguna operación correctora universal.
88	`[88] El modulo {1} ya esta cargado\x0D`	El módulo indicado ya está cargado	Partición/ mensajes	Depende de quien llama; no se especifica ninguna operación correctora universal.
89	`[89] Demasiados modulos re-entrantes\x0D`	Demasiados módulos reentrantes	Partición/ mensajes	Depende de quien llama; no se especifica ninguna operación correctora universal.
90	`[90] "{1}.{2}:{3}" salvado y fin edicion\x0D`	Archivo guardado y edición terminada	Estado del editor	Depende de quien llama; no se especifica ninguna operación correctora universal.
91	`[91] "{1}.{2}:{3}" salvado\x0D`	Archivo guardado	Estado del editor	Depende de quien llama; no se especifica ninguna operación correctora universal.
92	`[92] ** La particion ha sido parada ! **\x07\x0D`	La partición se ha detenido; incluye un pitido	Partición/ mensajes	Depende de quien llama; no se especifica ninguna operación correctora universal.

Registro	Texto almacenado exacto (controles con escape)	Significado	Subsistema	Causa o acción correctora
93	`[93] Esta cuenta esta siendo utilizada por la particion {1}\x0D`	Cuenta en uso por otra partición	Cuenta/sesión	Depende de quien llama; no se especifica ninguna operación correctora universal.
94	`[94] No se puede ESPIAR a si mismo !\x0D`	No se puede observar la propia consola con ESPIAR	Partición/mensajes	Depende de quien llama; no se especifica ninguna operación correctora universal.
95	`[95] No puede ejecutar FORZAR en su particion !\x0D`	No se puede usar FORZAR en la propia partición	Partición/mensajes	Depende de quien llama; no se especifica ninguna operación correctora universal.
96	`[96] Todas las particiones estan activas\x0D`	Todas las particiones están activas	Partición/mensajes	Depende de quien llama; no se especifica ninguna operación correctora universal.
97	`[97] Tiene mensajes esperando. Para recibirlos ejecute BUZON\x0D`	Hay mensajes almacenados en espera; BUZON los recupera	Partición/mensajes	BUZON recupera y elimina los mensajes almacenados; conserva el archivo si quieres examinarlo sin consumirlos.
98	`[98] Debe ser OASIS multi-usuario\x0D`	Se requiere OASIS multiusuario, según el texto original	Partición/mensajes	Depende de quien llama; no se especifica ninguna operación correctora universal.
99	`\x0D`	Registro con solo CR; sin texto de diagnóstico	Programa/ejecución	Depende de quien llama; no se especifica ninguna operación correctora universal.
100	`\x0D`	Registro con solo CR; sin texto de diagnóstico	Programa/ejecución	Depende de quien llama; no se especifica ninguna operación correctora universal.
101	`\x0D`	Registro con solo CR; sin texto de diagnóstico	Programa/ejecución	Depende de quien llama; no se especifica ninguna operación correctora universal.
102	`\x0D`	Registro con solo CR; sin texto de diagnóstico	Programa/ejecución	Depende de quien llama; no se especifica ninguna operación correctora universal.
103	`[103] El spooler esta planificando el siguiente trabajo\x0D`	El SPOOLER está seleccionando su siguiente trabajo	SPOOLER	Depende de quien llama; no se especifica ninguna operación correctora universal.
104	`[104] Form "{1} {2} {3} {4} {5} {6} {7} {8}" montado en spooler\x0D`	Formulario montado en el SPOOLER	SPOOLER	Depende de quien llama; no se especifica ninguna operación correctora universal.

Registro	Texto almacenado exacto (controles con escape)	Significado	Subsistema	Causa o acción correctora
105	`[105] El spooler esta parado\x0D`	SPOOLER detenido	SPOOLER	Depende de quien llama; no se especifica ninguna operación correctora universal.
106	`[106] El fichero de spool se esta imprimiendo actualmente\x0D`	Archivo de cola imprimiéndose en este momento	SPOOLER	Depende de quien llama; no se especifica ninguna operación correctora universal.
107	`[107] El fichero de spool esta abierto o no existe\x0D`	Archivo de cola abierto o ausente	SPOOLER	Depende de quien llama; no se especifica ninguna operación correctora universal.
108	`[108] Parametros spool: Cola={1}, Copias={2}, {3}\x0D`	Resumen de los parámetros de cola, copias y conservación	SPOOLER	Depende de quien llama; no se especifica ninguna operación correctora universal.
109	`[109] Conectado a las {1}, del {2}\x0D`	Hora y fecha de conexión	Cuenta/sesión	Depende de quien llama; no se especifica ninguna operación correctora universal.
110	`[110] Desconectado a las {1}, el dia {2}. Conectado {3} minutos\x0D`	Hora y fecha de desconexión y duración de la sesión	Cuenta/sesión	Depende de quien llama; no se especifica ninguna operación correctora universal.
111	`[111] Esperando parada del spooler\x0D`	Esperando a que se detenga el SPOOLER	SPOOLER	Depende de quien llama; no se especifica ninguna operación correctora universal.
112	`[112] Codigo de cola y formulario del spooler no coinciden\x0D`	Los códigos de cola y de formulario no coinciden	SPOOLER	Depende de quien llama; no se especifica ninguna operación correctora universal.
113	`[113] No se puede ejecutar FORZAR sobre el usuario\x0D`	FORZAR no permitido para ese usuario	Partición/mensajes	Depende de quien llama; no se especifica ninguna operación correctora universal.
114	`[114] Nombre de cuenta a conectarse: |\x0D`	Petición del nombre de cuenta para iniciar sesión	Cuenta/sesión	Depende de quien llama; no se especifica ninguna operación correctora universal.
115	`[115] Palabra de paso? |\x0D`	Petición de contraseña	Cuenta/sesión	Depende de quien llama; no se especifica ninguna operación correctora universal.
116	`[116] Overflow en la tabla {1}{2}-no. excesivo de atributos\x0D`	Desbordamiento de la tabla de atributos	Aplicación/consulta	Depende de quien llama; no se especifica ninguna operación correctora universal.
117	`|\x0D`	Registro de formato con barra vertical literal	Aplicación/consulta	Depende de quien llama; no se especifica ninguna operación correctora universal.
118	`[118] Imposible cargar el overlay requerido\x0D`	No se puede cargar una superposición necesaria	Aplicación/consulta	Comprueba el archivo de la superposición necesaria, su formato y la memoria de ejecución; la cobertura exacta de lo que lo provoca está incompleta.

Registro	Texto almacenado exacto (controles con escape)	Significado	Subsistema	Causa o acción correctora
119	`[119] Imposible abrir/cerrar el fichero despues del overlay1\x0D`	Falló la apertura o el cierre de un archivo después de overlay1	Aplicación/consulta	Depende de quien llama; no se especifica ninguna operación correctora universal.
120	`[120] Imposible cargar el ACB {1}.{2}\x0D`	No se puede cargar el ACB indicado; el contrato completo de los ACB no se ha resuelto	Aplicación/consulta	Depende de quien llama; no se especifica ninguna operación correctora universal.
121	`[121] {1}.{2} tipo de fichero no procesable por ACCESO\x0D`	ACCESO no puede procesar el tipo de archivo	Aplicación/consulta	Depende de quien llama; no se especifica ninguna operación correctora universal.
122	`[122] Imposible abrir el fichero {1}.{2}\x0D`	No se puede abrir el archivo indicado	Aplicación/consulta	Depende de quien llama; no se especifica ninguna operación correctora universal.
123	`[123] La frase {1}.{2} es erronea\x0D`	El archivo de consulta o de instrucciones es erróneo	Aplicación/consulta	Depende de quien llama; no se especifica ninguna operación correctora universal.
124	`[124] {1}.{2} no encontrado en {3}.{4}\x0D`	No se encuentra el elemento indicado en el archivo especificado	Aplicación/consulta	Depende de quien llama; no se especifica ninguna operación correctora universal.
125	`[125] Tipo de fichero invalido en {1}.{2}\x0D`	Tipo de archivo no válido	Aplicación/consulta	Depende de quien llama; no se especifica ninguna operación correctora universal.
126	`[126] Todas las entradas al vocabulario deben ser textos\x0D`	Las entradas del vocabulario deben ser texto	Aplicación/consulta	Depende de quien llama; no se especifica ninguna operación correctora universal.
127	`[127] Error en un operador de relacion\x0D`	Error de operador relacional	Aplicación/consulta	Depende de quien llama; no se especifica ninguna operación correctora universal.
128	`[128] Imposible leer el siguiente registro de "{1}.{2}"\x0D`	No se puede leer el registro siguiente	Aplicación/consulta	Depende de quien llama; no se especifica ninguna operación correctora universal.
129	`[129] Error de sintaxis en una correlacion\x0D`	Error de sintaxis en la correlación	Aplicación/consulta	Depende de quien llama; no se especifica ninguna operación correctora universal.
130	`[130] Correlacion "&STK" erronea\x0D`	Correlación &STK no válida	Aplicación/consulta	Depende de quien llama; no se especifica ninguna operación correctora universal.

Registro	Texto almacenado exacto (controles con escape)	Significado	Subsistema	Causa o acción correctora
131	`[131] Operador de correlacion indefinido\x0D`	Operador de correlación no definido	Aplicación/consulta	Depende de quien llama; no se especifica ninguna operación correctora universal.
132	`[132] Correlacion de relacion indefinida\x0D`	Correlación de relación no definida	Aplicación/consulta	Depende de quien llama; no se especifica ninguna operación correctora universal.
133	`[133] Una correlacion no puede referenciar a ese dato\x0D`	La correlación no puede hacer referencia a este dato	Aplicación/consulta	Depende de quien llama; no se especifica ninguna operación correctora universal.
134	`[134] Falta el delimitador de fin de literal\x0D`	Falta el delimitador de cierre del literal	Aplicación/consulta	Depende de quien llama; no se especifica ninguna operación correctora universal.
135	`[135] El fichero destino de la translacion debe ser I o D\x0D`	El destino de la traducción debe tener organización I o D	Aplicación/consulta	Depende de quien llama; no se especifica ninguna operación correctora universal.
136	`[136] Elemento referenciado del stack fuera de limites\x0D`	Elemento de la pila referenciado fuera de límites	Aplicación/consulta	Depende de quien llama; no se especifica ninguna operación correctora universal.
137	`[137] Codigo de retorno de la translacion erroneo\x0D`	Código de retorno de traducción no válido	Aplicación/consulta	Depende de quien llama; no se especifica ninguna operación correctora universal.
138	`[138] Error en el tipo de fichero de la correlacion\x0D`	Tipo de archivo erróneo en la correlación	Aplicación/consulta	Depende de quien llama; no se especifica ninguna operación correctora universal.
139	`[139] Error de lectura en la translacion\x0D`	Error de lectura en la traducción	Aplicación/consulta	Depende de quien llama; no se especifica ninguna operación correctora universal.
140	`[140] Imposible grabar en "{1}.{2}"\x0D`	No se puede escribir el archivo indicado	Aplicación/consulta	Depende de quien llama; no se especifica ninguna operación correctora universal.
141	`[141] \x0D`	Registro con solo un número; sin texto explicativo	Aplicación/consulta	El registro almacenado contiene solo su número entre corchetes y espacios.
142	`[142] No existe tal banco\x0D`	El banco solicitado no existe	Partición/mensajes	Depende de quien llama; no se especifica ninguna operación correctora universal.

Registro	Texto almacenado exacto (controles con escape)	Significado	Subsistema	Causa o acción correctora
143	`[143] Parametros numericos fuera de rango\x0D`	Parámetros numéricos fuera de intervalo	Comandos/ archivos	Depende de quien llama; no se especifica ninguna operación correctora universal.
144	`[144] El spooler no puede ser descargado !!\x0D`	El SPOOLER no puede descargarse	SPOOLER	Depende de quien llama; no se especifica ninguna operación correctora universal.
145	`[145] No se puede archivar sobre el disco del sistema !! \x0D`	No se puede archivar en el disco del sistema	Dispositivos/ soportes	Selecciona un destino de archivado independiente; no selecciones el disco del sistema como destino del archivado.

Resultados de comandos y errores de BASIC

Valor o texto	Significado establecido y acción
CSI CR0	Finalización normal indicada en las rutas seleccionadas; no demuestra el vaciado de búferes, la entrega ni la E/S física
CSI CR8	Varios rechazos de sintaxis, de dispositivo o de archivo; usa el texto que lo acompaña
CSI CR12	Varios rechazos de búsqueda, de almacenamiento o del SPOOLER; no es un error único del núcleo
CSI CR252	Denegación del prólogo estándar de privilegio/número de serie de los comandos
CSI CR253	Ruta de error de ejecución de BASIC, incluidos los fallos de política o de bifurcación calculada
CSI CR254	Rutas de interrupción activa o de anulación por error de disco; también el fallo de inicio de CLAVIN. El código por sí solo no identifica la causa. Un CLOSE protegido puede indicar en cambio 0
BASIC `[30] Fich. no Esta`	Archivo inexistente o inaccesible, incluidas las entradas ocultas por un 00 anterior
BASIC `[32] Directori.Llen`	Agotamiento de la búsqueda en el directorio; otros huecos del catálogo pueden seguir sin usarse
Error 33 de BASIC	Rechazo establecido por protección en OPEN/WRITE; distingue la operación y el propietario
BASIC `[40] Programa no esta en posicion 4CED`	RUN fallido observado; verifica el BASICCOM y la ruta del entorno de ejecución en lugar de tratar 4CED como una dirección universal fija
BASIC `[41] Format Fich Invalid`	Tipo de registro no válido; una E/S ignorada o un EOF registrado pueden dejar en el destino bytes antiguos que BASIC interpreta como un tipo
Error de ON calculado 0x14	Selector 0 en la ruta de bifurcación especificada; resultado normal 253

Diagnósticos del verificador nativo

Texto del diagnóstico o forma con parámetros	Significado	Acción
`***** Fin de fichero incorrecto`	El último sector S recorrido no coincide con el final indicado en el directorio	Haz una copia de seguridad de la cadena y de la entrada; F no garantiza la reconstrucción del último sector
`La longitud del fichero no coincide con el directorio`	La extensión redondeada de la cadena es distinta de la asignación declarada	Conserva ambas interpretaciones y los metadatos anteriores
`Sector <n> ocupado pero no pertenece a ningun fichero`	El sector numerado está asignado, pero no tiene propietario en el recorrido del verificador	Conserva los datos candidatos antes de que F libere los bits

El verificador no informa necesariamente de un recuento secuencial desactualizado, de una identidad duplicada ni de un hueco de EOF en el nivel de aplicación. Que no diga nada no es una certificación completa del sistema de archivos.

Estado del controlador de disco

El vocabulario especificado del controlador incluye 01 no preparado (`no prep.`), 02 protegido contra escritura, 03 sin inicializar, 04 error de transferencia, 07 fallo de sector o identificador, 08 fallo de pista y 09 fallo de cabeza. Los significados exactos varían según la traducción del controlador, y no todos los fallos físicos tienen un contrato completo definido. R reintenta; M invoca la ranura de recalibrado o reinicio del controlador y reintenta; I ignora; Q anula de forma no local. La ranura M instalada para WD es un RET. Ignorar no reconstruye los datos. Consulta el capítulo 27 para ver la política de devoluciones de llamada y la excepción del resultado cero en el CLOSE protegido.

Capítulo 34 — Dispositivos y límites del sistema

Los nombres lógicos designan usos dentro de una partición. Los nombres físicos seleccionan definiciones instaladas, que pueden describir hardware opcional no instalado en la máquina. Por tanto, la presencia de un nombre es una prueba más débil que una operación de dispositivo correcta.

Índice de asociaciones lógicas

Índice (hex.)	Nombre o función
00	S, disco del sistema
01–07	A–G, otros discos
08	Entrada de consola
09	Salida de consola
0A–0B	Huecos restantes no clasificados por completo aquí
0C–0F	IMPR1–IMPR4
10	COMU1 en la llamada establecida
11–13	Más huecos de comunicaciones; su uso completo no se ha probado de forma independiente
14	TAPE1 en la inicialización de DIS10
15–17	Huecos restantes; se desconoce la correspondencia completa con dispositivos activos

El byte de asociación es el índice del descriptor físico empezando en cero, con FF para sin asignar. El número de definición mostrado es índice+1. Así, el DISCO1/dispositivo 3 instalado se resuelve como el índice de descriptor 2, en 0x0069+2*32=0x00A9. ASOCIAR manipula la asignación entre dispositivos lógicos y físicos; ASIGNAR asigna canales de aplicación. Un nombre de archivo usa `nombre.tipo:unidad`, donde S y A–G se interpretan a través de la partición actual.

Nombres físicos

Nombre	Número de definición	Controlador/clase y disponibilidad
FLOPPY1, FLOPPY2	1,2	DIS1/2 con rutas de controladora independientes; el capítulo 11 trata el acceso y el formato FLOPPY2; el mecanismo instalado y la compatibilidad de arranque requieren un hardware que coincida
DISCO1–DISCO6	3–8	Definiciones de disco; la asociación S depende de la configuración
TAP	10	DIS10; el funcionamiento de la controladora y de la cinta física no se ha resuelto

Nombre	Número de definición	Controlador/clase y disponibilidad
IP, CDC	11,12	Controladores de impresora; definiciones de controladores de impresora
PAN9, PAN10; IS9, IS10	16,17	DIS16/17 en el SCC de SS2; anillos de entrada de 128 bytes y salida por sondeo; requieren el hardware y un formato de trama del terminal que coincida
PAN1–PAN4	18–21	Consolas ALP; cuatro puertos serie establecidos
IMP	22	Interfaz de impresora serie DIS22 con la disposición de registros del SCN2651 y señales de disponibilidad de módem; modelo de impresora física sin especificar
PAN5–PAN8; IS5–IS8	23–26	Segundo ALP opcional; hardware opcional
COM1–COM4	27–30	Definiciones; no se han establecido los controladores operativos completos correspondientes

IS2/IS3/IS4 comparten los archivos de controlador de PAN2/PAN3/PAN4 correspondientes. Sus nombres no implican que se hayan especificado todos los modos predeterminados ni las configuraciones simultáneas de alias. El apéndice B recoge los tamaños exactos de los controladores, los desplazamientos de entrada, los puertos y los índices de devolución de llamada.

Límites canónicos

Magnitud	Valor establecido	Ámbito
Espacio de direcciones de la CPU	65 536 bytes	Ventana visible del Z80
RAM implementada	256 KiB: 16 comunes + 5 × 48 en bancos	Configuración de ejemplo de cinco bancos
Disco físico de ejemplo	830 × 5 × 32 × 256 = 33 996 800 bytes	Geometría de ejemplo; verifica el dispositivo instalado
Sectores del volumen lógico	62 048	Geometría de volumen de ejemplo
Bloques de asignación	15 512 de 1024 bytes	Geometría de volumen de ejemplo
Sector/datos	256 físicos/lógicos; 254 de datos en S	Formatos establecidos
Huecos del catálogo	1208 para 151 sectores de directorio	151 × 8; el defecto de sondeo puede reducir la capacidad accesible
Nombre/tipo de archivo	Ocho bytes cada uno	Rellenados con espacios; se aplican restricciones de normalización
Años de las marcas de tiempo	1977–1992	Desplazamiento de año de cuatro bits; no todo el intervalo del RTC del hardware
Asociaciones	24 por partición	Disposición del contexto

Magnitud	Valor establecido	Ámbito
Índices de canales de aplicación	0–16	17 entradas; la ayuda de ASIGNAR para el usuario expone de 1 a 16
Filas del planificador	16	Tabla y numeración pública; no se ha demostrado que funcionen dieciséis particiones activas
Nombre de cuenta	Ocho caracteres en CUENTA	Para los nombres nuevos normales, usa una letra inicial seguida de letras o dígitos
Contraseña de la cuenta	Campo de ocho caracteres en CUENTA	La entrada elimina los espacios; se rechaza la coma; el capítulo 10 explica las respuestas en blanco y con #
Números de cuenta	CUENTA asigna números nuevos distintos hasta 250; la ayuda de CAMCU permite de 0 a 254; FF es especial	Intervalos distintos según el comando; no se ha establecido el número máximo de cuentas o sesiones simultáneas
Valores de privilegio	0 a 5, intervalo normal documentado/ especificado	Las comprobaciones internas adicionales de 6/FF no establecen niveles de usuario
Campos de terminal predeterminados	L79/P23	Presupuesto de entrada y umbrales del cursor, no dimensiones de pantalla genéricas
Argumentos de EXEC	16 × 8 bytes	El decimoséptimo campo indicado se excluye en el caso especificado
Variables de EXEC	16 celdas de nombre de 8 y valor de 8 por marco	Disposición; no se ha establecido todo el comportamiento ante el agotamiento
Marco de EXEC	512 bytes	Los archivos de trabajo instalados ofrecen cuatro marcos cada uno
Tabla de bloqueos de registros	64 utilizables + terminador	Código/disposición; no se ha provocado el comportamiento con la tabla llena
Tabla de archivos abiertos	129 filas físicas de 4 bytes, incluido el almacenamiento necesario para el terminador	Se desconoce el comportamiento efectivo de saturación
Campo de registro I/K	R=(packed&511)+1	Representable de 1 a 512; los límites de creación y de ejecución pueden ser más estrechos
Campo de clave I/K	K=(packed>>9)+1	Representable de 1 a 128; se desconoce el comportamiento completo con valores extremos
Decimal de BASIC	13 dígitos decimales significativos en la representación especificada	El redondeo y el intervalo aritméticos no están totalmente establecidos
Tamaño máximo de programa	Desconocido como constante única	Depende de la partición, del límite superior, de los controladores y clases y del entorno de ejecución
Máximo general de archivos abiertos simultáneamente	Desconocido	El número de canales y la capacidad de la tabla compartida son cosas distintas
Cola del SPOOLER	64 registros almacenados; el productor parece dar la vuelta en 1–63	Capacidad efectiva derivada del código; saturación nativa sin probar

Magnitud	Valor establecido	Ámbito
Intervalo seguro de la aritmética de colas	De 1 a 128 con un estado serializado válido	Evitación derivada de defectos concretos, no un máximo anunciado del sistema operativo

Los campos de capacidad no deben considerarse máximos utilizables garantizados sin comprobar las filas centinela, los registros reservados y los algoritmos. A la inversa, un ejemplo pequeño no establece el máximo del sistema.

Capítulo 35 — Referencia de grafía de comandos y de lenguajes

El sistema tiene varios lenguajes de comandos. CSI selecciona un comando del sistema; después, cada utilidad interpreta sus propios argumentos. EDITAR, EXEC, CONSULTA y BASIC reconocen cada uno un vocabulario distinto. Este capítulo aporta los detalles de grafía y de búsqueda después de las introducciones prácticas de los capítulos 5, 7 y 8. La referencia del lenguaje BASIC sigue en el apéndice D.

Nombres completos de los comandos y abreviaturas permitidas

Al seguir los procedimientos, usa los nombres completos de la columna Comando. La mayoría de los comandos aceptan cualquier prefijo entre su mínimo indicado y el nombre completo. La columna de grafías aceptadas enumera las alternativas exactas para que se vean las excepciones. El reconocimiento depende de que la abreviatura esté activada y de que esté seleccionada la tabla estándar. Un nombre reconocido no crea un ejecutable que no existe.

BASIC es una excepción: B selecciona BASIC, BA selecciona BACKUP, y de BAS a BASIC seleccionan BASIC. EXEC tiene una limitación aparte en la transferencia de argumentos: EX y EXE se reconocen, pero pueden ejecutar un archivo de comandos llamado EX o EXE en lugar del archivo solicitado. Escribe siempre EXEC completo. DESCARGAR es un nombre de la tabla de nueve caracteres limitado por un campo de comando de ocho caracteres; no tiene ningún ejecutable correspondiente disponible.

Comando	Grafías estándar aceptadas	Notas
ACTIVAR	`ACTIV`, `ACTIVA`, `ACTIVAR`	Comando estándar.
ARCHIVAR	`AR`, `ARC`, `ARCH`, `ARCHI`, `ARCHIV`, `ARCHIVA`, `ARCHIVAR`	Comando estándar.
ASIGNAR	`A`, `AS`, `ASI`, `ASIG`, `ASIGN`, `ASIGNA`, `ASIGNAR`	Comando estándar.
ASOCIAR	`ASO`, `ASOC`, `ASOCI`, `ASOCIA`, `ASOCIAR`	Comando estándar.
ATRIB	`ATR`, `ATRI`, `ATRIB`	Comando estándar.
AYUDA	`AY`, `AYU`, `AYUD`, `AYUDA`	Comando estándar.
BACKUP	`BA`, `BAC`, `BACK`, `BACKU`, `BACKUP`	Comando estándar.
BASIC	`B`, `BAS`, `BASI`, `BASIC`	BA selecciona BACKUP.
BORRAR	`BO`, `BOR`, `BORR`, `BORRA`, `BORRAR`	Comando estándar.
BUSCAR	`BUSC`, `BUSCA`, `BUSCAR`	Comando estándar.
BUZON	`BUZ`, `BUZO`, `BUZON`	Comando estándar.

Comando	Grafías estándar aceptadas	Notas
CAMBIAR	CAM, CAMB, CAMBI, CAMBIA, CAMBIAR	Comando estándar.
CAMPRO	CAMPRO	Comando estándar.
CARGAR	CAR, CARG, CARGA, CARGAR	Ejecutable ausente; referencia a la carga de residentes en SPOOLER.
CLASIF	CL, CLA, CLAS, CLASI, CLASIF	Ejecutable ausente; no se redirige a SORT.
COMPART	COMPART	Comando estándar.
CONECTAR	CONEC, CONECT, CONECTA, CONECTAR	Comando estándar.
COPIFI	C, CO, COP, COPI, COPIF, COPIFI	Comando estándar.
CREAR	CRE, CREA, CREAR	Comando estándar.
CUENTA	CUEN, CUENT, CUENTA	Comando estándar.
DESCARGAR	DESCAR, DESCARG, DESCARGA	Faltan el ejecutable y el archivo EXEC; nombre de búsqueda DESCARGA.
EDITAR	ED, EDI, EDIT, EDITA, EDITAR	Comando estándar.
ELIMINAR	ELIM, ELIMI, ELIMIN, ELIMINA, ELIMINAR	Comando estándar.
ESTADO	EST, ESTA, ESTAD, ESTADO	Comando estándar.
EXEC	EX, EXE, EXEC	Escribe EXEC completo; las formas cortas pueden seleccionar el archivo equivocado.
FIN	FIN	Comando estándar.
FORZAR	FORZ, FORZA, FORZAR	Comando estándar.
GENSIST	GENS, GENSI, GENSIS, GENSIST	Comando estándar.
IBMSEC	IBMSEC	Comando estándar.
INICIAR	INI, INIC, INICI, INICIA, INICIAR	Comando estándar.
LISDIR	LISD, LISDI, LISDIR	Comando estándar.
LISTAR	LIST, LISTA, LISTAR	Comando estándar.
MENSAJE	MENS, MENSA, MENSAJ, MENSAJE	Comando estándar.
MONTAR	MON, MONT, MONTA, MONTAR	Comando estándar.
PARAR	PARAR	Comando estándar.
PONER	PON, PONE, PONER	Comando estándar.
RECARGAR	RECAR, RECARG, RECARGA, RECARGAR	Comando estándar.
REPARAR	REP, REPA, REPAR, REPARA, REPARAR	Comando estándar.

Comando	Grafías estándar aceptadas	Notas
REVIVIR	REV, REVI, REVIV, REVIVI, REVIVIR	Comando estándar.
RUN	R, RU, RUN	Comando estándar.
SECIBM	SECIBM	Comando estándar.
SECTOR	SEC, SECT, SECTO, SECTOR	Comando estándar.
SPOOLER	SP, SPO, SPOO, SPOOL, SPOOLE, SPOOLER	Comando estándar.
TEDITAR	TED, TEDI, TEDIT, TEDITA, TEDITAR	Comando estándar.
VOLCAR	VOLC, VOLCA, VOLCAR	Comando estándar.

Los nombres de ejecutable siguientes no tienen abreviatura estándar. Ya son los nombres completos de sus comandos, aunque su grafía sea compacta:

ARCHDIR, BSC, CAMCU, CEDIT, CIFRA, CLAVIN, CONSULTA, COPFICH, CORLUZ, DIRIBM, DUPLICAR, ENVIAR, ENVIARA, ESPIAR, FISDEF, HASH, INICDISC, INICIBM, INITTAPE, LOGDEF, MODIMPR, O8PRECOM, ORGDIR, PANTALLA, RECIBIR, RECIBIRA, SERIAL, SORT, TECLAS, TERMINAL, TITULO, UNOSEC, VERFORM.

LCOMPART tiene un tema de ayuda, pero no tiene ejecutable correspondiente ni entrada en la tabla estándar. CARGAR, CLASIF y DESCARGAR conservan sus entradas en la tabla, pero no tienen ningún ejecutable ni archivo EXEC disponible; DESCARGAR se busca como DESCARGA. CLASIF no se redirige a SORT. A la inversa, CLAVIN está instalado, pero devuelve CR 254 durante el inicio en las invocaciones probadas. Sus entradas del capítulo 31 distinguen entre nombres reconocidos, archivos instalados y comandos utilizables.

Activa o desactiva la abreviatura

En CSI, introduce:

```
PONER ABREV ON
```

Así se activa la expansión de prefijos de comandos. Para exigir los nombres de comando completos, introduce:

```
PONER ABREV OFF
```

Los nombres de archivo exactos y los sinónimos configurados exactos siguen funcionando. Estos ajustes no desactivan las abreviaturas de las opciones de una utilidad.

Para seleccionar la tabla de abreviaturas integrada, introduce:

```
PONER SINONIM (S)
```

Para desactivar esa tabla, introduce:

```
PONER SINONIM (N)
```

Desactivar la tabla no cambia el nombre de los archivos de comandos. Para mostrar sus entradas, introduce:

```
ESTADO SINONIMO
```

Así se listan los nombres estándar y sus mínimos anunciados. Hay una limitación cuando se selecciona un archivo de sinónimos del usuario: CSI lee el tipo SINONIMO, pero este programa ESTADO busca SINONIM. Por tanto, el listado puede terminar con archivo no encontrado/CR 12 aunque los sinónimos personalizados funcionen. Mantén el tipo SINONIMO del archivo de definiciones que funciona.

Sinónimos del usuario y prioridad de los nombres de archivo

Un sinónimo aporta otro nombre para un comando. Por ejemplo, un archivo de definiciones llamado CORTO.SINONIMO puede contener esta línea:

```
ESTADO MIRA 2
```

Es contenido de un archivo, no un comando de CSI: ESTADO es el nombre real; MIRA, el sinónimo; y 2, la longitud mínima del sinónimo. Crea el archivo con el procedimiento de edición de texto del capítulo 7. Después, selecciónalo en CSI:

```
PONER SINONIM CORTO
```

Con la abreviatura activada, MIRA, MIR y MI seleccionan ESTADO, salvo que un archivo real tenga prioridad. Con la abreviatura desactivada, el sinónimo exacto MIRA sigue funcionando. Para anular las definiciones personalizadas, introduce:

```
PONER SINONIM (C)
```

Los sinónimos personalizados se tienen en cuenta antes que la expansión estándar. Dentro de cada asociación de disco en la que busca, CSI prueba el archivo COMANDO del token original y después su archivo EXEC, si la ejecución implícita está habilitada, antes de probar los nombres expandidos. Solo entonces continúa con otra asociación. Así, un ASO.COMANDO real o un ASO.EXEC habilitado pueden tener prioridad sobre la expansión a ASOCIAR. El comando EXEC completo sigue su propia ruta de intérprete anterior. Consulta el capítulo 30 para la carga.

Opciones de las utilidades: palabras clave completas y formas abreviadas

Las tablas siguientes registran las grafías aceptadas, no procedimientos de uso completos. Consulta la entrada del capítulo 31 de cada comando para ver la finalidad, los argumentos, las precauciones y los ejemplos. Que se reconozca una opción no establece que la operación de hardware solicitada pueda completarse. Las palabras clave completas aparecen primero; las abreviaturas aceptadas son alternativas, no nombres corregidos.

La mayoría de las opciones de las tablas aceptan los prefijos intermedios indicados. La selección sigue el orden de la tabla de cada utilidad, así que los prefijos que se solapan pueden seleccionar operaciones distintas. Por ejemplo, COPIFI PA selecciona PANT; usa PACK para solicitar el empaquetado. SPOOLER SAL selecciona una operación distinta de SALVAR, aunque S y SA seleccionan SALVAR. Estas excepciones se registran explícitamente a continuación.

Una palabra clave completa es la palabra literal que reconoce ese programa. ESTADO usa HISTOR e INIREL, mientras que PONER usa HISTORIA e INIRELOJ. Son interfaces distintas. El selector de minúsculas de COPIFI es LMINUSC; MINUSC y MINUSCU no son alias. La palabra clave de adición de COPIFI es literalmente A@ADIR, con arroba. Su forma abreviada A tiene la misma función de adición.

Opciones con análisis por caracteres

Algunas utilidades examinan letras individuales en lugar de usar las tablas siguientes. No amplíes estas formas a palabras inventadas.

Comando	Forma	Interpretación y restricciones
ASOCIAR	L79, P23, C2, B9600	Las formas de anchura, página, clase y velocidad llevan los números unidos. L 79 y LINEA79 no sustituyen a L79.
ASOCIAR	LFnn, FFnn	Campos de retardo específicos del dispositivo; independientes de las dimensiones de anchura y de página.
ASOCIAR	W8	Forma aceptada de tamaño de palabra; W7 se rechaza.
ASOCIAR	PE, PO, PP, ALF, PUBLIC	Indicadores de paridad, de tratamiento de páginas y líneas y de asociación pública; consulta las opciones específicas del dispositivo del comando.
ASOCIAR	ENAB1–ENAB4, O, H, R, STP, STS	Ajustes de dispositivo analizados uno a uno; los argumentos y los efectos dependen del dispositivo.
ASOCIAR	SY, SD, WA	Ramas reconocidas cuyos contratos completos para el operador siguen sin especificarse.
ATRIB	LE	Protección contra lectura. También se acepta L; LEER no. NOL requiere el privilegio 255 y no es una operación inversa sin restricciones.
ATRIB	E/NOE, B/NOB	Controles de protección contra escritura y contra borrado.
BORRAR	PANT/NOPANT, CONSUL/NOCONSUL	Controles de presentación y de pregunta de confirmación. El borrado por nombre exacto puede realizarse sin pregunta de confirmación.

Comando	Forma	Interpretación y restricciones
LISTAR	PA seguido del número de página	Selección de página; las demás formas que empiezan por P seleccionan la salida a la impresora, y un dígito final selecciona la impresora 1 a 4.
LISDIR	FN, FT, FD, FO, FI, AP, NOS, NOH	Selectores individuales de campo, de salida y de listado; consulta LISDIR en lugar de aplicar las reglas de abreviatura de CSI.

En los ejemplos, usa paréntesis de opciones equilibrados. El analizador también acepta un retorno de carro como final de las opciones, lo que explica que haya archivos de comandos que omitan el paréntesis de cierre. Esto no hace opcionales los espacios dentro de los argumentos numéricos: ASOCIAR usa L79, mientras que CREAR usa REGISTRO 128.

Ajustes de PONER

Un ajuste va detrás de PONER y admite sus propios argumentos. Por ejemplo:

```
PONER IMPEXEC ON
```

Así se activa la búsqueda implícita de archivos EXEC. No cambia las reglas de grafía dentro de un archivo EXEC.

Ajuste completo	Grafías aceptadas
HORA	HO, HOR, HORA
PRIVLEV	PRIV, PRIVL, PRIVLE, PRIVLEV
ERRNUM	ERRN, ERRNU, ERRNUM
ERRTEXT	ERRT, ERRTE, ERRTEX, ERRTEXT
FECHA	FE, FEC, FECH, FECHA
ABREV	AB, ABR, ABRE, ABREV
MENSAJE	MEN, MENS, MENSA, MENSAJ, MENSAJE
TIEMPO	TI, TIE, TIEM, TIEMP, TIEMPO
MEMORIA	MEM, MEMO, MEMOR, MEMORI, MEMORIA
MODULO	MO, MOD, MODU, MODUL, MODULO
VERSION	VER, VERS, VERSI, VERSIO, VERSION
HISTORIA	H, HI, HIS, HIST, HISTO, HISTOR, HISTORI, HISTORIA
DIA	DIA
SINONIM	SIN, SINO, SINON, SINONI, SINONIM
INCOM	INC, INCO, INCOM

Ajuste completo	Grafías aceptadas
IMPEXEC	IMP, IMPE, IMPEX, IMPEXE, IMPEXEC
ESCAPE	ESC, ESCA, ESCAP, ESCAPE
SUBIR	SU, SUB, SUBI, SUBIR
BAJAR	BA, BAJ, BAJA, BAJAR
IZQ	IZ, IZQ
DER	DE, DER
RUBOUT	RU, RUB, RUBO, RUBOU, RUBOUT
CANCEL	CA, CAN, CANC, CANCE, CANCEL
RTCAVAIL	RTC, RTCA, RTCAV, RTCAVA, RTCAVAI, RTCAVAIL
SISFECHA	SIS, SISF, SISFE, SISFEC, SISFECH, SISFECHA
INIRELOJ	INI, INIR, INIRE, INIREL, INIRELO, INIRELOJ
QET	QET
FORFECHA	FORFE, FORFEC, FORFECH, FORFECHA
ALEMAN	ALE, ALEM, ALEMA, ALEMAN

Opciones de CREAR

REGISTRO, NUMREG y LONCLA toman su argumento numérico en el campo siguiente. DIRECTO, INDEXADO y CLAVE seleccionan la organización del archivo; VACIAR cambia el contenido existente y requiere precaución. Los ejemplos completos de creación están en el capítulo 17.

Opción completa	Grafías aceptadas
DIRECTO	D, DI, DIR, DIRE, DIREC, DIRECT, DIRECTO
INDEXADO	I, IN, IND, INDE, INDEX, INDEXA, INDEXAD, INDEXADO
CLAVE	C, CL, CLA, CLAV, CLAVE
REGISTRO	R, RE, REG, REGI, REGIS, REGIST, REGISTR, REGISTRO
LONCLA	LONC, LONCL, LONCLA
NUMREG	N, NU, NUM, NUMR, NUMRE, NUMREG
VACIAR	VA, VAC, VACI, VACIA, VACIAR

Palabras clave de ARCHIVAR

Estas palabras clave pertenecen a ARCHIVAR. Varias filas pueden compartir una operación; la columna de grafías incluye los alias que seleccionan esa misma operación.

Palabra clave completa	Grafías aceptadas	Notas
PANT	P, PA, PAN, PANT	—
NOPANT	NOP, NOPA, NOPAN, NOPANT	—
VERIFI	V, VE, VER, VERI, VERIF, VERIFI	—
NOVERIFI	NOV, NOVE, NOVER, NOVERI, NOVERIF, NOVERIFI	—
CUENTA	C, CU, CUE, CUEN, CUENT, CUENTA	—
CONSU	CO, CON, CONS, CONSU	C selecciona CUENTA
NOCONSU	NOC, NOCO, NOCON, NOCONS, NOCONSU	—
INF	IN, INF	—
NOINF	NOI, NOIN, NOINF	—
VOLUM	VOL, VOLU, VOLUM	—

Palabras clave de BSC

Estas palabras clave pertenecen a BSC. Varias filas pueden compartir una operación; la columna de grafías incluye los alias que seleccionan esa misma operación.

Palabra clave completa	Grafías aceptadas	Notas
COM1	COM1	—
COM2	COM2	—
COM3	COM3	—
COM4	COM4	—
COMU1	COMU1	—
COMU2	COMU2	—
COMU3	COMU3	—
COMU4	COMU4	Se acepta la grafía exacta a pesar del umbral de prefijo almacenado.
TEST	TEST	—
IMP	IMP	—
IMPR	IMP, IMPR	—

Palabra clave completa	Grafías aceptadas	Notas
IMP1	IMP, IMP1	—
IMPR1	IMP, IMPR, IMPR1	—
IMP2	IMP2	—
IMPR2	IMPR2	—
IMP3	IMP3	—
IMPR3	IMPR3	—
IMP4	IMP4	—
IMPR4	IMPR4	—
PANT	PANT	—
MODIF	M, MO, MOD, MODI, MODIF	—

Palabras clave de COPIFI

Estas palabras clave pertenecen a COPIFI. Varias filas pueden compartir una operación; la columna de grafías incluye los alias que seleccionan esa misma operación.

Palabra clave completa	Grafías aceptadas	Notas
PANT	P, PA, PAN, PANT	—
NOPANT	NOP, NOPA, NOPAN, NOPANT	—
SISFECH	SISF, SISFE, SISFEC, SISFECH	—
ANTFECH	ANT, ANTF, ANTFE, ANTFEC, ANTFECH	—
DESD	DE, DES, DESD	—
NUM	NU, NUM	—
DSTRING	DST, DSTR, DSTRI, DSTRIN, DSTRING	—
HSTRING	HST, HSTR, HSTRI, HSTRIN, HSTRING	—
TRUNC	TR, TRU, TRUN, TRUNC	—
NOTRUNC	NOTR, NOTRU, NOTRUN, NOTRUNC	—
NOREEMP	NORE, NOREE, NOREEM, NOREEMP	—
REEMPLA	REE, REEM, REEMP, REEMPL, REEMPLA	—
A@ADIR	A, A@, A@A, A@AD, A@ADI, A@ADIR	—
PACK	PAC, PACK	PA selecciona PANT

Palabra clave completa	Grafías aceptadas	Notas
UNPACK	UNP, UNPA, UNPAC, UNPACK	—
MAYUSC	MA, MAY, MAYU, MAYUS, MAYUSC	—
LMINUSC	LMI, LMIN, LMINU, LMINUS, LMINUSC	—
MONTA	M, MO, MON, MONT, MONTA	—
VERIFI	V, VE, VER, VERI, VERIF, VERIFI	—
NOVERIFI	NOV, NOVE, NOVER, NOVERI, NOVERIF, NOVERIFI	—
TABLA	TABL, TABLA	—
CONSU	CONS, CONSU	—
NOCONSU	NOC, NOCO, NOCON, NOCONS, NOCONSU	—
PUBLIC	PUB, PUBL, PUBLI, PUBLIC	—
FILL	FI, FIL, FILL	—
SPECS	SP, SPE, SPEC, SPECS	—

Palabras clave de DIRIBM

Estas palabras clave pertenecen a DIRIBM. Varias filas pueden compartir una operación; la columna de grafías incluye los alias que seleccionan esa misma operación.

Palabra clave completa	Grafías aceptadas	Notas
IMPRESO	I, IM, IMP, IMPR, IMPRE, IMPRES, IMPRESO	—
IMPRESO	I, IM, IMP, IMPR, IMPRE, IMPRES, IMPRESO	—
IMP	I, IM, IMP	—
IMP1	I, IM, IMP, IMP1	—
IMPRESO2	IMPRESO2	IMPRE selecciona IMPRESO; IMPRES selecciona IMPRESO; IMPRESO selecciona IMPRESO
IMP2	IMP2	—
IMPRESO3	IMPRESO3	IMPRE selecciona IMPRESO; IMPRES selecciona IMPRESO; IMPRESO selecciona IMPRESO
IMP3	IMP3	—
IMPRESO4	IMPRESO4	IMPRE selecciona IMPRESO; IMPRES selecciona IMPRESO; IMPRESO selecciona IMPRESO

Palabra clave completa	Grafías aceptadas	Notas
IMP4	IMP4	—
PANT	P, PA, PAN, PANT	—

Palabras clave de ENVIAR

Estas palabras clave pertenecen a ENVIAR. Varias filas pueden compartir una operación; la columna de grafías incluye los alias que seleccionan esa misma operación.

Palabra clave completa	Grafías aceptadas	Notas
NOEOT	NOE, NOEO, NOEOT	—
COM1	COM1	—
COMU1	COMU1	—
COM2	COM2	—
COMU2	COMU2	—
COM3	COM3	—
COMU3	COMU3	—
COM4	COM4	—
COMU4	COMU4	—
CONSULTA	C, CO, CON, CONS, CONSU, CONSUL, CONSULT, CONSULTA	—
MENSAJE	ME, MEN, MENS, MENSA, MENSAJ, MENSAJE	—
NOCONSUL	NOC, NOCO, NOCON, NOCONS, NOCONSU, NOCONSUL	—
PUBLICO	PUB, PUBL, PUBLI, PUBLIC, PUBLICO	—

Palabras clave de ENVIARA

Estas palabras clave pertenecen a ENVIARA. Varias filas pueden compartir una operación; la columna de grafías incluye los alias que seleccionan esa misma operación.

Palabra clave completa	Grafías aceptadas	Notas
NOEOT	NOE, NOEO, NOEOT	—
COM1	COM1	—
COMU1	COMU1	—

Palabra clave completa	Grafías aceptadas	Notas
COM2	COM2	—
COMU2	COMU2	—
COM3	COM3	—
COMU3	COMU3	—
COM4	COM4	—
COMU4	COMU4	—
CONSULTA	C, CO, CON, CONS, CONSU, CONSUL, CONSULT, CONSULTA	—
MENSAJE	ME, MEN, MENS, MENSA, MENSAJ, MENSAJE	—
NOCONSUL	NOC, NOCO, NOCON, NOCONS, NOCONSU, NOCONSUL	—
PUBLICO	PUB, PUBL, PUBLI, PUBLIC, PUBLICO	—

Palabras clave de ESTADO

Estas palabras clave pertenecen a ESTADO. FRELOJ y ERRTEST no son palabras clave aceptadas; usa las consultas de la lista. Varias filas pueden compartir una operación; la columna de grafías incluye los alias que seleccionan esa misma operación.

Palabra clave completa	Grafías aceptadas	Notas
ALL	ALL	—
QUIEN	QU, QUI, QUIE, QUIEN	—
HORA	HO, HOR, HORA	—
FECHA	FE, FEC, FECH, FECHA	—
SISTEMA	SIS, SIST, SISTE, SISTEM, SISTEMA	—
FORFECHA	FORFE, FORFEC, FORFECH, FORFECHA	—
PRIVILEG	PRIV, PRIVI, PRIVIL, PRIVILE, PRIVILEG	—
RELOJ	R, RE, REL, RELO, RELOJ	—
DIA	DIA	—
MENSAJE	MEN, MENS, MENSA, MENSAJ, MENSAJE	—
TIEMPO	TI, TIE, TIEM, TIEMP, TIEMPO	—
USUARIOS	U, US, USU, USUA, USUAR, USUARI, USUARIO, USUARIOS	—
MEMORIA	MEM, MEMO, MEMOR, MEMORI, MEMORIA	—

Palabra clave completa	Grafías aceptadas	Notas
MODULO	MO, MOD, MODU, MODUL, MODULO	—
DISCO	DI, DIS, DISC, DISCO	—
ERRNUM	ERRN, ERRNU, ERRNUM	—
ERRTEXT	ERRT, ERRTE, ERRTEX, ERRTEXT	—
INCOM	INC, INCO, INCOM	—
ABREV	AB, ABR, ABRE, ABREV	—
IMPEXEC	IMP, IMPE, IMPEX, IMPEXE, IMPEXEC	—
HISTOR	HI, HIS, HIST, HISTO, HISTOR	—
RTCAVAIL	RTC, RTCA, RTCAV, RTCAVA, RTCAVAI, RTCAVAIL	—
SISFECHA	SISF, SISFE, SISFEC, SISFECH, SISFECHA	SIS selecciona SISTEMA
INIREL	INI, INIR, INIRE, INIREL	—
QET	QET	—
ALEMAN	ALE, ALEM, ALEMA, ALEMAN	—
ESCAPE	ESC, ESCA, ESCAP, ESCAPE	—
SUBIR	SU, SUB, SUBI, SUBIR	—
BAJAR	BA, BAJ, BAJA, BAJAR	—
IZQU	IZ, IZQ, IZQU	—
DERECHA	DE, DER, DERE, DEREC, DERECH, DERECHA	—
RUBOUT	RUB, RUBO, RUBOU, RUBOUT	—
CANCEL	CAN, CANC, CANCE, CANCEL	—
SINONIMO	SIN, SINO, SINON, SINONI, SINONIM, SINONIMO	—
VERSION	VER, VERS, VERSI, VERSIO, VERSION	—

Palabras clave de INICDISC

Estas palabras clave pertenecen a INICDISC. INCR es el selector aceptado de incremento/entrelazado; INTERL no es un alias. Varias filas pueden compartir una operación; la columna de grafías incluye los alias que seleccionan esa misma operación.

Palabra clave completa	Grafías aceptadas	Notas
ETIQU	E, ET, ETI, ETIQ, ETIQU	—

Palabra clave completa	Grafías aceptadas	Notas
VACIAR	V, VA, VAC, VACI, VACIA, VACIAR	—
DIRE	D, DI, DIR, DIRE	—
WP	W, WP	—
NOWP	NOW, NOWP	—
FORMAT	F, FO, FOR, FORM, FORMA, FORMAT	—
INCR	I, IN, INC, INCR	—
TIEM	TI, TIE, TIEM	—
CARA	C, CA, CAR, CARA	—
PISTA	P, PI, PIS, PIST, PISTA	—
CILINDRO	CI, CIL, CILI, CILIN, CILIND, CILINDR, CILINDRO	—
SECTOR	S, SE, SEC, SECT, SECTO, SECTOR	—
HSECT	H, HS, HSE, HSEC, HSECT	—
REPUES	RE, REP, REPU, REPUE, REPUES	—
DEVTYPE	DEV, DEVT, DEVTY, DEVTYP, DEVTYPE	—

Palabras clave de INICIAR

Estas palabras clave pertenecen a INICIAR. NOPE desactiva la protección contra escritura por software; NOWP pertenece a INICDISC. Varias filas pueden compartir una operación; la columna de grafías incluye los alias que seleccionan esa misma operación.

Palabra clave completa	Grafías aceptadas	Notas
ETIQU	ET, ETI, ETIQ, ETIQU	—
VACIA	V, VA, VAC, VACI, VACIA	—
DIRE	D, DI, DIR, DIRE	—
WP	W, WP	—
NOPE	NOP, NOPE	—
FORMAT	F, FO, FOR, FORM, FORMA, FORMAT	—
INCR	I, IN, INC, INCR	—
TIEM	TI, TIE, TIEM	—
CARA	C, CA, CAR, CARA	—

Palabra clave completa	Grafías aceptadas	Notas
PISTA	PI, PIS, PIST, PISTA	—
CILINDRO	CI, CIL, CILI, CILIN, CILIND, CILINDR, CILINDRO	—
SECTOR	S, SE, SEC, SECT, SECTO, SECTOR	—
HSECT	H, HS, HSE, HSEC, HSECT	—
SPARE	SP, SPA, SPAR, SPARE	—

Palabras clave de PONER

Estas palabras clave pertenecen a PONER. Varias filas pueden compartir una operación; la columna de grafías incluye los alias que seleccionan esa misma operación.

Palabra clave completa	Grafías aceptadas	Notas
NUL	NUL	—
SOH	SOH	—
STX	STX	—
ETX	ETX	—
EOT	EOT	—
ENQ	ENQ	—
ACK	ACK	—
BEL	BEL	—
BS	BS	Se acepta la grafía exacta a pesar del umbral de prefijo almacenado.
HT	HT	Se acepta la grafía exacta a pesar del umbral de prefijo almacenado.
LF	LF	Se acepta la grafía exacta a pesar del umbral de prefijo almacenado.
VT	VT	Se acepta la grafía exacta a pesar del umbral de prefijo almacenado.
FF	FF	Se acepta la grafía exacta a pesar del umbral de prefijo almacenado.
CR	CR	Se acepta la grafía exacta a pesar del umbral de prefijo almacenado.
SO	SO	Se acepta la grafía exacta a pesar del umbral de prefijo almacenado.
SI	SI	Se acepta la grafía exacta a pesar del umbral de prefijo almacenado.
DLE	DLE	—
DC1	DC1	—
DC2	DC2	—

Palabra clave completa	Grafías aceptadas	Notas
DC3	DC3	—
DC4	DC4	—
NAK	NAK	—
SYN	SYN	—
ETB	ETB	—
CAN	CAN	—
EM	EM	Se acepta la grafía exacta a pesar del umbral de prefijo almacenado.
SUB	SUB	—
ESC	ESC	—
FS	FS	Se acepta la grafía exacta a pesar del umbral de prefijo almacenado.
GS	GS	Se acepta la grafía exacta a pesar del umbral de prefijo almacenado.
RS	RS	Se acepta la grafía exacta a pesar del umbral de prefijo almacenado.
US	US	Se acepta la grafía exacta a pesar del umbral de prefijo almacenado.
DEL	DEL	—

Palabras clave de RECARGAR

Estas palabras clave pertenecen a RECARGAR. Varias filas pueden compartir una operación; la columna de grafías incluye los alias que seleccionan esa misma operación.

Palabra clave completa	Grafías aceptadas	Notas
CUENTA	C, CU, CUE, CUEN, CUENT, CUENTA	—
RESCUE	RESC, RESCU, RESCUE	—
REEMPLA	R, RE, REE, REEM, REEMP, REEMPL, REEMPLA	—
NOREEM	N, NO, NOR, NORE, NOREE, NOREEM	—
COPREEM	CO, COP, COPR, COPRE, COPREE, COPREEM	C selecciona CUENTA
CONSU	CON, CONS, CONSU	C selecciona CUENTA; CO selecciona COPREEM
NOCONSU	NOC, NOCO, NOCON, NOCONS, NOCONSU	—
PANT	P, PA, PAN, PANT	—
NOPANT	NOP, NOPA, NOPAN, NOPANT	—
INF	IN, INF	—

Palabra clave completa	Grafías aceptadas	Notas
NOINF	NOI, NOIN, NOINF	—
VERIFI	V, VE, VER, VERI, VERIF, VERIFI	—
NOVERIFI	NOV, NOVE, NOVER, NOVERI, NOVERIF, NOVERIFI	—
VOLUM	VO, VOL, VOLU, VOLUM	—

Palabras clave de SERIAL

Estas palabras clave pertenecen a SERIAL. Varias filas pueden compartir una operación; la columna de grafías incluye los alias que seleccionan esa misma operación.

Palabra clave completa	Grafías aceptadas	Notas
SERIAL	S, SE, SER, SERI, SERIA, SERIAL	—
V	No se ha establecido ningún prefijo aceptado.	—
S	No se ha establecido ningún prefijo aceptado.	—
M	No se ha establecido ningún prefijo aceptado.	—
N	No se ha establecido ningún prefijo aceptado.	—
E	No se ha establecido ningún prefijo aceptado.	—
PANT	P, PA, PAN, PANT	—
NOPANT	NOP, NOPA, NOPAN, NOPANT	—

Palabras clave de SPOOLER

Estas palabras clave pertenecen a SPOOLER. Varias filas pueden compartir una operación; la columna de grafías incluye los alias que seleccionan esa misma operación.

Palabra clave completa	Grafías aceptadas	Notas
INIC	I, IN, INI, INIC	—
FINAL	F, FI, FIN, FINA, FINAL	—
LIST	L, LI, LIS, LIST	—
FORM	FO, FOR, FORM	F selecciona FINAL
COLA	C, CO, COL, COLA	—
COPIAS	COP, COPI, COPIA, COPIAS	CO selecciona COLA
COPI	COP, COPI	CO selecciona COLA

Palabra clave completa	Grafías aceptadas	Notas
ESTADO	E, ES, EST, ESTA, ESTAD, ESTADO	—
RETE	R, RE, RET, RETE	—
NORETE	NOR, NORE, NORET, NORETE	—
PARA	PAR, PARA	—
ACTIV	ACT, ACTI, ACTIV	—
SAL	SAL	—
MUINIT2	MUINIT2	—
MUINIT3	MUINIT3	—
CAMBIA	CA, CAM, CAMB, CAMBI, CAMBIA	—
ELIM	ELIM	—
SALVAR	S, SA, SALV, SALVA, SALVAR	SAL selecciona SAL
BACKSPAC	B, BA, BAC, BACK, BACKS, BACKSP, BACKSPA, BACKSPAC	—
IMPR	IM, IMP, IMPR	I selecciona INIC
ALINE	AL, ALI, ALIN, ALINE	—
NOMB	N, NO, NOM, NOMB	—
RELANZA	REL, RELA, RELAN, RELANZ, RELANZA	R selecciona RETE; RE selecciona RETE

Palabras clave de SPOOLER

Estas palabras clave pertenecen a SPOOLER. Varias filas pueden compartir una operación; la columna de grafías incluye los alias que seleccionan esa misma operación.

Palabra clave completa	Grafías aceptadas	Notas
RETE	RE, RET, RETE	—
NORETE	NOR, NORE, NORET, NORETE	—

Palabras clave de TERMINAL

Estas palabras clave pertenecen a TERMINAL. Varias filas pueden compartir una operación; la columna de grafías incluye los alias que seleccionan esa misma operación.

Palabra clave completa	Grafías aceptadas	Notas
AUTOLF	AU, AUT, AUTO, AUTOL, AUTOLF	—

Palabra clave completa	Grafías aceptadas	Notas
RUB	RUB	—
CTL	CTL	—
NULL	NUL, NULL	—
ESC	ESC	—
BEEHIVE	B, BE, BEE, BEEH, BEEHI, BEEHIV, BEEHIVE	—
ADDS	AD, ADD, ADDS	—
SOROC	S, SO, SOR, SORO, SOROC	—
ADM3A	ADM, ADM3, ADM3A	—
HAZELTIN	HAZ, HAZE, HAZEL, HAZELT, HAZELTI, HAZELTIN	—
COM1	COM1	—
COM2	COM2	—
COM3	COM3	—
COM4	COM4	—
COMU1	COMU1	—
COMU2	COMU2	—
COMU3	COMU3	—
COMU4	COMU4	—
HALF	HA, HAL, HALF	—
VDM	V, VD, VDM	—
RDRON	RDRON	—
RDROFF	RDROF, RDROFF	—
PCHON	PCHON	—
PCHOFF	PCHOF, PCHOFF	—
MASTER	M, MA, MAS, MAST, MASTE, MASTER	—
GO	G, GO	—

Palabras clave de TERMINAL

Estas palabras clave pertenecen a TERMINAL. Varias filas pueden compartir una operación; la columna de grafías incluye los alias que seleccionan esa misma operación.

Palabra clave completa	Grafías aceptadas	Notas
CONSOLE	CON, CONSOLE	—
READER	READER	—
PUNCH	PUNCH	—
PRINTER1	PRINT, PRINTE, PRINTER, PRINTER1	—
PRINTER2	PRINTER2	—
PRINTER3	PRINTER3	—
PRINTER4	PRINTER4	—
COMU1	COM, COMU, COMU1	—
COMU2	COMU2	—
COMU3	COMU3	—
COMU4	COMU4	—
TAPE1	TAP, TAPE, TAPE1	—
TAPE2	TAPE2	—
TAPE3	TAPE3	—
TAPE4	TAPE4	—
PRINTER	PRINT, PRINTE, PRINTER	—
COMU	COM, COMU	—
TAPE	TAP, TAPE	—
CON	CON	—
RDR	RDR	—
PCH	PCH	—
PRT1	PRT1	—
PRT2	PRT2	—
PRT3	PRT3	—
PRT4	PRT4	—
COM1	COM, COM1	—
COM2	COM2	—
COM3	COM3	—
COM4	COM4	—

Palabra clave completa	Grafías aceptadas	Notas
TAP1	TAP, TAP1	—
TAP2	TAP2	—
TAP3	TAP3	—
TAP4	TAP4	—

Referencia de instrucciones de EXEC

Las directivas de EXEC van dentro de un archivo de texto EXEC. El signo & las distingue de los comandos normales que el archivo pide ejecutar a CSI. El capítulo 8 enseña la creación de archivos, la invocación, las variables, las comparaciones y los bucles. El índice siguiente da cuenta de los diecinueve nombres del despachador; END es un delimitador de bloque adicional, no una entrada del despachador.

Directiva	Operación o límite de la interfaz
&TYPE	Mostrar campos y valores sustituidos; capítulo 8.
&BEGTYPE	Mostrar líneas literales hasta END; capítulo 8.
&SPACE	Espaciado de líneas de salida; véase el ejemplo siguiente.
&CONTINUE	Seguir leyendo la siguiente línea del archivo de comandos; no reanuda la ejecución de BASIC.
&QUIT	Terminar este nivel de EXEC, devolviendo opcionalmente un resultado; capítulo 8.
&STACK	Poner en cola una línea para la entrada de un programa.
&BEGSTACK	Poner en cola líneas literales hasta END.
&SKIP	Mover la posición de lectura del archivo de comandos un número de líneas.
&GOTO	Bifurcar a una etiqueta del archivo de comandos; capítulo 8.
&CONTROL	Seleccionar el tratamiento del eco y del control y de la entrada en cola.
&IF	Comprobar una condición antes de ejecutar el resto de la instrucción; capítulo 8.
&CRT	Enviar una salida de control de pantalla o colocar el cursor.
&ESC	Controlar determinados tratamientos de Esc; el contrato completo de los modos sigue sin especificarse.
&READ	Leer una línea de entrada en variables de EXEC con nombre.
&WAIT	Solicitar la pausa de paginación del ejecutivo; véase el ejemplo siguiente.
&WHILE	Repetir mientras se cumpla una condición, emparejado con REPEAT; capítulo 8.
&UNTIL	Repetir hasta que se cumpla una condición, emparejado con REPEAT.

Directiva	Operación o límite de la interfaz
`&REPEAT`	Volver a la instrucción WHILE o UNTIL que controla el bucle.
`&ERROR`	Registrar el tratamiento de errores para resultados de comandos posteriores; capítulo 8.

SPACE: separar líneas de salida

Esta ilustración de sintaxis va en un archivo de comandos:

```
&SPACE 2
```

SPACE realiza dos operaciones de espaciado de líneas a través del ejecutivo. Sin recuento, realiza una; el cero también se trata como uno. El recuento se procesa mediante un valor de ocho bits, así que no debe tratarse como una cantidad de maquetación de página sin restricciones. SPACE es distinto de la función de cadena SPACE$ de BASIC.

CONTINUE: pasar a la línea siguiente

Por ejemplo, un destino con etiqueta puede contener:

```
-LISTO &CONTINUE
```

Cuando la ejecución llega a LISTO, CONTINUE pasa a la siguiente línea del archivo de comandos. No reinicia un programa terminado ni realiza la operación CONTINUE de BASIC.

READ: obtener una respuesta

Esta forma aparece en procedimientos de comandos interactivos:

```
&READ &RESP
```

READ obtiene la entrada y asigna un campo a RESP. En los ejemplos para principiantes, limita las respuestas a una sola palabra dentro de la anchura de valor de ocho caracteres de EXEC. El tratamiento de varias variables y de la entrada vacía requiere más cuidado que esta forma básica; un READ sin variable sigue otra ruta de procesamiento de comandos y no es la misma operación.

STACK: proporcionar entrada al siguiente programa

Un procedimiento de comandos puede poner en cola una respuesta en lugar de esperar a que la escriba un operador. Por ejemplo, un procedimiento que va a entrar en BASIC puede poner en cola su comando de salida:

```
&STACK FIFO QUIT
```

El texto en cola es QUIT. FIFO significa «primero en entrar, primero en salir»: las líneas más antiguas de la cola se consumen primero. La forma opcional LIFO coloca en cambio la entrada más reciente delante de la más antigua. Si se omite el selector de orden, se usa FIFO. Poner en cola no ejecuta QUIT por sí mismo; el programa que consume la entrada debe interpretarlo. BASIC reconoce QUIT, mientras que otro programa podría interpretar esa misma línea de otra forma.

Para entrar en BASIC después de poner en cola esa respuesta, la siguiente línea del archivo es:

```
BASIC
```

Esto ilustra el mecanismo de entrada. Úsalo solo después de guardar el trabajo de BASIC. Una respuesta en cola puede contestar a una confirmación destructiva con la misma facilidad que a una pregunta de edición; no reutilices colas de entrada sin conocer el diálogo del programa receptor.

BEGSTACK: proporcionar varias líneas de entrada literales

Esta ilustración de sintaxis aporta instrucciones BASIC como entrada en cola:

```
&BEGSTACK FIFO
PRINT 1
QUIT
&END
```

BEGSTACK lee líneas literales hasta END y las pone en cola en el orden seleccionado. Estas líneas son entrada para un programa posterior, no instrucciones de EXEC, y el bloque no sustituye las variables de EXEC. Un comando BASIC posterior consumiría las líneas PRINT y QUIT. Es distinto de repetir instrucciones STACK, en las que las sustituciones pueden construir cada respuesta.

CONTROL: política de eco y de entrada en cola

Los procedimientos de comandos suelen empezar así:

```
&CONTROL OFF NOSTACK
```

OFF suprime el eco de los comandos. NOSTACK selecciona un modo de control de la entrada en cola; no prohíbe instrucciones STACK posteriores. Activa el bit 3 del byte de control de ejecución de la partición, mientras que STACK borra ese bit. El efecto completo de este indicador en los pasos posteriores sigue sin especificarse. Los procedimientos existentes combinan este modo con respuestas en cola para las utilidades que invocan. STACK selecciona el modo de control opuesto:

```
&CONTROL STACK
```

Estas palabras de CONTROL son independientes de FIFO/LIFO, que eligen el orden de inserción en la cola. Las interacciones completas con interrupciones, intérpretes anidados y terminaciones anómalas de programas siguen sin especificarse; usa la secuencia deliberada de un procedimiento existente en lugar de cambiar de modo por analogía con otro lenguaje de comandos.

SKIP: saltar líneas

La instrucción siguiente aparece en los procedimientos de inserción de código fuente:

```
&SKIP 2
```

Avanza más allá de las dos líneas siguientes del archivo de comandos. El recuento se refiere a líneas físicas del archivo, así que cambiar comentarios o texto insertado puede alterar el destino. Un GOTO con etiqueta es más claro en los procedimientos nuevos cuando el destino debe sobrevivir a ese tipo de ediciones. Existe tratamiento para recuentos negativos, pero no es una interfaz básica de bucle hacia atrás.

UNTIL: repetir hasta que una condición sea verdadera

Esta ilustración de sintaxis usa el mismo mecanismo de condiciones y de REPEAT que el capítulo 8:

```
&A = 1
&UNTIL &A > 3
&TYPE &A
&A = &A + 1
&REPEAT
&QUIT
```

UNTIL comprueba la condición al principio del grupo. El grupo continúa mientras A no sea mayor que 3; REPEAT vuelve a esa comprobación. Por tanto, es distinto de los lenguajes en los que la comprobación de UNTIL se produce siempre después de la primera iteración. Los procedimientos de inserción de código fuente usan esta forma para recoger un número fijo de argumentos.

CRT, ESC y WAIT

CRT CLEAR se usa antes de las presentaciones de procedimientos interactivos:

```
&CRT CLEAR
```

Solicita la operación de borrado del terminal. La forma numérica de CRT toma las posiciones horizontal y vertical; el intérprete resta uno a cada una antes de emitir la secuencia de control de posicionamiento del terminal. Por ejemplo:

```
&CRT 47 14
```

Así se solicita la posición 47 de la línea 14 en un terminal compatible. El tratamiento del dispositivo sigue siendo importante; no es una API portable de gráficos de pantalla.

Los procedimientos también usan esta forma de control de Esc en torno a operaciones que tienen sus propios diagnósticos:

```
&ESC O OFF
```

La forma correspondiente es:

```
&ESC O ON
```

Estas formas seleccionan y restauran un indicador de control de modo O. El significado completo de ese indicador y de todos los modos ESC sigue sin especificarse; estas dos líneas son formas que conviene reconocer al leer procedimientos, no una recomendación para suprimir en general los errores o las interrupciones. WAIT invoca el servicio de pausa de paginación del ejecutivo. Su forma básica es:

```
&WAIT
```

Llama al servicio 0x31 y reanuda la lectura del archivo de comandos cuando ese servicio vuelve. No se analiza ninguna duración; no es una instrucción de espera temporizada. El comportamiento exacto de la pausa y de la cancelación sigue la política de paginación de la consola activa; consulta el servicio 0x31.

Operandos especiales

Los operandos especiales aportan valores o forman parte de expresiones; no son comandos para introducir en CSI.

Operando	Significado o límite de la interfaz
&RETCODE	Resultado del comando o del procedimiento hijo anterior; véase el capítulo 8.
&INDEX	Número de argumentos posicionales, hasta dieciséis; véase el capítulo 8.
&NULL	Valor vacío, relleno de espacios; se usa en comparaciones de argumentos y variables.
&PAGE, &LINE	Valores obtenidos mediante el servicio de geometría de la consola 0x3B; no es un contador de páginas del documento. La interpretación completa para el operador sigue sin especificarse.
&TYP, &LEN, &LIT	Nombres de expresiones reconocidos; los contratos completos de argumentos, resultados y entrecomillado siguen sin especificarse.
&CAT	Operando de construcción de cadenas que usan los procedimientos de vocabulario; las reglas completas generales de entrecomillado y concatenación siguen sin especificarse.
&SUB	Selección de subcadena con inicio y fin incluidos; véase el capítulo 8.

Por ejemplo, este fragmento de procedimiento comprueba si ARG no tiene ningún valor:

```
&IF &ARG EQ &NULL &GOTO -FALTA
```

FALTA debe ser una etiqueta del mismo procedimiento. No sustituyas &NULL por la palabra literal NULL. Para unir cadenas en general, usa la expresión documentada con barra vertical del capítulo 8 hasta que se disponga del contrato completo de CAT.

Comandos internos de EDITAR

EDITAR edita texto secuencial y muestra su propio indicador de asterisco. Sus nombres de comando son independientes de las abreviaturas de CSI y de los comandos del editor de BASIC. Empieza con el procedimiento práctico de CASE U, INPUT y FILE del capítulo 7.

La tabla siguiente conserva los cuarenta y siete nombres internos. Un mínimo almacenado es un umbral del analizador, no una garantía de que cualquier prefijo seleccione esa fila: las filas coincidentes anteriores pueden tener prioridad. Los nombres completos evitan esa ambigüedad. Un nombre y un manejador identifican una rama disponible del analizador, pero no establecen por sí solos sus argumentos, su intervalo predeterminado ni sus efectos de modificación. Cuando esos detalles no están especificados, no se ofrece ningún comando de edición inventado.

Comando completo del editor	Longitud mínima almacenada	Operación o limitación
`AGAIN`	1	Comando del editor reconocido; el contrato completo de argumentos y efectos no está especificado.
`TABSET`	2	Comando del editor reconocido; el contrato completo de argumentos y efectos no está especificado.
`HELP`	1	Comando del editor reconocido; el contrato completo de argumentos y efectos no está especificado.
`NEXT`	1	Comparte el manejador de DOWN; los argumentos completos de desplazamiento no están especificados.
`UP`	1	Comando del editor reconocido; el contrato completo de argumentos y efectos no está especificado.
`BOTTOM`	1	Comando del editor reconocido; el contrato completo de argumentos y efectos no está especificado.
`TYPE`	1	Comparte el manejador de LIST; la sintaxis completa de intervalo y salida no está especificada.
`FILE`	4	Escribe y sale del editor; consulta el capítulo 7 para el comportamiento de la interrupción pendiente.
`SAVE`	4	Comando del editor reconocido; el contrato completo de argumentos y efectos no está especificado.
`LOCATE`	1	Los procedimientos de inserción de código fuente usan LOCATE seguido de una cadena marcadora. Las reglas completas de coincidencia no están especificadas.
`ZONE`	2	Comando del editor reconocido; el contrato completo de argumentos y efectos no está especificado.

Comando completo del editor	Longitud mínima almacenada	Operación o limitación
UNNUMBER	2	Comando del editor reconocido; el contrato completo de argumentos y efectos no está especificado.
HTAB	2	Comando del editor reconocido; el contrato completo de argumentos y efectos no está especificado.
X	1	Comando del editor reconocido; el contrato completo de argumentos y efectos no está especificado.
Y	1	Comando del editor reconocido; el contrato completo de argumentos y efectos no está especificado.
Z	1	Comando del editor reconocido; el contrato completo de argumentos y efectos no está especificado.
VERIFY	1	Comando del editor reconocido; el contrato completo de argumentos y efectos no está especificado.
WRAPMODE	1	Comando del editor reconocido; el contrato completo de argumentos y efectos no está especificado.
FIND	1	Comando del editor reconocido; el contrato completo de argumentos y efectos no está especificado.
RENUMBER	3	Los procedimientos de inserción de código fuente usan RENUMBER 10 10 con el modo de líneas activo. Las reglas completas de intervalo y de colisión no están especificadas.
PAGE	1	Comando del editor reconocido; el contrato completo de argumentos y efectos no está especificado.
TOP	2	Se usa para situar las operaciones de inserción de código fuente; el modo de edición posterior importa.
QUIT	4	Ruta de salida; no des por hecho que guarda los cambios. Usa el procedimiento FILE del capítulo 7.
?	1	Comando del editor reconocido; el contrato completo de argumentos y efectos no está especificado.
INPUT	1	INPUT texto inserta el texto que sigue; capítulo 7.
REPLACE	1	Comando del editor reconocido; el contrato completo de argumentos y efectos no está especificado.
DELETE	2	Los procedimientos de inserción de código fuente lo usan con un argumento numérico; el intervalo predeterminado y la recuperación no están especificados.
MODIFY	2	Comando del editor reconocido; el contrato completo de argumentos y efectos no está especificado.
CASE	2	CASE U conserva las mayúsculas de la entrada; capítulo 7.
COLUMN	2	Comando del editor reconocido; el contrato completo de argumentos y efectos no está especificado.
COMBINE	3	Comando del editor reconocido; el contrato completo de argumentos y efectos no está especificado.

Comando completo del editor	Longitud mínima almacenada	Operación o limitación
CSI	3	Comando del editor reconocido; el contrato completo de argumentos y efectos no está especificado.
TRUNCATE	2	Comando del editor reconocido; el contrato completo de argumentos y efectos no está especificado.
NAME	2	Comando del editor reconocido; el contrato completo de argumentos y efectos no está especificado.
LINEMODE	3	Las formas ON/OFF aparecen en los procedimientos de inserción de código fuente; sus efectos sobre la numeración y la posición requieren cuidado.
PROMPT	3	Comando del editor reconocido; el contrato completo de argumentos y efectos no está especificado.
CHANGE	1	Comando del editor reconocido; el contrato completo de argumentos y efectos no está especificado.
DUP	2	Comando del editor reconocido; el contrato completo de argumentos y efectos no está especificado.
LIST	2	Comparte el manejador de TYPE; la sintaxis completa de intervalo y salida no está especificada.
DOWN	1	Comparte el manejador de NEXT; los argumentos completos de desplazamiento no están especificados.
GET	3	Los procedimientos de inserción de código fuente usan GET nombre BASIC S para importar código fuente secuencial. Las reglas completas de posición de inserción y de errores no están especificadas.
PUTD	4	Comando del editor reconocido; el contrato completo de argumentos y efectos no está especificado.
NUMBER	2	Comando del editor reconocido; el contrato completo de argumentos y efectos no está especificado.
PUT	3	Los procedimientos de inserción de código fuente usan PUT nombre BASIC S seguido de un límite numérico. La semántica completa del límite no está especificada.
ERROR	2	Comando del editor reconocido; el contrato completo de argumentos y efectos no está especificado.
SKIP	2	Comando del editor reconocido; el contrato completo de argumentos y efectos no está especificado.
SPLIT	2	Comando del editor reconocido; el contrato completo de argumentos y efectos no está especificado.

Para una primera inserción segura en el indicador de asterisco, introduce:

```
INPUT UNA LINEA CORTA
```

Así se inserta UNA LINEA CORTA en el texto actual. No se ejecuta ese texto. Guarda con FILE como se describe en el capítulo 7. Los procedimientos avanzados de inserción de código fuente cambian deliberadamente LINEMODE, la posición, importan texto y renumeran líneas; esas secuencias no son intercambiables con los comandos del editor de BASIC que llevan los mismos nombres. CEDIT y TEDITAR son editores independientes cuyos diálogos internos completos siguen sin especificarse.

Vocabulario de CONSULTA y dependencias del diccionario

CONSULTA interpreta textos de consulta en español frente a definiciones de archivos y atributos. CONTROL.VOC aporta el vocabulario que mantienen los procedimientos VOC y GENVOC. CONSULTA usa un archivo de trabajo cuyo nombre termina con el número de la partición de ejecución, como CONSULTA.TRAB1 para la partición 1. Consulta el apéndice C para esas herramientas de apoyo y el capítulo 31 para los requisitos de entrada.

La instrucción siguiente es la forma de listado del diccionario que usa VOC. Pertenece a CONSULTA, no a CSI. Requiere un CONTROL.VOC correctamente inicializado y un almacenamiento de trabajo compatible:

```
LISTAR CONTROL VOC VIS NOMBRE ATRIBUTO CAMPO LONGITUD FORMATO ROTULO SUP.ID CABECERA 'Diccionario
del fichero CONTROL' @FECHA
```

LISTAR selecciona el listado; CONTROL VOC designa el archivo de diccionario. VIS selecciona la salida por terminal. Los nombres de atributo siguientes eligen los campos que se muestran. SUP.ID y CABECERA controlan la presentación; el texto entre comillas aporta un encabezado y @FECHA solicita la sustitución de la fecha. Estos nombres de atributo proceden del diccionario y no son palabras reservadas universales para todos los archivos de aplicación. El contrato completo de anchura, paginación y errores de consulta no está especificado. El ejemplo de envoltorio siguiente evita escribir una consulta larga en el indicador del sistema.

En CSI, esta forma de procedimiento solicita el mismo listado del diccionario:

```
EXEC VOC LIST
```

VOC pone en cola su consulta y entra en CONSULTA. Requiere que el diccionario exista; no es una instrucción para crearlo automáticamente. Para imprimir en lugar de mostrar, la función IMPR del envoltorio usa una impresora configurada.

Vocabulario reservado

La tabla agrupa las palabras con bytes de resultado del analizador idénticos. Así se ve dónde el analizador da el mismo token a varias grafías; no garantiza que las instrucciones completas puedan reordenarse libremente. La puntuación es significativa: POR.DESC, NO.COMP, @PAGINA y los símbolos de comparación son formas literales. SON aparece dos veces en el vocabulario, con pares de resultado distintos.

Palabras	Bytes de resultado del analizador	Límite de interpretación
`LISTAR`, `BUSCAR`	0x00, 0x00	Mismo resultado del analizador; sigue aplicándose la gramática de la instrucción que lo rodea.
`CLASIFICAR`, `ORDENAR`, `ORGANIZAR`	0x00, 0x02	Mismo resultado del analizador; sigue aplicándose la gramática de la instrucción que lo rodea.
`SELECC`	0x00, 0x01	Token reservado; úsalo en su contexto de consulta.
`CSELECC`	0x00, 0x03	Token reservado; úsalo en su contexto de consulta.
`DESDE`	0x0F, 0x00	Token reservado; úsalo en su contexto de consulta.
`TIENEN`, `CON`	0x14, 0x00	Mismo resultado del analizador; sigue aplicándose la gramática de la instrucción que lo rodea.
`IMP`	0x1E, 0x00	Token reservado; úsalo en su contexto de consulta.
`VIS`	0x1E, 0x01	Token reservado; úsalo en su contexto de consulta.
`NO.COMP`	0x21, 0x00	Token reservado; úsalo en su contexto de consulta.
`COMPLETO`	0x22, 0x00	Token reservado; úsalo en su contexto de consulta.
`TOTAL`	0x32, 0x00	Token reservado; úsalo en su contexto de consulta.
`POR`	0x46, 0x41	Token reservado; úsalo en su contexto de consulta.
`POR.DESC`	0x46, 0x44	Token reservado; úsalo en su contexto de consulta.
`SALTAR.EN`, `SALTO`, `SUBTOTAL`, `ROMPER`	0x5A, 0x00	Mismo resultado del analizador; sigue aplicándose la gramática de la instrucción que lo rodea.
`CABECERA`	0x6E, 0x00	Token reservado; úsalo en su contexto de consulta.
`PIE.DER`	0x6F, 0x00	Token reservado; úsalo en su contexto de consulta.
`PIE.IZQ`	0x70, 0x00	Token reservado; úsalo en su contexto de consulta.
`@PAGINA`, `@FECHA`, `@HORA`	0x18, 0x00	Mismo resultado del analizador; sigue aplicándose la gramática de la instrucción que lo rodea.

Palabras	Bytes de resultado del analizador	Límite de interpretación
`SUP.ID`	0x82, 0x00	Token reservado; úsalo en su contexto de consulta.
`ID`	0x87, 0x00	Token reservado; úsalo en su contexto de consulta.
`SIN`	0x15, 0x00	Token reservado; úsalo en su contexto de consulta.
`NO`, `NINGUN`, `NINGUNO`	0x13, 0x00	Mismo resultado del analizador; sigue aplicándose la gramática de la instrucción que lo rodea.
`=`, `IGUAL`, `COMO`, `EN`, `SIMILAR`, `SEMEJANTE`	0x17, 0x01	Mismo resultado del analizador; sigue aplicándose la gramática de la instrucción que lo rodea.
`>`, `MAYOR`, `DESPUES`, `SUPERIOR`, `MAS`, `POSTERIOR`	0x17, 0x02	Mismo resultado del analizador; sigue aplicándose la gramática de la instrucción que lo rodea.
`<`, `MENOR`, `ANTERIOR`, `MENOS`, `INFERIOR`	0x17, 0x03	Mismo resultado del analizador; sigue aplicándose la gramática de la instrucción que lo rodea.
`#`, `<>`, `NI`, `DISTINTO`, `DESIGUAL`	0x17, 0x04	Mismo resultado del analizador; sigue aplicándose la gramática de la instrucción que lo rodea.
`MENOR.IGUAL`, `<=`	0x17, 0x05	Mismo resultado del analizador; sigue aplicándose la gramática de la instrucción que lo rodea.
`MAYOR.IGUAL`, `>=`	0x17, 0x06	Mismo resultado del analizador; sigue aplicándose la gramática de la instrucción que lo rodea.
`CONTENIENDO`, `QUE.CONTENGA`, `INCLUYENDO`, `INCLUYE`	0x17, 0x07	Mismo resultado del analizador; sigue aplicándose la gramática de la instrucción que lo rodea.
`Y`	0x19, 0x01	Token reservado; úsalo en su contexto de consulta.
`O`	0x19, 0x00	Token reservado; úsalo en su contexto de consulta.
`UN`, `UNA`, `SON`, `CUALQUIER`, `CADA`, `TODOS`, `FICHERO`, `PARA`, `CUYO`, `REGISTROS`, `DE`, `LOS`, `ES`, `DEL`, `A`, `QUE`, `EL`, `DONDE`, `TAMBIEN`, `CUYOS`, `SON`, `AQUELLOS`, `CUYA`, `POR.FAVOR`	0x0A, 0x00	Mismo resultado del analizador; sigue aplicándose la gramática de la instrucción que lo rodea.

El vocabulario incluye operaciones, palabras de selección, sinónimos de comparación, controles de salida, sustituciones y palabras de enlace. No es una lista de comandos del sistema independientes. En concreto, el CLASIFICAR de CONSULTA no establece un comando de CSI llamado CLASIF. Todavía no se dispone de una gramática completa para consultas arbitrarias; la forma de listado del diccionario anterior no debe ampliarse con convenciones de SQL.

PANTALLA: controles de pantalla que distinguen mayúsculas y minúsculas

PANTALLA diseña pantallas de entrada de datos. Sus controles de punto y coma en mayúsculas se enumeran en el capítulo 31. Además, se anuncian los siguientes controles en minúsculas para el aspecto de las etiquetas:

Controles literales	Atributo de la etiqueta
;i / ;j	Vídeo inverso
;s / ;t	Subrayado
;p / ;q	Parpadeo

Por ejemplo, el control literal es:

```
;i
```

Pertenece al par de vídeo inverso dentro de PANTALLA. No están especificados el sentido completo de activación o desactivación, la persistencia ni el comportamiento en cada terminal; no des por hecho que equivale a ;I en mayúscula, que inserta un campo. Del mismo modo, ;s en minúscula es distinto de ;S en mayúscula, que guarda el diseño.

El movimiento de ventanas se anuncia con esta forma:

```
;<flecha>nn<intro>
```

Después del punto y coma, pulsa una tecla de flecha, introduce el número de posiciones y pulsa Intro. Las expresiones entre signos de menor y mayor designan teclas, no texto literal. La codificación exacta de las flechas y los límites de movimiento dependen del terminal y no están especificados por completo.

Apéndice A. Formatos de programa, de lenguaje y de archivado

Este apéndice distingue la reubicación de ejecutables, el código fuente BASIC tokenizado, las instrucciones BASIC compiladas, los valores en tiempo de ejecución y los marcos de trabajo de EXEC. Bytes de token similares pueden significar cosas distintas en estos espacios de nombres. No hay establecida ninguna especificación general de objetos o bibliotecas de ensamblador solo porque los mensajes heredados mencionen ABS, REL, COM, símbolos o PAB.

Formatos de ejecutables nativos y de submódulos

Una imagen de carga R empieza en el desplazamiento de archivo 0 e incluye código y datos inicializados. La ruta de ejecución de comandos entra en un comando por su base de carga, pero cargar un submódulo R no especifica por sí solo dónde empezar la ejecución. Las superposiciones de BASIC tienen un byte de componente seguido de un hueco JP en el desplazamiento 0x0001; las superposiciones de SORT colocan sus huecos de entrada después de un área de control de 0xF0 bytes. Quien llama debe seguir la interfaz específica del componente descrita en el apéndice C. Directorio+0x1E aporta la longitud de la imagen; +0x11, el total de sectores almacenados; y +0x15, el primer LBA. Cuando existen, los registros de reubicación empiezan en `ceil(image_bytes/256)*256`; un módulo que no necesite correcciones puede no tener sector de reubicación. Cada sector de reubicación tiene un recuento de un byte de 1 a 127, seguido de ese número de desplazamientos de imagen LE16. Suma la base de carga a la palabra LE16 de cada desplazamiento, módulo 65 536. Valida que ambos bytes estén dentro de la imagen antes de aplicar una corrección en un cargador o en un lector del formato.

Las imágenes A se cargan en directorio+0x1E. El contrato general completo de ejecución de las imágenes A está menos especificado que la carga de R. Un prólogo `CF41` opcional va precedido del privilegio mínimo y de tres bytes de número de serie, como se describe en el capítulo 30; no es una cabecera de archivo obligatoria.

El descriptor compacto de submódulo del servicio 0x62 tiene siete bytes: dirección de carga LE16 en +0, longitud de la imagen LE16 en +2, byte de sectores almacenados en +4 y LBA relativo LE16 en +5. Quien lo llama aporta el LBA base. Los módulos BASICUSR pueden volver a través de un envoltorio de BASIC definido; esto no establece una cadena general de herramientas de enlazador y bibliotecas.

Registros de código fuente BASICOBJ

Desplazamiento en el registro	Tamaño	Significado
00	1	Longitud total del registro N, incluidos este byte y el terminador
01	2	Número de línea del código fuente, big-endian
03	1	Número de espacios entre el número de línea y la instrucción en el código fuente especificado
04	N−5	Cuerpo de la instrucción tokenizado
N−1	1	CR (0Dh)

Un byte de longitud de registro cero termina el flujo de código fuente; le sigue 0x1A como relleno secuencial. La longitud de un registro debe incluir su CR final. Una longitud mal formada o la falta de terminador es un error de formato del código fuente; no intentes adivinar los límites de las líneas siguientes.

Bytes	Interpretación probada
20 seguido del texto restante	Instrucción REM; se conservan la grafía y los espacios del comentario
29	Instrucción END
3A	Instrucción PRINT
05 alto bajo	Literal entero, palabra big-endian; el signo depende del contexto del código fuente
06 longitud texto...	Literal de cadena; longitud de un byte seguida de los caracteres exactos
1C antes de un literal numérico	Menos unario en el código fuente especificado
03 seguido de 8 bytes	Literal numérico decimal empaquetado, también usado por encima de 32 767

Cabecera y expresiones de BASICCOM

Desplazamiento	Tamaño	Significado
00	1	Cero obligatorio que comprueba el entorno de ejecución; no es el número mágico de los ejecutables R
01	1	Política de base de matrices: 00 establece 0, 01 establece 1; FF mantiene la política actual
02	2 BE	Cabeza de la lista enlazada de definiciones, relativa al programa; cero significa que no hay lista
04	2 BE	Cabeza de DATA relativa al programa, copiada en IX+4B/+4C; cero si no hay DATA
06	variable	Empieza el flujo principal de instrucciones compiladas

Código de operación bajo	Bytes siguientes	Función
03	8	Literal decimal empaquetado
04	2 BE	Palabra de literal o valor entero
06	1 de longitud + ese número de bytes	Literal de cadena
07	Nombre codificado hasta su primer byte con el bit alto activado	Referencia a una variable
08	ninguno	Menos unario
0F/10/11	ninguno	NOT/AND/OR
12/13/14	ninguno	Comparaciones de mayor, menor e igual
18/19/1A/1B/1C	ninguno	Potencia, multiplicación, división, suma y resta
1D	ninguno	Acceso a una matriz unidimensional después de los operandos de variable e índice
1F	ninguno	Concatenación de cadenas
30	ninguno	Marcador de la función ABS dentro de una expresión
21	ninguno	Aplicación de una función nativa con un argumento en la forma especificada
26	ninguno	Aplicación de una función de usuario con un argumento en la forma especificada

El bit 7 del código de operación de expresión marca el final de la expresión. Es distinto del bit alto que termina un nombre de variable codificado. Por ejemplo, 2+3*4 compilado es `04 00 02 04 00 03 04 00 04 19 9B`: los operandos preceden a la multiplicación y a la suma, y la suma final lleva el bit de final. Por tanto, las constantes de PRINT compiladas pueden usar 84 para entero, 83 para decimal y 86 para cadena sin introducir tres nuevos tipos de literal subyacentes.

Los nombres codificados probados incluyen A=`C1`, AB=`41 C2`, I%=`89`, S$=`F3`, FNT=`46 4E D4` y X=`D8`. Una referencia ARR concreta se reduce a `40 80`; no se ha establecido un algoritmo general completo de codificación de nombres. No lo deduzcas solo a partir de estos ejemplos.

Bifurcaciones y estado modificable de los bucles

Forma compilada	Campos después del código de operación	Significado en tiempo de ejecución
2E IF	expresión de condición; destino BE16 para falso	Cero selecciona el destino; un valor distinto de cero salta sus dos bytes.
2D GOTO	destino BE16	Transferencia al destino; también se emite para saltar ELSE.
2C GOSUB	destino BE16	Guarda la continuación y transfiere el control.
40 RETURN	ninguno	Elimina el registro de continuación de tres bytes especificado y la reanuda.
20 RETURN explícito	destino BE16	Anulación del destino establecida por el código; el comportamiento completo de anulación del destino no está especificado.
2A FOR con intervalo	Desplazamiento de NEXT BE16; nombre codificado; expresiones inicial, de límite y de paso	Asigna el valor inicial, inicializa el estado de NEXT, comprueba el intervalo y entra en el cuerpo o lo salta.
53 FOR con lista de expresiones	Desplazamiento de NEXT BE16; nombre codificado; expresiones terminadas en 0Dh	Evalúa una expresión en cada iteración del cuerpo.

Desplazamiento desde NEXT	Entero 34	Real 63	Lista 64
+01	Desplazamiento BE16 del nombre codificado en FOR	igual	igual
+03	Desplazamiento BE16 del cuerpo del bucle	igual	igual
+05	Límite BE16 con signo	límite decimal empaquetado de ocho bytes	Cursor BE16 de la siguiente expresión
+07 / +0D	Paso BE16 con signo en +07	paso de ocho bytes en +0D	ninguno
Total de bytes del registro	9	21	7

Posición	Tamaño	Significado
+00	1	36h, código de operación de ON calculado
+01	variable	Expresión selectora, terminada con el bit 7 de su código de operación de expresión
después de la expresión	1	2Dh GOTO o 2Ch GOSUB en las formas documentadas

Posición	Tamaño	Significado
siguiente	1	Número de destinos sin signo N
siguiente	2×N	Desplazamientos de destino big-endian relativos al byte cero del BASICCOM, incluida su cabecera de seis bytes
después de la tabla	—	Siguiente instrucción y continuación del GOSUB calculado

El GOSUB compilado apila una continuación de tres bytes: `FF` más un desplazamiento big-endian relativo al programa; el puntero activo a la cima de las continuaciones en IX-2 es little-endian. Las llamadas calculadas interpretadas usan en cambio una continuación de código fuente de siete bytes, cuyos cuatro bytes finales de contexto solo tienen significados concretos. El estado de NEXT es modificable en la RAM y empieza a cero en los campos correspondientes del archivo compilado.

El selector de ON empieza en uno. Un destino que falta se almacena como 0000 y la ejecución continúa; el GOSUB calculado vuelve después de la tabla completa. El selector 0 sigue la ruta de error 0x14 observada y el resultado 253. ON ERROR usa el código de operación 55 más un destino BE16; su puntero activo está en IX-0x0C/-0x0B. La gramática completa de todas las combinaciones de errores y bifurcaciones sigue incompleta.

Decimal de ocho bytes y registros de datos

Desplazamiento	Tamaño	Significado
00	1 byte	Exponente con sesgo 80h; 00 es la convención para el cero aritmético
01 bit 7	1 bit	Signo: 0 positivo, 1 negativo
01 bits 6–4	3 bits	Cero en la salida normalizada especificada; la normalización del resultado en tiempo de ejecución los borra
01 bits 3–0	1 dígito decimal	Dígito significativo inicial
02–07	6 bytes	Doce dígitos restantes, nibble alto y después nibble bajo en cada byte

Sea E=byte 0-0x80 y C el coeficiente decimal de 13 dígitos formado por el nibble bajo del byte 1 y los doce dígitos BCD siguientes. Un valor distinto de cero es `sign*C*10^(E-13)`. El byte de exponente cero es la convención para el cero aritmético. Ejemplos: 45,5=`82 04 55 00 00 00 00 00`; -0,5=`80 85 00 00 00 00 00 00`. Los bits 6–4 del byte 1 son cero en la salida normalizada especificada; no se ha establecido el comportamiento con entradas mal formadas arbitrarias.

Los campos de WRITE de BASIC usan 03 más los datos decimales, 04 más un entero BE16 con signo y 06 más la longitud de cadena de un byte y los caracteres. El byte 00 termina un registro; 0x1A indica EOF en un límite de campo o tipo. Los literales del código fuente usan en cambio 05 para el entero pequeño especificado y un token de menos unario independiente donde se ha observado. Estas diferencias evitan que un descodificador de código fuente interprete mal los registros de datos.

Las cadenas de DATA usan la cabeza situada en cabecera+4 del archivo compilado. El código de operación 54 de READ lleva nombres codificados con 07; el código de operación 3F de RESTORE aporta un destino BE16 resuelto. Los literales de DATA no usan el bit alto de final de expresión. A continuación se recogen los desplazamientos de IX en tiempo de ejecución y las estructuras escalares.

Relativo a IX	Tamaño	Función respaldada por el código
−34h/−33h	2 LE	Base del programa compilado
−02h/−01h	2 LE	Cima actual de los registros de continuación de BASIC; los registros de GOSUB compilado son FF+desplazamiento relativo BE16, mientras que los registros interpretados de 230 ocupan siete bytes
−30h/−2Fh	2 LE	Puntero actual a la pila de expresiones o temporal
−2Ch/−2Bh	2 LE	Cabeza de la lista enlazada de variables y definiciones
+1Dh	1	Byte de base de matrices 0/1, que también se compara con cabecera+1 del archivo compilado

Relativo a IX	Tamaño	Función respaldada por el código
+4Bh/+4Ch	2 LE	Cursor de DATA relativo al programa, inicializado a partir de cabecera+4

Indicadores	Datos después de los indicadores
03	Decimal empaquetado de ocho bytes
04	Entero BE16 con signo
06	Puntero BE16 al descriptor de cadena
0B	Índice superior BE16; elementos decimales consecutivos de ocho bytes

Desplazamiento	Bytes	Significado observado
+00	1	Desplazamiento del token de continuación dentro del registro de código fuente que llama; 14h en su CR final en estas llamadas
+01	2	Línea de código fuente que llama, big-endian
+03	1	Copia de IX+1Eh del contexto anterior a la llamada; aquí se desconoce su significado más amplio
+04	1	Copia de IX+1Fh
+05	1	Copia de IX+20h
+06	1	Copia de IX+3Ah

Marco de nivel de EXEC

Cada nivel usa 512 bytes en un archivo de trabajo directo. Los argumentos ocupan 0x000–0x07F como dieciséis valores de ocho bytes. Las variables ocupan 0x080–0x17F como dieciséis pares de nombre de 8 y valor de 8. Los bytes 0x180–0x1F1 siguen sin clasificar. El byte 0x1F2 guarda contexto+0x38 con el bit 0 borrado; 0x1F3–0x1F5 guardan contexto+0x2A–0x2C; 0x1F6–0x1FF guardan contexto+0x2D–0x36. Dentro de estos últimos, 0x1F6–0x1FD contienen el nombre del script y 0x1FE es una palabra opaca.

El nivel N empieza en el byte 512*(N-1), es decir, en el registro directo 4*N-3 con registros de 128 bytes. Los archivos originales EXEC1–EXEC4 contienen cada uno dieciséis registros de 128 bytes, suficientes para cuatro marcos de 512 bytes por archivo. Es una asignación de la instalación, no una profundidad máxima de anidamiento documentada universalmente. CONECTAR deriva la capacidad del doble del byte bajo del recuento de bloques; los casos mal formados o extremos siguen siendo desconocidos.

Plantillas de etiquetas de cinta

Las estructuras de 80 bytes siguientes se especifican a partir de plantillas y código originales de las utilidades y del controlador. El flujo físico completo del soporte no está especificado. Conserva los bytes sin clasificar, los CR incrustados y los nombres literales del productor, en lugar de imponer la norma de etiquetas de otro sistema.

VOL1

Desplazamiento	Tamaño	Interpretación
00	4	`VOL1` literal
04	8	Nombre copiado por INITTAPE a partir de 0137, de hasta 8 bytes, que se detiene en un CR o un espacio; el sufijo inicial que no se toca son espacios
0C	25	Espacios en la plantilla original; se desconoce cualquier otro significado
25	14	Texto del propietario copiado en INITTAPE 0358
33	28	Espacios en la plantilla original
4F	1	`1` literal en ASCII; la interpretación semántica de la versión no está especificada

Primera etiqueta de archivo

Desplazamiento	Tamaño	Interpretación
00	3	Prefijo HDR, EOV o EOF
03	1	`1` literal
04	17	Tramo de comparación del identificador de archivo
15	6	Bytes de plantilla sin clasificar; incluye un CR en algunos originales
1B	4	Primer contador de etiquetas que se incrementa; aparentemente, la sección
1F	4	Segundo contador de etiquetas que se incrementa; aparentemente, la secuencia
23	6	Bytes de plantilla sin clasificar
29	6	AAMMDD derivado de los bytes de la fecha actual
2F	7	Bytes de plantilla sin clasificar
36	6	Recuento de bloques escrito en decimal
3C	13	Aquí empieza el texto del productor; la plantilla contiene OASIS u OASIS-ARCHIVE

Desplazamiento	Tamaño	Interpretación
49	7	Espacios en las plantillas observadas

Variaciones de las plantillas

Origen / desplazamiento	Desplazamientos de los CR incrustados	Bytes del productor
DIS10+0AC5	13	OASIS
BACKUP+0FA5	13,15	OASIS
ARCHIVAR+1493	15	OASIS-ARCHIVE
RECARGAR+1197	15	OASIS

Segunda etiqueta de archivo

Desplazamiento	Tamaño	Contenido observado	Interpretación
00	4	HDR2, con el prefijo sustituido en las etiquetas finales	—
04	1	F	Indicador provisional de registro fijo
05	5	04096	tramo numérico; función exacta del campo: desconocida
0A	5	04096	tramo numérico; función exacta del campo: desconocida
0F	65	Espacios	plantilla

Las cadenas OASIS y OASIS-ARCHIVE ayudan a identificar el linaje; no demuestran la compatibilidad con las utilidades de cinta de otra versión. El orden de los datos del archivado, el protocolo de intercambio con la controladora, los reintentos y la restauración completa siguen sin resolverse. Las anchuras de etiqueta 04096 identifican tramos numéricos de la plantilla, no garantías universales de tamaño de registro documentadas de forma independiente.

Apéndice B. Mapas de memoria, E/S, interrupciones y rutinas

Las direcciones se refieren aquí a la imagen común inicializada, salvo que se especifique un banco, una superposición o un desplazamiento relativo de un controlador. Las direcciones dinámicas de los controladores deben obtenerse de los descriptores. Los números de puerto bajos identifican grupos de dispositivos descodificados; las instrucciones de E/S del Z80 generan direcciones de 16 bits y la descodificación completa de la parte alta de la dirección en el hardware sigue sin especificarse. Ten en cuenta la dirección completa que presenta la instrucción.

CPU y memoria residente

Intervalo de la CPU	Correspondencia	Notas
0000–0FFF en el reinicio	Alias de la ROM de arranque	—
0000–3FFF tras eliminar el alias	RAM común	—
4000–FFFF	Banco de 48 KiB seleccionado, nibble bajo de FD	bancos 0–4
8000–8FFF con el bit 7 de FD activado	Superposición de lectura de la ROM de arranque	—

Intervalo	Descripción nativa
0000–39C1	NUCLEO, 14 786 bytes en ESTADO MEMORIA
39C2–4C8A	SPOOLER, 4809 bytes en ESTADO MEMORIA
0000–4C8A	Región de NUCLEO en ESTADO MODULO, 19 595 bytes
4C8B–FFFF	Módulos 00–03, de 45 941 bytes cada uno
50C3–E3EB	Módulo 04, 37 673 bytes

Dirección	Observación
0008	JP al despachador de servicios RST 08BD
Palabra 0033	Puntero al contexto de la partición actual, 4C8B en la sesión principal
Byte 046B	Registro de bancos de la CPU que usa el asignador de correspondencias 21DE
Palabra 048A	Puntero al registro actual del planificador
048C–04BB	16 registros del planificador de tres bytes; primer byte de indicadores en 048E; PONER impone los números públicos 1–16
0551/0553	Puntero/índice del origen normal del recorrido, distinto de la ejecución actual durante las devoluciones de llamada
0554–0557	Cuanto restante, indicación de devolución de llamada, cuanto guardado e indicador temporal del origen del recorrido
Desde 0513	Candidatos de límites de módulo de cuatro bytes
0713,0717,071B	Descriptores de almacenamiento de las superposiciones del núcleo
08E0–09BD	Tabla de destinos de llamadas al sistema de 111 entradas
424A	Ventana de ejecución compartida para las superposiciones del núcleo
5157	CSI cargado en la partición principal tras la inicialización

Dirección	Observación
FC67	Dirección de carga del controlador PAN1 observada anteriormente; dinámica, no una constante universal

Variables globales compartidas

Dirección	Significado
0x0003	Día de la semana
0x0006	Bloqueo de byte fijo que usan 59/5A
0x0007	Número de reintentos instalado: 4
0x000B	Byte de cuanto predeterminado
0x000C / 0x000D	Índice del registro actual / número configurado
0x000E	Indicadores globales de presentación y configuración
0x0013–0x0015	Mes/día/año en BCD
0x0016	Puntero a la cabeza de la lista de temporizadores
0x001B–0x001D	Hora/minuto/segundo en BCD
0x003B–0x003D	Bytes del número de serie de la instalación
0x0044	Puntero al SPOOLER activo
0x0046	Calibración del retardo activo
0x004A / 0x004C	Punteros a las devoluciones de llamada de lectura de hora y fecha, instalados 2E4C/2E6E
0x0062 / 0x0064	Punteros a las devoluciones de llamada de escritura de hora y fecha, instalados 2E88/2E98
0x0069	Matriz de descriptores físicos, paso de 32 bytes
0x02A6 / 0x02A7	Mapas de bits de disponibilidad del DMA de ALP
0x0485	Propietario de la cabeza de los temporizadores
0x0551 / 0x0553	Puntero/índice del origen normal del planificador
0x0554–0x0557	Cuanto residual, indicación de devolución de llamada, cuanto guardado e indicador temporal del origen
0x2078	Puntero de pila guardado para la copia entre bancos
0x208A	Cima de la pila común de copia; también es la entrada ejecutable de adquisición de bloqueos en la correspondencia y el uso normales

Dirección	Significado
0x21DC / 0x21DD	Byte de transferencia de mensajes / bloqueo de propietario
0x23FF+3*i	Fórmula de las filas del registro de devoluciones de llamada

Mapa de dispositivos físicos y puertos

Puerto bajo	R/W	Hardware / significado de los bits	Notas
00–0F	R/W	Primer 8237 de ALP: dirección/recuento 00–07, estado/comando 08, petición 09, máscara 0A, modo 0B, puntero 0C, reinicio 0D	—
10–1F	R/W	Segundo 8237 de ALP, con los mismos desplazamientos de registros	—
20–23	W	Báscula de banco DMA para cada par de canales de terminal; el nibble bajo selecciona el banco	función; resto de los bits: desconocidos
24–27	W	Control de flujo de recepción por terminal, bit 0=preparado	Cableado probable de RTS
28–2B	W	Borrado de otras fuentes de interrupción del nibble alto de ALP	Desconocido, sin correspondencia
2C	R	Resumen de ALP: bit 0 primer SCC, bit 1 segundo SCC, bit 2 báscula TC de DMA0, bit 3 báscula TC de DMA1	nibble bajo
2E/2F	W	Borrado de la báscula de fin de cuenta de DMA0/DMA1	—
30/31	R/W	SCC0 B control/datos, PAN2	—
32/33	R/W	SCC0 A control/datos, PAN1	—
34/35	R/W	SCC1 B control/datos, PAN4	—
36/37	R/W	SCC1 A control/datos, PAN3	—
60	R/W	Código de origen SS2: E0 serie, C0 temporizador; escribir A0 reconoce el temporizador	Rutas de despacho originales; prioridad física y cableado completo de la máscara sin especificar
61	R	Lectura de la configuración: bit 0 idioma, bit 1 omitir la prueba ASI, bit 2 omitir la prueba del disco duro, bit 3 activar los diagnósticos	significados del firmware; lógica física de los DIP/desplazamiento: desconocida
61	W	Control de la báscula de configuración/desplazamiento	Desconocido, sin correspondencia
74–77	R/W	Contadores 0/1/2/control del 8254	registros; reloj de 1 MHz: probable
78–7B	R/W	Interfaz serie IMP: 78 datos, 79 estado, 7A modos alternos, 7B comando	Disposición del SCN2651; cableado completo de reloj y módem sin especificar
7C–7F	R/W	SCC de SS2: B control/datos 7C/7D para PAN10; A control/datos 7E/7F para PAN9	DIS16/17; despacho por origen RR3 a través de 7E

Puerto bajo	R/W	Hardware / significado de los bits	Notas
80–9F	R/W	Contadores, estado y control del RTC MM58167	Probable; ruta de interrupción con comportamiento físico completo sin especificar
B8	R/W	FIFO de datos del WD1001	256 bytes por sector
B9	R/W	Error/precompensación del WD	compatible con el dispositivo
BA	R/W	Número de sectores del WD	—
BB	R/W	Identificador de sector del WD, 0..31	—
BC/BD	R/W	Cilindro bajo/alto del WD	—
BE	R/W	SDH del WD: bits 6:5 tamaño de sector, 4:3 unidad, 2:0 cabeza	Tamaño 00=256 bytes, 01=512, 11=128; 10 sin especificar
BF	R/W	Estado/comando del WD: ocupado 80, DRQ 08, error 01; preparado 50 tras la transferencia; restaurar 13/16, leer 20, escribir 30, formatear 50, buscar 70	Tras la finalización hay que examinar el estado y el error
C0/C1	R/W	Estado principal/datos del uPD765A para DIS2	Los bytes de comando y de resultado usan C1
C2	R/W	Adaptador de transferencia de DIS2: escribe el banco, la parte alta y la parte baja de la dirección; se sondea el bit 7 de lectura para detectar la finalización	Secuencia de software establecida; cableado físico completo y acarreo en los límites sin especificar
FD	W	Nibble bajo: banco; bit 7: superposición de la ROM en 8000; la primera escritura libera el alias de reinicio en 0000	comportamiento observado; bits sin usar: desconocidos
FE	W	Máscara de interrupciones activa a nivel alto	Probable, ejecución validada

RST 0x00 cambia durante el arranque: el núcleo recién reubicado salta a través de él a 0x5157 antes de la inicialización; en el estado de inicio de sesión observado, su destino es 0x34F7. Son fases sucesivas del arranque, no direcciones de entrada activas intercambiables. Véase el capítulo 21.

Para FORMAT 0x50 del WD, el puerto 0xB8 recibe un búfer de tabla del tamaño de un sector que contiene pares de indicador e identificador de sector. Con 32 sectores y un tamaño de 256 bytes, 64 bytes describen las entradas y el sistema sigue aportando el búfer completo de 256 bytes. El recuento está en 0xBA. La señalización normal de finalización puede acompañar a un error; examina 0xBF y el registro de error en 0xB9. La política de Ignorar del sistema operativo original puede devolver después éxito sin datos válidos, como se explica en el capítulo 27.

Mapa de interrupciones y DMA

Ruta	Código de operación/vector suministrado	Uso establecido
SS2 nivel 2	D7 / RST10 /0x0010	Origen serie E0 y origen de temporizador C0 seleccionados mediante el puerto 60
WD nivel 4	E7 / RST20 /0x0020	Disco
ALP nivel 5	EF / RST28 /0x0028	Recepción del SCC y finalización del DMA
Relacionado con la cinta	RST18 /0x0018	Ruta del registro 03; entrega física sin especificar
Gancho del contexto	RST38 /0x0038	Gancho en línea en contexto+0x39

En el modelo de interrupciones especificado, el puerto 0xFE aporta una máscara activa a nivel alto. El arbitraje por el nivel activo más alto sigue siendo provisional. El temporizador usa el contador 1 del 8254=8000, modo 2; una fuente de 1 MHz da un pulso de 8 ms. Para los valores de temporización de este apartado se usa una configuración de CPU de 4 MHz; el encaminamiento físico completo del reloj no está especificado.

Los canales 0/1 de DMA0 atienden la TX/RX de PAN1 y los canales 2/3, la de PAN2; los canales 0/1 de DMA1 atienden PAN3 y los 2/3, PAN4. La ruta de recepción instalada usa interrupciones del SCC y lecturas de la CPU, no DMA de RX. La TX usa DTR/REQ; las básculas de banco de cada par de canales seleccionan la memoria del DMA independientemente del puerto FD. La fuente serie de 2,4576 MHz admite la configuración de baudios observada; el encaminamiento de la fuente de SS2 sigue siendo una deducción.

Catálogo de controladores externos

Archivo original	Bytes cargados	Nombres instalados coincidentes	Desplazamientos de destino de JP en 00/03/06/09/0C
SISTEMA.DIS1	605	FLOPPY1	000D / 0045 / 004A / 005B / no es JP
SISTEMA.DIS2	575	FLOPPY2	000D / 0044 / 0049 / 005A / no es JP
SISTEMA.DIS10	2980	TAP	0029 / 0029 / 0029 / 002B / 005B
SISTEMA.DIS11	237	IP	000F / 002A / 002B / 004D / 0061
SISTEMA.DIS12	438	CDC	000F / 00B6 / 00B7 / 0151 / 018B
SISTEMA.DIS16	650	PAN9, IS9	000F / 004C / 005D / 0093 / 0105
SISTEMA.DIS17	650	PAN10, IS10	000F / 004C / 005D / 0093 / 0105
SISTEMA.DIS18	887	PAN1	000F / 0029 / 003D / 006B / 0118
SISTEMA.DIS19	887	PAN2, IS2	000F / 0029 / 003D / 006B / 0118
SISTEMA.DIS20	887	PAN3, IS3	000F / 0029 / 003D / 006B / 0118
SISTEMA.DIS21	887	PAN4, IS4	000F / 0029 / 003D / 006B / 0118
SISTEMA.DIS22	314	IMP	000F / 0070 / 0071 / 008D / 00F9
SISTEMA.DIS23	887	PAN5, IS5	000F / 0029 / 003D / 006B / 0118
SISTEMA.DIS24	887	PAN6, IS6	000F / 0029 / 003D / 006B / 0118
SISTEMA.DIS25	887	PAN7, IS7	000F / 0029 / 003D / 006B / 0118
SISTEMA.DIS26	887	PAN8, IS8	000F / 0029 / 003D / 006B / 0118

Parámetros de la plantilla ALP

Todas las entradas siguientes están en hexadecimal, salvo el número de archivo DIS.

Número DIS	Puertos de estado/ datos	Byte / bit común fijo de DMA	Índice de recepción de 67	Índice de propietario de DMA de 67
18	32/33	02A6 / 0	11	15
19	30/31	02A6 / 2	12	16
20	36/37	02A6 / 4	13	17
21	34/35	02A6 / 6	14	18
23	3A/3B	02A7 / 0	19	1D
24	38/39	02A7 / 2	1A	1E
25	3E/3F	02A7 / 4	1B	1F
26	3C/3D	02A7 / 6	1C	20

Índice de rutinas internas

Estos nombres son anotaciones descriptivas. No son símbolos exportados originales ni una ABI pública estable de CALL. Usa el servicio de RST08 cuando exista; las direcciones internas dependen de esta imagen y de esta superposición exactas.

Rutas de devolución de llamada y de control

Intervalo de direcciones	Función
2EA4–3061	Programar/sustituir/cancelar una devolución de llamada temporizada
3062–30EE	Vencimiento del temporizador y actualización del cuanto
3564–35C1	Búsqueda en tablas de palabras clave abreviables
35D8–3618	Filtrado de escapes en la entrada de dispositivos
3619–3864	Despacho de controles del terminal

Otros puntos de entrada de devoluciones de llamada

Intervalo de direcciones	Función
3435–346F	Guardar/desactivar la devolución de llamada de registros por partición
3470–3497	Restaurar la devolución de llamada y entregar la interrupción pendiente
3671–3679,36C0–36C4	Marcar como pendiente la interrupción Q cuando la devolución de llamada es cero
superposición 3:448C–4545	Limpieza genérica de comandos, reinicio del estado de los canales y liberación de bloqueos

Capas de la consola

Intervalo de direcciones	Función
1461–149A	Envoltorio de política de la consola 05
149B–161A	Estado de flujo, cursor y control de la consola 07
161B–1634	Salida de clase +3 y alternativa en bruto
1728–1757,1861–1873	Ruta 40 en bruto al dispositivo con la asociación 9
35D8–3618	Entrada 66 y devolución de llamada de clase +0
37CF–37DF	Esc seguida de C devuelve 18 y marca la cancelación

Rutas de flujo del terminal

Intervalo de direcciones	Función
1392–13A6	Eco opcional de imprimibles de 04 mediante 05
1476–147A	Comprobación del bit de supresión de salida y 07 condicional
34C2–34D8	Eliminar la inhibición de entrada y vaciar la asociación 8
36DC–3701	Bucle de pausa/reanudación S por partición con cesión del control
37BC–37CE	Conmutar el bit 1 de 0A del contexto para O
2CC9–2CF1,30E5–30EE	Habilitación/deshabilitación del temporizador y selección del número de particiones

EXTEND secuencial

Dirección enlazada en la superposición 2	Función
4303–435D	Selección del modo de OPEN y tratamiento del bit 2
4398–439D	EXTEND con un archivo inexistente selecciona la creación
440B–4463	Lectura del extremo S existente y restauración de los metadatos del canal
4464–447C	Recorrido hacia atrás de los 1A y puntero de adición/recuento libre
4750–476A	Escritura del sector actual en CLOSE y puesta a cero del resto del último bloque

Rutas de errores de disco

Dirección común	Función
09BF–0A58	Modo de un sector, bloqueo del disco, petición guardada y validación de la asociación
0BE0–0C30	Resultado del controlador, aritmética del recuento restante y retorno a la aplicación
0C31–0C5A	Liberar el propietario del disco y el estado de espera
0C5B–0C8D	Diagnóstico de asociación no válida y anulación
0C8E–0CC5	Devolución de llamada opcional del contexto 26 y contador automático de intentos
0CC6–0D9D	Texto del error de disco, coordenadas lógicas y política R/M/I/Q
0D9E–0DC8	Llamada a controlador+3, anulación con 4E y búsqueda en el contexto 26
0F99–0FCA /1079–10A1	Transferencia de lectura del WD antes de comprobar el estado, traducción de errores y reversión del puntero
3199–31A5	Registro de la devolución de llamada de error de disco del contexto 26 mediante 4A

Traducción de nombres de archivo y de la entrada

Dirección común	Función
1428–144D	El bit 6 de contexto+0A controla la conversión de minúsculas y guion bajo→Supr
249E–24C6	Recorrer los 16 bytes de nombre y tipo de quien llama: conservar espacios, $, dígitos y A–Z; convertir a–z; lo demás→$
24C8–24E1	Ajustar el primer byte de cada campo; por debajo de A, salvo $, suma 11h
Desde 24E2	Dispersión de los campos normalizados y búsqueda en el directorio
27CA–2810	Copiar/normalizar/crear la entrada de directorio y devolver éxito
2811–281E	Una creación fallida devuelve FF/acarreo

Mapa de reintentos de directorio y de comprobación de etiquetas

Dirección común	Función
2463–2480	Iteración por unidades con FF; restaura los punteros entre fallos, pero conserva los campos transformados
248B–24E1	Normalización repetida de los campos de quien llama
2510–2513	Llamar a la comprobación de etiqueta cuando la cuenta actual es igual a la cuenta de búsqueda
25C9–25F2	Recordar el primer hueco FF y volver a leer su sector en caso de fallo
2600–261D	Alternativa pública, salvo que el bit 6 de contexto+0C la inhiba
33D0–33ED	Leer la cabecera 1, comparar los ocho bytes de la etiqueta montada y restaurar BC/DE/HL

Mecanismos de sincronización independientes

Dirección / región	Función
1C9C–1D1B	Adquirir una entrada compartida o exclusiva de archivo abierto; el acarreo indica conflicto
1D1C–1D79	Eliminar una entrada de archivo abierto coincidente del propietario actual
1D7A–1DA7	Eliminar las entradas abiertas del propietario actual durante la limpieza de 00
1DA8–1FAB	Tabla de archivos abiertos de cuatro bytes por fila, 516 bytes
208A–20AB	Adquisición de byte 54, espera mediante el bit 4 del planificador
20AC–20C8	Liberación de byte 55, borrado del bit 4 por difusión
262B–2658	El envoltorio de borrado intenta la adquisición exclusiva, pero ignora el acarreo

Dirección / región	Función
3498–34BF	Bucle de admisión de OPEN; cede el control en 34AC si la función auxiliar devuelve acarreo
Superposición 2:481C	Llamada de CLOSE a 1D1C
Superposición 3:44A7–44C8	Lista fija de limpieza de bloqueos de byte

Rutas de las colas de bytes

Dirección	Función
3320–3329 /3352–3355	Comprobación de vacío y retorno con acarreo activado de 47
332B–334F	Lectura mediante desplazamiento de ocho bits, decremento, actualización condicional del índice y EI
335E–3364	Guardar en caché la ocupación y luego comparar si está llena sin volver a leerla
3366–3375	Suma de ocupación e índice de ocho bits, módulo y desplazamiento de los datos
337A–3385	Restaurar la A de quien llama, escribir, incrementar cola+01, EI y restaurar los registros

Eventos de dispositivo y devoluciones de llamada

Intervalo de direcciones	Función
1728–1898	Despacho de dispositivos de caracteres y retorno si falta la asociación
21E4–220D	Registro indexado de propietario/dirección 67
22B5–2334	Encaminamiento de las interrupciones de recepción y DMA de ALP
2335–2351,23F1–23FC	Selección del SCC y búsqueda de una devolución de llamada distinta de cero
2379–2385	Obtención de propietario/dirección y salto directo a 063F
063F–06E9	Entrar en una devolución de llamada en el contexto seleccionado
2352–2368,23DA–23F0	Combinación del estado de DMA preparado y despertar del propietario con el bit 5
06EA–0702	6B: EI/HALT con una sola partición o espera en el planificador con el bit 5

Transferencias de memoria entre particiones

Intervalo de direcciones	Función
1FAC–1FEF	68: del banco del registro seleccionado al banco de quien llama
1FF0–2033	69: del banco de quien llama al banco del registro seleccionado
2034–2055	Funciones auxiliares de entrada relacionadas que omiten la selección pública de registros

Intervalo de direcciones	Función
2056–2077	Comparación de mismo banco, selección de pila compartida o LDIR
2078–2089	Palabra de SP guardado y aparente área de pila común de 16 bytes
21DE–21E3	OUT(FD),A y actualización de 046B
inicialización 5C3D–5D30	Búsqueda del banco del registro temporal y comprobaciones de complemento y relectura

Rutinas de mensajería

Dirección	Finalidad interpretada
2141–21BD	Comprobaciones de destino del servicio 58, conversión del texto a siete bits, envío serializado y liberación
21BE–21DB	Devolución de llamada del destinatario que conserva los registros principales guardados
21DC/21DD	Byte de transferencia marcado / bloqueo de propietario
063F–06E9	Inyección en el contexto de destino; entrada automodificada y restauración del marco guardado

Añadidos sobre búsqueda de residentes y propiedad de temporizadores

Dirección	Función / contrato observado
04BC–050B	Ocho filas de componentes con nombre inicializadas; no se impone ningún límite numérico a la búsqueda
20DB–2107	6C: búsqueda exacta de ocho bytes; éxito AF0044/HL=valor, fallo AF0101/HL original
3559–3563	5D: comparación de ocho bytes que usa 6C
2EBE–2EF1	35: el bit 0 de la A de entrada omite la comprobación de propietario solo para una cabeza numéricamente coincidente; accede al nodo en la correspondencia actual
2F2B–2F3E	La eliminación de un temporizador intermedio sigue requiriendo el propietario actual

Paquete de disquete y traducción de los retornos

Las entradas privadas son A=recuento, B=cabeza, C=sector lógico dentro de la pista, DE=pista, HL=búfer e IY=descriptor. El adaptador de transferencia recibe el byte de banco común 0x046B y la dirección del búfer. DIS2 programa el puerto 0xC2 con el banco, el byte alto de la dirección y el byte bajo de la dirección. La ruta correcta del controlador hace avanzar C y H según el recuento sin ninguna propagación general del acarreo establecida; el acarreo del adaptador físico al cruzar 64 KiB sigue sin especificarse.

Desplazamiento en el paquete	Significado / construcción
00	Lectura 06 o escritura 05; OR 40 cuando N=1
01	Unidad guardada con el bit 2 borrado, OR B rotado dos veces a la izquierda
02	Pista E
03	Cabeza B
04	Primer sector físico: C+1 si N=1; 2*C+1 si N=0
05	Código de tamaño N: 1 para sectores de 256 bytes; 0 para sectores de 128 bytes
06	Último sector físico: primero+recuento−1 si N=1; primero+2*recuento−1 si N=0
07	Valor de separación 0E si N=1; 07 si N=0
08	Valor de longitud de datos 00 si N=1; 80 si N=0

Condición	A devuelta	Mensaje exacto del núcleo común
Bit 3 de ST0	01	`no prep.`
Bit 1 de ST1	02	`prot. escritura`
Bit 0 de ST1	03	`no inicializado`
Bit 5 de ST2	04	`error transferencia`
Bit 4 de ST2	08	`no existe pista`
Bit 1 o bit 0 de ST2	03	`no inicializado`
Bit 4 de ST1, **solo DIS1**	09	`error cabeza`
Bit 5 de ST1	09	`error cabeza`
Bit 2 de ST1	07	`no existe sector`
Ninguna condición especificada	00	Ruta de éxito

Operaciones privadas de la cinta

A selecciona una de las operaciones siguientes mediante controlador+0x0F. Los destinos y los comandos de la controladora están en hexadecimal y son relativos al controlador original. No son servicios de RST08. Los significados muy probables y los provisionales se mantienen separados de la ejecución física documentada.

A	Destino	Comando(s) de la controladora	Significado y notas
80	0165	06 y después selector 0–3	inicialización del selector/estado; E determina el selector. Interpretación física del selector: desconocida
81	01EA	03	Rebobinado probable, según las secuencias de las utilidades y la ayuda de TENSION
82	0229	0C	el código lee DE bytes en HL usando A1, con una ruta asíncrona especial; transferencia completa al soporte sin especificar
83	032D	0D; reintento 0E	el código escribe DE bytes desde HL mediante A1; transferencia completa al soporte sin especificar
84	0410	09	Espaciado hacia atrás probable, según su uso en los reintentos de lectura y escritura; unidad física exacta sin especificar
85	0462	08	Espaciado hacia delante provisional; puede devolver A05 en la condición de final compuesta
86	04A1	12	Acción de borrado probable en la secuencia de reintento; extensión: desconocida
87	0514	0F	Escritura probable de una marca de cinta, según las llamadas de etiquetas y de finales
88	05D9	06 y después 80	Retensado probable; INITTAPE TENSION lo invoca y espera A05
89	05F8	02 consulta de estado	estado compuesto y selector actual; determinadas combinaciones de estado especificadas
8A	04EC	07	Acción de reintento relacionada con el borrado; significado exacto: desconocido
8B	0637	0A	Búsqueda probable hacia delante de una marca de cinta a través de los selectores; el sospechoso retorno del último selector se describe más abajo
8C	066A	04 más secuencia de reinicio	secuencia de reinicio en el código; efecto físico sin especificar

Puertos de la cinta y estado compuesto

Byte bajo del puerto	Uso original	Notas
A0	escritura de comandos, escritura del selector, lecturas de estado	código y ruta de software especificada
A1	bytes de registro con INIR/OTIR	código; sin transferencia de datos nativa
A2	escrituras de control y lecturas de estado/protocolo de los bits 7/6	—
A3	escrituras de control FF/F0/5F/40/50/DF	código; significados de chips y patillas: desconocidos

Origen	Bit de salida
segundo estado 0B97, bit 3	bit 5 de A/C
segundo estado, bit 4	bit 3 de A/C
segundo estado, bits 6, 1 y 0 todos activados	bit 4 de A/C
bit 6 del control A2 borrado	bit 2 de A/C
constante	bits 7, 1 y 0 de A/C activados

Apéndice C. Archivos del sistema operativo y componentes de los lenguajes

Los archivos del sistema aportan el ejecutivo, el entorno de comandos, los lenguajes de programación y la compatibilidad con los dispositivos. Otros archivos contienen las definiciones de cuentas, los mensajes y el estado de trabajo temporal. Este apéndice agrupa los componentes relacionados para que una familia numerada pueda entenderse en conjunto. Los archivos COMANDO de las utilidades se describen a través de sus comandos en el capítulo 31.

Los tamaños y las fechas siguientes identifican versiones concretas de los componentes; los programas de sustitución y los archivos de trabajo que crecen pueden tener valores distintos. Los componentes opcionales no tienen por qué estar presentes en todos los sistemas.

Lectura de los detalles de los archivos

Cada tabla siguiente identifica un archivo en la columna Componente, seguido de su formato de almacenamiento, la información de tamaño y la marca de tiempo del catálogo. Lee la columna Formato antes de interpretar Contenido (bytes): el significado del tamaño depende de la organización del archivo.

Columna Formato

Los códigos siguientes se usan en la columna Formato de las tablas de componentes de este apéndice. Describen cómo se almacena o se carga un archivo, con independencia de su nombre o del sufijo de tipo.

Código de formato	Significado	Qué mide Contenido (bytes)
A	Programa absoluto en código máquina, cargado en una dirección especificada	Bytes almacenados en sectores completos de 256 bytes; el total puede incluir relleno.
D	Archivo directo con registros de longitud fija	Capacidad de registros: número de registros multiplicado por los bytes por registro.
R	Programa reubicable en código máquina, ajustado a su dirección de carga	Longitud de la imagen de carga, incluidos el código y los datos inicializados, pero sin la información de reubicación ni el relleno de sectores.
S	Archivo secuencial, que se lee como texto o como registros binarios serializados	Las tablas de texto cuentan el contenido incluidos los CR, sin el relleno final. Las tablas con la etiqueta «datos binarios almacenados» cuentan todos los datos enlazados, incluido el relleno.
I	Archivo indexado con registros con clave y un índice ordenado	Campos de datos de los registros activos; excluye las claves, las entradas de índice y los huecos de registro sin usar.

Por ejemplo, una R junto a SISTEMA.CSI significa que contiene código máquina reubicable. Una D junto a SISTEMA.EXEC1 significa que proporciona almacenamiento de trabajo de registros directos, no otra copia del intérprete EXEC. Estas letras describen formatos de archivo; la S de esta columna no tiene relación con la asociación del volumen del sistema `:S` de un nombre de archivo.

Contenido, espacio asignado y fechas

Contenido (bytes) indica el tamaño definido por la tabla de formatos anterior. En los archivos de texto de formato S, el recuento incluye todos los registros completos terminados en CR anteriores al relleno final 0x1A. Se excluyen los enlaces entre sectores, el relleno y la asignación sin usar. Esta interpretación se aplica a estos archivos de texto; no es una instrucción general para truncar cualquier archivo binario de formato S en su primer byte 0x1A.

Asignado (bytes) indica el espacio en disco reservado para el archivo en bloques de 1024 bytes. Por ejemplo, SISTEMA.CSI tiene una imagen de carga de 3104 bytes con código y datos inicializados, pero ocupa 4096 bytes de espacio asignado, que también alojan la información de reubicación y el relleno. SISTEMA.AYUDA contiene 37 676 bytes de texto y delimitadores de registro dentro de una asignación de 38 912 bytes. En los archivos D, Contenido es la capacidad de registros, no el número de registros ocupados. En las tablas adicionales de formato S binario, el apartado identifica explícitamente el recuento como datos binarios almacenados: cada sector enlazado aporta 254 bytes de datos después de su enlace de dos bytes. El relleno se incluye porque no se ha especificado el final exacto del programa o de los datos. No son longitudes de ejecutables descodificadas.

La protección contra lectura es un atributo independiente del formato nativo. Los programas compilados de apoyo de BASIC usan almacenamiento S con protección contra lectura; una presentación del catálogo puede mostrar SR. No son programas Z80 reubicables de formato R. El archivo indexado MAESTRO.VOC usa otra medida distinta: sus seis campos de datos activos de 512 bytes suman 3072 bytes, sin contar las claves ni la estructura del índice.

La fecha y hora del catálogo indican la marca de tiempo almacenada en la entrada de catálogo del archivo. Son metadatos del archivo, no necesariamente una fecha de compilación ni de publicación del software. Los archivos de trabajo pueden cambiar a medida que se usa el sistema, y sus fechas y tamaños de asignación también.

Dónde se almacenan los tamaños de los programas y de los registros

La información de carga de los programas está en la entrada de catálogo de 32 bytes, no en una cabecera universal al principio de cada programa. En los archivos R, el desplazamiento +0x1E de la entrada contiene la longitud de la imagen; +0x11, el número de sectores almacenados; y +0x15, el sector inicial. Cuando hay registros de reubicación, siguen a la imagen redondeada hasta un límite de 256 bytes. Algunos módulos R pequeños no necesitan registros de reubicación. En los archivos A, +0x1E contiene en cambio la dirección de carga absoluta. En los archivos D, +0x11 es el número de registros y +0x17 la longitud del registro. Estos campos deben interpretarse según el formato. El apéndice A explica la carga y los formatos de los lenguajes; el apéndice E recoge las disposiciones de bytes.

Ejecutivo y entorno de comandos

SISTEMA.NUCLEO contiene el ejecutivo, el material de inicialización y las superposiciones del núcleo. El cargador de arranque lo carga antes del entorno de comandos. Su dirección de carga absoluta es 0x0000; el componente almacenado ocupa 113 sectores, es decir, 28 928 bytes. GENSIST puede guardar la configuración en este componente, así que no es un archivo de configuración de texto para editarlo a mano.

SISTEMA.STARTUP es código de inicialización ejecutable que prepara el estado del sistema, incluido el almacenamiento de las superposiciones. SISTEMA.CSI es el procesador de comandos que lee los comandos y selecciona los programas. Cuando termina un programa, el sistema puede volver a cargar CSI en lugar de volver a una pila permanente del procesador de comandos. SISTEMA.EXECLANG interpreta los procedimientos EXEC, incluidas las variables, las bifurcaciones y la ejecución anidada. Lo usa el EXEC HOLA del capítulo 8; no se necesita ningún archivo EXEC.COMANDO.

SISTEMA.STARTUP empieza guardando AF, DE, BC, HL e IY: sus seis primeros bytes son `F5 D5 C5 E5 FD E5`. Su imagen de carga de 548 bytes va seguida del relleno del sector y de un sector de reubicación con 41 desplazamientos. La longitud almacenada completa es de 1024 bytes. No hay ninguna cabecera inicial de salto y versión.

SISTEMA.IPL es un componente de formato absoluto con la dirección de carga 0xE000. Su código busca SISTEMA.NUCLEO, accede a las familias de puertos de la controladora de disquetes y copia 0x7000 bytes en la RAM baja antes de transferir el control a través del vector de reinicio. Esto indica una función de carga del núcleo, pero sus requisitos de lanzamiento y su participación en la ruta de arranque desde disco duro siguen sin especificarse. Su longitud de copia es distinta de los 0x7100 bytes almacenados del componente NUCLEO que se lista aquí. No es un procedimiento de reinicio documentado para el operador.

Componente	Formato	Contenido (bytes)	Asignado (bytes)	Fecha y hora del catálogo
SISTEMA.NUCLEO	A	28 928	29 696	15/04/1986 10:48
SISTEMA.STARTUP	R	548	1024	04/02/1985 16:07
SISTEMA.CSI	R	3104	4096	11/12/1981 07:54
SISTEMA.EXECLANG	R	5146	7168	11/12/1981 07:55
SISTEMA.IPL	A	1792	2048	09/01/1984 16:59

Definiciones de cuentas, ayuda y dispositivos

SISTEMA.CUENTA es la base de datos secuencial de cuentas. Contiene los nombres, los identificadores, las contraseñas cuando se han asignado y los privilegios; su byte de propietario especial es 0xFF. Usa los procedimientos de CUENTA para listar las cuentas, crear o eliminar una cuenta y cambiar una contraseña. Un listado puede mostrar las contraseñas almacenadas y debe mantenerse en privado.

CONECTAR.COMANDO realiza la conexión de la cuenta y el procesamiento del perfil. Durante la conexión, lee y muestra SISTEMA.CONECTAR como texto secuencial de bienvenida. El mensaje de bienvenida son datos de presentación, no código ejecutable de conexión; su texto puede ser específico de cada instalación.

SISTEMA.AYUDA contiene los temas de ayuda de los comandos. AYUDA lee esos temas, con la limitación de la vía alternativa descrita en la referencia de comandos. SISTEMA.MENSERR contiene 200 registros fijos de 64 bytes, lo que da una capacidad de mensajes de 12 800 bytes. Los números de registro de los mensajes son distintos de los códigos de error de BASIC y de los resultados de finalización de los comandos; el catálogo de mensajes almacenados permite consultarlos.

Las definiciones de cuentas de SISTEMA.CUENTA y las de dispositivos de SISTEMA.NOMBDISP son configuración del sistema, cuyo contenido depende de la instalación. SISTEMA.AYUDA y SISTEMA.MENSERR aportan en cambio textos de referencia del sistema. Que los usen de forma compartida las funciones del sistema no convierte los nombres de cuenta, el texto de bienvenida ni las elecciones de dispositivos en valores predeterminados universales.

SISTEMA.NOMBDISP contiene definiciones secuenciales de nombres de dispositivo. Aportan el vocabulario de nombres físicos que se usa en las asociaciones de dispositivos; una definición no implica que el hardware correspondiente esté instalado. El capítulo 12 explica las asociaciones activas.

Componente	Formato	Contenido (bytes)	Asignado (bytes)	Fecha y hora del catálogo
SISTEMA.CUENTA	S	40	1024	01/01/1990 00:37
SISTEMA.CONECTAR	S	436	1024	07/01/1984 20:14
CONECTAR.COMANDO	R	2141	3072	24/09/1982 00:15
SISTEMA.AYUDA	S	37 676	38 912	19/06/1985 14:09
SISTEMA.MENSERR	D	12 800	13 312	07/12/1983 16:17
SISTEMA.NOMBDISP	S	783	1024	01/01/1984 04:29

Archivos de trabajo de EXEC: SISTEMA.EXEC1–SISTEMA.EXEC4

Estos cuatro archivos tienen la misma finalidad para distintas particiones de ejecución. El sufijo numérico selecciona el número de partición mostrado: EXEC1 pertenece a la partición 1, EXEC2 a la partición 2, y así sucesivamente. Contienen el estado de anidamiento de EXEC y los gestiona el intérprete. No son cuatro versiones de EXEC ni contienen procedimientos de comandos escritos por el usuario. Los marcos guardados pueden quedar después de su uso; que haya bytes de trabajo distintos de cero no establece por sí solo que un procedimiento siga en ejecución, y el almacenamiento de trabajo relleno de ceros puede seguir siendo necesario.

Cada archivo de la lista contiene dieciséis registros directos de 128 bytes, lo que da una capacidad de 2048 bytes. EXEC usa un marco de 512 bytes por nivel de anidamiento, así que esta asignación contiene cuatro marcos. El tamaño de registro y el tamaño de marco son unidades distintas. Los cuatro marcos describen esta asignación, no una profundidad máxima universal de anidamiento. La falta de capacidad de trabajo, o que no sea utilizable, puede impedir los perfiles automáticos de las cuentas. La disposición del marco se describe en el apéndice A.

Componente	Formato	Contenido (bytes)	Asignado (bytes)	Fecha y hora del catálogo
SISTEMA.EXEC1	D	2048	2048	14/10/1983 08:21
SISTEMA.EXEC2	D	2048	2048	01/01/1987 00:04
SISTEMA.EXEC3	D	2048	2048	01/01/1987 00:04
SISTEMA.EXEC4	D	2048	2048	01/01/1987 00:05

Programas de entrada y entorno de ejecución de BASIC

BASIC.COMANDO selecciona la edición y la interpretación interactivas cuando se invoca sin nombre de archivo de código fuente, o la compilación cuando se le da un archivo de código fuente BASICOBJ guardado. RUN.COMANDO inicia los programas BASICCOM compilados a través del entorno de ejecución de BASIC. BASIC.LOADFILE aporta el entorno de ejecución de los programas compilados, incluidos el tratamiento de la cabecera compilada, la evaluación de instrucciones y expresiones y la carga de extensiones BASICUSR con nombre. Es un componente de la implementación, no un programa de usuario guardado.

Sigue el capítulo 15 para ejecutar el mismo programa de forma interactiva y después de compilarlo. Conserva el código fuente BASICOBJ para editarlo: BASICCOM contiene instrucciones preparadas para el entorno de ejecución, no código Z80 autónomo.

Componente	Formato	Contenido (bytes)	Asignado (bytes)	Fecha y hora del catálogo
BASIC.COMANDO	R	854	2048	11/12/1981 08:00
BASIC.LOADFILE	R	24 175	27 648	11/04/1983 15:02
RUN.COMANDO	R	1842	3072	11/12/1981 07:59

BASIC.OVERLAY1–BASIC.OVERLAY4

Estos archivos contienen distintas partes del entorno BASIC. Una superposición aporta código cuando se necesita, para que no todas las funciones del lenguaje tengan que ocupar a la vez el área de ejecución. Sus sufijos identifican componentes que selecciona BASIC, no números de usuario, particiones de ejecución ni versiones sucesivas de un mismo archivo. BASIC los carga según sea necesario; no son comandos independientes que se invoquen en CSI.

Las responsabilidades siguientes explican por qué la familia contiene varias imágenes distintas. Describen funciones identificadas, no una asignación completa de cada comando del editor o de cada pasada numerada del compilador.

Componente	Función
BASIC.OVERLAY1	Análisis y tokenización del código fuente, tablas de palabras clave del editor y del lenguaje y ayuda de sintaxis incrustada.
BASIC.OVERLAY2	Informes de referencias del código fuente, incluidas las referencias a líneas y etiquetas y a variables y constantes.
BASIC.OVERLAY3	Compilación y construcción de la salida compilada, incluidas la emisión de bifurcaciones y las estadísticas del compilador.
BASIC.OVERLAY4	Ejecución interpretada de instrucciones, incluidas las operaciones con archivos y la carga de BASICUSR con nombre.

Cada superposición empieza con su número de componente, de 0x01 a 0x04, seguido de un hueco de entrada JP en el desplazamiento de archivo 0x0001. BASIC.LOADFILE empieza de forma similar con `04 C3 0A 01`. El byte de componente es un dato, no una instrucción que deba ejecutarse como un comando normal. El cargador de BASIC selecciona la entrada adecuada. Conserva la familia coincidente al mantener el entorno del lenguaje.

Componente	Formato	Contenido (bytes)	Asignado (bytes)	Fecha y hora del catálogo
`BASIC.OVERLAY1`	R	22 482	26 624	11/12/1981 07:55
`BASIC.OVERLAY2`	R	7757	9216	11/12/1981 07:56
`BASIC.OVERLAY3`	R	9438	11 264	07/05/1982 17:30
`BASIC.OVERLAY4`	R	28 458	32 768	11/04/1983 14:45

Controladores de dispositivos: la familia SISTEMA.DIS

Los números DIS identifican componentes de controlador a través de las definiciones de dispositivos físicos. No designan una secuencia de revisiones de controladores. ASOCIAR selecciona los dispositivos a través de esas definiciones; el controlador correspondiente aporta las rutinas de entrada/salida. Controladores de terminal similares pueden tener la misma longitud aunque atiendan puertos de terminal distintos. Que tengan la misma longitud no los convierte en archivos duplicados: sus bytes de puerto y de configuración son distintos.

SISTEMA.DIS1 y SISTEMA.DIS2 empiezan con cuatro huecos de entrada JP en los desplazamientos 0x00, 0x03, 0x06 y 0x09. Los controladores de terminal, impresora y cinta que se enumeran a continuación tienen cinco huecos iniciales hasta 0x0C. Son tablas de entrada específicas de cada controlador; no deben interpretarse como la cabecera de versión de un programa de comando.

Controlador	Dispositivo físico o función
SISTEMA.DIS1	FLOPPY1
SISTEMA.DIS2	FLOPPY2
SISTEMA.DIS10	Dispositivo de cinta TAP
SISTEMA.DIS11	Impresora IP
SISTEMA.DIS12	Impresora CDC
SISTEMA.DIS16	PAN9/IS9; controlador de terminal SS2
SISTEMA.DIS17	PAN10/IS10; controlador de terminal SS2
SISTEMA.DIS18	PAN1; controlador de terminal ALP
SISTEMA.DIS19	PAN2/IS2
SISTEMA.DIS20	PAN3/IS3
SISTEMA.DIS21	PAN4/IS4
SISTEMA.DIS22	Impresora IMP
SISTEMA.DIS23	PAN5/IS5
SISTEMA.DIS24	PAN6/IS6
SISTEMA.DIS25	PAN7/IS7
SISTEMA.DIS26	PAN8/IS8

Son programas reubicables. El apéndice B recoge los desplazamientos de entrada de los controladores y los puertos de hardware; el capítulo 12 explica el acceso del operador a los dispositivos. Un archivo de controlador por sí solo no aporta el equipo físico necesario.

Componente	Formato	Contenido (bytes)	Asignado (bytes)	Fecha y hora del catálogo
SISTEMA.DIS1	R	605	1024	05/12/1983 16:10
SISTEMA.DIS2	R	575	1024	05/12/1983 15:53
SISTEMA.DIS10	R	2980	4096	15/07/1985 09:13
SISTEMA.DIS11	R	237	1024	04/02/1984 03:57
SISTEMA.DIS12	R	438	1024	07/04/1983 01:11
SISTEMA.DIS16	R	650	1024	01/01/1984 00:40
SISTEMA.DIS17	R	650	1024	01/01/1984 00:59
SISTEMA.DIS18	R	887	2048	24/10/1984 17:19
SISTEMA.DIS19	R	887	2048	24/10/1984 17:23
SISTEMA.DIS20	R	887	2048	24/10/1984 17:27
SISTEMA.DIS21	R	887	2048	24/10/1984 17:31
SISTEMA.DIS22	R	314	1024	23/08/1984 09:54
SISTEMA.DIS23	R	887	2048	25/10/1984 09:01
SISTEMA.DIS24	R	887	2048	24/10/1984 17:38
SISTEMA.DIS25	R	887	2048	24/10/1984 17:43
SISTEMA.DIS26	R	887	2048	24/10/1984 17:46

Clases de terminal: SISTEMA.CLASE1 y SISTEMA.CLASE2

Una clase de terminal traduce las operaciones lógicas de presentación a secuencias de control específicas del terminal. CLASE1 usa secuencias de coordenadas ESC G; CLASE2 usa secuencias de cursor similares a ANSI. Los archivos son rutinas de traducción ejecutables, no listas editables de ajustes del terminal. Selecciona con ASOCIAR la clase adecuada para el terminal conectado, como se describe en el capítulo 12. Sustituir una clase repetidamente puede consumir memoria de la partición.

Componente	Formato	Contenido (bytes)	Asignado (bytes)	Fecha y hora del catálogo
SISTEMA.CLASE1	R	259	1024	31/05/1984 19:10
SISTEMA.CLASE2	R	328	1024	04/12/1984 23:59

Módulos de extensión de BASIC

Un archivo BASICUSR aporta una rutina reubicable a la que se puede llamar mediante la interfaz USR de BASIC. OPTION USR selecciona un módulo; la rutina seleccionada determina los argumentos y el resultado. Compartir este tipo de archivo no hace que dos rutinas sean intercambiables.

GETPRIV devuelve el privilegio actual, GETID devuelve el número de cuenta y GETUSER rellena una cadena de ocho bytes con el nombre de la cuenta. Sus llamadas básicas se muestran después de la tabla. Otros módulos tienen funciones de implementación entendidas en parte, que se enumeran a continuación, aunque sus contratos generales de argumentos y efectos secundarios siguen incompletos. Son rutinas de extensión que carga BASIC, no miembros automáticos del ejecutivo ni interfaces USR intercambiables.

Módulo	Operación establecida y límites
CAMPAR	Llama al servicio de planificación y cesión 0x4F y vuelve. Su imagen completa tiene tres bytes, `CF 4F C9`; ocupa un sector almacenado y no tiene sector de reubicación.
PUTQUET	Conserva HL, pasa HL = 0 al servicio 0x39 de establecimiento de devoluciones de llamada, restaura HL y vuelve. Su nombre no establece una operación de cola.
CLAVOUT	Selecciona temporalmente la búsqueda en la cuenta pública para CLAVE.CUENTA, restaura el byte de cuenta de quien llama y lee el primer sector en el almacenamiento de quien llama. Si la búsqueda falla, pone a cero el primer byte de quien llama. El contrato general de llamada sigue incompleto.
LINPAN	Analiza un texto de entrada y cambia campos del descriptor del dispositivo; no es una consulta de solo lectura del número de líneas.
ESPERA	Instala una devolución de llamada, cambia el estado del puerto 0x91, espera mediante el servicio 0x6B y elimina la devolución de llamada. Sus requisitos generales de entrada y de dispositivo no están especificados.
OCONV, OCONV2 y KEYSET	Rutinas de extensión que usan las funciones de apoyo a las aplicaciones; sus gramáticas de transformación completas no están especificadas.

NOMVOL, TAMDIS y DIREC también carecen de un contrato de llamada completo y documentado. Los ejemplos básicos de GETPRIV, GETID y GETUSER que siguen no deben adaptarse a otro módulo cambiando solo el nombre del módulo.

Componente	Formato	Contenido (bytes)	Asignado (bytes)	Fecha y hora del catálogo
`GETPRIV.BASICUSR`	R	6	1024	01/06/1981 12:37
`GETID.BASICUSR`	R	6	1024	17/06/1981 21:20
`GETUSER.BASICUSR`	R	25	1024	16/04/1981 12:38
`CAMPAR.BASICUSR`	R	3	1024	16/11/1984 14:26
`PUTQUET.BASICUSR`	R	8	1024	25/05/1983 14:22

Componente	Formato	Contenido (bytes)	Asignado (bytes)	Fecha y hora del catálogo
NOMVOL.BASICUSR	R	18	1024	27/04/1983 10:58
TAMDIS.BASICUSR	R	41	1024	01/06/1983 09:25
DIREC.BASICUSR	R	91	1024	18/05/1984 11:36
CLAVOUT.BASICUSR	R	61	1024	01/06/1983 08:44
LINPAN.BASICUSR	R	89	1024	13/09/1984 12:59
ESPERA.BASICUSR	R	65	1024	22/06/1983 08:09
OCONV.BASICUSR	R	116	1024	28/07/1981 12:38
OCONV2.BASICUSR	R	119	1024	14/09/1981 10:21
KEYSET.BASICUSR	R	1539	2048	17/11/1982 15:58

CLAVIN, CLAVOUT y CLAVE.CUENTA

CLAVIN.COMANDO es el componente asociado instalado, del lado de los comandos, de CLAVOUT.BASICUSR. Su cabecera dice `CLAVIN ver 1.0 01JUN83`; su imagen de carga es de 6146 bytes con 686 registros de reubicación, y su prólogo requiere el privilegio 5. El cuerpo principal aparente tiene rutas de escritura de argumentos y de lectura sin argumentos asociadas a CLAVE.CUENTA. La pareja parece intercambiar una cadena con recuento o un valor, cuyo significado para las aplicaciones sigue sin especificarse.

El comando falla durante el inicio con `CR = 254`, antes de esas operaciones aparentes del cuerpo principal. Su código almacenado y sus registros de reubicación son incoherentes: la reubicación en el desplazamiento de archivo 0x010D modifica bytes que incluyen el código de operación de una instrucción, y otra entrada en 0x0191 lleva a un valor no deseado del puntero de pila en la ruta de inicio observada. Siguen sin conocerse el origen del fallo y una reparación auténtica. Crear CLAVE.CUENTA no corrige este fallo. Consulta CLAVIN para ver la invocación y sus límites.

CLAVE.CUENTA no existe en la instalación documentada y es distinto de la base de datos de cuentas SISTEMA.CUENTA. Usa CUENTA para la administración de cuentas y contraseñas; ninguna de las dos funciones auxiliares ofrece una alternativa establecida.

Lee el privilegio actual

En el indicador de BASIC, selecciona la rutina:

```
OPTION USR "GETPRIV"
```

OPTION USR carga la extensión indicada para las llamadas siguientes. Ahora llámala e imprime su resultado numérico:

```
PRINT USR(0,0)
```

El número que se muestra es el privilegio actual. El control vuelve a BASIC.

Lee el número de cuenta

Selecciona la rutina del número de cuenta:

```
OPTION USR "GETID"
```

Así se sustituye la extensión seleccionada anteriormente. Llama a la nueva rutina:

```
PRINT USR(0,0)
```

El resultado es el número de cuenta actual, que es distinto del número de la partición de ejecución.

Lee el nombre de la cuenta

Selecciona la rutina que devuelve una cadena:

```
OPTION USR "GETUSER"
```

Proporciona un área de trabajo de ocho caracteres e imprime el nombre devuelto:

```
PRINT USR$(0,SPACE$(8))
```

SPACE$(8) aporta los ocho bytes que rellena GETUSER. USR$ recupera un resultado de cadena; USR, en los ejemplos anteriores, recupera un número.

Superposiciones de SORT y de CONSULTA

SORT.OVERLAY1 y SORT.OVERLAY2 aportan partes distintas de la utilidad de ordenación. Ambas empiezan con el literal `SORTCB` y un área compartida de control y datos de 0xF0 bytes (240 bytes). Los huecos de entrada JP ejecutables siguen en los desplazamientos 0xF0, 0xF3 y 0xF6. SORT copia el área de control en la superposición seleccionada antes de llamarla y la vuelve a copiar después. OVERLAY2 parece realizar el trabajo de fusión y salida; el algoritmo de ordenación completo y la división en fases siguen sin especificarse.

CONSULTA.COMANDO solicita CONSULTA.OVERLAY1, OVERLAY2 y OVERLAY3; OVERLAY3 solicita a su vez OVERLAY4. Estas dependencias no implican que todas las consultas carguen los cuatro componentes. No se dispone de una descripción funcional completa de cada superposición. En ambas familias, los números distinguen componentes de código que selecciona la utilidad, no comandos independientes ni versiones sucesivas.

Componente	Formato	Contenido (bytes)	Asignado (bytes)	Fecha y hora del catálogo
SORT.OVERLAY1	R	4183	6144	19/02/1982 10:17
SORT.OVERLAY2	R	4982	7168	18/02/1982 09:07
CONSULTA.OVERLAY1	R	21 320	24 576	07/02/1983 13:34
CONSULTA.OVERLAY2	R	19 547	22 528	27/01/1983 16:15
CONSULTA.OVERLAY3	R	20 252	23 552	24/09/1982 00:50
CONSULTA.OVERLAY4	R	15 811	18 432	07/02/1983 13:37

CONSULTA.TRAB1–CONSULTA.TRAB4

Son archivos de trabajo directos que usa CONSULTA para estado estructurado temporal. A diferencia de las superposiciones de CONSULTA, contienen datos y no código. Cada archivo de la lista proporciona 400 registros de 256 bytes, es decir, 102 400 bytes de capacidad. Su disposición completa de registros y su regla de selección no están especificadas; el sufijo numérico por sí solo no basta para garantizar la misma correspondencia con las particiones que tienen los archivos de trabajo de EXEC. CONSULTA mantiene su contenido, que no puede sustituirse por texto de EDITAR. Los registros de trabajo almacenados pueden quedar después de una consulta; su presencia por sí sola no establece que haya actividad en curso.

Componente	Formato	Contenido (bytes)	Asignado (bytes)	Fecha y hora del catálogo
CONSULTA.TRAB1	D	102 400	102 400	01/01/1985 09:52
CONSULTA.TRAB2	D	102 400	102 400	01/01/1987 00:04
CONSULTA.TRAB3	D	102 400	102 400	01/01/1987 00:04
CONSULTA.TRAB4	D	102 400	102 400	01/01/1987 00:05

Componentes del SPOOLER e informes generados

SPOOLER.OVERLAY1 aporta código ejecutable del SPOOLER. SISTEMA.SPOOLCOL es el archivo directo de la cola: sus 64 registros de 16 bytes describen los informes, las particiones que los envían, las fechas, los indicadores, las colas y los números de páginas. Su capacidad de 1024 bytes son metadatos de la cola, no la capacidad conjunta de los informes.

Componente	Formato	Contenido (bytes)	Asignado (bytes)	Fecha y hora del catálogo
SPOOLER.OVERLAY1	R	1617	3072	27/04/1984 12:44
SISTEMA.SPOOLCOL	D	1024	1024	11/12/1981 07:59

El contenido de los informes se almacena por separado en archivos generados llamados SISTEMA.SP$ seguidos de un número de informe. Sus longitudes y fechas cambian con el trabajo enviado. La cola relaciona cada informe con su estado y su número de páginas. Usa el SPOOLER para examinar la cola actual:

```
SPOOLER LIST
```

Así se muestra la información de la cola. LIST se detiene en el terminador de la cola; los bytes distintos de cero situados más allá de ese terminador no tienen por qué representar informes activos. No borres los archivos de respaldo de los informes mientras haya entradas de la cola que hagan referencia a ellos. El capítulo 29 explica la codificación de los informes y las reglas de reinicio.

Datos de apoyo para rótulos en letras grandes

MAESTRO.TITULO es un archivo de datos directos al que hace referencia TITULO.COMANDO, la utilidad para imprimir rótulos en letras grandes. Sus registros compactos parecen aportar datos de patrones de caracteres. Siguen sin especificarse la interpretación completa de los registros y si este archivo concreto lo distribuyó el fabricante sin cambios. Su relación con la utilidad permite tratarlo como probable apoyo de la utilidad, en lugar de clasificarlo solo por el nombre MAESTRO.

El archivo proporciona 100 registros de 50 bytes, lo que da una capacidad de 5000 bytes. Su longitud almacenada redondeada a sectores y su asignación son ambas de 5120 bytes. Son datos que lee una utilidad, no un programa para invocar directamente. La referencia de TITULO explica el comando y ofrece un ejemplo en la consola.

Componente	Formato	Contenido (bytes)	Asignado (bytes)	Fecha y hora del catálogo
MAESTRO.TITULO	D	5000	5120	08/02/1983 00:06

Buzón y área de trabajo de duplicación

SISTEMA.BUZON

El buzón secuencial contiene cabeceras de destinatario y emisor y los cuerpos de los mensajes. Su cabecera de mensaje ocupa 33 bytes; el apéndice E documenta los campos. Es un archivo de servicio generado, así que un tamaño de programa o una fecha de catálogo fijos serían engañosos. No tiene por qué existir antes de usar la mensajería almacenada.

Para recuperar los mensajes de la cuenta actual, introduce:

```
BUZON
```

La recuperación elimina los mensajes que muestra. Esta operación no es una vista de solo lectura repetible del buzón.

SISTEMA.DUPLICAR

Esta área de trabajo directa permite la duplicación de disquetes. El procedimiento de preparación asigna 4000 registros de 256 bytes, una capacidad de 1 024 000 bytes. Es un tamaño de área de trabajo solicitado, no la longitud de un programa ejecutable. Su fecha de catálogo depende de su creación; no se especifica ninguna fecha fija. El capítulo 11 recoge los pasos de preparación y de duplicación. Un área de trabajo por sí sola no establece que haya preparada una unidad de disquete compatible.

SISTEMA.BUZON, SISTEMA.DUPLICAR y los archivos de informe generados por el SPOOLER son archivos de servicio condicionales; no debe asignárseles un tamaño instalado fijo solo porque un programa pueda usarlos. CONECTAR también hace referencia a SISTEMA.HISTORIA, cuyo contrato completo de registros y cuya disponibilidad no están especificados. CLAVE.CUENTA no existe; su relación con CLAVIN y CLAVOUT se describe con los módulos de extensión de BASIC, más arriba.

Identificación incrustada en los componentes

Estas cadenas literales identifican componentes concretos. El desplazamiento se mide desde el principio del archivo antes de la reubicación. No es una ubicación de cabecera universal. Una fecha dentro de una marca se reproduce tal como está escrita y es distinta de la fecha del catálogo de las tablas de tamaños.

Componente	Desplazamiento en el archivo	Texto de identificación literal
SISTEMA.CSI	0x0003	`CSI ver 5.5 Copyright by Tim Williams`
SISTEMA.EXECLANG	0x0003	`EXEC ver 5.5 Copyright by T.S.Williams`
BASIC.COMANDO	0x0003	`BASIC VER 1.0 11DIC81`
RUN.COMANDO	0x0003	`RUN VER 1.0 11DIC81`
SORT.COMANDO	0x0003	`SORT ver 1.0A 18FEB82`
SORT.OVERLAY1	0x00FC	`SORTLTAG Ver 2.0 08SEP81`
SORT.OVERLAY2	0x00FC	`SORTMTAG Ver 1.0 02APR81`
CONSULTA.COMANDO	0x0003	`ACCESS ver 1.0D 01OCT82 Copyright de SECOINSA-D.TECNICA`
CONSULTA.OVERLAY1	0x0003	`CONSULTA ver 1.0 01OCT83 Cpyright de SECOINSA-D.TECNICA`
CONSULTA.OVERLAY2	0x0003	`CONSULTA ver 1.0 01OCT82 Cpyright de SECOINSA-D.TECNICA`
CONSULTA.OVERLAY3	0x0003	`CONSULTA ver 1.0 01OCT82 Cpyright de SECOINSA-D.TECNICA`
CONSULTA.OVERLAY4	0x0003	`CONSULTA ver 1.0 01OCT82 Cpyright de SECOINSA-D.TECNICA`

CONSULTA.OVERLAY1 contiene la fecha 01OCT83, aunque su marca de tiempo en el catálogo es 07/02/1983 13:34. OVERLAY3 tiene una fecha de catálogo de 24/09/1982, anterior a su fecha incrustada 01OCT82. Estas discrepancias se conservan; ninguno de los dos campos establece una cronología de compilación fiable. ACCESS y Cpyright son grafías literales de los componentes. Las versiones de componentes como CSI 5.5 y BASIC 1.0 son distintas de la versión 6.2 del sistema operativo.

Archivos de código fuente y de programa creados por el usuario

HOLA.BASICOBJ y HOLA.BASICCOM se crean con el ejemplo del capítulo 15. BASICOBJ contiene el código fuente guardado para el editor de BASIC; BASICCOM contiene la entrada compilada para el entorno de ejecución. NOTA.TEXTO y HOLA.EXEC son, del mismo modo, archivos didácticos. No son componentes incluidos con el sistema operativo.

Procedimientos de apoyo y herramientas compiladas opcionales

Los archivos siguientes amplían el entorno de programación en BASIC y de archivos de gestión. Un procedimiento EXEC aporta preguntas, colas de entrada y llamadas a programas; un archivo BASICCOM contiene instrucciones compiladas que consumen RUN y el entorno de ejecución de BASIC. Por tanto, una misma raíz de nombre no tiene por qué designar programas duplicados. Por ejemplo, ACTUCAP.EXEC prepara los argumentos para ACTUCAP.BASICCOM. Un RUN ACTUCAP directo omite la preparación de ese envoltorio.

Estos componentes son software de apoyo opcional, no funciones residentes del ejecutivo. Su presencia por sí sola no establece que estén disponibles todos los diccionarios, definiciones de pantalla, impresoras o programas asociados necesarios. Las descripciones siguientes distinguen el envoltorio invocable del componente compilado e identifican las operaciones que modifican datos. No se ha establecido que todos los componentes opcionales sean productos originales del fabricante.

Definiciones de captura y generación de código fuente

Una definición de captura describe los campos y el comportamiento de un programa de entrada de datos. Las herramientas de apoyo almacenan esas definiciones en LOCAL.MACROS, mantienen el vocabulario en CONTROL.VOC y pueden generar código fuente BASIC. No constituyen un ensamblador ni un enlazador Z80 de uso general.

ACTUCAP.EXEC acepta un nombre de programa o de pantalla opcional, lo pone en cola y llama a ACTUCAP.BASICCOM. Sin el nombre, el programa compilado lo solicita. Su ayuda describe la creación o actualización de la pantalla, del archivo y de los criterios de modificación en LOCAL.MACROS antes de la expansión del código fuente. ACTUCAP2.BASICCOM aporta la compatibilidad con la definición de procesos especiales; el paso completo a ese programa no está especificado. ACTUCAP.PANTALLA y ACTUCAP2.PANTALLA contienen datos de pantalla de apoyo.

Empieza solicitando la ayuda del envoltorio:

```
EXEC ACTUCAP AYUDA
```

Así se selecciona su rama explicativa en lugar de abrir el diálogo de mantenimiento de definiciones. La forma operativa se ilustra con:

```
EXEC ACTUCAP ENTRADA
```

ENTRADA es un nombre de definición elegido, no una aplicación suministrada. Esta forma puede modificar LOCAL.MACROS y requiere archivos de apoyo inicializados. Las definiciones de campos deben incluir todos los campos clave y no clave a los que vaya a acceder el programa generado. El diálogo interactivo completo no se especifica aquí.

EXPAND.EXEC lee una definición existente y controla EXPAND.BASICCOM, seguido de la carga, la sangría y el guardado del código fuente BASIC. La compilación opcional crea una salida BASICCOM. Sus opciones anunciadas son PANT, COMPILAR, NOCOM, DISCO=df, NOB y BORRAR. Algunas ramas borran archivos intermedios BASIC o BASICOBJ, así que la expansión es una operación de mantenimiento que genera código fuente, no una vista previa de solo lectura.

Para leer primero la información de invocación disponible, introduce:

```
EXEC EXPAND AYUDA
```

Una ilustración de sintaxis para la generación de código fuente es:

```
EXEC EXPAND ENTRADA
```

Requiere una definición de captura ENTRADA compatible, los archivos de macros y el modelo, y los componentes de BASIC. No es un ejemplo completo que cree esas definiciones desde cero. EXPAND.MACMODEL aporta texto BASIC con marcadores de posición como {PNAME}, {READ}, {WRITE}, {DELETE} y {SPECPROC}. USOIMPR.BASICMAC aporta código fuente BASIC relacionado con la impresora que abre IMPR1 y reconstruye las líneas de pantalla para la salida. Ninguno de los dos es un comando ejecutable ni una biblioteca de objetos reubicables.

TESTCAP.EXEC pone en cola un nombre de pantalla opcional y llama a TESTCAP.BASICCOM. Su ayuda describe la prueba de una pantalla con una sola clave: el primer campo de datos es la clave y el último dato es una etiqueta de fin de pantalla. Indica que esta herramienta no mantiene archivos y que la clave no puede cambiarse en el modo de modificación. Para leer esas restricciones, introduce:

```
EXEC TESTCAP AYUDA
```

IMPCAP.EXEC pone en cola un nombre de pantalla obligatorio y llama a IMPCAP.BASICCOM para imprimir los datos de su definición. Para una pantalla llamada ENTRADA y una impresora asociada correctamente, la sintaxis es:

```
EXEC IMPCAP ENTRADA
```

Es una petición de impresión, no la ejecución de la pantalla. IMPCAP devuelve el resultado de sintaxis 8 cuando se omite el nombre obligatorio. SCAPTURA.BASICCOM y GENPAN.BASICCOM pertenecen al apoyo de pantallas y capturas, pero sus contratos completos de invocación independiente siguen sin especificarse. SCAPTURA incluye una llamada a DIREC.BASICUSR con nombre; esa dependencia no lo convierte en un comando general de inspección de archivos.

Las entradas de EXEC y de modelos siguientes cuentan los bytes de texto incluidos los CR.

Componente	Formato	Contenido (bytes)	Asignado (bytes)	Fecha y hora del catálogo
ACTUCAP.EXEC	S	945	1024	25/11/1981 11:18
EXPAND.EXEC	S	2577	3072	06/08/1982 00:41
TESTCAP.EXEC	S	818	1024	25/11/1981 09:21
IMPCAP.EXEC	S	485	1024	24/11/1981 16:13
EXPAND.MACMODEL	S	15 898	16 384	25/06/1984 04:59
USOIMPR.BASICMAC	S	661	1024	27/11/1981 08:58

Las entradas S siguientes cuentan los datos binarios almacenados, incluido el relleno, no longitudes de programa descodificadas. Los programas BASICCOM requieren RUN o sus envoltorios; los archivos PANTALLA son datos de apoyo y no son automáticamente intercambiables con la salida del editor PANTALLA de tipo PANT.

Componente	Formato	Contenido (bytes)	Asignado (bytes)	Fecha y hora del catálogo
ACTUCAP.BASICCOM	S	4318	5120	01/12/1981 17:48
ACTUCAP2.BASICCOM	S	3556	4096	16/08/1982 11:38
ACTUCAP.PANTALLA	S	1270	2048	27/11/1981 11:40
ACTUCAP2.PANTALLA	S	762	1024	25/01/1982 08:47
EXPAND.BASICCOM	S	5588	6144	17/09/1982 00:32
TESTCAP.BASICCOM	S	3556	4096	16/08/1982 11:49
IMPCAP.BASICCOM	S	1778	2048	30/11/1984 14:15
SCAPTURA.BASICCOM	S	6096	6144	19/06/1984 13:59
GENPAN.BASICCOM	S	6096	6144	16/05/1984 18:33

Creación de diccionarios y mantenimiento del vocabulario

GENFICH.EXEC crea los archivos de trabajo que usan las funciones de apoyo de captura y de consulta. Acepta una letra de disco lógico opcional, que de forma predeterminada es S. Solicita primero su ayuda:

```
EXEC GENFICH AYUDA
```

La forma de creación es:

```
EXEC GENFICH S
```

Úsala solo al preparar un nuevo entorno de apoyo con nombres de archivo de trabajo que no estén en uso. Crea los archivos indexados siguientes, copia el vocabulario inicial a través de un archivo temporal CONTROL.ANTVOC, llama a COPIVOC.BASICCOM y borra ese archivo temporal. El contenido existente y una creación interrumpida requieren una revisión aparte; no es un procedimiento de reparación automática.

Archivo de trabajo	Bytes por registro	Capacidad inicial de registros	Bytes de clave
LOCAL.MACROS	512	100	8
PROMPT.MACROS	34	200	11
CONTROL.VOC	512	100	30

MAESTRO.VOC aporta el vocabulario inicial. Su asignación indexada tiene espacio para once campos de datos de 512 bytes; seis están activos en el componente descrito aquí. Cada hueco contiene además la clave de 30 bytes y un enlace de dos bytes, lo que da 544 bytes por hueco. El almacenamiento del índice y la asignación de bloques ocupan espacio adicional. La cifra de contenido solo refleja los seis campos de datos activos.

GENVOC.EXEC crea entradas de vocabulario a partir de una pantalla de captura. Acepta un nombre de pantalla y un nombre de archivo de destino opcional; si se omite este último, se usa el nombre de la pantalla. La opción NOC suprime las preguntas por cada dato. Su ayuda limita la creación a archivos con una sola clave de acceso. Para una pantalla ENTRADA existente, la ilustración de sintaxis es:

```
EXEC GENVOC ENTRADA
```

Así se añaden definiciones a CONTROL.VOC; requiere que existan los datos de la pantalla y el diccionario. No crea un archivo de aplicación arbitrario. Para examinar el texto introductorio completo del procedimiento, usa su argumento AYUDA.

VOC.EXEC gestiona las relaciones entre los atributos con nombre y los datos almacenados. Sin función, inicia VOC.BASICCOM para el mantenimiento. LIST o PANT lista el diccionario en la pantalla, IMPR lo imprime y BORRAR elimina grupos de atributos mediante MANTVOC.BASICCOM. FICHERO limita opcionalmente esas operaciones a un archivo con nombre. Un ejemplo que no borra nada es:

```
EXEC VOC LIST
```

Así se invoca CONSULTA con la consulta preparada de listado del diccionario que se explica en el capítulo 35. Requiere un CONTROL.VOC utilizable y el archivo de trabajo de CONSULTA de la partición. VOC.PANTALLA respalda la interfaz de mantenimiento.

COPIVOC.EXEC amplía o reconstruye un diccionario. Pide las asociaciones de disco antigua y nueva y una nueva capacidad de registros, elimina la protección contra borrado, cambia el nombre del diccionario antiguo a CONTROL.ANTVOC, crea un nuevo CONTROL.VOC, ejecuta COPIVOC.BASICCOM y, si tiene éxito, elimina el archivo antiguo temporal. Su rama de error intenta la restauración. Estos pasos cambian nombres, protecciones y datos; no hacen que cualquier migración interrumpida sea recuperable automáticamente. Revisa el diccionario existente y conserva una copia de seguridad independiente antes de usar este procedimiento interactivo.

Las entradas de procedimientos S siguientes cuentan bytes de texto; MAESTRO.VOC cuenta los campos de datos indexados activos.

Componente	Formato	Contenido (bytes)	Asignado (bytes)	Fecha y hora del catálogo
GENFICH.EXEC	S	943	1024	09/07/1982 15:52
GENVOC.EXEC	S	1327	2048	25/11/1981 12:04
VOC.EXEC	S	2060	3072	18/02/1982 09:22
COPIVOC.EXEC	S	937	1024	04/02/1983 08:47
MAESTRO.VOC	I	3072	7168	01/01/1983 00:02

Las entradas siguientes cuentan los datos binarios almacenados, incluido el relleno. GENFICH también llama a COPIVOC.BASICCOM; MANTVOC.BASICCOM implementa la ruta de eliminación del vocabulario. No deben invocarse como sustitutos de sus envoltorios sin preparar el estado esperado.

Componente	Formato	Contenido (bytes)	Asignado (bytes)	Fecha y hora del catálogo
GENVOC.BASICCOM	S	1524	2048	02/03/1983 01:05
VOC.BASICCOM	S	4572	5120	18/02/1982 10:20
MANTVOC.BASICCOM	S	508	1024	01/12/1981 16:27
COPIVOC.BASICCOM	S	254	1024	01/12/1981 16:26
VOC.PANTALLA	S	1524	2048	18/02/1982 09:51

Generación de informes e inserción de código fuente

INFORME.EXEC coordina la generación de programas de informes. Obtiene la partición actual mediante GETPART.BASICCOM, usa el resultado de finalización para seleccionar el archivo de trabajo numerado de esa partición (por ejemplo, CONSULTA.TRAB1 para la partición 1), vacía ese archivo de trabajo, pone en cola los argumentos indicados e inicia INFORME1.BASICCOM. INFORME1–INFORME5 son miembros de la familia de apoyo a los informes, no cinco versiones independientes de un mismo comando. Sigue sin especificarse qué etapas posteriores de la generación corresponden a los miembros 2 a 5.

Solicita la ayuda del envoltorio antes de crear una definición de informe:

```
EXEC INFORME AYUDA
```

Así se toma la rama de ayuda antes de GETPART y del vaciado del archivo de trabajo. La forma de generación documentada toma un nombre de programa y una A o una C opcionales para un informe abierto o cerrado. Su sintaxis se ilustra con:

```
EXEC INFORME RESUMEN A
```

Puede sustituir el estado de trabajo de informes de la partición y genera código fuente BASIC a partir de definiciones aportadas por el operador. Necesita el archivo de trabajo de CONSULTA numerado correspondiente, BASIC y los componentes de informes. No es un informe ya preparado sobre archivos arbitrarios. INFORME1 usa los módulos BASICUSR OCONV2 y KEYSET. GETPART es una función auxiliar compilada cuyo resultado es un selector de partición; no es una instrucción para crear una partición.

INSER.EXEC pone en cola los argumentos indicados, ejecuta INSER1.BASICCOM y pregunta si se desea realizar otra inserción. INSER1.EXEC es un procedimiento de comandos independiente: controla BASIC y EDITAR para copiar un intervalo de instrucciones de código fuente, renumerarlas y guardar la salida BASICOBJ en S. La distinción entre INSER1.EXEC e INSER1.BASICCOM es importante al elegir un punto de entrada.

INSER1.EXEC toma cinco campos: programa de destino, punto de inserción, programa de origen, primera instrucción de origen y número de instrucciones. Una forma ilustrativa es:

```
EXEC INSER1 DESTINO 0 ORIGEN 100 5
```

Aquí, el cero solicita la inserción al principio de DESTINO. ORIGEN y DESTINO ya deben ser programas BASIC/BASICOBJ guardados y compatibles; 100 identifica la primera instrucción de origen y 5 el número de instrucciones que se insertan. Esta operación reescribe el código fuente guardado y usa archivos temporales INSER$. Cuando el origen y el destino son el mismo, elimina las instrucciones originales después de moverlas para evitar duplicados. Guarda copias independientes antes de usarla; las posiciones de edición predeterminadas, los intervalos mal formados y la recuperación tras una interrupción no están especificados por completo.

Las tres entradas de procedimientos siguientes cuentan bytes de texto.

Componente	Formato	Contenido (bytes)	Asignado (bytes)	Fecha y hora del catálogo
INFORME.EXEC	S	1133	2048	01/10/1983 00:34
INSER.EXEC	S	186	1024	02/08/1982 18:02
INSER1.EXEC	S	2042	3072	02/08/1982 15:03

Las entradas compiladas siguientes cuentan los datos binarios almacenados, incluido el relleno. Usa el envoltorio de informes o de inserción para la preparación que aportan.

Componente	Formato	Contenido (bytes)	Asignado (bytes)	Fecha y hora del catálogo
GETPART.BASICCOM	S	254	1024	17/10/1983 16:30
INFORME1.BASICCOM	S	4826	5120	01/10/1983 00:39
INFORME2.BASICCOM	S	6096	6144	19/05/1983 13:34
INFORME3.BASICCOM	S	6350	7168	19/05/1983 13:35
INFORME4.BASICCOM	S	7112	7168	19/05/1983 13:54
INFORME5.BASICCOM	S	5334	6144	23/09/1983 01:33
INSER1.BASICCOM	S	1524	2048	24/12/1982 12:54

Configuración del sistema, copia y migración de versiones

CIPL.EXEC configura las cuentas, las particiones, las asociaciones, el estado de la impresora y del SPOOLER y los archivos de trabajo. Es software de configuración administrativa, no un comando previo necesario para cada inicio de sesión. Invoca otro procedimiento llamado CIPL1 cuya implementación no está disponible, así que no puede prescribirse su ejecución completa como procedimiento normal de arranque.

Lo siguiente es solo una ilustración de sintaxis:

```
EXEC CIPL ALP4 A IP
```

ALP4 selecciona la rama de terminales ALP; no significa exactamente cuatro usuarios. Esa rama empieza con la partición 2 y el terminal PAN2. De NP1 a NP7 se reduce el límite de activación; el límite predeterminado es ocho. El argumento A solicita una asociación pública del disco lógico A con la definición física 4, que ya debe describir un almacenamiento disponible adecuado. No crea ni formatea ningún volumen. IP selecciona el dispositivo de impresora en paralelo en lugar de la selección predeterminada IMP. Úsalo solo con el hardware y las definiciones necesarios. El procedimiento asigna o ajusta particiones e inicializa el estado del SPOOLER y de trabajo, así que no es una consulta de estado inofensiva. La administración de cuentas y de particiones se explica por separado en los capítulos 10 y 12.

COPSIS.EXEC y COPSIS2.EXEC copian componentes del sistema. Son procedimientos alternativos, no particiones numeradas. COPSIS pide el origen y el destino y puede inicializar el soporte de destino. COPSIS2 contiene una lista explícita de componentes que incluye destinos opcionales que no están disponibles en todas las instalaciones. Ninguno de los dos debe sustituir a los procedimientos planificados de copia de seguridad y restauración del capítulo 13 sin comprobar su origen, su destino y los componentes necesarios.

CAMBVERS.EXEC implementa una ruta de migración de 6.1/6.1A a 6.2. Copia componentes e invoca O8PRECOM para cambiar el estado de precompensación del disco. No es necesario en un sistema que ya ejecuta la 6.2. O8PRECOM escribe en la controladora; su contrato completo de invocación y de recuperación sigue sin especificarse. Por eso, la entrada del comando ofrece un ejemplo de inspección en lugar de un comando de migración para probar.

Estas entradas de procedimientos cuentan los bytes de texto, incluidos los CR.

Componente	Formato	Contenido (bytes)	Asignado (bytes)	Fecha y hora del catálogo
`CIPL.EXEC`	S	1548	2048	01/01/1984 04:41
`COPSIS.EXEC`	S	6847	7168	18/01/1985 12:04
`COPSIS2.EXEC`	S	6912	7168	02/01/1984 01:21
`CAMBVERS.EXEC`	S	1729	2048	01/01/1986 19:03

Rutinas de código fuente BASIC y otros archivos de apoyo

RUTINA1.BASIC a RUTINA4.BASIC son bibliotecas de código fuente con finalidades distintas. No son comandos de CSI independientes ni números de revisión de una misma rutina. Aportan material para incorporarlo a un programa BASIC con las variables, el estado de archivos y los supuestos de hardware necesarios.

Archivo de código fuente	Finalidad y requisitos previos
RUTINA1.BASIC	FNDICO%: búsqueda binaria en un archivo directo; requiere las convenciones de registro y clave de la rutina.
RUTINA2.BASIC	FNVFEC$: cálculo de fechas de vencimiento con un año comercial; no des por supuesta la aritmética del calendario civil.
RUTINA3.BASIC	Cargar y mostrar etiquetas a partir de definiciones PANT; usa la interfaz de información de archivos DIREC.
RUTINA4.BASIC	Dibujo de rectángulos descrito para una pantalla 8500; no se ha establecido su aplicabilidad a otro terminal.

Para examinar una biblioteca de código fuente antes de incorporarla, introduce:

```
LISTAR RUTINA1.BASIC:S
```

Así se muestra el código fuente y sus comentarios explicativos; no se ejecuta FNDICO%. Lee los datos que necesita la rutina antes de copiarla en otro programa. Las herramientas de inserción de código fuente anteriores pueden modificar programas, así que su presencia no elimina la necesidad de entender la rutina copiada.

Componente	Formato	Contenido (bytes)	Asignado (bytes)	Fecha y hora del catálogo
RUTINA1.BASIC	S	18 697	19 456	18/05/1984 13:19
RUTINA2.BASIC	S	17 655	18 432	16/05/1984 14:05
RUTINA3.BASIC	S	16 949	17 408	12/11/1984 08:34
RUTINA4.BASIC	S	4392	5120	05/10/1984 12:23

Los demás archivos compilados requieren una interpretación individual. LONCLA.BASICCOM pide un nombre de archivo y contiene un mensaje de resultado de longitud de clave, pero su contrato completo de entrada y errores no está especificado. Es distinto de la opción LONCLA de CREAR. DISCO.BASICCOM contiene una cadena de comando VERFORM A; esto establece una relación con la comprobación de soportes, no una interfaz general de gestión de discos. RESUME.BASICCOM no tiene una finalidad independiente suficientemente especificada y es distinto de la instrucción RESUME de BASIC.

TRAFIC.BASICCOM presenta un diálogo de transformación con archivos de entrada, auxiliar y de salida, y puede crear un archivo de control EXEC. Sus preguntas describen la selección de claves, campos y formato de salida. No establecen una receta completa y segura de transformación. En concreto, la selección de la salida puede escribir archivos, y un procedimiento de control generado opcional puede repetir esas elecciones más adelante. No se prescribe ningún ejemplo de RUN directo para estas herramientas especificadas de forma incompleta.

Estas entradas cuentan los datos binarios almacenados, incluido el relleno.

Componente	Formato	Contenido (bytes)	Asignado (bytes)	Fecha y hora del catálogo
DISCO.BASICCOM	S	254	1024	04/01/1985 21:39
LONCLA.BASICCOM	S	508	1024	16/10/1984 14:42
RESUME.BASICCOM	S	254	1024	16/06/1983 08:42
TRAFIC.BASICCOM	S	5842	6144	17/10/1983 13:20

MAESTRO.POLIZA es una plantilla de formato directo conservada cuyo consumidor y cuya finalidad completa siguen sin especificarse; su nombre por sí solo no identifica una aplicación admitida. Su recuento de Contenido es la capacidad de registros fijos, no el número de registros ocupados. MAESTRO.TITULO se describe por separado más arriba y no es otro miembro ausente de este grupo.

Componente	Formato	Contenido (bytes)	Asignado (bytes)	Fecha y hora del catálogo
MAESTRO.POLIZA	D	612	1024	16/11/1982 15:33

Apéndice D. Comandos, instrucciones y funciones de BASIC

Usa esta referencia después del ejemplo del primer programa del capítulo 15. Los comandos del editor se introducen en el indicador de guion de BASIC. Las instrucciones numeradas se convierten en líneas de programa almacenadas. Las funciones producen valores dentro de expresiones, que a menudo se muestran con PRINT. Ninguno de estos ejemplos es un comando de CSI, salvo que se indique expresamente.

Cada forma documentada tiene a continuación su propio ejemplo básico. Un fragmento que menciona una línea de código fuente, una matriz o un canal de archivo requiere que ese objeto exista. No es un programa completo por sí solo. Los capítulos 15 a 17 aportan flujos de trabajo y programas completos.

Límites de la interfaz. Los tutoriales principales ofrecen un comportamiento definido que funciona. Varias formas adicionales solo tienen un contrato incompleto. Sus entradas distinguen la forma documentada de los argumentos de una función probable o de un efecto desconocido. Una ilustración de sintaxis no garantiza el resultado mostrado, la operación del dispositivo ni el diálogo completo del editor. En concreto, no deben atribuirse a las funciones de cadena, fecha y dispositivo poco conocidas la semántica de otro dialecto de BASIC.

Los signos de menor y mayor de la sintaxis identifican marcadores de posición. El ejemplo los sustituye cuando se conoce el dominio del argumento. Un marcador de dispositivo que permanece significa expresamente que no se especifica ningún ejemplo de dispositivo ejecutable. La ayuda original tiene corchetes desemparejados en algunas formas; solo se han normalizado cuando la puntuación opcional no es ambigua.

Comandos del editor y del depurador

AUTO

Generar automáticamente los números de línea del código fuente.

Sintaxis: `AUTO [<inicio>[,<incr>]]`

Ejemplo

```
AUTO 10,10
```

10 es el primer número y 10 el incremento. Introduce el código fuente en los números que se ofrecen. El diálogo completo de cancelación no está especificado.

BOTTOM

Seleccionar el final del área de edición del código fuente.

Sintaxis: `BOTTOM`

Ejemplo

```
BOTTOM
```

Es un comando de posicionamiento del editor, no de ejecución del programa. El redibujado exacto de la pantalla y la interacción de edición posterior no están especificados.

BREAK AT

Establecer un punto de interrupción asociado a una línea de código fuente.

Sintaxis: `BREAK [AT <línea> [AFTER <recuento>]]`

Ejemplo

```
BREAK AT 20
```

Así se solicita una parada en la línea 20 del programa cargado. AFTER añade una condición de recuento; las reglas completas de recuento y continuación no están especificadas.

BREAK ON (cambio)

Solicitar un punto de interrupción relacionado con una variable.

Sintaxis: `BREAK ON <var> [CHANGE] [AFTER <recuento>]`

Ejemplo

```
BREAK ON A CHANGE
```

Esta forma menciona A y solicita la vigilancia de cambios. Usa un programa desechable; la temporización exacta de la vigilancia y las reglas de la comparación inicial no están especificadas.

BREAK ON (comparación)

Solicitar un punto de interrupción cuando una variable cumple una comparación.

Sintaxis: `BREAK [ON <var> <relac> <valor>]`

Ejemplo

```
BREAK ON A = 7
```

La forma compara A con 7. El ciclo de vida completo del punto de interrupción no está especificado; no es un comando condicional de CSI.

CHANGE

Sustituir texto en el código fuente.

Sintaxis: `CHANGE [/<de>/<a>[/ <intervalo>]]`

Ejemplo

```
CHANGE /HOLA/ADIOS/
```

Los campos delimitados por barras identifican el texto antiguo y el nuevo. Guarda antes una copia; el intervalo predeterminado y las reglas de confirmación interactiva no están especificados.

CONTINUE

Continuar una ejecución de BASIC suspendida.

Sintaxis: `CONTINUE`

Ejemplo

```
CONTINUE
```

Se usa después de una parada de BASIC que se pueda reanudar. No reanuda un comando terminado con FIN o con Esc seguida de Q. Las reglas completas de reanudación no están especificadas.

DELETE (código fuente)

Eliminar líneas de código fuente.

Sintaxis: `DELETE [<intervalo>]`

Ejemplo

```
DELETE 20
```

Así se selecciona la línea de código fuente 20 para eliminarla. Guarda antes el código fuente. Es distinto de DELETE #canal,clave, que afecta a los registros de un archivo.

INDENT

Controlar el ajuste de sangría del código fuente.

Sintaxis: `INDENT [<valor>]`

Ejemplo

```
INDENT 2
```

Así se indica un valor de sangría de 2. No está especificado exactamente cómo se aplica a las líneas existentes frente a las que se introduzcan después.

LENGTH

Solicitar la función de longitud del código fuente de BASIC.

Sintaxis: `LENGTH`

Ejemplo

```
LENGTH
```

En la forma documentada, el comando no tiene argumentos. Las unidades de salida y las reglas de cómputo no están especificadas; no las equipares con bloques de disco asignados.

LIST

Mostrar líneas de código fuente.

Sintaxis: `LIST [<intervalo>]`

Ejemplo

```
LIST
```

Así se muestra el programa en memoria. No lo ejecuta ni lo escribe en disco. LIST 10 selecciona una línea concreta en la forma de intervalo documentada.

LOAD

Leer en el editor código fuente BASIC guardado.

Sintaxis: `LOAD <nombre>[.<tipo>][:<unidad>]`

Ejemplo

```
LOAD HOLA:S
```

Así se selecciona HOLA.BASICOBJ en S. Guarda el trabajo actual antes de cargar otro programa.

LOCATE

Buscar texto en el código fuente.

Sintaxis: `LOCATE [/cadena[/<intervalo>]]`

Ejemplo

```
LOCATE /HOLA/
```

Así se indica HOLA como cadena de búsqueda. El intervalo predeterminado, la colocación del cursor y el comportamiento de la búsqueda repetida no están especificados.

LPnLIST

Enviar un listado del código fuente a la impresora n.

Sintaxis: `LPnLIST`

Ejemplo

```
LP1LIST
```

Así se selecciona la impresora 1. La impresora lógica debe estar asociada a un hardware preparado; que el comando se acepte no demuestra que se haya impreso.

LPnXREF

Enviar un listado de referencias cruzadas a la impresora n.

Sintaxis: `LPnXREF`

Ejemplo

```
LP1XREF
```

Así se solicita la salida de referencias cruzadas en la impresora 1. La disposición exacta del informe no está especificada.

MODIFY

Entrar en la modificación del código fuente de un intervalo seleccionado.

Sintaxis: `MODIFY [<intervalo>]`

Ejemplo

```
MODIFY 10
```

Así se selecciona la línea 10. Las pulsaciones de edición completas y el diálogo de finalización no están especificados; volver a introducir una línea numerada es el método básico de sustitución que se explica en el capítulo 15.

NAME

Establecer o consultar el nombre de programa que usa el editor.

Sintaxis: `NAME [<nombre>[.<tipo>][:<unidad>]]`

Ejemplo

```
NAME HOLA:S
```

Así se indican un nombre y una unidad. No sustituye a SAVE; las interacciones de persistencia y de nombre predeterminado más allá de esta forma no están especificadas.

NEW

Empezar un nuevo programa de código fuente.

Sintaxis: `NEW`

Ejemplo

```
NEW
```

Guarda el trabajo antes de usar NEW, porque sustituye el contexto de edición actual. No crea ningún BASICOBJ en disco hasta que guardes.

QUIT (comando)

Salir del entorno BASIC.

Sintaxis: `QUIT [<código>|<comando CSI>]`

Ejemplo

```
QUIT
```

Así se vuelve a CSI. Las formas opcionales con código de resultado y con comando de CSI están documentadas, pero su interacción completa con los archivos abiertos no está especificada. Cierra los archivos explícitamente.

RENUMBER

Renumerar las líneas de código fuente.

Sintaxis: `RENUMBER [<primero> [<incr> [<inicio> [<fin>]]]]`

Ejemplo

```
RENUMBER 10 10
```

Así se solicita una numeración que empieza en 10 con incrementos de 10. Haz una copia de seguridad antes de usarlo; el tratamiento de todos los destinos incrustados y del intervalo opcional no está especificado.

RUN (comando)

Ejecutar código fuente BASIC mediante el intérprete.

Sintaxis: `RUN [<nombre>][ <línea>]`

Ejemplo

```
RUN
```

Así se ejecuta el código fuente actual. Es distinto del RUN de CSI, que carga un BASICCOM. La ejecución no guarda el código fuente.

SAVE

Escribir el código fuente BASIC guardado.

Sintaxis: `SAVE [<nombre>[.<tipo>][:<unidad>]]`

Ejemplo

```
SAVE HOLA:S
```

Así se crea o actualiza HOLA.BASICOBJ:S. Guarda código fuente; no produce un BASICCOM compilado.

STEP

Solicitar un paso de depuración acotado.

Sintaxis: `STEP [<recuento>]`

Ejemplo

```
STEP 1
```

Así se pasa el recuento 1 a la función de ejecución paso a paso. Las unidades exactas de los pasos y el estado de suspensión necesario no están especificados.

TOP

Seleccionar el principio del área de edición del código fuente.

Sintaxis: `TOP`

Ejemplo

```
TOP
```

Es una petición de posicionamiento del editor. No es el comando RUN de ejecución desde la primera línea.

TRACE

Activar la traza de ejecución.

Sintaxis: `TRACE [VARS]`

Ejemplo

```
TRACE
```

La traza afecta a la ejecución, no al guardado del código fuente. VARS solicita la variante documentada relacionada con las variables; la presentación exacta y la granularidad de la traza no están especificadas.

UNBREAK

Eliminar la selección de un punto de interrupción.

Sintaxis: `UNBREAK [AT <línea>|ON <var>]`

Ejemplo

```
UNBREAK AT 20
```

Así se elimina el punto de interrupción asociado a la línea 20. La forma ON variable se dirige a la vigilancia de una variable en lugar de a una línea.

UNTRACE

Desactivar la traza.

Sintaxis: `UNTRACE`

Ejemplo

```
UNTRACE
```

Úsalo después de una sesión de traza para volver a solicitar la presentación normal de la ejecución.

VARS

Mostrar variables.

Sintaxis: `VARS [<lista-var>]`

Ejemplo

```
VARS A
```

Así se selecciona la variable A para examinarla. Si se omite la lista, se solicita la presentación general de variables; el formato exacto no está especificado.

XREF

Solicitar las referencias cruzadas del código fuente.

Sintaxis: `XREF`

Ejemplo

```
XREF
```

Es una ayuda de navegación por los identificadores del código fuente y sus usos. Por sí sola no compila ni cambia el programa.

Presentación por número de línea

Mostrar una línea de código fuente por su número.

Sintaxis: `nnnn (mostrar línea)`

Ejemplo

```
10
```

En el indicador de BASIC, así se solicita la línea 10. Distingue un número solo de 10 PRINT ..., que aporta texto de código fuente.

Ctrl + Z

Subir en la edición del código fuente BASIC.

Sintaxis: `ctl-z (línea arriba)`

Ejemplo

```
Pulsa Ctrl + Z
```

Es una acción de tecla, no texto literal. El mismo byte no es un comando de historial de CSI.

Tecla LF

Bajar en la edición del código fuente BASIC.

Sintaxis: `lf-key (línea abajo)`

Ejemplo

```
Pulsa la tecla de salto de línea
```

Envía LF, el carácter 0x0A. No des por hecho que la tecla Intro de un teclado moderno envíe LF; el terminador de comandos habitual es CR.

Tecla Intro

Mostrar la línea de código fuente actual en el modo de edición documentado.

Sintaxis: `ret-key (mostrar línea)`

Ejemplo

```
Pulsa Intro
```

Envía CR, el carácter 0x0D. Su significado en el editor depende de estar en el contexto de edición de BASIC y no en CSI.

Instrucciones de programa

CASE

Introducir un caso dentro de una selección estructurada.

Sintaxis: `CASE <expr>`

Ejemplo

```
20 CASE 1
```

Así se indica el valor 1 para la comparación dentro de una construcción SELECT. Las reglas completas de coincidencia de SELECT/CASE no están especificadas; el fragmento no es un programa completo.

CEND

Terminar una construcción de selección estructurada.

Sintaxis: `CEND`

Ejemplo

```
40 CEND
```

CEND va con SELECT/CASE. No termina todo el programa BASIC como END.

CHAIN

Solicitar la transferencia a otro programa BASIC.

Sintaxis: `CHAIN <expr-cad>`

Ejemplo

```
10 CHAIN "NEXT:S"
```

La cadena indica el programa siguiente. La selección exacta entre código fuente y compilado y la herencia de variables y opciones no están especificadas; no des por supuesto el contrato de CHAIN de otro dialecto de BASIC.

CLEAR

Borrar el estado de determinadas variables.

Sintaxis: `CLEAR [<lista-var>]`

Ejemplo

```
10 CLEAR A
```

Así se selecciona A en la forma documentada de lista de variables. El tratamiento exacto de las matrices, las cadenas y las listas omitidas no está especificado.

CLOSE

Cerrar un canal de archivo abierto.

Sintaxis: `CLOSE #<canal>`

Ejemplo

```
CLOSE #1
```

Úsalo cuando termines con el canal 1. Completa el tratamiento de la salida y libera la asociación abierta; no significa cerrar la sesión.

COMMON

Declarar variables para compartirlas entre programas.

Sintaxis: `COMMON <lista-var>`

Ejemplo

```
10 COMMON A,S$
```

La declaración menciona la numérica A y la de cadena S$. Las reglas exactas de duración y disposición con CHAIN/LINK no están especificadas, así que es solo un ejemplo de declaración.

CSI

Solicitar desde BASIC una operación del procesador de comandos.

Sintaxis: `CSI <expr-cad>`

Ejemplo

```
10 CSI "ESTADO QUIEN"
```

La cadena aporta un comando de CSI. El comportamiento completo de retorno, conservación de variables y canales abiertos no está especificado; usa el ejemplo directo de CSI para las comprobaciones de identidad habituales.

DATA

Almacenar valores literales para instrucciones READ posteriores.

Sintaxis: `DATA <lista-lit>`

Ejemplo

```
10 DATA 11,2.5,"ABC",-4
```

READ consume estos valores en orden. DATA no es una operación de escritura en archivos.

DEF FN

Definir una función de usuario.

Sintaxis: `DEF FN<nombre>[(<argumentos>)][=<expresión>]`

Ejemplo

```
10 DEF FNT(X)=X+1
```

La función toma X y devuelve X+1. PRINT FNT(2) la usa cuando la definición está disponible. FNEND cierra la forma independiente de varias líneas.

DELETE (registro)

Borrar un registro con clave a través de un canal abierto.

Sintaxis: `DELETE #<canal>,<clave>`

Ejemplo

```
DELETE #1,"MMMM"
```

Así se elimina la referencia activa de la clave MMMM de un archivo indexado o con clave compatible que esté abierto. No borra la línea de código fuente MMMM ni todo el archivo.

DIM

Declarar una matriz.

Sintaxis: `DIM <lista-var-dim>`

Ejemplo

```
DIM ARR(2)
```

Con OPTION BASE 0, ARR tiene los índices 0 a 2; con BASE 1, los índices 1 a 2. Establece la base antes de declarar la matriz.

ELSE

Introducir la acción alternativa de una condición.

Sintaxis: `ELSE [<instrucción>|<ref-línea>]`

Ejemplo

```
20 IF A=7 THEN PRINT "SI" ELSE PRINT "NO"
```

ELSE selecciona PRINT "NO" cuando la condición no se cumple. Pertenece a la instrucción IF; no es una acción incondicional independiente.

END

Terminar la ejecución del programa.

Sintaxis: `END`

Ejemplo

```
20 END
```

El control vuelve del programa a su entorno de ejecución. No guarda el código fuente ni desconecta la cuenta.

FNEND

Terminar una definición de función de varias líneas.

Sintaxis: `FNEND`

Ejemplo

```
100 FNEND
```

Es el terminador dentro de esa forma de definición. El contrato completo del valor devuelto en varias líneas no está especificado; usa primero la forma básica DEF FNT(X)=X+1.

FOR

Iniciar un bucle con contador o con lista de valores.

Sintaxis: `FOR <var>=[<lista-expr>|<expr-num> TO <expr-num>[ STEP <expr-num>]]`

Ejemplo

```
10 FOR I%=1 TO 3
```

NEXT cierra el bucle. El intervalo usa un paso implícito de 1; una forma de lista como FOR I%=7,9,11 aporta directamente los valores sucesivos.

GET

Leer mediante la forma de acceso a dispositivos del lenguaje.

Sintaxis: `GET <dispositivo> <expr-num>,<lista-var>`

Ejemplo

```
GET <dispositivo> 1,A
```

El marcador de dispositivo debe sustituirse por un designador admitido. El vocabulario de dispositivos válido y el contrato de transferencia no están especificados, así que solo es sintaxis, no un ejemplo de acceso al hardware ejecutable.

GOSUB

Llamar a una subrutina situada en una línea de código fuente.

Sintaxis: `GOSUB <ref-línea>`

Ejemplo

```
10 GOSUB 100
```

La línea 100 debe existir. RETURN en la subrutina reanuda la ejecución después de este GOSUB; consulta el programa completo del capítulo 16.

GOTO

Transferir la ejecución a una línea de código fuente.

Sintaxis: `GOTO <ref-línea>`

Ejemplo

```
10 GOTO 100
```

La ejecución continúa en la línea 100 sin guardar un retorno de subrutina. El destino debe existir.

IF

Elegir una acción según una condición.

Sintaxis: `IF <expr-num>[ THEN <instrucción>[ ELSE <instrucción>]]`

Ejemplo

```
20 IF A=7 THEN PRINT "SIETE"
```

El PRINT solo se ejecuta cuando A es igual a 7. ELSE puede añadir una alternativa.

IFEND

Terminar una construcción IF de bloque.

Sintaxis: `IFEND`

Ejemplo

```
50 IFEND
```

Úsalo después de las secciones THEN y ELSE (opcional) del bloque. Es distinto de END, que termina el programa.

INPUT

Leer valores en variables.

Sintaxis: `INPUT [<expr-cad>,|#<canal>[,<clave>]:]<lista-var>`

Ejemplo

```
10 INPUT "VALOR",A
```

Es la forma documentada de entrada con pregunta: muestra la pregunta y solicita un valor para A. El INPUT de archivo usa en cambio #canal y dos puntos. Las reglas completas de conversión y de reintento no están especificadas.

LET

Asignar un valor.

Sintaxis: `[LET] <var>[ [<expr-num>:<expr-num>] ]=<expr>`

Ejemplo

```
10 LET A=7
```

LET es opcional: A=7 también es una asignación. Almacena el valor en A, en lugar de comparar A con 7 en una condición.

LINK

Solicitar el enlace con un programa mediante una especificación de cadena.

Sintaxis: `LINK <expr-cad>`

Ejemplo

```
10 LINK "NEXT:S"
```

La forma aporta un nombre de programa. Su contrato de retorno y de compartición de estado no está especificado; no debe confundirse con un enlazador nativo de archivos objeto.

LINPUT

Leer texto orientado a líneas en una variable.

Sintaxis: `LINPUT [[<expr-cad>,][USING <expr-cad>]|#<canal>[,<clave>]:]<var>`

Ejemplo

```
10 LINPUT "NAME",A$
```

Así se indican una pregunta y un destino de cadena en la forma documentada. Las reglas exactas de delimitadores y de edición no están especificadas; INPUT y LINPUT no tienen por qué analizar el texto de la misma forma.

MAT (copia)

Asignar una matriz mediante una instrucción matricial.

Sintaxis: `MAT <var>=<var>`

Ejemplo

```
20 MAT A=B
```

Declara antes matrices compatibles. La forma identifica B como origen y A como destino; las reglas de coincidencia de dimensiones y de conversión no están especificadas.

MAT (relleno)

Asignar una expresión escalar mediante una instrucción matricial.

Sintaxis: `MAT <var>=(<expr>)`

Ejemplo

```
20 MAT A=(0)
```

Los paréntesis distinguen esta forma de expresión. Los límites de la matriz y la semántica exacta del relleno no están especificados por completo.

MAT INPUT

Solicitar la entrada de una matriz.

Sintaxis: `MAT INPUT [#<canal>[,<clave>]:] <var>`

Ejemplo

```
20 MAT INPUT A
```

A debe ser una matriz declarada. El orden completo de los elementos, las preguntas y el comportamiento ante errores no están especificados.

MAT PRINT

Solicitar la salida de una matriz.

Sintaxis: `MAT PRINT [#<canal>[,<clave>]:] <lista-var>`

Ejemplo

```
20 MAT PRINT A
```

La instrucción selecciona A para imprimirla. La disposición exacta de los elementos y los separadores no están especificados.

MAT READ

Solicitar valores de matriz al origen de READ.

Sintaxis: `MAT READ [#<canal>[,<clave>]:] <var>`

Ejemplo

```
20 MAT READ A
```

Declara la matriz y aporta DATA compatibles antes de usar un programa completo. Los detalles del recorrido de los elementos y de la conversión no están especificados.

MAT WRITE

Escribir valores de una matriz a través de un canal de archivo.

Sintaxis: `MAT WRITE #<canal>[,<clave>]:<var>`

Ejemplo

```
20 MAT WRITE #1:A
```

El canal 1 ya debe estar abierto para una salida compatible. La serialización exacta de los registros matriciales no está especificada; el WRITE normal se explica en el capítulo 17.

MOUNT

Solicitar el tratamiento de soportes mediante una especificación de cadena.

Sintaxis: `MOUNT <expr-cad>`

Ejemplo

```
10 MOUNT "A"
```

La cadena aporta una especificación similar a una unidad. La gramática completa de cadenas aceptada y su relación con el MONTAR de CSI no están especificadas, así que solo es sintaxis.

NEXT

Avanzar y repetir un bucle FOR.

Sintaxis: `NEXT [<var>]`

Ejemplo

```
30 NEXT I%
```

Así se cierra el bucle FOR I% del capítulo 16. No es un salto general a la siguiente línea de código fuente.

ON ERROR

Seleccionar la línea de un manejador de errores.

Sintaxis: `ON ERROR GOTO <ref-línea> | 0`

Ejemplo

```
10 ON ERROR GOTO 100
```

La línea 100 debe contener el manejador. ON ERROR GOTO 0 desactiva la ruta del manejador. Los errores anidados y todas las interacciones con RESUME no están especificados.

ON GOTO

Elegir una bifurcación de una lista de destinos numerada desde uno.

Sintaxis: `ON <expr-num> GOTO <lista-ref-línea>`

Ejemplo

```
20 ON 2 GOTO 100,200,300
```

Así se selecciona la línea 200. El selector cero está fuera de intervalo; aporta todos los destinos en el programa completo.

ON GOSUB

Elegir una subrutina de una lista numerada desde uno.

Sintaxis: `ON <expr-num> GOSUB <lista-ref-línea>`

Ejemplo

```
20 ON 2 GOSUB 100,200,300
```

Así se llama a la línea 200; RETURN reanuda después de toda la instrucción ON. Los destinos deben ser subrutinas válidas.

OPEN

Conectar un canal a un archivo con un modo y una organización.

Sintaxis: `OPEN #<canal>: <expr-cad>,<modo><método>[<opciones>]`

Ejemplo

```
OPEN #1: "DIRECTO.DATOS:S", UPDATE DIRECT
```

Crea primero el archivo directo como en el capítulo 17. Los dos puntos después de #1 son obligatorios. OPEN puede tener efectos secundarios inmediatos, especialmente la sustitución secuencial.

OPTION

Seleccionar una opción del lenguaje o del entorno de ejecución.

Sintaxis: `OPTION [BASE 0|1]|[PROMPT <expr-cad>]|[USR <cad>]|[CASE <cad>]`

Ejemplo

```
OPTION BASE 1
```

Así se selecciona la indexación de matrices desde uno para las declaraciones posteriores. Las demás opciones tienen ejemplos independientes más adelante.

OTHERWISE

Introducir la rama alternativa de una selección estructurada.

Sintaxis: `OTHERWISE`

Ejemplo

```
30 OTHERWISE
```

Va dentro de SELECT/CASE/CEND. El contrato completo de coincidencia y de paso al caso siguiente no está especificado; el fragmento no es un programa independiente.

PRINT

Mostrar valores o escribir una salida con formato en un canal.

Sintaxis: `PRINT [#<canal>[,<clave>]:][USING <expr-cad>,][<lista-expr>]`

Ejemplo

```
PRINT "HOLA"
```

Las comillas identifican texto literal. PRINT A, en cambio, evalúa la variable A. La forma básica de consola no crea ningún archivo en disco.

PUT

Escribir mediante la forma de acceso a dispositivos del lenguaje.

Sintaxis: `PUT <dispositivo><expr-num>,<lista-expr>`

Ejemplo

```
PUT <dispositivo> 1,65
```

El marcador de dispositivo y el selector requieren un contrato de dispositivo establecido. Ese contrato no está especificado aquí; no lo escribas como una escritura de puerto supuesta.

QUIT (instrucción)

Salir de BASIC desde la ejecución de un programa, indicando opcionalmente un resultado.

Sintaxis: `QUIT [<expr>]`

Ejemplo

```
20 QUIT 0
```

Así se solicita una finalización correcta al entorno que lo rodea. Cierra antes los canales de archivo.

RANDOMIZE

Solicitar la inicialización de la generación de números aleatorios.

Sintaxis: `RANDOMIZE`

Ejemplo

```
10 RANDOMIZE
```

La forma no tiene argumentos. Las reglas de selección de la semilla y de reproducibilidad no están especificadas; no confíes en una secuencia concreta.

READ

Consumir valores de DATA o leer un registro de archivo seleccionado.

Sintaxis: `READ [#<canal>,<clave>:]<lista-var>`

Ejemplo

```
20 READ A,B,S$,I%
```

Con DATA 11,2.5,"ABC",-4, estas variables reciben los cuatro valores en orden. La forma #canal,clave lee en cambio de un archivo compatible.

READNEXT

Leer el siguiente registro indexado y devolver su clave.

Sintaxis: `READNEXT #<canal>,<var>:<lista-var>`

Ejemplo

```
READNEXT #1,K$:A$
```

Con un canal indexado compatible situado después de la clave AAAA, así se recupera la clave siguiente en K$ y sus datos en A$. En el archivo de ejemplo del capítulo 17, lo siguiente es MMMM/MEDIO.

REM

Incluir un comentario en el código fuente.

Sintaxis: `REM <cualquier-texto>`

Ejemplo

```
10 REM SALUDO CON PRINT
```

BASIC conserva el comentario como texto de código fuente. No es una operación PRINT.

RESTORE

Restablecer la posición de lectura de DATA.

Sintaxis: `RESTORE [<ref-línea>]`

Ejemplo

```
RESTORE
```

El siguiente READ vuelve a empezar en el primer DATA. RESTORE con una referencia de línea selecciona esa línea DATA; los destinos inexistentes o que no son DATA no están especificados.

RESUME

Continuar desde un contexto de tratamiento de errores.

Sintaxis: `RESUME [<ref-línea>|0]`

Ejemplo

```
100 RESUME 20
```

Así se documenta una línea de destino especificada para la reanudación. La semántica completa de reintento y de errores anidados no está especificada; no sustituye a corregir la causa del error.

RETURN

Volver de una subrutina GOSUB.

Sintaxis: `RETURN [<ref-línea>]`

Ejemplo

```
110 RETURN
```

La ejecución se reanuda después del GOSUB que hizo la llamada. La variante opcional con referencia de línea tiene un contrato especificado de forma incompleta.

RUN (instrucción)

Iniciar la ejecución mediante la forma RUN en el nivel de programa.

Sintaxis: `RUN [<expr-cad>|<línea>]`

Ejemplo

```
10 RUN "NEXT:S"
```

La cadena selecciona otro programa. Las reglas exactas de conservación y de carga de esta forma no están especificadas; el RUN interactivo normal es el método de ejecución introductorio.

SELECT

Iniciar una selección estructurada según un valor.

Sintaxis: `SELECT <expr>`

Ejemplo

```
10 SELECT A
```

Las entradas CASE aportan las alternativas y CEND cierra la construcción. Las reglas completas de conversión y de coincidencia no están especificadas; para decisiones básicas, usa primero IF.

SLEEP

Solicitar un retardo o un intervalo de suspensión.

Sintaxis: `SLEEP <expr-num>`

Ejemplo

```
10 SLEEP 1
```

El argumento es numérico. Su unidad de tiempo, el redondeo y las condiciones de despertar no están especificados, así que no interpretes 1 como un segundo sin un contrato definido.

STOP

Suspender la ejecución del programa mediante BASIC.

Sintaxis: `STOP [<expr>]`

Ejemplo

```
20 STOP
```

Es distinto de END y del PARAR de CSI. El comportamiento completo de continuación y de los archivos abiertos no está especificado.

THEN

Introducir la rama verdadera de IF.

Sintaxis: `THEN [<instrucción>|<ref-línea>]`

Ejemplo

```
20 IF A=7 THEN PRINT "SI"
```

Se llega a THEN cuando la condición se cumple. Forma parte de la gramática condicional, no es un comando incondicional independiente.

WAIT

Solicitar la operación de espera sin argumentos.

Sintaxis: `WAIT`

Ejemplo

```
10 WAIT
```

La condición de despertar no está especificada. No la uses como si fuera una pregunta de teclado ni un retardo temporizado.

WAIT (dispositivo)

Esperar usando un dispositivo y condiciones numéricas.

Sintaxis: `WAIT <dispositivo><expr-num>,<expr-num>[,<expr-num>]`

Ejemplo

```
WAIT <dispositivo> 1,1
```

El designador admitido y la interpretación de los bits de condición no están especificados. Es solo sintaxis y no debe usarse como una receta inventada de control de dispositivos.

WEND

Cerrar un bucle WHILE.

Sintaxis: `WEND`

Ejemplo

```
40 WEND
```

La ejecución vuelve a la condición WHILE correspondiente. Las reglas exactas de anidamiento y de salida excepcional no están especificadas.

WHILE

Iniciar un bucle controlado por una condición.

Sintaxis: `WHILE <expr-num>`

Ejemplo

```
10 WHILE A<3
```

El bucle previsto continúa mientras su condición sea verdadera y se cierra con WEND. Es una ilustración de la forma; FOR/NEXT es el ejemplo de bucle introductorio completo.

WRITE

Serializar valores en un canal de archivo abierto.

Sintaxis: `WRITE #<canal>[,<clave>]:<lista-expr>`

Ejemplo

```
WRITE #1: "PRIMERO"
```

El canal 1 debe estar abierto para salida. Ciérralo después con CLOSE. WRITE de BASIC almacena registros con tipo, en lugar de garantizar bytes de texto simple.

Funciones

Las entradas marcadas como «Interfaz incompleta» ofrecen una forma documentada de los argumentos y una interpretación funcional probable. Las notas que las siguen identifican el contrato que falta. No des por supuesto un resultado esperado cuando no se indica ninguno.

ABS — valor absoluto

Valor absoluto. El uso básico siguiente tiene un contrato definido.

Sintaxis: `ABS(X)`

Ejemplo

```
PRINT ABS(-3)
```

El resultado numérico en este caso básico es 3.

ASC — conversión de carácter a número

Interfaz incompleta.

Sintaxis: `ASC(A$)`

Ejemplo de la forma documentada

```
PRINT ASC("A")
```

La cadena es el argumento; el tratamiento exacto de la cadena vacía y de varios caracteres no está especificado.

AT$ — cadena de posición del cursor

Interfaz incompleta.

Sintaxis: `AT$(X,Y)`

Ejemplo de la forma documentada

```
PRINT AT$(1,1)
```

Se indican dos coordenadas numéricas. El orden y la base de las coordenadas y los bytes de terminal generados no están especificados.

ATN — arcotangente

Interfaz incompleta.

Sintaxis: `ATN(X)`

Ejemplo de la forma documentada

```
PRINT ATN(1)
```

Se indica un argumento numérico. Las unidades angulares, la precisión y el comportamiento en los límites no están especificados.

BIN — conversión de una cadena binaria en número

Interfaz incompleta.

Sintaxis: `BIN(A$)`

Ejemplo de la forma documentada

```
PRINT BIN("101")
```

El argumento es texto. El alfabeto aceptado, la anchura y el tratamiento del desbordamiento no están especificados.

BINOF$ — conversión de un número en texto binario

Interfaz incompleta.

Sintaxis: `BINOF$(X)`

Ejemplo de la forma documentada

```
PRINT BINOF$(5)
```

Un argumento numérico produce una forma de cadena; la anchura y la representación con signo no están especificadas.

CHR$ — carácter a partir de un valor numérico de byte

Carácter a partir de un valor numérico de byte. El uso básico siguiente tiene un contrato definido.

Sintaxis: `CHR$(X)`

Ejemplo

```
PRINT CHR$(65)
```

CHR$(222) aporta el delimitador de campo 0xDE real en el ejemplo de EXT$. Los valores numéricos no se escriben como nombres de caracteres literales.

COS — coseno

Interfaz incompleta.

Sintaxis: `COS(X)`

Ejemplo de la forma documentada

```
PRINT COS(0)
```

La forma básica toma un argumento numérico. Las reglas generales de unidades, precisión e intervalo no están especificadas.

CRT$ — conversión a cadena de control del terminal

Interfaz incompleta.

Sintaxis: `CRT$(A$)`

Ejemplo de la forma documentada

```
PRINT CRT$("A")
```

El argumento documentado es una cadena; el vocabulario de controles aceptado y los bytes resultantes no están especificados. Es solo sintaxis.

DATE$ — función de cadena de fecha

Interfaz incompleta.

Sintaxis: `DATE$(X)`

Ejemplo de la forma documentada

```
PRINT DATE$(0)
```

El significado del selector numérico y el formato de salida no están especificados. Esta ilustración no promete ningún resultado con la fecha actual.

DAY — función de conversión de fecha/día

Interfaz incompleta.

Sintaxis: `DAY(A$)`

Ejemplo de la forma documentada

```
PRINT DAY("01/01/90")
```

El texto de la fecha es ilustrativo; el formato aceptado y el significado del resultado numérico no están especificados.

DEL$ — familia de borrado en cadenas delimitadas

Interfaz incompleta.

Sintaxis de la forma demostrada: `DEL$(A$,X,Y)`

Ejemplo de la forma documentada

```
PRINT DEL$("ABC",1,1)
```

Este ejemplo de tres argumentos se acepta e imprime una cadena vacía, visible como una línea de salida vacía. Un cuarto argumento provoca FALTAN PARENTESIS y entra en la edición de corrección del código fuente. Las reglas de delimitadores, posiciones y límites siguen incompletas; este resultado no es un borrado normal de caracteres que produzca AC.

DTE$ — función de conversión de texto de fecha

Interfaz incompleta.

Sintaxis: `DTE$(A$)`

Ejemplo de la forma documentada

```
PRINT DTE$("01/01/90")
```

La gramática de fechas de entrada aceptada y la representación de salida no están especificadas.

EOF — condición de final de un canal de archivo

Condición de final de un canal de archivo. El uso básico siguiente tiene un contrato definido.

Sintaxis: `EOF(X)`

Ejemplo

```
PRINT EOF(1)
```

Consúltala después de una lectura en el canal abierto 1. Una condición de final puede valer -1; es distinta del resultado de CSI y del final físico de la cadena.

ERL — valor de la ubicación del error

Interfaz incompleta.

Sintaxis: `ERL`

Ejemplo de la forma documentada

```
PRINT ERL
```

Úsalo en un contexto de error establecido. El valor no tiene por qué ser cero antes de provocar deliberadamente un error. El valor exacto fuera de un manejador y la correspondencia entre interpretado y compilado no están especificados.

ERR — valor del código de error

Interfaz incompleta.

Sintaxis: `ERR`

Ejemplo de la forma documentada

```
PRINT ERR
```

El valor pertenece al estado de error de BASIC, no necesariamente a un número de registro de SISTEMA.MENSERR. No tiene por qué ser cero antes de provocar deliberadamente un error. Las reglas completas de su duración no están especificadas.

EXP — función exponencial

Interfaz incompleta.

Sintaxis: `EXP(X)`

Ejemplo de la forma documentada

```
PRINT EXP(1)
```

Se indica un argumento numérico. Los límites numéricos, el redondeo y el comportamiento ante el desbordamiento no están especificados.

EXT$ — extraer un campo o subcampo delimitado

Extraer un campo o subcampo delimitado. El uso básico siguiente tiene un contrato definido.

Sintaxis: `EXT$(A$,X,Y)`

Ejemplo

```
PRINT EXT$("UN"&CHR$(222)&"DOS",2,0)
```

Devuelve DOS: el campo 2, completo, seleccionado con el subcampo 0. Los campos usan 0xDE y los subcampos 0xDD.

FIX — función de conversión/truncamiento numérico

Interfaz incompleta.

Sintaxis: `FIX(X)`

Ejemplo de la forma documentada

```
PRINT FIX(3.5)
```

La diferencia con INT y FLOAT no está totalmente especificada. Esta forma por sí sola no promete ningún resultado de redondeo.

FLOAT — conversión a un valor numérico real

Interfaz incompleta.

Sintaxis: `FLOAT(X)`

Ejemplo de la forma documentada

```
PRINT FLOAT(3)
```

La entrada numérica aceptada y las reglas de conversión excepcionales no están especificadas por completo.

FORMAT$ — formato numérico en una cadena

Interfaz incompleta.

Sintaxis: `FORMAT$(X,A$)`

Ejemplo de la forma documentada

```
PRINT FORMAT$(7,"###")
```

El segundo argumento es una cadena de formato. Su gramática de patrones no está especificada; las almohadillas ilustran un argumento de cadena, no una máscara admitida garantizada.

HEX — conversión de texto hexadecimal en número

Interfaz incompleta.

Sintaxis: `HEX(A$)`

Ejemplo de la forma documentada

```
PRINT HEX("FF")
```

Los prefijos aceptados, las mayúsculas y minúsculas, la anchura y las reglas de desbordamiento no están especificados.

HEXOF$ — conversión de un número en texto hexadecimal

Interfaz incompleta.

Sintaxis: `HEXOF$(X)`

Ejemplo de la forma documentada

```
PRINT HEXOF$(255)
```

La función toma un número; la anchura de la cadena y la representación con signo no están especificadas.

INP — función numérica relacionada con la entrada

Interfaz incompleta.

Sintaxis: `INP`

Ejemplo de la forma documentada

```
PRINT INP
```

La forma documentada no tiene argumentos explícitos. Su origen, su comportamiento de bloqueo y el significado del resultado se desconocen; es solo sintaxis.

INS$ — familia de inserción en cadenas delimitadas

Interfaz incompleta.

Sintaxis: `INS$(A$,X,Y,B$)`

Ejemplo de la forma documentada

```
PRINT INS$("ABC",1,1,"X")
```

Se indican cuatro argumentos. El posicionamiento exacto por campos y subcampos y el comportamiento de la inserción no están especificados.

INT — conversión numérica a valor entero

Interfaz incompleta.

Sintaxis: `INT(X)`

Ejemplo de la forma documentada

```
PRINT INT(3.5)
```

No deduzcas del nombre el redondeo de los números negativos. FIX e INT pueden ser distintos; sus contratos completos no están especificados.

LEFT$ — selección por la izquierda de una cadena

Interfaz incompleta.

Sintaxis: `LEFT$(A$,X)`

Ejemplo de la forma documentada

```
PRINT LEFT$("ABC",2)
```

Está documentada la forma con cadena y longitud numérica. Los límites y el comportamiento con longitudes excepcionales no están especificados.

LEN — consulta de la longitud de una cadena

Interfaz incompleta.

Sintaxis: `LEN(A$)`

Ejemplo de la forma documentada

```
PRINT LEN("ABC")
```

El argumento es una cadena. El tratamiento de los bytes de control incrustados y los límites de longitud no están especificados por completo.

LINE — consulta del estado de la línea

Interfaz incompleta.

Sintaxis: `LINE(X)`

Ejemplo de la forma documentada

```
PRINT LINE(1)
```

El dominio del selector y si designa un canal o un dispositivo de salida no están especificados. Es solo sintaxis.

Sin ningún canal abierto, este ejemplo muestra [27] Fich. Cerrado. No es un ejemplo completo de consulta positiva: el contexto necesario con un canal abierto y el contrato útil del valor devuelto siguen sin especificarse por completo.

LOG — logaritmo

Interfaz incompleta.

Sintaxis: `LOG(X)`

Ejemplo de la forma documentada

```
PRINT LOG(1)
```

La base, el dominio de entrada y el tratamiento de errores no están totalmente especificados en esta interfaz.

LPAD$ — relleno de una cadena por la izquierda

Interfaz incompleta.

Sintaxis: `LPAD$(A$,X)`

Ejemplo de la forma documentada

```
PRINT LPAD$("A",4)
```

El argumento numérico es un valor parecido a un tamaño. El carácter de relleno, el truncamiento y las convenciones de anchura no están especificados.

LRL — familia de rotación a la izquierda

Interfaz incompleta.

Sintaxis: `LRL(X,Y)`

Ejemplo de la forma documentada

```
PRINT LRL(1,1)
```

Están documentados dos argumentos numéricos. La anchura en bits, la interpretación del sentido y del recuento y el comportamiento con signo no están especificados.

LRR — familia de rotación a la derecha

Interfaz incompleta.

Sintaxis: `LRR(X,Y)`

Ejemplo de la forma documentada

```
PRINT LRR(2,1)
```

Están documentados dos argumentos numéricos; las reglas exactas de la operación de bits no están especificadas.

LSL — familia de desplazamiento a la izquierda

Interfaz incompleta.

Sintaxis: `LSL(X,Y)`

Ejemplo de la forma documentada

```
PRINT LSL(1,1)
```

El nombre sugiere un desplazamiento, pero la anchura del operando y los límites del recuento no están especificados. No se afirma ningún resultado numérico.

LSR — familia de desplazamiento a la derecha

Interfaz incompleta.

Sintaxis: `LSR(X,Y)`

Ejemplo de la forma documentada

```
PRINT LSR(2,1)
```

El comportamiento lógico frente a la propagación del signo y la anchura del operando no están especificados.

LTRIM$ — recorte de una cadena por la izquierda

Interfaz incompleta.

Sintaxis: `LTRIM$(A$)`

Ejemplo de la forma documentada

```
PRINT LTRIM$("  A")
```

El conjunto exacto de caracteres que se eliminan y el tratamiento de la cadena vacía no están especificados.

MATCH — coincidencia de cadenas

Interfaz incompleta.

Sintaxis: `MATCH(A$,B$)`

Ejemplo de la forma documentada

```
PRINT MATCH("ABC","A")
```

El lenguaje de patrones y la convención del resultado numérico no están especificados. No des por supuesta la sintaxis de las expresiones regulares modernas.

MAX — selección del valor mayor

Interfaz incompleta.

Sintaxis: `MAX(X,Y)`

Ejemplo de la forma documentada

```
PRINT MAX(2,3)
```

La forma básica tiene dos argumentos numéricos. Las reglas de conversión con tipos mixtos y valores excepcionales no están especificadas.

MID$ — selección interior de una cadena

Interfaz incompleta.

Sintaxis: `MID$(A$,X,Y)`

Ejemplo de la forma documentada

```
PRINT MID$("ABC",2,1)
```

Se indican una cadena y dos argumentos numéricos. Las reglas exactas de indexación y de valores fuera de intervalo no están especificadas por completo.

MIN — selección del valor menor

Interfaz incompleta.

Sintaxis: `MIN(X,Y)`

Ejemplo de la forma documentada

```
PRINT MIN(2,3)
```

Se indican dos argumentos numéricos; las reglas generales de conversión y de límites no están especificadas.

MOD — función de resto

Interfaz incompleta.

Sintaxis: `MOD(X,Y)`

Ejemplo de la forma documentada

```
PRINT MOD(7,2)
```

El comportamiento con signo, con fracciones y con divisor cero no está especificado. No lo deduzcas solo de la división entera.

NBR — consulta numérica con una función sin especificar

Interfaz incompleta.

Sintaxis de la forma demostrada: `NBR(A$)`

Ejemplo de la forma documentada

```
PRINT NBR("1")
```

Este ejemplo con un argumento de cadena imprime -1. Un argumento numérico produce NECESITA EXPRESION STRING y entra en la edición de corrección. El significado del valor devuelto sigue siendo desconocido; este ejemplo no establece una conversión numérica ni ninguna otra interpretación general.

OCT — conversión de texto octal en número

Interfaz incompleta.

Sintaxis: `OCT(A$)`

Ejemplo de la forma documentada

```
PRINT OCT("10")
```

El alfabeto de entrada aceptado, los prefijos y el tratamiento del desbordamiento no están especificados.

OCTOF$ — conversión de un número en texto octal

Interfaz incompleta.

Sintaxis: `OCTOF$(X)`

Ejemplo de la forma documentada

```
PRINT OCTOF$(8)
```

La anchura de salida y la representación de los valores negativos no están especificadas.

OVR$ — familia de sobrescritura en cadenas delimitadas

Interfaz incompleta.

Sintaxis: `OVR$(A$,X,Y,B$)`

Ejemplo de la forma documentada

```
PRINT OVR$("ABC",1,1,"X")
```

Está documentada la forma de cuatro argumentos; las reglas exactas de campos y subcampos y de sustitución no están especificadas.

PAGE — consulta del estado de la página

Interfaz incompleta.

Sintaxis: `PAGE(X)`

Ejemplo de la forma documentada

```
PRINT PAGE(1)
```

El dominio del selector, las unidades del contador y la relación con el dispositivo de salida no están especificados.

Sin ningún canal abierto, este ejemplo muestra [27] Fich. Cerrado. No es un ejemplo completo de consulta positiva: el contexto necesario con un canal abierto y el contrato útil del valor devuelto siguen sin especificarse por completo.

PI — constante numérica pi

Interfaz incompleta.

Sintaxis: `PI`

Ejemplo de la forma documentada

```
PRINT PI
```

Esta constante con forma de función no admite argumentos. La precisión almacenada exacta no está especificada.

POS — consulta del estado de la posición

Interfaz incompleta.

Sintaxis: `POS(X)`

Ejemplo de la forma documentada

```
PRINT POS(1)
```

El selector numérico y la convención de la coordenada devuelta no están especificados.

Sin ningún canal abierto, este ejemplo muestra [27] Fich. Cerrado. No es un ejemplo completo de consulta positiva: el contexto necesario con un canal abierto y el contrato útil del valor devuelto siguen sin especificarse por completo.

REP$ — familia de sustitución en cadenas delimitadas

Interfaz incompleta.

Sintaxis: `REP$(A$,X,Y,B$)`

Ejemplo de la forma documentada

```
PRINT REP$("ABC",1,1,"X")
```

El intervalo exacto de sustitución y el comportamiento por campos y subcampos no están especificados; no la uses en lugar de EXT$.

RIGHT$ — selección por la derecha de una cadena

Interfaz incompleta.

Sintaxis: `RIGHT$(A$,X)`

Ejemplo de la forma documentada

```
PRINT RIGHT$("ABC",2)
```

La forma documentada acepta una cadena y un número; los límites exactos y las longitudes excepcionales no están especificados.

RND — generación de números aleatorios

Interfaz incompleta.

Sintaxis: `RND`

Ejemplo de la forma documentada

```
PRINT RND
```

La forma documentada no admite argumentos. Las reglas de distribución, intervalo y reproducibilidad no están especificadas.

ROUND — redondeo numérico

Interfaz incompleta.

Sintaxis: `ROUND(X,Y)`

Ejemplo de la forma documentada

```
PRINT ROUND(1.25,1)
```

La escala del segundo argumento y la regla del punto medio no están especificadas. No se promete ningún resultado redondeado concreto.

RPAD$ — relleno de una cadena por la derecha

Interfaz incompleta.

Sintaxis: `RPAD$(A$,X)`

Ejemplo de la forma documentada

```
PRINT RPAD$("A",4)
```

El carácter de relleno, el truncamiento y la interpretación de la anchura no están especificados.

RPT$ — repetir una cadena

Repetir una cadena. El uso básico siguiente tiene un contrato definido.

Sintaxis: `RPT$(X,A$)`

Ejemplo

```
PRINT RPT$(3,"X")
```

Primero va el recuento y después la cadena. Así se producen tres caracteres X. No se acepta invertir los tipos de los argumentos.

RTRIM$ — recorte de una cadena por la derecha

Interfaz incompleta.

Sintaxis: `RTRIM$(A$)`

Ejemplo de la forma documentada

```
PRINT RTRIM$("A  ")
```

El conjunto exacto de caracteres que se eliminan y los límites no están especificados.

SCH — familia de búsqueda en cadenas

Interfaz incompleta.

Sintaxis: `SCH(X,A$,B$)`

Ejemplo de la forma documentada

```
PRINT SCH(1,"ABC","B")
```

El orden y la interpretación de los operandos de búsqueda y la convención del resultado no están especificados; es solo una ilustración de la forma.

SEC — conversión de una cadena relacionada con el tiempo

Interfaz incompleta.

Sintaxis: `SEC(A$)`

Ejemplo de la forma documentada

```
PRINT SEC("00:00:01")
```

La gramática de horas aceptada y la unidad del resultado numérico no están especificadas.

SGN — clasificación del signo

Interfaz incompleta.

Sintaxis: `SGN(X)`

Ejemplo de la forma documentada

```
PRINT SGN(-3)
```

Está documentada la forma de un argumento; los valores exactos del resultado y las reglas numéricas mixtas no están especificados por completo.

SIN — seno

Interfaz incompleta.

Sintaxis: `SIN(X)`

Ejemplo de la forma documentada

```
PRINT SIN(0)
```

Las unidades angulares, la precisión y los límites numéricos no están especificados.

SPACE$ — crear una cadena de espacios

Crear una cadena de espacios. El uso básico siguiente tiene un contrato definido.

Sintaxis: `SPACE$(X)`

Ejemplo

```
PRINT SPACE$(8)
```

Así se obtienen ocho espacios. Es útil como área de trabajo de ocho bytes para GETUSER.

SQR — familia de la raíz cuadrada

Interfaz incompleta.

Sintaxis: `SQR(X)`

Ejemplo de la forma documentada

```
PRINT SQR(4)
```

Este ejemplo muestra 1.999999999999, una aproximación de 2. No uses los dígitos mostrados como garantía de igualdad exacta. El dominio numérico completo, la precisión y el comportamiento ante errores siguen sin especificarse por completo.

STR$ — convertir un valor numérico en texto

Interfaz incompleta.

Sintaxis: `STR$(X)`

Ejemplo de la forma documentada

```
PRINT STR$(7)
```

Las convenciones de signo, espaciado y formato decimal no están especificadas.

TAN — tangente

Interfaz incompleta.

Sintaxis: `TAN(X)`

Ejemplo de la forma documentada

```
PRINT TAN(0)
```

Las unidades angulares, la precisión y el tratamiento de las singularidades no están especificados.

TIME$ — función de cadena de hora

Interfaz incompleta.

Sintaxis: `TIME$(X)`

Ejemplo de la forma documentada

```
PRINT TIME$(0)
```

El selector y el formato de salida no están especificados; el ejemplo no implica ningún contrato con la hora actual.

TRIM$ — recorte de cadenas

Interfaz incompleta.

Sintaxis: `TRIM$(A$)`

Ejemplo de la forma documentada

```
PRINT TRIM$(" A ")
```

El conjunto exacto de caracteres eliminados y la relación con LTRIM$/RTRIM$ no están especificados por completo.

USR — llamar a una extensión numérica de BASIC

Llamar a una extensión numérica de BASIC. El uso básico siguiente tiene un contrato definido.

Sintaxis: `USR(X,Y)`

Ejemplo

```
PRINT USR(0,0)
```

Selecciona primero GETPRIV o GETID con OPTION USR. El desplazamiento 0 y el argumento numérico 0 siguen los contratos de esas funciones de consulta; otros módulos requieren sus propias interfaces.

USR$ — llamar a una extensión de cadena de BASIC

Llamar a una extensión de cadena de BASIC. El uso básico siguiente tiene un contrato definido.

Sintaxis: `USR$(X,A$)`

Ejemplo

```
PRINT USR$(0,SPACE$(8))
```

Selecciona primero GETUSER con OPTION USR. La cadena aporta un área de trabajo de ocho bytes; el resultado contiene el nombre de la cuenta.

VAL — convertir texto numérico en un valor

Interfaz incompleta.

Sintaxis: `VAL(A$)`

Ejemplo de la forma documentada

```
PRINT VAL("7")
```

Los espacios en blanco aceptados, los signos, la notación decimal y el comportamiento ante un final no válido no están especificados.

Controles adicionales y opciones de compilación

OPTION BASE 0

Selecciona índices de matriz desde cero antes de DIM. Así, DIM ARR(2) admite 0, 1 y 2.

Ejemplo

```
OPTION BASE 0
```

OPTION PROMPT

Indica la cadena de la pregunta en la forma de opción documentada. El alcance exacto en las instrucciones de entrada no está especificado.

Ejemplo

```
OPTION PROMPT "? "
```

OPTION CASE

La opción documentada acepta una cadena. Los valores admitidos y si se acepta U no están especificados; solo ilustra la forma del argumento, no es una receta establecida para fijar mayúsculas o minúsculas.

Ejemplo

```
OPTION CASE "U"
```

OPTION USR

Selecciona la extensión del número de cuenta antes de PRINT USR(0,0). La selección y la invocación son operaciones distintas.

Ejemplo

```
OPTION USR "GETID"
```

OPTION PRIV

Exige al menos el privilegio 1 cuando se ejecute esta instrucción compilada. Una cuenta con privilegio inferior se rechaza; la opción no eleva el privilegio.

Ejemplo

```
OPTION PRIV 1
```

OPTION SERIAL

Ilustra una vinculación a un número de serie en dos partes. El valor debe coincidir con el del sistema previsto; 22-0 es un ejemplo, no un ajuste universal. Una vinculación todo a ceros omite la comparación de BASIC.

Ejemplo

```
OPTION SERIAL 22-0
```

UNLOCK

Libera los tramos de registros del canal 1. La posesión exclusiva obtenida con OPEN se mantiene hasta CLOSE.

Ejemplo

```
UNLOCK #1
```

OPEN INPUT

Abre para lectura un archivo de datos BASIC secuencial existente. Debe contener registros adecuados para la operación de entrada de BASIC prevista.

Ejemplo

```
OPEN #1: "LINEAS.DATOS:S", INPUT SEQUENTIAL
```

OPEN OUTPUT

Abre una salida secuencial. Elige un nombre de archivo nuevo o acepta deliberadamente la sustitución; cierra el canal después de escribir.

Ejemplo

```
OPEN #1: "LINEAS.DATOS:S", OUTPUT SEQUENTIAL
```

OPEN UPDATE DIRECT

Abre el archivo de práctica de registros fijos para acceso directo. Es distinto de la sustitución secuencial.

Ejemplo

```
OPEN #1: "DIRECTO.DATOS:S", UPDATE DIRECT
```

OPEN UPDATE SEQUENTIAL

Sustituye el archivo existente borrándolo y volviéndolo a crear en OPEN. No es un modo de edición de texto en el sitio.

Ejemplo

```
OPEN #1: "LINEAS.DATOS:S", UPDATE SEQUENTIAL
```

OPEN EXTEND

Añade datos a un archivo secuencial cerrado correctamente. EXTEND es una opción final; no la uses con metadatos sin terminar.

Ejemplo

```
OPEN #1: "LINEAS.DATOS:S", OUTPUT SEQUENTIAL, EXTEND
```

OPEN LOCK

Solicita la admisión de apertura exclusiva. Quien espera para abrir puede continuar después de CLOSE; UNLOCK por sí solo no basta.

Ejemplo

```
OPEN #1: "DIRECTO.DATOS:S", UPDATE DIRECT, LOCK
```

LIST del compilador

En CSI, compila con un listado del código fuente en la consola.

Ejemplo

```
BASIC HOLA.BASICOBJ:S (LIST)
```

NOLIST del compilador

En CSI, suprime el listado de compilación.

Ejemplo

```
BASIC HOLA.BASICOBJ:S (NOLIST)
```

IMPRn del compilador

En CSI, selecciona la impresora 1 para el listado. La impresora debe estar asociada y preparada.

Ejemplo

```
BASIC HOLA.BASICOBJ:S (IMPR1)
```

XREF del compilador

En CSI, solicita un listado de referencias cruzadas de la compilación.

Ejemplo

```
BASIC HOLA.BASICOBJ:S (XREF)
```

NOOBJ del compilador

NOOBJ suprime la generación del BASICCOM; por sí sola no solicita un listado del código fuente. Pueden seguir apareciendo las estadísticas del compilador. Para listar el código fuente sin producir salida compilada, combina LIST y NOOBJ en CSI. Puede quedar un BASICCOM anterior, que no debe confundirse con una salida recién generada.

Ejemplo

```
BASIC HOLA.BASICOBJ:S (LIST NOOBJ)
```

LIST solicita el listado del código fuente; NOOBJ suprime de forma independiente el archivo compilado. NOOBJ por sí sola no implica LIST ni suprime el procesamiento de la compilación.

NOPANT del compilador

En CSI, suprime la salida por consola. Evítalo mientras aprendes o diagnosticas una compilación.

Ejemplo

```
BASIC HOLA.BASICOBJ:S (NOPANT)
```

OBJ del compilador

En CSI, selecciona A para la salida compilada. A debe ser una asociación de disco accesible y con permiso de escritura.

Ejemplo

```
BASIC HOLA.BASICOBJ:S (OBJ=A)
```

COMMA del compilador

La ayuda suministrada anuncia COMMA para el formato con coma decimal, pero esta versión del compilador la rechaza. Lo siguiente es una invocación rechazada, no un procedimiento de compilación que funcione.

Ejemplo

```
BASIC HOLA.BASICOBJ:S (COMMA)
```

El resultado es Opcion no valida, CR = 8. La grafía alternativa del compilador COMA también se rechaza. Usa el punto decimal en el código fuente de los ejemplos de programación; no se ha establecido que exista una opción del compilador para la coma decimal que funcione.

TRACE en tiempo de ejecución

La ayuda suministrada anuncia TRACE para trazar las direcciones en tiempo de ejecución. La invocación siguiente ejecuta el saludo compilado o el ejemplo numérico, pero no produce ninguna traza visible de direcciones de ejecución en esos programas. No debe tratarse como un procedimiento demostrado de depuración en tiempo de ejecución.

Ejemplo

```
RUN HOLA.BASICCOM:S (TRACE)
```

CR = 0 establece que el programa ha terminado, no que se haya reconocido esta opción. Una opción arbitraria también termina con la misma salida numérica. Otros efectos internos o condiciones de ejecución siguen sin especificarse.

Para una traza de compilación, usa la invocación independiente del compilador:

```
BASIC HOLA.BASICOBJ:S (TRACE)
```

Así se muestran de PASO # 1 a PASO # 8 y los números de línea del código fuente durante la compilación. No traza la ejecución del programa compilado. El comando TRACE del editor interactivo de BASIC es también una interfaz independiente; consulta TRACE.

COMA en tiempo de ejecución

La ayuda suministrada anuncia COMA como opción de coma decimal en tiempo de ejecución. Se escribe con una sola M. El programa numérico de demostración sigue imprimiendo 1.5 con esta opción, así que no es una forma establecida de cambiar la presentación de los decimales.

Para este ejemplo, usa HOLA compilado a partir de las dos líneas de código fuente siguientes, siguiendo el procedimiento de guardado y compilación del capítulo 15:

```
10 PRINT 1.5
20 END
```

Esta versión numérica sustituye al saludo usado antes. Compílala antes de comprobar la presentación de los decimales; un ejecutable anterior del saludo no puede demostrar una opción de formato numérico.

Ejemplo

```
RUN HOLA.BASICCOM:S (COMA)
```

Tanto RUN normal como RUN con COMA muestran 1.5 y devuelven CR = 0. Esto no establece el comportamiento de la conversión de entrada ni excluye otros efectos internos. PRINT 1,5 pasa dos elementos a PRINT; no es un literal con coma decimal. COMMA y COMA del compilador se rechazan por separado, como se ha descrito antes.

Otras palabras clave de organización, opciones y expresiones

Los diecisiete nombres siguientes completan el vocabulario más allá de las formas anteriores. Su presencia en la tabla de palabras clave de BASIC establece su grafía; no establece las convenciones de llamada de otro dialecto de BASIC. KEYED e INDEXED tienen un contexto de apertura de archivos, y NOT, AND y OR tienen operaciones identificadas en las expresiones compiladas. Otros nombres siguen careciendo de un contrato completo visible para el programador. Se mantienen aquí para que pueda reconocerse el código fuente sin sugerir ejemplos no admitidos.

INDEXED

INDEXED selecciona la organización de archivo indexado en OPEN. Para el archivo INDICE.DATOS existente creado en el capítulo 17, introduce en BASIC:

```
OPEN #1: "INDICE.DATOS:S", UPDATE INDEXED
```

Así se conecta el canal 1 al archivo indexado existente para actualizarlo. No crea su índice ni define sus tamaños de registro y de clave. Sigue el capítulo 17 para la creación, la escritura y la lectura por clave, y cierra el canal antes de reutilizarlo.

KEYED

KEYED selecciona el acceso por clave sin el área de índice ordenado del formato I. Una ilustración de sintaxis para un archivo existente con formato CLAVE es:

```
OPEN #1: "CLAVES.DATOS:S", UPDATE KEYED
```

La ilustración requiere que CLAVES.DATOS se haya creado con la organización con clave adecuada y con las dimensiones de registro y clave correspondientes. No es intercambiable con la apertura de un archivo indexado. El capítulo 17 distingue las organizaciones; la creación detallada y la disposición de los registros deben coincidir con el archivo que realmente se abre.

NOT

NOT es la operación unaria de negación lógica en el flujo de expresiones compiladas, representada por el byte 0x0F. Actúa sobre una expresión, no introduce una instrucción. La conversión numérica, los efectos de la anchura de los enteros y las reglas completas de precedencia siguen sin especificarse; aquí no se promete ningún resultado numérico concreto.

AND

AND combina dos operandos en una expresión; el byte compilado 0x10 selecciona su operación. En el plano conceptual, la conjunción exige que se cumplan ambas condiciones. El contrato completo de codificación de los valores lógicos, de conversión y de orden de evaluación sigue sin especificarse. No des por supuesto que se omite el segundo operando cuando el primero es falso.

OR

OR combina dos operandos en una expresión; el byte compilado 0x11 selecciona su operación. En el plano conceptual, la disyunción permite que se cumpla cualquiera de las dos condiciones. El contrato numérico completo y el de orden de evaluación siguen sin especificarse. Su marcador de expresión es distinto de cualquier palabra clave de CSI o de CONSULTA. El apéndice A sitúa estas operaciones dentro del formato compilado.

QUOTE

QUOTE es una palabra clave de opción. El diálogo de salida de TRAFIC describe el uso de OPTION QUOTE al abrir una salida de tipo PROCETEX. Es un uso concreto de una aplicación, no una especificación completa del entrecomillado, del escape ni de los delimitadores de registro. No des por supuesto que añadir QUOTE crea un formato de texto intercambiable universalmente; su sintaxis y su alcance completos siguen sin especificarse.

RECORDSIZE

RECORDSIZE se reconoce en la tabla de opciones y funciones de BASIC. Su contexto permitido, la forma de sus argumentos, sus unidades y el comportamiento del resultado aún no están especificados. No debe sustituir a la opción REGISTRO de CREAR solo porque los nombres describan conceptos relacionados.

FILESIZE

FILESIZE es una palabra clave reconocida del lenguaje cuyos argumentos y unidades siguen sin especificarse. Todavía no puede documentarse como función de recuento de bytes ni como ajuste de capacidad de registros. Las distinciones de tamaño del apéndice C explican por qué importan esas alternativas.

KEYSIZE

KEYSIZE es una palabra clave reconocida del lenguaje. Su relación completa con un canal abierto o con una petición de creación de archivos sigue sin especificarse; usa la sintaxis establecida CREAR LONCLA para definir una longitud de clave en CSI.

XOR

XOR aparece en el vocabulario de expresiones y operadores. Los tipos de sus operandos, su representación numérica y su comportamiento de evaluación siguen sin especificarse. La grafía por sí sola no basta para importar un ejemplo a nivel de bits o booleano de otro BASIC.

IMP

IMP aparece junto a los nombres de operadores de expresión XOR y EQV. Su contrato completo de expresión sigue sin especificarse. Esta palabra clave de BASIC debe distinguirse del nombre de impresora física IMP y de las opciones de salida de grafía similar de otras utilidades.

EQV

EQV pertenece al mismo vocabulario de expresiones y operadores. Su representación de los valores lógicos, la conversión de los operandos y la precedencia siguen sin especificarse. No es un sustituto documentado de una comparación de igualdad.

DEVICE

DEVICE es una palabra clave reconocida del lenguaje. Su gramática de argumentos y su relación con OPEN o con la información de dispositivos siguen sin especificarse. Los nombres de dispositivo usados en una especificación de archivo de OPEN no establecen la firma de una función DEVICE.

MEMORY

MEMORY es una palabra clave reconocida del lenguaje cuyo alcance y argumentos siguen sin especificarse. Es independiente del ajuste de gestión de particiones PONER MEMORIA de CSI. No la trates como un asignador documentado ni como una operación de acceso directo a memoria.

PORT

PORT es una palabra clave reconocida del lenguaje. Aquí no se especifica ningún contrato completo de privilegio, argumentos o acceso al hardware. Su nombre por sí solo no establece una interfaz de BASIC segura para los puertos de E/S enumerados en el apéndice B.

TAB

TAB es una palabra clave reconocida del lenguaje. Su contexto permitido de expresión o de salida, la base de su argumento y su comportamiento de posicionamiento siguen sin especificarse. Su presencia no justifica importar la sintaxis PRINT TAB de otro dialecto.

UPCASE

UPCASE es una palabra clave reconocida del lenguaje, pero el tipo de su argumento y de su resultado y el alcance de la conversión a mayúsculas siguen sin especificarse. Debe distinguirse del modo de entrada CASE U de EDITAR y de la opción de conversión de archivos MAYUSC de COPIFI.

Apéndice E. Atlas de estructuras binarias

Esta es la referencia canónica de disposiciones. Los desplazamientos de estas tablas están en hexadecimal, salvo que se indique explícitamente una cantidad decimal. Las palabras de varios bytes son little-endian, salvo la marca de tiempo de tres bytes, los campos de BASIC y los campos marcados con BE. Un campo marcado como desconocido debe conservarse, no ponerse a cero, durante una recuperación. Las tablas del sistema de archivos usan una geometría de ejemplo de 151 sectores de directorio que empiezan en el sector 9; los valores reales de la cabecera determinan los límites de otro volumen. Las disposiciones del contexto y de los controladores describen la interfaz del sistema.

Geometría y cabecera del volumen

La correspondencia física en bruto se define en el capítulo 27. Con esa geometría, el sector 1 inicia la región de cabecera y mapa de bits, los sectores 9 a 159 contienen 1208 registros de directorio y el sector 160 inicia la asignación de archivos. La etiqueta de la cabecera no es una firma mágica. Un lector que incluya incondicionalmente el sector 160 en el directorio fabrica entradas falsas a partir de datos.

Desplazamiento	Tamaño	Significado	Notas
00	8	Etiqueta de volumen rellenada con espacios	—
08	3	Sin resolver; consérvalo tal cual	Desconocido
0B	8	Etiqueta de la cinta de archivado	Probable; ESTADO muestra bytes coincidentes
13	4	Metadatos relacionados con el archivado o la fecha	Provisional; empaquetado exacto incompleto
17	2	Nibble alto del byte 17 = cabezas lógicas; nibble bajo seguido del byte 18 = número de cilindros de 12 bits	núcleo 0AE2–0AF8 y ESTADO 09BC–0A37; 17 93 = 1 cabeza, 1939 cilindros
19	1	Sectores por pista (20h=32)	cargador A0D6
1A	1	Sectores de directorio (97h=151)	nativo: 1208 entradas / ocho por sector
1B	2	Sin resolver	Desconocido
1D	2	Bloques de asignación libres, little-endian	—
1F	1	Los seis bits bajos más 2 dan el primer sector del directorio	cargador A0DC–A0E7
Desde 20	variable	Mapa de bits de asignación con el bit más significativo primero	comportamiento de asignación y borrado

Registro de directorio

Cada registro ocupa 32 bytes, así que un sector de 256 bytes contiene exactamente ocho. El formato 00 es un terminador temprano de la búsqueda; FF es un hueco borrado. El formato activo es distinto de los bits de protección.

Desplazamiento	Tamaño	Significado	Notas
00	1	Bits de formato y atributos; FF marca un hueco sin usar o borrado	correspondencia básica de formatos; demás indicadores incompletos
01	8	Nombre de archivo rellenado con espacios	—
09	8	Tipo de archivo rellenado con espacios	—
11	2	Número de registros; número de sectores en A/R; número de páginas en los informes SP$ creados por el SPOOLER	comparaciones nativas y cierre original del SPOOLER 3E0C–3E15; véase el apartado del SPOOLER
13	2	Bloques de 1 KiB asignados	—
15	2	Primer sector lógico	cargador, SECTOR y copia
17	2	Longitud máxima de escritura contada para la salida S especificada; longitud de registro en D; campo de longitud en A/R; longitudes empaquetadas de registro y clave en I/K	determinados casos nativos y de código; el valor de S no es necesariamente la longitud completa de un registro BASIC
19	3	Fecha y hora empaquetadas, campo de bits big-endian	—
1C	1	Número de la cuenta propietaria; cuenta destinataria en un alias de uso compartido; espacio de nombres especial FF	cambios de propiedad nativos, búsqueda y controles de uso compartido; véanse más abajo las disposiciones del alias y del buzón
1D	1	Cuenta de destino distinta de cero para una redirección de búsqueda de uso compartido	concesión y revocación nativas de COMPART y acceso resuelto; véase más abajo la disposición del alias
1E	2	Bytes de la imagen de carga en R; último sector enlazado en S; dirección de carga en A; capacidad de huecos en I/K	R/S en los archivos especificados; A según el cargador

Valor de formato	Letra nativa	Interpretación del almacenamiento
0x01	R	Ejecutable reubicable
0x02	A	Ejecutable absoluto
0x04	S	Datos secuenciales enlazados
0x08	D	Registros directos de longitud fija
0x10	I	Huecos indexados ordenados

Valor de formato	Letra nativa	Interpretación del almacenamiento
0x18	K	Huecos con clave por dispersión
0x20	Protección contra lectura	Atributo, no compresión
0x40	Protección contra borrado	Atributo
0x80	Protección contra escritura	Atributo; exactamente 0x80 puede ser un alias de uso compartido

Para la marca de tiempo big-endian de tres bytes D: mes=`(D>>20)&15`, día=`(D>>15)&31`, año=`1977+((D>>11)&15)`, hora=`(D>>6)&31`, minuto=`D&63`. No hay campo de segundos. Su intervalo de años es 1977–1992. No amplíes su intervalo suponiendo una época moderna.

Alias de uso compartido

Desplazamiento	Tamaño	Significado específico del alias
00	1	80h en el alias de uso compartido especificado
01	8	El mismo nombre de archivo que el destino
09	8	El mismo tipo que el destino
11–1B	11	Cero en el alias creado; sin recuento de registros, extensión ni marca de tiempo propios
1C	1	Número de la cuenta destinataria
1D	1	Número de la cuenta propietaria del destino
1E	2	Cero en el alias creado

Estructuras de datos

Formato	Regla a nivel de bytes
Sector S	+0x00..+0xFD datos de 254 bytes; +0xFE LE16 LBA siguiente; un enlace cero termina
Registro D n	Empieza en el byte `(n-1)*R` desde el inicio del archivo; sectores completos de 256 bytes; n empieza en uno
Prefijo I	N números de hueco LE16; rellena el prefijo hasta `B=ceil(2*N/1024)*1024`
Hueco I s	En B+(s-1)*(K+R+2): clave de K bytes, datos de R bytes, hueco siguiente LE16
Hueco K s	En (s-1)*(K+R): clave de K bytes seguida de datos de R bytes

En I/K, la palabra del directorio en +0x17 empaqueta `R=(word&0x1FF)+1` y `K=(word>>9)+1`; la palabra en +0x1E es la capacidad de huecos N. Los bloques de asignación de I son `ceil(2*N/1024)+ceil(N*(K+R+2)/1024)`. Las primeras C entradas del prefijo, con C tomado de directorio+0x11, identifican los huecos activos ordenados por clave. K usa el algoritmo de dispersión y sondeo del capítulo 26. Un primer byte de clave cero termina un sondeo sin usar; FF marca una lápida.

Los registros de reubicación R y las disposiciones de los objetos BASIC aparecen en el apéndice A. El espacio sobrante de la asignación y el relleno secuencial no son longitudes de datos de las aplicaciones.

Contexto de partición

La base del contexto es el puntero actual en 0x0033 o el puntero de una fila del planificador. Su área fija de 0x4CC bytes precede al programa. La tabla siguiente incluye deliberadamente regiones desconocidas.

Desplazamiento	Bytes	Significado	Notas
0x00	2	SP guardado	—
0x02	2	Base del contexto	—
0x04	2	Límite superior de memoria	—
0x06	2	Extremo asignado; cero representa 0x10000	—
0x08	2	Bytes de cabecera sin clasificar	Desconocido
0x0A	1	Política de la consola; bit 1 supresión, bit 6 modo de conversión/borrado	—
0x0B	1	bit 3 pausa; bit 4 interrupción diferida; bit 5 evento/cancelación	—
0x0C	1	Política de consola, fecha y búsqueda; bit 5 versión, bit 6 inhibir la búsqueda pública, bit 7 inhibición de mensajes	—
0x0D–0x0E	2	Coordenadas del cursor registradas	—
0x0F	1	Sin clasificar	Desconocido
0x10	1	bit 1 posibilidad de estado conectado; bit 2 IMPEXEC	—
0x11	1	Privilegio	—
0x12–0x19	8	Nombre de cuenta	—
0x1A	1	Número de cuenta	—
0x1B	1	Capacidad de niveles de EXEC	—
0x1C	2	Inicio del archivo de trabajo de EXEC	—
0x1E–0x25	8	Otro estado de ejecución	Clasificado de forma incompleta
0x26	2	Devolución de llamada de error de disco	—
0x28	2	Devolución de llamada de interrupción y control	—
0x2A–0x36	13	Estado conservado por los marcos de EXEC	Subcampos opacos
0x37	1	Nivel de EXEC	—
0x38	1	Indicadores de continuación de EXEC y de comandos	—

Desplazamiento	Bytes	Significado	Notas
0x39	3	Gancho RST38 en línea; C9 lo desactiva	—
0x3C	1	bit 5 estado de salida de escape	—
0x3D–0x67	43	Controles y estado; byte de escape en +0x40	Clasificado en parte
0x68	1	Último resultado de comando	—
0x69–0x7C	20	Otro estado del contexto	Incompleto
0x7D–0x94	24	Mapa de asociaciones	—
0x95–0xAB	23	Resto de la cabecera	Incompleto
0xAC–0x2CB	544	Diecisiete canales de 32 bytes	—
0x2CC–0x3CB	256	Área de trabajo de sector	—
0x3CC–0x4CB	256	Búfer de comandos	—
Desde 0x4CC	variable	Programa	—

Marco de ejecución guardado

En el SP guardado, las palabras son HL′, DE′, BC′, AF′, IY, IX, HL, DE, BC, AF y el PC de continuación, en ese orden, en los desplazamientos 0x00, 02, 04, 06, 08, 0A, 0C, 0E, 10, 12 y 14. F es el byte bajo de cada palabra AF. Son 22 bytes; pueden seguir más marcos. Aquí no se establece ningún campo I/R/IFF ni de modo de interrupción. La entrada de una devolución de llamada deja la continuación en SP=SP guardado+0x14 para el RET.

Petición de canal de quien llama

Desplazamiento	Tamaño	Significado	Notas
00	1	Índice del canal, intervalo aceptado 0..16	núcleo 34D9
01	1	Indicadores de operación y método; 38h usado por UPDATE INDEXED de BASIC	valor observado; bits completos parciales
02	2	Puntero little-endian al área de trabajo de sector o índice propiedad de quien llama	superposición 1:4692 y llamadas de E/S

Canal indexado interno

Desplazamiento	Tamaño	Significado en archivos indexados	Notas
00	1	Índice de la unidad asociada	se pasa como B a los servicios de disco
01	8	Nombre de archivo rellenado con espacios	nombre copiado y memoria activa
09	8	Tipo de archivo rellenado con espacios	—
11	1	Selector de dispositivo o clase; 01 para el archivo de disco especificado	Enumeración completa de clases desconocida
12	1	Indicadores de estado del canal	Significados parciales de los bits más abajo
13	2	Longitudes empaquetadas de registro y clave, como en directorio+17	superposición 2:461F
15	2	Capacidad de huecos N, como en directorio+1E	superposición 2:462B
17	2	Sector lógico inicial del área de índice	superposición 2:4642
19	2	Sector lógico inicial del área de registros	superposición 2:4637–4652
1B	2	Número de registros activos	superposición 2:4610 y borrado 4C05
1D	2	Siguiente hueco para el recorrido secuencial	READ/READNEXT indexados
1F	1	Selector de unidad original guardado durante la búsqueda de OPEN	almacenamiento en la superposición 2:42FD; duración completa parcial

Cola del canal secuencial

Los desplazamientos siguientes sustituyen a los significados indexados; son una unión, no campos adicionales.

Desplazamiento	Tamaño	Significado observado
13	2	Puntero al siguiente byte del área de trabajo de quien llama
15	1	Bytes de datos sin usar que quedan en el sector actual
16	2	Sector lógico actual en el búfer
18	1	Sectores que quedan en el bloque actual, incluido el actual
19	2	Número de escrituras contadas correctas
1B	2	Número de bloques de 1 KiB asignados
1D	2	Longitud máxima de escritura contada

Estructuras de sincronización y de devoluciones de llamada

Estructura	Campos	Límites e interpretación
Fila del planificador	+0 contexto LE16; +2 banco/estado	16 filas inicializadas de 3 bytes; nibble bajo: banco; alto: bits de espera/parada
Bloqueo de registros	+0 dispositivo (3 bits bajos)/canal (5 bits altos); +1 primer LBA LE16; +3 último LBA LE16; +5 propietario	Propietario=índice+1; el bit 7 termina; 64 filas utilizables según el código y la disposición
Fila de archivo abierto	+0 dispositivo físico; +1 primer LBA LE16; +3 propietario/exclusivo	Propietario=índice+1; bit 7 exclusivo; sin campo de canal
Bloqueo de byte	byte de propietario	0 libre; índice+1 con propietario; sin recuento de recursión
Nodo de temporizador	+0 diferencial LE16; +2 desconocido; +3 propietario siguiente; +4 puntero siguiente LE16; +6 código	Lista enlazada de diferencias; al vencer, se borran los 6 primeros bytes
Registro de devoluciones de llamada	+0 propietario; +1 puntero LE16	Fila en 0x23FF+3*índice; índices 0x02–0x20
Componente con nombre	+0 clave de 8; +8 dirección LE16	Clave binaria exacta; un primer byte 00 o espacio termina la tabla
Cola en memoria	+0 capacidad; +1 ocupación; +2 índice de lectura; +3 datos	Defectos de la aritmética de ocho bits; véase el capítulo 25

SPOOLER y buzón

Estructura	Desplazamiento	Bytes	Significado
Ranura del recolector	0x00	1	Indicadores
Ranura del recolector	0x01	1	Bytes libres del búfer
Ranura del recolector	0x02	2	Puntero al búfer
Ranura del recolector	0x04	2	LBA actual
Ranura del recolector	0x06	2	Páginas
Ranura del recolector	0x08	2	Bloques
Ranura del recolector	0x0A	2	Número de cola
Ranura del recolector	0x0C	1	Selección de cola
Ranura del recolector	0x0D	1	Indicadores/copias
Ranura del recolector	0x0E	1	Estado de línea
Ranura del recolector	0x0F	1	Desconocido
Registro de cola	0x00	8	Etiqueta del informe
Registro de cola	0x08	1	Índice+1 de la partición que lo envía
Registro de cola	0x09	3	Marca de tiempo empaquetada
Registro de cola	0x0C	1	Indicadores de estado, conservación, copias e impresora
Registro de cola	0x0D	1	Letra de la cola
Registro de cola	0x0E	2	Número de páginas
Cabecera de correo	0x00	8	Nombre exacto del destinatario
Cabecera de correo	0x08	8	Nombre del emisor
Cabecera de correo	0x10	8	Texto de la fecha
Cabecera de correo	0x18	8	Texto de la hora
Cabecera de correo	0x20	1	CR

Los registros de cola del SPOOLER y las ranuras del recolector tienen ambos 16 bytes, pero disposiciones distintas. Los cuerpos de los mensajes usan líneas terminadas en CR y relleno de límite 0x7F; los sectores S que los contienen siguen llevando 254 bytes de datos y enlaces de dos bytes. Ninguno de estos formatos sustituye a la estructura del sistema de archivos.

Uniones de los descriptores físicos y almacenamiento privado de los controladores

Un descriptor físico tiene 32 bytes y empieza en 0x0069+32*índice. Las tablas siguientes describen los campos identificados para la familia indicada; los huecos siguen sin clasificar. El campo de geometría y espacio libre de un disco no debe descodificarse con la interpretación de longitud de clase de un dispositivo de caracteres.

Descriptor de terminal

Desplazamiento	Tamaño	Función documentada
00	2 LE	Entrada del controlador DIS cargado
02/03	1 each	Umbral horizontal de salto de línea/presupuesto de entrada y límite de filas para LF; nativo L79/P23, bytes 4F/17
04	1	Número de clase, 1 o 2
05 bit 7	1 bit	Habilitar el filtrado de la entrada
05 bit 6	1 bit	Prefijo de escape pendiente
0E	2 LE	Longitud cargada del controlador, aquí 0377
10	2 LE	Longitud cargada de la clase, 0103/0148
16	2 LE	Entrada de la clase cargada; entrada de datos +0, salida +3

Descriptor de impresora

Desplazamiento en el descriptor	Tamaño	Significado / pruebas	Notas
00	2 LE	Base del controlador cargado	—
02 / 03	1 each	Anchura de línea / longitud de página configuradas; nativo 84/3C	—
04	1	Número de clase de terminal o impresora; cero en las rutas especificadas	—
05 nibble bajo	4 bits	Selector de la tabla de velocidades de IMP	—
06 / 07	1 each	Selectores de retardo de CR/LF y de salto de página en 1848–185E	código; cero en nativo
08 bits 7/6	2 bits	Selección de paridad serie que usa DIS22; PO produjo 10b y PE, 11b	código y opciones

Desplazamiento en el descriptor	Tamaño	Significado / pruebas	Notas
08 bit 5	1 bit	Indicador de tamaño de palabra; W8 nativo lo activa	—
08 bits 1/2/3	3 bits	Recuento común de líneas por página, política de LF automático y tratamiento de la paridad de página	código; combinaciones sin especificar
09	1	Se suma a la longitud de página configurada en la inicialización	—
0A	1	Contador modificable de líneas restantes en la página	código; 3C inicial en nativo
0B	1	Último byte de salida o control registrado por el envoltorio común	—
0C	1	Longitud de página inicial más 09; el bit 7 se conmuta en los límites de página	código y determinados cambios de FF
0E	2 LE	Longitud del controlador cargado	—

Anillo de entrada de ALP

Desplazamiento relativo al controlador	Tamaño	Significado
0240	1	Capacidad 50h, 80 bytes
0241	1	Ocupación
0242	1	Índice de lectura
0243–0292	80	Datos

Almacenamiento de salida de ALP

Desplazamiento relativo al controlador	Tamaño	Significado
0293–02E5	83	Primer búfer de salida
02E6–0338	83	Segundo búfer de salida
0339	1	Selector de búfer, alterna 00/FF
033A	2 LE	Puntero actual al búfer de adición
033C	2 LE	Puntero al descriptor físico
033E	2 LE	Número de bytes en el búfer

Campos de inicialización de SS2

Desplazamiento	Tamaño	Significado	Notas
01CC	32	Dieciséis valores de divisor LE16 indexados por el nibble bajo de 05 del descriptor	—
01EC	1	Número de pares de registros de inicialización: 0A	—
01ED	20	Secuencia de números de registro y valores	—
01F2 /01F4	1 each	Bytes bajo y alto del divisor seleccionado en esa secuencia	—
01F8	1	Valor del registro 4; bits de paridad derivados de los bits 7/6 de 08 del descriptor	—
01FA	1	Valor en caché del registro 5; el bit 6 sigue al bit 5 de 08 del descriptor; el umbral de recepción borra el bit 7	—
01FC	1	Valor en caché del registro 3; el bit 7 sigue al bit 5 de 08 del descriptor	—
0201	2 LE	Dirección del descriptor físico recordada para la devolución de llamada de recepción	—

Anillo y devoluciones de llamada guardadas de SS2

Desplazamiento	Tamaño	Significado
0203	1	Capacidad 80h, 128 bytes; constante original
0204	1	Ocupación, 0..128 en estado válido
0205	1	Índice de lectura, 0..127
0206–0285	128	Huecos de datos
0286	2 LE	Contexto+28 guardado de la ruta de estado
0288	2 LE	Contexto+28 guardado de la ruta de salida

Estado privado de los disquetes

Desplazamiento en DIS1	Desplazamiento en DIS2	Tamaño	Significado
024A	022C	2	BC guardado
024C	022E	2	DE guardado
024E	0230	2	HL guardado
0250	0232	3	Tres primeros bytes de resultado de la controladora
0253	0235	9	Paquete de comando / parámetros modificables
025C	023E	1	C3 original; función: desconocida

Estado modificable de la cinta

Desplazamiento	Tamaño	Función / notas
093F	2	Operando JP automodificado hacia +0F del controlador seleccionado; documentado en nativo
0A73	2	Contador de avance de registros/bloques; código original
0A75	80	Búfer de volumen y de lectura de etiquetas; código original
0AC5	80	Plantilla/búfer de la primera etiqueta de archivo
0B15	80	Plantilla/búfer de la segunda etiqueta de archivo
0B67	2	Valor de progresión de las secciones de etiquetas; interpretación parcial
0B93	2	Contador de reintentos, que imprime la devolución de llamada
0B95	1	Recuento de control de marcas y reintentos, inicializado a 2
0B96/0B97	1 each	Primer y segundo estado en bruto; determinados valores nativos documentados
0B98	1	Indicador de finalización distinto de cero; inicializado a 1 en la imagen original
0B99	1	Selector actual 0–3; documentado en nativo
0B9A	2	Dirección del búfer de registros
0B9C	1	Indicador de la ruta corta/síncrona con D=0
0B9D	1	Indicador de devolución de llamada de escritura pendiente
0B9E	1	Estado del canal de lectura
0B9F	1	Indicador de devolución de llamada de lectura pendiente
0BA0	1	Cuenta atrás de reintentos de lectura
0BA1/0BA2	1 each	Indicadores de inicialización del nombre de comando de escritura/lectura
0BA3	1	Byte inicial del nombre de comando en caché

Glosario

Las definiciones usan las distinciones propias del sistema. Los números de referencia identifican capítulos; las letras identifican apéndices. El capítulo 31 define los nombres de comandos y las opciones, el apéndice D define los comandos y las instrucciones de BASIC, y el apéndice C explica cada archivo del sistema. Este glosario aporta los conceptos compartidos, las abreviaturas y la terminología de dispositivos necesarios para leer esas referencias.

ABI — Interfaz binaria de aplicación (application binary interface): el acuerdo a nivel de máquina sobre registros, disposiciones de memoria, puntos de entrada y retornos. La ABI de un controlador no tiene por qué ser la misma que la de los servicios del sistema. Véase 19, 24, 27, 28.

ABS / REL / COM — Etiquetas heredadas de objetos o de cadenas de herramientas que aparecen en el vocabulario de diagnóstico. Su presencia no establece un formato de objeto admitido ni un procedimiento completo de ensamblador y enlazador. COM también debe distinguirse de los nombres de dispositivo físico COM1–COM4. Véase 18, 33; A.

acarreo — Indicador del Z80 que algunas interfaces usan para indicar error o vacío, y otras para la disponibilidad de un dispositivo. No tiene un significado universal en todo el sistema operativo. Véase 19, 32.

AF — Par de registros del Z80 que combina el acumulador A y los indicadores F. Un servicio puede devolver un valor en A y una condición aparte en un indicador. La notación AF′ designa el par alternativo. Véase 19, 24, 32; E.

alias de COMPART — Redirección del directorio que concede a una cuenta destinataria acceso al archivo privado de otra cuenta sin transferir la propiedad. Véase 10, 22; E.

ALP — Interfaz de terminales representada por canales SCC, controladoras de DMA y puertos de control de flujo. En esta arquitectura se establecen cuatro consolas. Véase 28; B.

apertura exclusiva — Política de admisión en la tabla de archivos abiertos. Sobrevive a UNLOCK y no impide de forma fiable el borrado nativo. Véase 17, 25.

API — Interfaz de programación de aplicaciones (application programming interface): las operaciones y convenciones de datos que se exponen al software de aplicación. Aquí, los servicios del sistema, las funciones de BASIC y las interfaces de dispositivos tienen cada uno sus propios contratos. Véase 17, 19, 32.

archivo público — Archivo de la cuenta 0 que tiene en cuenta la búsqueda alternativa, salvo que esté inhibida. La cuenta especial FF es distinta. Véase 5, 10, 26.

área de trabajo — Almacenamiento temporal propiedad de quien llama o del contexto, que usa un servicio; deben indicarse su propietario y su duración. Véase 17, 24; E.

ASCII — American Standard Code for Information Interchange, una codificación de caracteres usada para letras, dígitos y caracteres de control. El valor de un carácter y la acción del terminal asignada a él son cuestiones distintas. Véase 4, 19, 28.

ASI — Etiqueta de un diagnóstico del firmware cuya omisión controla un bit de configuración. Aquí no se establecen su desarrollo completo ni su alcance en el hardware. Véase B.

asociación — Correspondencia, local de cada partición, entre un uso lógico de dispositivo y un descriptor físico. Véase 12, 23, 34.

banco — Región de RAM de respaldo de 48 KiB que se selecciona en las direcciones 4000–FFFF de la CPU. Un índice del planificador no tiene por qué coincidir con un número de banco. Véase 20, 22, 25.

BASIC — El lenguaje de programación del sistema y su entorno de desarrollo integrado, identificado como BASIC S20. Ofrece edición del código fuente, interpretación, compilación y ejecución. BASICOBJ, BASICCOM y BASICUSR designan representaciones distintas que usa ese entorno. Véase 15–17; D.

BASICCOM — Flujo compilado de instrucciones y expresiones BASIC que interpreta el entorno de ejecución instalado; no es un ejecutable R. Véase 15, 16; A.

BASICOBJ — Código fuente BASIC tokenizado y guardado, pese a que el nombre del tipo sugiera un objeto. Véase 15; A.

BASICUSR — Extensión en código máquina de formato R seleccionada con OPTION USR de BASIC. Véase 19; A.

BC — Par de registros del Z80 formado por B y C. Las interfaces lo usan para recuentos, selectores y otros valores; su significado se especifica por separado en cada llamada. Véase 19, 32; E.

BCD — Decimal codificado en binario: cada nibble representa un dígito decimal. Se usa en el estado del reloj y en los datos decimales de BASIC. Véase 19; A, B.

BE — Orden de bytes big-endian: el byte más significativo de un valor de varios bytes va primero. Varios campos de BASIC y la marca de tiempo empaquetada del directorio usan este orden, aunque las palabras normales del Z80 sean little-endian. Véase A, E.

bloque de asignación — Cuatro sectores de 256 bytes, en total 1024 bytes. El mapa de bits del volumen lleva la cuenta de la asignación. Véase 23, 26; E.

bloqueo de registros — Tramo de sectores con extremos incluidos, con propietario y asociado a un canal; no es una entrada de apertura exclusiva de archivo. Véase 17, 25; E.

BUZON — Utilidad de recuperación del buzón; también es el contexto de tipo y nombre del buzón persistente secuencial. La recuperación elimina los mensajes. Véase 9, 29, 31.

byte de formato — Campo del directorio que combina la organización nativa con los atributos de protección. Véase 23; E.

canal — Uno de los diecisiete registros de archivo o dispositivo de cada partición, indexados de 0 a 16 en la interfaz del núcleo. Véase 17, 24; E.

carácter CR — Retorno de carro, 0x0D. Es distinto de la abreviatura CR del resultado de comando. Véase 4, 33.

cargador de arranque (bootstrap) — Cargador en disco que lee el firmware antes del ejecutivo completo. Las coordenadas físicas del cargador de arranque son distintas de las direcciones lógicas del volumen. Véase 3, 21, 27.

catálogo — Los registros fijos del directorio del volumen. El sondeo por nombre exacto y la enumeración con comodines lo recorren de forma distinta. Véase 5, 23, 26.

CDC — Definición de impresora física atendida por SISTEMA.DIS12. Es distinta de los nombres de impresora lógica como IMPR1. Véase 12, 28, 34; B.

cesión del control — Entrada voluntaria en la planificación mediante 4F. La apropiación por el temporizador también existe de forma independiente. Véase 22, 25.

CHS — Coordenadas de cilindro, cabeza y sector (cylinder, head, sector) de un disco físico. La interfaz de disco duro usa identificadores de sector que empiezan en cero. Véase 27.

CLASE — Programa de clase de terminal que traduce los controles lógicos de la consola. Es independiente del controlador físico DIS. Véase 4, 28.

clase de terminal — Véase CLASE. Véase 28.

COM1–COM4 — Definiciones de dispositivos físicos de comunicaciones. Son distintas de los nombres COMU locales de cada partición y de los tipos de archivo. Una definición no establece que estén disponibles el hardware necesario y un comportamiento completo del controlador. Véase 34; B.

COMANDO — Tipo de archivo que usan la mayoría de las utilidades de CSI; el formato ejecutable nativo se registra de forma independiente en el directorio. Véase 7, 30, 31.

COMU — Familia de nombres lógicos de dispositivos de comunicaciones, como COMU1, que se resuelven mediante asociaciones. No los sustituyas por un nombre físico como COM1 sin establecer su asociación. Véase 23, 34.

CONSOLA — Nombre lógico de la consola usado en la sintaxis de los comandos. La entrada y la salida se resuelven mediante asociaciones de consola y no tienen por qué tratarse como el nombre de un modelo de terminal físico. Véase 12, 23, 34.

contexto — La cabecera de la partición, los canales y las áreas de trabajo, junto con los punteros a su estado de ejecución. Véase 24, 25; E.

CONTROL.VOC — Diccionario indexado de atributos que usa el software de apoyo de consultas y vocabulario; es distinto de la directiva CONTROL de EXEC. Véase 35; C.

CPU — Unidad central de proceso (central processing unit): el procesador que ejecuta las instrucciones. El Z80 tiene una ventana de direcciones de 64 KiB; la selección de banco cambia qué memoria es visible a través de parte de esa ventana. Véase 2, 20, 22.

CR — Aparecen dos abreviaturas distintas. En la E/S de caracteres, CR significa retorno de carro (carriage return), el carácter 0x0D, que envía Intro para enviar una línea de comandos. En los mensajes de finalización de los comandos, CR significa resultado del comando, un estado numérico. Véanse carácter CR y resultado de comando; 4, 9, 33.

CRT — Tubo de rayos catódicos (cathode-ray tube), la tecnología de pantalla del terminal IS 9502. CRT también aparece como palabra de EXEC especificada de forma incompleta, y CRT$ es una función de cadena de control del terminal de BASIC cuyo vocabulario de controles completo no está especificado. Véase 4, 8; D.

CSI — El procesador de comandos: el programa que muestra el indicador del sistema >, lee las líneas de comandos, busca los comandos e inicia su ejecución. De manera informal, también se llama «indicador del sistema» a este entorno de comandos; en sentido estricto, el indicador es el carácter visible >, mientras que CSI es el programa que hay detrás. Su archivo del sistema es SISTEMA.CSI. La versión 5.5 de CSI es distinta de la versión 6.2 del sistema operativo. Véase 1, 5, 30; C.

cuenta — Identidad de usuario con nombre, con número de cuenta y privilegio. Es distinta de una partición planificada. Véase 9, 10, 22.

DE — Par de registros del Z80 formado por D y E. Muchos servicios usan DE para un puntero o un valor, pero no tiene una función universal como argumento. Véase 19, 32; E.

definición de captura — Descripción de pantalla, campos y acceso a archivos que usan las herramientas de apoyo opcionales ACTUCAP y EXPAND para preparar un programa de entrada de datos. Véase C.

DEL — Carácter de borrado (delete), 0x7F. CSI acepta Supr para eliminar el carácter de entrada anterior; otros programas pueden aplicar reglas de entrada distintas. Véase 4, 28.

descriptor — Registro de dispositivo físico de 32 bytes cuyos campos dependen de la familia del dispositivo. Véase 23, 28; E, B.

devolución de llamada (callback) — Rutina en la que se entra cuando se produce un evento, a veces en el contexto guardado de otra partición. No es necesariamente un nuevo hilo de ejecución. Véase 19, 25, 28.

DIRAL — Columna de ESTADO MEMORIA que muestra el límite superior de memoria actual menos uno, tomado del desplazamiento 0x04 del contexto. Los controladores o clases cargados pueden reducir este límite dentro de la región asignada. Véase 12, 20; E.

DIRBA — Columna de ESTADO MEMORIA que muestra la dirección base de la partición, tomada del desplazamiento 0x02 del contexto. Es una dirección de memoria, no un número de directorio de disco. Véase 12; E.

DIRECT — Organización de archivo de BASIC que usa registros de longitud fija seleccionados por número de registro. Corresponde al almacenamiento nativo de formato D. Véase formato D; 17, 26; D, E.

DIRMX — Columna de ESTADO MEMORIA que muestra la dirección final de la región asignada a la partición: la palabra del desplazamiento 0x06 del contexto menos uno. Puede superar el límite superior utilizable actual que muestra DIRAL. Véase 12; E.

DIS — Familia de archivos de controladores del sistema, por ejemplo SISTEMA.DIS10 para la cinta y SISTEMA.DIS18 para PAN1. El sufijo numérico selecciona una definición de controlador; no es un número de llamada al sistema. Véase 23, 28, 34; B, C.

DISCO1–DISCO6 — Definiciones de discos físicos. Las letras lógicas S y A–G adquieren su significado físico mediante asociaciones con definiciones como estas. Véase 11, 23, 34.

dispositivo físico — Extremo de hardware y controlador seleccionado por un descriptor; puede existir una definición sin que el hardware esté instalado. Véase 23, 28, 34.

dispositivo lógico — Un uso local de la partición, como S, la entrada de consola o IMPR1, que se resuelve mediante una asociación. Véase 12, 23, 34.

DMA — Acceso directo a memoria (direct memory access) usado en la salida establecida de ALP, con básculas de banco por par de canales independientes de la selección de banco de la CPU. Véase 28; B.

DRQ — Petición de datos (data request): señal de estado de la controladora de disco que se usa para coordinar las transferencias a través del registro de datos. Consulta el mapa de la controladora para ver el bit de estado y el sentido correspondientes. Véase 27; B.

DTR / REQ — Señales de terminal de datos preparado (data terminal ready) y de petición (request) mencionadas en la interfaz serie y de DMA. Se refieren al protocolo de intercambio del hardware, no a si una cuenta tiene la sesión iniciada. Véase 28; B.

EBCDIC — Extended Binary Coded Decimal Interchange Code, la codificación de caracteres que usan las utilidades de intercambio IBM 3741. SECIBM convierte desde ASCII el texto secuencial saliente; IBMSEC realiza la conversión inversa. Véase 11, 31.

entrada de directorio — Registro de 32 bytes que contiene el nombre de archivo, el tipo, el formato, la asignación, la marca de tiempo y la propiedad. Véase 26; E.

EOF — Condición de fin de archivo (end of file). El agotamiento de la cadena en el núcleo, una marca secuencial 1A y la función EOF de BASIC son capas distintas. Véase 17, 26, 33.

EOV — Prefijo de etiqueta de cinta que se usa al formar las etiquetas finales de fin de volumen (end of volume); EOF se usa en las etiquetas correspondientes de fin de archivo. Estos prefijos de etiqueta son distintos de la condición de retorno EOF de un programa. Véase A.

E/S contada — Transferencia secuencial de bytes mediante 5E/5F. El número de llamadas de escritura contada no es necesariamente el número de registros de una aplicación. Véase 17, 26, 32.

ESC — Carácter de escape, 0x1B, que en las instrucciones de teclado se escribe Esc. Introduce varias acciones de control de CSI y secuencias de salida de las clases de terminal. Esos usos tienen consumidores distintos. Véase 4, 28.

espacio sobrante — Bytes asignados más allá de los datos actuales. Están asignados, pero no pertenecen al contenido útil actual del archivo. Véase 13, 26.

ETX — Carácter de fin de texto (end of text), 0x03. En la ruta de salida de ALP, es uno de los caracteres que pueden provocar el vaciado de la salida con búfer. Su nombre no lo convierte en un comando universal de terminación de programas. Véase 28.

EXEC — Intérprete de archivos de comandos y tipo de archivo secuencial. Escribirlo completo evita el defecto establecido de transferencia de tokens de EX/EXE. Véase 8, 30.

EXECLANG — Ejecutable instalado que implementa EXEC; es distinto de los archivos de trabajo EXECn. Véase 8, 21.

EXTEND — Modo de adición secuencial. Localiza el último sector y recorta los 1A finales; un extremo cero puede dañar metadatos reservados. Véase 17, 26.

FF — En las disposiciones hexadecimales, FF designa el byte 0xFF: según el contexto, una entrada borrada, un propietario especial, una asociación sin asignar u otro valor centinela. En los nombres de controles del terminal, FF significa salto de página (form feed), el carácter 0x0C. Distingue el nombre del carácter del valor hexadecimal del byte. Véase 28; E.

FIFO — Primero en entrar, primero en salir (first in, first out): una cola en la que el elemento sin leer más antiguo se consume primero. El término también designa una interfaz de datos de la controladora; no especifica su capacidad ni su comportamiento ante errores. Véase 25, 27; B.

FIN — Comando de desconexión de la cuenta, no un apagado global documentado. Véase 3, 9, 14.

FLOPPY1 / **FLOPPY2** — Definiciones de dispositivos físicos de disquete atendidas por SISTEMA.DIS1 y SISTEMA.DIS2. Una letra de disco lógico como A debe asociarse con la definición prevista antes de acceder a los archivos. Véase 11, 27, 34.

FM-7 — Ordenador de Fujitsu que también se ofreció con la marca Secoinsa durante la entrada de esta en la informática personal y educativa. Es un producto distinto del ordenador Secoinsa Serie 20. Véase el prefacio.

formato A — Formato ejecutable absoluto. Su entrada de directorio aporta la dirección de carga; es distinto de la unidad A. Véase 18, 30; A.

formato D — Almacenamiento directo de registros de longitud fija. Los números de registro empiezan en uno. Véase 17, 26; E.

formato I — Almacenamiento indexado con un prefijo ordenado de números de hueco y enlaces al hueco siguiente. Véase 17, 26; E.

formato K — Huecos de clave y datos por dispersión, sin el prefijo de índice ni el enlace final al hueco siguiente del formato I. Véase 17, 26; E.

formato R — Imagen reubicable en código máquina con una longitud definida en el directorio y registros de reubicación por sectores. Véase 18, 30; A.

formato S — Sectores secuenciales enlazados con 254 bytes de datos cada uno. Véase 17, 23, 26; E.

Fujitsu — Empresa tecnológica japonesa que fue socia fundadora de Secoinsa y aportó tecnología a su gama de productos. Más tarde, Fujitsu España absorbió Secoinsa. Véase el prefacio.

HDD — Unidad de disco duro (hard-disk drive). Esta abreviatura aparece en las descripciones de los diagnósticos de arranque y de la configuración del hardware. Véase 21, 27; B.

HL — Par de registros del Z80 formado por H y L, que se usa habitualmente para punteros, recuentos y resultados. HL′ es su par alternativo. Interprétalo según cada interfaz. Véase 19, 32; E.

IBM 3741 — El formato de intercambio de disquetes que manejan INICIBM, DIRIBM, SECIBM e IBMSEC. Sus cabeceras, sus registros y la conversión de caracteres requieren esas utilidades especializadas. Véase 11, 31.

IFF — Estado de habilitación de interrupciones del Z80, al que se suele hacer referencia mediante IFF1 e IFF2. Es independiente de los indicadores de AF. El marco del planificador documentado no establece campos IFF guardados. Véase 24, 25; E.

IM — Modo de interrupción del Z80. Véase IM 0 para el modo que usa la interfaz de interrupciones documentada; no deduzcas otro modo solo a partir de la dirección de un vector. Véase 24; B.

IM 0 — Modo de interrupción del Z80 que usa un código de operación suministrado; la interfaz de interrupciones aporta instrucciones de reinicio. Véase 24; B.

IMP — Definición de impresora física asociada a SISTEMA.DIS22. Normalmente, una aplicación selecciona una impresora lógica como IMPR1, que ASOCIAR conecta con IMP. Véase 12, 28, 34.

IMPEXEC — Ajuste que permite la ejecución implícita de archivos de comandos. Véase 8, 31.

IMPR1–IMPR4 — Nombres de impresoras lógicas disponibles mediante las asociaciones de la partición. PRINT1 es una opción que selecciona una impresora en comandos como LISTAR; IMP es una definición de impresora física. Véase 12, 23, 34.

INDEXED — Organización de archivo de BASIC con claves ordenadas y un prefijo de índice. Corresponde al almacenamiento nativo de formato I y es distinta de KEYED. Véase formato I; 17, 26; D, E.

indicador Z — Indicador de cero del Z80. Puede informar de un resultado cero o de una condición específica de la interfaz; algunos servicios devuelven A y los indicadores con significados distintos. Véase 19, 32.

índice del planificador — Identificador del registro de ejecución, empezando en cero, distinto del número de partición mostrado y del selector de banco. Véase 22, 25.

INI — Instituto Nacional de Industria, el grupo industrial del Estado español y socio fundador de Secoinsa. Véase el prefacio.

IP — Definición de impresora física en paralelo asociada a SISTEMA.DIS11. Aquí, IP es un nombre de dispositivo, no una dirección de red de Internet. Véase 28, 34; B.

IPL — Carga inicial del programa (initial program load) / ruta de reinicio, incluida Esc seguida de I. Rechazar la petición especificada finaliza el comando en lugar de reanudarlo. Véase 14, 21.

IS 9502 — El terminal informático de Secoinsa que acompañaba al ordenador Secoinsa Serie 20, con teclado independiente y una pantalla monocroma verde de 24 líneas por 80 caracteres. Véase el prefacio; 4.

IX — Registro índice del Z80. Su función varía según la interfaz; no des por supuesto que apunte siempre al mismo tipo de estructura. Véase 19, 24, 32; E.

IY — Registro índice del Z80. En las rutinas documentadas suele apuntar a datos del ejecutivo, del contexto o de un descriptor, pero el significado necesario depende de quien llama. Véase 19, 24, 32; E.

KEYED — Organización de archivo de BASIC que usa claves por dispersión en lugar del índice ordenado de los archivos INDEXED. Corresponde al almacenamiento nativo de formato K. Véase formato K; 17, 26; D, E.

lápida — Marca FF de un hueco con clave o de una entrada de catálogo borrados; su interpretación depende de la estructura que la rodea. Véase 26; E.

LBA — Número de sector lógico (logical block address) dentro de un volumen. Tradúcelo mediante el sentido y la tabla de sustitución de ese volumen para obtener las coordenadas CHS. Véase 26, 27.

LE — Orden de bytes little-endian: el byte menos significativo va primero. LE16 significa un valor de 16 bits y dos bytes en ese orden. El orden de bytes debe leerse de la estructura especificada, no suponerse a partir del tipo de archivo. Véase A, E.

LE16 — Valor little-endian de 16 bits que ocupa dos bytes. Véase LE; A, E.

LF — Carácter de salto de línea (line feed), 0x0A. La salida al terminal lo usa para avanzar de línea; Intro envía la entrada de CSI con CR. Las opciones de dispositivo que empiezan por LF pueden especificar el retardo del salto de línea. Véase 4, 12, 28.

LIFO — Último en entrar, primero en salir (last in, first out): la entrada que se pone en cola más recientemente se coloca delante de la anterior. Es independiente de CONTROL STACK/NOSTACK de EXEC. Véase 35.

mapa de bits — Mapa de asignación con los bits en orden del más significativo al menos significativo, uno por cada bloque de 1 KiB. Véase 26; E.

marco — Datos guardados de registros de la CPU y de continuación, o el estado de interpretación guardado de 512 bytes de un nivel de EXEC; indica de cuál se trata. Véase 8, 25; E, A.

MENSERR — Archivo directo que contiene 200 registros fijos de mensajes de 64 bytes. Véase 24, 33.

MM58167 — Dispositivo de reloj de tiempo real mencionado en el mapa del hardware. Sus contadores y sus registros de control son independientes de la representación compacta de las marcas de tiempo del sistema de archivos. Véase B.

módulo — Término que se usa en las presentaciones nativas de memoria y también para unidades de código cargadas; no es automáticamente una partición del planificador. Véase 12, 20.

NUCLEO — Nombre del archivo del núcleo y de la región de memoria nativa. La imagen de arranque almacenada incluye material de inicialización además del cuerpo residente. Véase 20, 21.

núcleo (kernel) — El ejecutivo residente y las superposiciones del núcleo asociadas de SISTEMA.NUCLEO; las presentaciones nativas muestran por separado la extensión del SPOOLER. Véase 20, 24.

NUL — Carácter nulo o byte cero, 0x00. Su función depende de la estructura: puede terminar una cadena o marcar otra condición. No es intercambiable con un espacio, un CR o un EOF. Véase 19, 32; A, E.

número de serie de la instalación — Vinculación por software de tres bytes que comprueban determinadas protecciones de los comandos y de BASIC; no tiene relación con los baudios del puerto serie. Véase 10, 16, 30.

número de servicio en línea — Byte situado inmediatamente después de RST08 que identifica uno de los 111 servicios. Véase 18, 24, 32.

OASIS — Linaje que indican los derechos de autor originales y las plantillas de cinta. Se desconoce la compatibilidad exacta de versión o de API. Véase A.

OPEN — Operación de preparación de un canal cuyo modo puede sustituir inmediatamente un archivo secuencial. No es intrínsecamente de solo lectura. Véase 17, 26, 32.

OS — Sistema operativo (operating system): el ejecutivo, los servicios y el entorno de comandos asociado que se describen en este documento. La versión identificada es S.O. Multi-usuario versión 6.2. Véase 1, 20.

PAB — Término presente en los mensajes originales cuyo significado completo como objeto o en la cadena de herramientas no se ha establecido en este sistema operativo. Véase 33; A.

PAN — Familia de definiciones de terminales físicos, como PAN1 y PAN2. PAN1–PAN4 usan los controladores de consola ALP establecidos; otros números pueden designar interfaces opcionales. Son nombres de dispositivo, no nombres de cuenta. Véase 12, 28, 34.

partición — El contexto de ejecución planificado del sistema operativo, con memoria, canales, asociaciones e identidad. En líneas generales es comparable a un proceso, pero sin presuponer una ABI ni una frontera de protección modernas. Véase 12, 22, 25.

PC — Contador de programa (program counter), el registro del Z80 que identifica la siguiente instrucción. PC en las tablas de estado de ejecución y de memoria no significa ordenador personal. Véase 12, 24, 25; E.

procesador de comandos — Lee la entrada de comandos, resuelve los comandos e inicia las rutas de ejecutables o intérpretes. Véase 5, 30 y CSI.

proceso fantasma — Término de la ayuda original para un proceso en segundo plano. La sintaxis completa de activación y su ciclo de vida siguen sin especificarse. Véase 7, 31.

prólogo — Secuencia de entrada opcional CF41 de los comandos, con los datos de privilegio y número de serie que la preceden. Véase 10, 30.

propietario — Byte de cuenta en una entrada de archivo, o marca derivada del índice del planificador en un bloqueo. Las dos representaciones son distintas. Véase 10, 22, 25.

RAM — Memoria de acceso aleatorio (random-access memory) que se usa para el ejecutivo, las particiones, los búferes y los programas en ejecución. La arquitectura la divide en memoria común y regiones seleccionadas por banco. Véase 2, 20; B.

RAM común — La región baja de 16 KiB compartida por todas las selecciones de banco una vez eliminado el alias de reinicio. Véase 20; B.

residente — Que se mantiene en la disposición de memoria establecida durante la ejecución normal de comandos; no implica que todos los bytes sean comunes a todos los bancos. Véase 20, 29.

resultado de comando — El valor indicado como CR tras ejecutar un comando. No certifica la persistencia en el soporte ni la entrega de mensajes. Véase 13, 33.

reubicación — Suma de la base de carga seleccionada a las palabras de dirección identificadas en una imagen R. Véase 18, 30; A.

ROM — Memoria de solo lectura (read-only memory) que contiene el firmware de arranque. Su función de diagnóstico y de arranque es independiente del ejecutivo y del entorno BASIC que se cargan desde disco. Véase 2, 21; B.

RST — Instrucción de reinicio (restart) del Z80 que transfiere el control a un vector fijo. RST08 lleva las llamadas al sistema operativo; los demás vectores tienen usos distintos. Véase 18, 24; B.

RTC — Reloj de tiempo real (real-time clock), el hardware que mantiene la información de fecha y hora. Véase MM58167; B.

RTS — Petición de envío (request to send), una señal de control de la interfaz serie. La interpretación del cableado del control de flujo de recepción de ALP como RTS sigue siendo probable, no totalmente especificada. Véase 28; B.

RX / TX — Recepción y transmisión, respectivamente, en los mapas de las controladoras serie y de interrupciones. La entrada de recepción y la finalización de la transmisión pueden usar búferes y devoluciones de llamada distintos. Véase 28; B.

S20 — Designación de componente en el nombre BASIC S20. Identifica el entorno BASIC del ordenador Secoinsa Serie 20; su número de versión es independiente de la versión del sistema operativo. Véase 1, 15.

SCC — Controladora de comunicaciones serie (serial communications controller). Las rutas serie de ALP usan canales SCC para la comunicación con los terminales, con un apoyo de DMA independiente para la salida. Véase 28; B.

Secoinsa — Sociedad Española de Comunicaciones e Informática, S.A., la empresa española de informática y comunicaciones cuyo ordenador Secoinsa Serie 20 se describe aquí. Véase el prefacio.

sector — Unidad de 256 bytes en este sistema de archivos lógico y en la interfaz de disco duro; la traducción de los disquetes puede implicar sectores físicos de 128 bytes. Véase 23, 27.

SEQUENTIAL — Organización de archivo de BASIC que usa registros secuenciales sobre sectores enlazados de formato S. El comportamiento de sustitución, adición y posicionamiento depende del modo de OPEN seleccionado. Véanse formato S y EXTEND; 17, 26; D, E.

sinónimo — Nombre alternativo de comando configurado, opcionalmente con una longitud mínima de prefijo. Los sinónimos personalizados y los archivos de comando reales pueden tener prioridad sobre la abreviatura estándar. Véase 5, 30, 35.

SIO / PIO / PSIO / CPSIO — Etiquetas de controlador o de clase en las definiciones de dispositivos físicos. Identifican el comportamiento y los valores predeterminados de la interfaz; usa la definición y el controlador asociado en lugar de deducir un contrato completo del dispositivo solo a partir de las letras. Véase 12, 23, 28, 34; C.

SISTEMA — La cuenta de administración que se usa en los procedimientos y, por otro lado, una raíz común de los nombres de archivo de los componentes del sistema operativo. SISTEMA como cuenta, SISTEMA.CSI como archivo y S como disco lógico son identificadores distintos. Véase 3, 5, 10; C.

S.O. Multi-usuario — El nombre que muestra el sistema operativo; este documento describe la versión 6.2. CSI, EXEC y BASIC llevan sus propios números de versión de componente. Véase 1.

SP — Puntero de pila (stack pointer), el registro del Z80 que localiza la pila actual. El SP guardado de una partición apunta a las palabras de registros y a una dirección de continuación que se usan al reanudar la ejecución. Véase 19, 24, 25; E.

SPOOLCOL — Archivo directo de cola con registros de dieciséis bytes que usa el SPOOLER residente. Véase 29; E.

SPOOLER — Recolector residente de salida diferida y gestor de impresoras. La recogida nativa está establecida; la impresión física y el reinicio completos siguen incompletos. Véase 29, 31.

SS2 — Agrupación de interfaces que incluye, en esta arquitectura, las rutas de temporizador e interrupciones y rutas serie opcionales; el encaminamiento físico completo sigue siendo incierto. Véase 28; B.

ST0 / ST1 / ST2 — Bytes de resultado y estado de la controladora de disquetes. Interpreta cada bit mediante la traducción de errores documentada del controlador; no son valores de resultado de comando de CSI. Véase 27; B.

STARTUP — Componente ejecutable de inicialización del sistema, SISTEMA.STARTUP. Reubica e inicializa el código del sistema; no es un archivo de comandos EXEC editable. Véase 2, 21; C.

superposición (overlay) — Código reemplazable que se carga en una ventana de ejecución compartida. Identifica su número antes de interpretar una dirección. Véase 20, 24.

TAP — Definición física del controlador de cinta, asociada a SISTEMA.DIS10. Normalmente se conecta al nombre lógico TAPE1. Véase 13, 28, 34.

TAPE1 — Nombre lógico del dispositivo de cinta que usan las utilidades de cinta y la inicialización del controlador. Se resuelve mediante una asociación con TAP; no es una letra de disco ni un directorio de un sistema de archivos. Véase 13, 28, 34.

TC — Fin de cuenta (terminal count), una indicación de finalización del DMA. En los mapas del hardware, se refiere a la finalización de una transferencia y no debe leerse como un recuento de usuarios de terminal ni de cuentas. Véase 28; B.

TE — Campo de tiempo transcurrido que se imprime en los informes de finalización de los comandos, expresado en minutos. Es independiente de CR, el resultado del comando, y no es otro código de error.

Telesincro — Empresa española de electrónica y de ordenadores de gestión cuya gama de productos Factor y cuya experiencia técnica pasaron a formar parte del grupo Secoinsa. Véase el prefacio.

tipo de archivo — El segundo campo de ocho bytes del nombre de archivo. No determina por sí solo el formato de almacenamiento. Véase 5, 23.

tramo libre — Bloques consecutivos sin asignar disponibles para una asignación contigua. Puede ser menor que el espacio libre total. Véase 11, 26.

UPDATE — Modo de acceso de BASIC. UPDATE secuencial sin EXTEND sustituye el archivo; UPDATE directo usa registros numerados. Véase 17, 26.

uso compartido — Acceso al archivo del propietario a través de un alias del destinatario. Es distinto de la transferencia de propiedad y de la admisión de aperturas compartidas. Véase 10, 17, 25.

VOL1 / HDR1 / HDR2 — Identificadores de las estructuras de etiquetas de cinta de 80 bytes. VOL1 es una etiqueta de volumen; HDR1 y HDR2 son registros de etiqueta de archivo. Que los nombres de las etiquetas resulten conocidos no establece por sí solo la compatibilidad con otro formato de cinta. Véase 13; A.

volumen — Sistema de archivos lógico con etiqueta, geometría, directorio y mapa de bits; sus límites y su correspondencia se guardan en los metadatos del volumen. Véase 5, 23, 27.

volumen del sistema — La unidad S, cuyo sistema de archivos asociado aporta el núcleo, CSI, los mensajes y otros componentes necesarios. Véase 11, 21, 27.

WD / WD1001 — Nombres que se usan para la controladora de disco duro y su interfaz de registros. Los reintentos del controlador, las transferencias por el registro de datos y los bits de estado deben interpretarse según esa interfaz. Véase 27; B.

Z80 — La familia de procesadores que ejecuta este sistema operativo. Los ejemplos de programación nativa usan sus registros, sus instrucciones de reinicio y su espacio de direcciones de 16 bits. Véase 18–20; B, E.

Índice alfabético

Los enlaces llevan directamente al apartado correspondiente. Las referencias a capítulos y apéndices siguen siendo útiles en la versión impresa.

Conceptos

Término	Referencia
acarreo	19, 32
alias de COMPART	10, 22; E
ALP	28; B
apertura exclusiva	17, 25
archivo público	5, 10, 26
área de trabajo	17, 24; E
asociación	12, 23, 34
banco	20, 22, 25
BASICCOM	15, 16; A
BASICOBJ	15; A
BASICUSR	Llamadas a extensiones, 19; A
BCD	19; A, B
bloque de asignación	23, 26; E
bloqueo de registros	17, 25; E
BUZON	9, 29, 31
byte de formato	23; E
canal	17, 24; E
carácter CR	4, 33
cargador de arranque (bootstrap)	3, 21, 27
catálogo	5, 23, 26
cesión del control	22, 25
CHS	27
CLASE	Configuración del terminal, 12, 28
clase de terminal	28
COMANDO	7, 30, 31
contexto	24, 25; E

Término	Referencia
CSI	1, 5, 30; Glosario
cuenta	9, 10, 22
descriptor	23, 28; E, B
devolución de llamada (callback)	19, 25, 28
dispositivo físico	23, 28, 34
dispositivo lógico	12, 23, 34
DMA	28; B
entrada de directorio	26; E
EOF	17, 26, 33
E/S contada	17, 26, 32
espacio sobrante	13, 26
EXEC	8, 30
EXECLANG	8, 21
EXTEND	17, 26
FF	E
FIN	3, 9, 14
formato A	18, 30; A
formato D	17, 26; E
formato I	17, 26; E
formato K	17, 26; E
formato R	18, 30; A
formato S	17, 23, 26; E
IM 0	24; B
IMPEXEC	8, 31
índice del planificador	22, 25
IPL	14, 21
IY/IX	19, 24; E, A
lápida	26; E
LBA	26, 27

Término	Referencia
mapa de bits	26; E
marco	8, 25; E, A
MENSERR	24, 33
módulo	12, 20
NUCLEO	20, 21
núcleo (kernel)	20, 24
número de serie de la instalación	10, 19, 30
número de servicio en línea	18, 24, 32
OASIS	A
OPEN	17, 26, 32
PAB	33; A
partición	12, 22, 25
procesador de comandos	5, 30
proceso fantasma	7, 31
prólogo	10, 30
propietario	10, 22, 25
RAM común	20; B
residente	20, 29
resultado de comando	13, 33
reubicación	18, 30; A
RST	18, 24; B
sector	23, 27
SPOOLCOL	29; E
SPOOLER	29, 31
SS2	28; B
superposición (overlay)	20, 24
tipo de archivo	5, 23
tramo libre	11, 26
UPDATE	17, 26

Término	Referencia
uso compartido	10, 17, 25
volumen	5, 23, 27
volumen del sistema	11, 21, 27

Procedimientos administrativos

Tarea	Referencia
Listado de cuentas: cuentas registradas y usuarios conectados	10
Creación de cuentas	10
Eliminación de cuentas	10
Cambio, conservación y eliminación de contraseñas	10
Listado de particiones	12
Creación y activación de particiones	12
Detención de particiones	12
Eliminación de particiones	12
Asociación y montaje de disquetes	11
Copia de archivos en disquetes	11
Formateo de disquetes	11
Formateo y verificación del disco duro	11
Volumen adicional, preparación condicional	11
Archivado en disquetes y restauración	13
Asociación de la cinta	13
Inicialización y tensado de cintas	13
Copia de seguridad en cinta	13
Restauración desde cinta	13

Comandos

Término	Referencia
ACTIVAR	Activar una partición o un comando en segundo plano
ARCHDIR	Listar un disquete de archivado
ARCHIVAR	Archivar archivos, cuentas o volúmenes
ASIGNAR	Asignar canales
ASOCIAR	Asociar dispositivos lógicos y físicos
ATRIB	Renombrar o proteger archivos
AYUDA	Mostrar la ayuda de los comandos
BACKUP	Copiar un soporte completo de disco o de cinta
BASIC	Editar, interpretar o compilar BASIC
BORRAR	Borrar archivos y liberar almacenamiento
BSC	Configurar las comunicaciones 2780/3780
BUSCAR	Comprobar si existe un archivo
BUZON	Leer y eliminar los mensajes almacenados
CAMBIAR	Cambiar el privilegio mínimo de un comando
CAMCU	Cambiar el número de una cuenta existente
CAMPRO	Transferir la propiedad de archivos privados
CARGAR	Ejecutable ausente; entrada conservada en la tabla de CSI
CEDIT	Editar registros directos, indexados o con clave
CIFRA	Calcular un valor de comprobación de un archivo
CLASIF	Ejecutable ausente; no es un alias de SORT
CLAVIN	Función auxiliar CLAVE.CUENTA instalada; al iniciarse devuelve CR 254
COMPART	Conceder, revocar o listar el acceso compartido
CONECTAR	Conectarse a otra cuenta
CONSULTA	Entrar en el lenguaje de consulta en español
COPFICH	Copiar registros con clave entre archivos
COPIFI	Copiar archivos

Término	Referencia
CORLUZ	Volver a contar los registros indexados o con clave
CREAR	Crear archivos estructurados
CUENTA	Mantener las cuentas de usuario
DESCARGAR	Ejecutable ausente; nombre de búsqueda DESCARGA
DIRIBM	Listar un disquete IBM-3741
DUPLICAR	Duplicar disquetes a través del almacenamiento en disco
EDITAR	Editar texto secuencial
ELIMINAR	Eliminar una entrada de catálogo dañada
ENVIAR	Enviar un archivo de forma síncrona
ENVIARA	Enviar un archivo de forma asíncrona
ESPIAR	Mostrar la actividad de la consola de otro usuario
ESTADO	Examinar los ajustes y el estado del sistema
EXEC	Ejecutar un archivo de comandos
FIN	Desconectar la cuenta
FISDEF	Mostrar las pistas físicas defectuosas
FORZAR	Inyectar un comando en otra partición
GENSIST	Guardar la configuración actual del sistema
HASH	Examinar la distribución de claves
IBMSEC	Importar registros IBM-3741
INICDISC	Inicializar un disco o cambiar su etiqueta
INICIAR	Inicializar un disquete
INICIBM	Escribir cabeceras de disquete compatibles con IBM
INITTAPE	Inicializar o borrar un cartucho
LCOMPART	Examinar o reconstruir el uso compartido y las cuentas
LISDIR	Listar las entradas del catálogo
LISTAR	Mostrar o imprimir un archivo
LOGDEF	Mostrar las pistas lógicas defectuosas
MENSAJE	Enviar mensajes en directo o almacenados
MODIMPR	Cambiar las dimensiones de la impresora

Término	Referencia
MONTAR	Notificar al sistema un cambio de soporte
O8PRECOM	Utilidad con una interfaz de uso sin especificar
ORGDIR	Reorganizar el catálogo
PANTALLA	Diseñar pantallas de entrada de datos
PARAR	Detener una partición
PONER	Cambiar los ajustes del sistema o de la sesión
RECARGAR	Restaurar archivos, cuentas o volúmenes archivados
RECIBIR	Recibir archivos síncronos
RECIBIRA	Recibir archivos asíncronos
REPARAR	Comprobar y conciliar los metadatos del sistema de archivos
REVIVIR	Recuperar una entrada de catálogo ASCII borrada
RUN	Ejecutar BASIC compilado
SECIBM	Exportar registros secuenciales a IBM-3741
SECTOR	Listar los sectores ocupados por un archivo
SERIAL	Editar la vinculación de un ejecutable a un número de serie
SORT	Ordenar registros por claves
SPOOLER	Gestionar la impresión diferida
TECLAS	Emitir secuencias de programación del terminal
TEDITAR	Editar archivos pequeños
TERMINAL	Usar una consola como terminal remoto
TITULO	Imprimir rótulos en letras grandes
UNOSEC	Reconstruir el sector lógico 1
VERFORM	Comprobar el formato de los soportes
VOLCAR	Volcar los bytes de un archivo o de un sector lógico

BASIC y controles de compilación

Término	Referencia
AUTO	Explicación y ejemplo
BOTTOM	Explicación y ejemplo
BREAK AT	Explicación y ejemplo
BREAK ON (cambio)	Explicación y ejemplo
BREAK ON (comparación)	Explicación y ejemplo
CHANGE	Explicación y ejemplo
CONTINUE	Explicación y ejemplo
DELETE (código fuente)	Explicación y ejemplo
INDENT	Explicación y ejemplo
LENGTH	Explicación y ejemplo
LIST	Explicación y ejemplo
LOAD	Explicación y ejemplo
LOCATE	Explicación y ejemplo
LPnLIST	Explicación y ejemplo
LPnXREF	Explicación y ejemplo
MODIFY	Explicación y ejemplo
NAME	Explicación y ejemplo
NEW	Explicación y ejemplo
QUIT (comando)	Explicación y ejemplo
RENUMBER	Explicación y ejemplo
RUN (comando)	Explicación y ejemplo
SAVE	Explicación y ejemplo
STEP	Explicación y ejemplo
TOP	Explicación y ejemplo
TRACE	Explicación y ejemplo
UNBREAK	Explicación y ejemplo

Término	Referencia
UNTRACE	Explicación y ejemplo
VARS	Explicación y ejemplo
XREF	Explicación y ejemplo
Presentación por número de línea	Explicación y ejemplo
Ctrl + Z	Explicación y ejemplo
Tecla LF	Explicación y ejemplo
Tecla Intro	Explicación y ejemplo
CASE	Explicación y ejemplo
CEND	Explicación y ejemplo
CHAIN	Explicación y ejemplo
CLEAR	Explicación y ejemplo
CLOSE	Explicación y ejemplo
COMMON	Explicación y ejemplo
CSI	Explicación y ejemplo
DATA	Explicación y ejemplo
DEF FN	Explicación y ejemplo
DELETE (registro)	Explicación y ejemplo
DIM	Explicación y ejemplo
ELSE	Explicación y ejemplo
END	Explicación y ejemplo
FNEND	Explicación y ejemplo
FOR	Explicación y ejemplo
GET	Explicación y ejemplo
GOSUB	Explicación y ejemplo
GOTO	Explicación y ejemplo
IF	Explicación y ejemplo
IFEND	Explicación y ejemplo
INPUT	Explicación y ejemplo
LET	Explicación y ejemplo

Término	Referencia
LINK	Explicación y ejemplo
LINPUT	Explicación y ejemplo
MAT (copia)	Explicación y ejemplo
MAT (relleno)	Explicación y ejemplo
MAT INPUT	Explicación y ejemplo
MAT PRINT	Explicación y ejemplo
MAT READ	Explicación y ejemplo
MAT WRITE	Explicación y ejemplo
MOUNT	Explicación y ejemplo
NEXT	Explicación y ejemplo
ON ERROR	Explicación y ejemplo
ON GOTO	Explicación y ejemplo
ON GOSUB	Explicación y ejemplo
OPEN	Explicación y ejemplo
OPTION	Explicación y ejemplo
OTHERWISE	Explicación y ejemplo
PRINT	Explicación y ejemplo
PUT	Explicación y ejemplo
QUIT (instrucción)	Explicación y ejemplo
RANDOMIZE	Explicación y ejemplo
READ	Explicación y ejemplo
READNEXT	Explicación y ejemplo
REM	Explicación y ejemplo
RESTORE	Explicación y ejemplo
RESUME	Explicación y ejemplo
RETURN	Explicación y ejemplo
RUN (instrucción)	Explicación y ejemplo
SELECT	Explicación y ejemplo
SLEEP	Explicación y ejemplo

Término	Referencia
STOP	Explicación y ejemplo
THEN	Explicación y ejemplo
WAIT	Explicación y ejemplo
WAIT (dispositivo)	Explicación y ejemplo
WEND	Explicación y ejemplo
WHILE	Explicación y ejemplo
WRITE	Explicación y ejemplo
ABS	Explicación y ejemplo
ASC	Explicación y ejemplo
AT$	Explicación y ejemplo
ATN	Explicación y ejemplo
BIN	Explicación y ejemplo
BINOF$	Explicación y ejemplo
CHR$	Explicación y ejemplo
COS	Explicación y ejemplo
CRT$	Explicación y ejemplo
DATE$	Explicación y ejemplo
DAY	Explicación y ejemplo
DEL$	Explicación y ejemplo
DTE$	Explicación y ejemplo
EOF	Explicación y ejemplo
ERL	Explicación y ejemplo
ERR	Explicación y ejemplo
EXP	Explicación y ejemplo
EXT$	Explicación y ejemplo
FIX	Explicación y ejemplo
FLOAT	Explicación y ejemplo
FORMAT$	Explicación y ejemplo
HEX	Explicación y ejemplo

Término	Referencia
HEXOF$	Explicación y ejemplo
INP	Explicación y ejemplo
INS$	Explicación y ejemplo
INT	Explicación y ejemplo
LEFT$	Explicación y ejemplo
LEN	Explicación y ejemplo
LINE	Explicación y ejemplo
LOG	Explicación y ejemplo
LPAD$	Explicación y ejemplo
LRL	Explicación y ejemplo
LRR	Explicación y ejemplo
LSL	Explicación y ejemplo
LSR	Explicación y ejemplo
LTRIM$	Explicación y ejemplo
MATCH	Explicación y ejemplo
MAX	Explicación y ejemplo
MID$	Explicación y ejemplo
MIN	Explicación y ejemplo
MOD	Explicación y ejemplo
NBR	Explicación y ejemplo
OCT	Explicación y ejemplo
OCTOF$	Explicación y ejemplo
OVR$	Explicación y ejemplo
PAGE	Explicación y ejemplo
PI	Explicación y ejemplo
POS	Explicación y ejemplo
REP$	Explicación y ejemplo
RIGHT$	Explicación y ejemplo
RND	Explicación y ejemplo

Término	Referencia
ROUND	Explicación y ejemplo
RPAD$	Explicación y ejemplo
RPT$	Explicación y ejemplo
RTRIM$	Explicación y ejemplo
SCH	Explicación y ejemplo
SEC	Explicación y ejemplo
SGN	Explicación y ejemplo
SIN	Explicación y ejemplo
SPACE$	Explicación y ejemplo
SQR	Explicación y ejemplo
STR$	Explicación y ejemplo
TAN	Explicación y ejemplo
TIME$	Explicación y ejemplo
TRIM$	Explicación y ejemplo
USR	Explicación y ejemplo
USR$	Explicación y ejemplo
VAL	Explicación y ejemplo
OPTION BASE 0	Explicación y ejemplo
OPTION PROMPT	Explicación y ejemplo
OPTION CASE	Explicación y ejemplo
OPTION USR	Explicación y ejemplo
OPTION PRIV	Explicación y ejemplo
OPTION SERIAL	Explicación y ejemplo
UNLOCK	Explicación y ejemplo
OPEN INPUT	Explicación y ejemplo
OPEN OUTPUT	Explicación y ejemplo
OPEN UPDATE DIRECT	Explicación y ejemplo
OPEN UPDATE SEQUENTIAL	Explicación y ejemplo
OPEN EXTEND	Explicación y ejemplo

Término	Referencia
OPEN LOCK	Explicación y ejemplo
LIST del compilador	Explicación y ejemplo
NOLIST del compilador	Explicación y ejemplo
IMPRn del compilador	Explicación y ejemplo
XREF del compilador	Explicación y ejemplo
NOOBJ del compilador	Explicación y ejemplo
NOPANT del compilador	Explicación y ejemplo
OBJ del compilador	Explicación y ejemplo
COMMA del compilador	Explicación y ejemplo
TRACE en tiempo de ejecución	Explicación y ejemplo
COMA en tiempo de ejecución	Explicación y ejemplo

Servicios del sistema

Término	Referencia
0x00	Finalizar el comando / restaurar CSI
0x01	Entrada de línea por consola
0x02	Mostrar una cadena
0x03	Sondear la entrada de consola
0x04	Leer un carácter de la consola
0x05	Salida de caracteres por consola
0x06	Leer un carácter de la consola con eco directo
0x07	Salida de caracteres mediante la clase de terminal
0x08	Salida de caracteres a la asociación de impresora
0x09	Invalidar los campos del descriptor de disco en caché
0x0A	Leer un sector lógico
0x0B	Escribir un sector lógico
0x0C	Devolver la firma del núcleo inicializado
0x0D	Crear/escribir una entrada de directorio
0x0E	Analizar un hexadecimal sin signo
0x0F	Analizar un decimal sin signo
0x10	Dar formato hexadecimal a un byte
0x11	Dar formato decimal a una palabra sin signo
0x12	Nueva línea en la consola
0x13	Retardo activo calibrado
0x14	Buscar una entrada de directorio
0x15	Obtener el descriptor del dispositivo asociado
0x16	Cargar un ejecutable con nombre sin entrar en él
0x17	Salida de cadenas a la asociación de impresora
0x18	Inicializar el nombre/clase de un canal
0x19	Convertir un carácter de unidad en índice de asociación

Término	Referencia
0x1A	Convertir una asociación de disco en carácter
0x1B	Asignar un número exacto de bloques contiguos
0x1C	Liberar un intervalo de bloques contiguos
0x1D	Borrar un archivo con nombre
0x1E	Cargar y ejecutar un comando
0x1F	Renombrar un archivo con nombre
0x20	Abrir un canal
0x21	Cerrar un canal
0x22	Leer un registro de un canal
0x23	Escribir un registro en un canal
0x24	Dar formato a la fecha actual
0x25	Dar formato a la hora actual
0x26	División sin signo
0x27	Multiplicación sin signo
0x28	Leer un registro directo numerado
0x29	Escribir un registro directo numerado
0x2A	Analizar un decimal o un hexadecimal con sufijo H
0x2B	Leer un registro indexado/con clave
0x2C	Leer el siguiente registro indexado/con clave
0x2D	Escribir un registro indexado/con clave
0x2E	Empaquetar la fecha/hora actuales
0x2F	Resolver una etiqueta de volumen o una unidad de un carácter
0x30	Obtener el contexto de la partición actual
0x31	Pausa de paginación
0x32	Leer varios sectores lógicos
0x33	Escribir varios sectores lógicos
0x34	Mostrar una cadena con determinados indicadores de modo suprimidos
0x35	Programar, sustituir o cancelar una devolución de llamada temporizada
0x36	Sustituir la entrada de comandos y volver a CSI

Término	Referencia
0x37	Obtener el límite superior de memoria de la partición
0x38	Establecer el límite superior de memoria de la partición
0x39	Establecer la devolución de llamada de la palabra 28 del contexto
0x3A	Resolver una asociación
0x3B	Obtener los campos de geometría del terminal asociado
0x3C	Borrar un registro indexado/con clave
0x3D	Despacho de inicialización de dispositivos
0x3E	Despacho de estado de dispositivos
0x3F	Despacho de entrada de dispositivos
0x40	Despacho de salida de dispositivos
0x41	Prólogo estándar de privilegio/número de serie de los comandos
0x42	Copiar la etiqueta del volumen
0x43	Establecer la entrada del controlador del dispositivo físico
0x44	Terminar/descargar el controlador asociado
0x45	Sondear y consumir el indicador de evento de consola
0x46	Sustituir la entrada de comandos con límite superior de continuación
0x47	Leer un byte de una cola en memoria
0x48	Añadir un byte a una cola en memoria
0x49	Cambiar los bits de modo de la consola
0x4A	Establecer la devolución de llamada de error de disco
0x4B	Inicializar o cambiar el volumen del sistema
0x4C	Retardo activo escalado
0x4D	Obtener una entrada de la tabla de canales
0x4E	Despachar un comando de control del terminal
0x4F	Ceder el control / planificar
0x50	Obtener la base global
0x51	Leer el byte global 003B
0x52	Obtener el índice actual del planificador
0x53	Liberar los bloqueos de registros del canal

Término	Referencia
0x54	Adquirir un bloqueo de byte
0x55	Liberar un bloqueo de byte
0x56	Entrar en una devolución de llamada en la partición seleccionada
0x57	Obtener la base del mapa de asociaciones
0x58	Enviar texto a la consola de otra partición
0x59	Adquirir el bloqueo global fijo
0x5A	Liberar el bloqueo global fijo
0x5B	Obtener el área de trabajo de sector
0x5C	Obtener el byte de privilegio
0x5D	Comparar memoria
0x5E	Leer bytes secuenciales contados
0x5F	Escribir bytes secuenciales contados
0x60	Mostrar un mensaje del sistema numerado
0x61	Mostrar un mensaje numerado y finalizar el comando
0x62	Cargar un submódulo R a partir de un descriptor compacto
0x63	Asignar un tramo de bloques con tamaño mínimo
0x64	Buscar en una tabla de palabras clave abreviables
0x65	Obtener el número de cuenta
0x66	Filtrar la entrada del dispositivo para detectar controles de escape
0x67	Registrar una devolución de llamada indexada
0x68	Copiar desde el banco del registro seleccionado
0x69	Copiar al banco del registro seleccionado
0x6A	Dar formato a una fecha BCD proporcionada
0x6B	Esperar un evento de dispositivo o una interrupción
0x6C	Buscar un componente residente con nombre
0x6D	Escribir la hora global actual mediante la devolución de llamada del RTC
0x6E	Escribir la fecha global actual mediante la devolución de llamada del RTC

Archivos del sistema

Archivo	Referencia
SISTEMA.NUCLEO	Detalles del componente
SISTEMA.STARTUP	Detalles del componente
SISTEMA.CSI	Detalles del componente
SISTEMA.EXECLANG	Detalles del componente
SISTEMA.CUENTA	Detalles del componente
SISTEMA.CONECTAR	Detalles del componente
CONECTAR.COMANDO	Detalles del componente
SISTEMA.AYUDA	Detalles del componente
SISTEMA.MENSERR	Detalles del componente
SISTEMA.NOMBDISP	Detalles del componente
SISTEMA.IPL	Detalles del componente
SISTEMA.SPOOLCOL	Detalles del componente
SISTEMA.BUZON	Detalles del componente
SISTEMA.DUPLICAR	Detalles del componente
BASIC.COMANDO	Detalles del componente
BASIC.LOADFILE	Detalles del componente
RUN.COMANDO	Detalles del componente
SISTEMA.EXEC1	Detalles del componente
SISTEMA.EXEC2	Detalles del componente
SISTEMA.EXEC3	Detalles del componente
SISTEMA.EXEC4	Detalles del componente
BASIC.OVERLAY1	Detalles del componente
BASIC.OVERLAY2	Detalles del componente
BASIC.OVERLAY3	Detalles del componente
BASIC.OVERLAY4	Detalles del componente
SISTEMA.DIS1	Detalles del componente

Archivo	Referencia
SISTEMA.DIS2	Detalles del componente
SISTEMA.DIS10	Detalles del componente
SISTEMA.DIS11	Detalles del componente
SISTEMA.DIS12	Detalles del componente
SISTEMA.DIS16	Detalles del componente
SISTEMA.DIS17	Detalles del componente
SISTEMA.DIS18	Detalles del componente
SISTEMA.DIS19	Detalles del componente
SISTEMA.DIS20	Detalles del componente
SISTEMA.DIS21	Detalles del componente
SISTEMA.DIS22	Detalles del componente
SISTEMA.DIS23	Detalles del componente
SISTEMA.DIS24	Detalles del componente
SISTEMA.DIS25	Detalles del componente
SISTEMA.DIS26	Detalles del componente
SISTEMA.CLASE1	Detalles del componente
SISTEMA.CLASE2	Detalles del componente
GETPRIV.BASICUSR	Detalles del componente
GETID.BASICUSR	Detalles del componente
GETUSER.BASICUSR	Detalles del componente
CAMPAR.BASICUSR	Detalles del componente
PUTQUET.BASICUSR	Detalles del componente
NOMVOL.BASICUSR	Detalles del componente
TAMDIS.BASICUSR	Detalles del componente
DIREC.BASICUSR	Detalles del componente
CLAVOUT.BASICUSR	Detalles del componente
LINPAN.BASICUSR	Detalles del componente
ESPERA.BASICUSR	Detalles del componente
OCONV.BASICUSR	Detalles del componente

Archivo	Referencia
OCONV2.BASICUSR	Detalles del componente
KEYSET.BASICUSR	Detalles del componente
SORT.OVERLAY1	Detalles del componente
SORT.OVERLAY2	Detalles del componente
CONSULTA.OVERLAY1	Detalles del componente
CONSULTA.OVERLAY2	Detalles del componente
CONSULTA.OVERLAY3	Detalles del componente
CONSULTA.OVERLAY4	Detalles del componente
SPOOLER.OVERLAY1	Detalles del componente
CONSULTA.TRAB1	Detalles del componente
CONSULTA.TRAB2	Detalles del componente
CONSULTA.TRAB3	Detalles del componente
CONSULTA.TRAB4	Detalles del componente
MAESTRO.TITULO	Detalles del componente

Grafía de los comandos y lenguajes de los componentes

Término	Referencia
Abreviaturas, nombres completos y sinónimos	Grafía de los comandos, 5, 30, 35
Palabras clave de opciones y prefijos que se solapan	Opciones de las utilidades, 35
Vocabulario interno de EDITAR	Referencia del editor, 7, 35
Directivas de EXEC, FIFO/LIFO y entrada en cola	Referencia de EXEC, 8, 35
Tokens de consulta de CONSULTA y listado del diccionario	Referencia de consultas, 35
Controles en minúsculas de PANTALLA	Controles de pantalla, 35
Palabras clave adicionales de opciones y operadores de BASIC	Palabras clave adicionales, D
ACTUCAP, EXPAND, IMPCAP y TESTCAP	Apoyo a la captura, C
CONTROL.VOC, GENFICH, GENVOC y VOC	Apoyo al vocabulario, C
CIPL, COPSIS, INFORME, INSER y las bibliotecas de código fuente RUTINA	Componentes de apoyo, C

Archivos de apoyo adicionales

Componente	Referencia
ACTUCAP.BASICCOM	Descripción del componente, C
ACTUCAP.EXEC	Descripción del componente, C
ACTUCAP.PANTALLA	Descripción del componente, C
ACTUCAP2.BASICCOM	Descripción del componente, C
ACTUCAP2.PANTALLA	Descripción del componente, C
CAMBVERS.EXEC	Descripción del componente, C
CIPL.EXEC	Descripción del componente, C
COPIVOC.BASICCOM	Descripción del componente, C
COPIVOC.EXEC	Descripción del componente, C
COPSIS.EXEC	Descripción del componente, C
COPSIS2.EXEC	Descripción del componente, C
DISCO.BASICCOM	Descripción del componente, C
EXPAND.BASICCOM	Descripción del componente, C
EXPAND.EXEC	Descripción del componente, C
EXPAND.MACMODEL	Descripción del componente, C
GENFICH.EXEC	Descripción del componente, C
GENPAN.BASICCOM	Descripción del componente, C
GENVOC.BASICCOM	Descripción del componente, C
GENVOC.EXEC	Descripción del componente, C
GETPART.BASICCOM	Descripción del componente, C
IMPCAP.BASICCOM	Descripción del componente, C
IMPCAP.EXEC	Descripción del componente, C
INFORME.EXEC	Descripción del componente, C
INFORME1.BASICCOM	Descripción del componente, C
INFORME2.BASICCOM	Descripción del componente, C
INFORME3.BASICCOM	Descripción del componente, C

Componente	Referencia
INFORME4.BASICCOM	Descripción del componente, C
INFORME5.BASICCOM	Descripción del componente, C
INSER.EXEC	Descripción del componente, C
INSER1.BASICCOM	Descripción del componente, C
INSER1.EXEC	Descripción del componente, C
LONCLA.BASICCOM	Descripción del componente, C
MAESTRO.POLIZA	Descripción del componente, C
MAESTRO.VOC	Descripción del componente, C
MANTVOC.BASICCOM	Descripción del componente, C
RESUME.BASICCOM	Descripción del componente, C
RUTINA1.BASIC	Descripción del componente, C
RUTINA2.BASIC	Descripción del componente, C
RUTINA3.BASIC	Descripción del componente, C
RUTINA4.BASIC	Descripción del componente, C
SCAPTURA.BASICCOM	Descripción del componente, C
TESTCAP.BASICCOM	Descripción del componente, C
TESTCAP.EXEC	Descripción del componente, C
TRAFIC.BASICCOM	Descripción del componente, C
USOIMPR.BASICMAC	Descripción del componente, C
VOC.BASICCOM	Descripción del componente, C
VOC.EXEC	Descripción del componente, C
VOC.PANTALLA	Descripción del componente, C

Emulación

Término	Referencia
MAME: inicio, menús y cierre	Emulación en MAME
Archivos de diferencias del disco duro y copias de seguridad	Emulación en MAME
Inserción, sustitución y formateo de disquetes	Emulación en MAME
Captura de la impresora serie	Emulación en MAME

Emulación en MAME

Uso del SECOINSA Serie 20 y de S.O. Multi-Usuario 6.2 en MAME.

Usa una versión de MAME que contenga el controlador `secoinsa20` y la entrada de la lista de software `somu62` (S.O. Multi-Usuario 6.2). La emulación funcional está disponible a partir de MAME 0.290.

Los comandos que empiezan por `mame` se introducen en el ordenador anfitrión. Los comandos de S.O. Multi-Usuario se introducen en el indicador > del sistema emulado.

Instalación de las imágenes e inicio de MAME

Coloca el conjunto de ROM de la máquina y el disco del sistema en la ruta de búsqueda de ROM de MAME. Una organización adecuada es:

```
roms/
    secoinsa20.zip
    secoinsa20/
        somu62/
            somu62.chd
```

Inicia la máquina con:

```
mame secoinsa20 somu62 -window
```

Así se seleccionan el disco duro con S.O. Multi-Usuario 6.2 preinstalado y el terminal serie compatible predeterminado. Indicar solo `secoinsa20` no selecciona automáticamente el disco del sistema.

Deja que termine la secuencia de arranque original e inicia sesión como:

```
SISTEMA
```

Esta cuenta no tiene contraseña. La imagen contiene el volumen del sistema, accesible como `:S`.

Por ejemplo:

```
LISDIR SISTEMA.*:S
AYUDA
```

Cambio entre el terminal y los menús de MAME

Durante el funcionamiento normal, el teclado controla el terminal emulado. Para usar los menús de MAME:

1. Pulsa **Bloq Despl** (Scroll Lock) para activar los controles de la interfaz de MAME.
2. Pulsa **Tab** para abrir el menú.
3. Después de hacer los cambios, cierra el menú y desactiva los controles de la interfaz para volver a escribir en el terminal.

En macOS, la tecla predeterminada para activar y desactivar la interfaz es **Supr** (Forward Delete). También puede reasignarse en los ajustes de entrada de MAME.

La consola que se muestra es un terminal serie emulado. El terminal predeterminado entiende las secuencias de control de SECOINSA que usa el sistema; no se necesita ninguna aplicación de terminal aparte ni configurar manualmente la línea serie.

Guardar el trabajo y volver a un sistema limpio

Cuando se selecciona `somu62` mediante la lista de software, MAME guarda los cambios del disco duro en un archivo de diferencias independiente, normalmente:

```
diff/somu62.dif
```

El `somu62.chd` original no cambia. Si vuelves a iniciar MAME con el mismo directorio de diferencias, se sigue usando el disco modificado.

Para mantener una instalación de trabajo independiente:

```
mame secoinsa20 somu62 -window -diff_directory "work/main/diff"
```

Para empezar un experimento desde la imagen limpia del sistema:

```
mame secoinsa20 somu62 -window -diff_directory "work/experiment1/diff" -noautosave
```

Elige un directorio que no contenga ya un archivo de diferencias. Si reutilizas ese directorio, se reanuda el contenido del disco del experimento.

Haz la copia de seguridad del archivo de diferencias con MAME cerrado, junto con su CHD padre correspondiente. El CHD padre por sí solo no contiene el trabajo posterior. Los estados guardados de MAME no sustituyen a las copias de seguridad del disco.

Inicio con un disquete insertado

Para empezar con una imagen de disquete existente:

```
mame secoinsa20 somu62 -window -flop1 "media/work.mfi"
```

En la configuración predeterminada de la máquina, el dispositivo `-flop1` de MAME corresponde a **`FLOPPY2` dentro de S.O. Multi-Usuario**.

Después de iniciar sesión, asocia una letra de disco lógico que no esté en uso y notifica al sistema operativo el soporte insertado:

```
ASOCIAR A FLOPPY2
MONTAR A
LISDIR *.*:A
```

Estas instrucciones suponen que la letra de disco lógico `A` está disponible. Introduce `ASOCIAR` sin argumentos para examinar las asociaciones actuales.

MFI es el formato probado para las imágenes de disquete con escritura. Conserva las densidades de grabación y los tamaños de sector mixtos que usa la utilidad de formateo original. Una imagen de disco con formato de PC normal no es equivalente.

Inserción, extracción y sustitución de disquetes en funcionamiento

Para insertar un disquete sin reiniciar:

1. Activa los controles de la interfaz de MAME y abre **Tab → File Manager** (gestor de archivos).
2. Selecciona la unidad de disquete.
3. Busca la imagen y selecciónala.
4. Elige el modo de acceso cuando se te pida.
5. Vuelve al terminal y usa `ASOCIAR` y `MONTAR` como se ha mostrado antes.

Entre los modos de acceso disponibles están:

Modo	Efecto
Read-only (solo lectura)	Protege la imagen seleccionada contra escrituras.
Read-write (lectura y escritura)	Guarda las escrituras directamente en la imagen seleccionada.
Read this image, write to another image (leer esta imagen y escribir en otra)	Usa una imagen de salida independiente y conserva el origen seleccionado.

Las escrituras en disquete no usan el directorio de diferencias del disco duro. Usa el acceso de solo lectura o una imagen de salida independiente cuando quieras conservar un disquete maestro.

Antes de expulsarlo, termina las aplicaciones que usen el disquete y libera su asociación:

```
ASOCIAR A
```

Después, abre **File Manager**, selecciona la unidad de disquete y elige **[empty slot]** (ranura vacía). En una sesión multiusuario, asegúrate antes de que los demás usuarios también hayan terminado de acceder al disco.

Para insertar un disquete de sustitución, selecciona su imagen en el mismo menú e introduce:

```
ASOCIAR A FLOPPY2
MONTAR A
```

`MONTAR` indica a S.O. Multi-Usuario que el soporte ha cambiado. No inserta una imagen en MAME ni formatea un disco virgen.

Creación y formateo de un disquete nuevo

En **File Manager**, selecciona la unidad de disquete y elige **[create]** (crear). Indica un nuevo nombre de archivo terminado en `.mfi`, selecciona el formato MFI de MAME y, si se ofrece, elige el soporte de 5¼ pulgadas de doble cara y alta densidad sin formatear.

La nueva imagen representa un disco virgen. Vuelve a S.O. Multi-Usuario e introduce:

```
ASOCIAR A FLOPPY2
INICIAR A (FORMAT CARA 2)
```

FORMAT solicita el formateo físico; CARA 2 selecciona dos caras.

El formateo borra el disquete asociado con `A`. Confirma que está insertada la nueva imagen prevista antes de responder a las preguntas de confirmación de la utilidad.

Indica una etiqueta de disco cuando se te pida y deja que termine el formateo. Después, comprueba el resultado:

```
MONTAR A
VERFORM A
LISDIR *.*:A
```

Así se usa el formato nativo probado. La selección de la unidad emulada no establece el fabricante ni el modelo exacto de la unidad física original.

Prueba con otra imagen de disco duro

Cierra MAME antes de cambiar el disco duro del sistema. Para seleccionar directamente un CHD compatible:

```
mame secoinsa20 -harddisk "images/alternative.chd" -window -diff_directory "work/alternative/
diff" -noautosave
```

Así se sustituye la selección de `somu62` en esa ejecución.

La imagen alternativa debe contener una disposición de disco y un software de arranque compatibles con la máquina emulada. Una extensión `.chd` por sí sola no establece la compatibilidad.

Usa una copia de trabajo de una imagen que no conozcas. Las imágenes con escritura conectadas directamente pueden modificarse en el sitio; seleccionar un directorio de diferencias no obliga a todas las imágenes a usar una superposición. El procedimiento con la lista de software `somu62` descrito antes ofrece la organización definida de imagen maestra y diferencias.

Captura de la salida de la impresora serie

Conecta la impresora serie y selecciona un nuevo nombre de archivo de salida:

```
mame secoinsa20 somu62 -window -printerport printer -printout "output.prn"
```

Dentro de S.O. Multi-Usuario:

```
ASOCIAR IMPR1 IMP
LISTAR SISTEMA.NOMBDISP:S (PRINT1 NOHEAD)
```

El archivo de salida contiene los bytes que recibe la impresora, incluidos los caracteres de control. No es un PDF renderizado. Usa un nombre de archivo nuevo para conservar las salidas anteriores.

Fin de una sesión

Termina las aplicaciones en ejecución y libera los soportes extraíbles antes de salir. Introduce:

```
FIN
```

`FIN` cierra la sesión de S.O. Multi-Usuario; no cierra MAME. Cuando los demás usuarios hayan terminado, activa los controles de la interfaz de MAME y sal de la forma normal. Así se pueden guardar las escrituras pendientes en las imágenes.

En el momento de redactar este documento, quedan pendientes las siguientes características de la emulación: el funcionamiento de la cinta no está implementado; no se ha establecido el arranque desde disquete; y el terminal emulado todavía no reproduce por completo la funcionalidad del firmware del terminal original Secoinsa IS 9502.